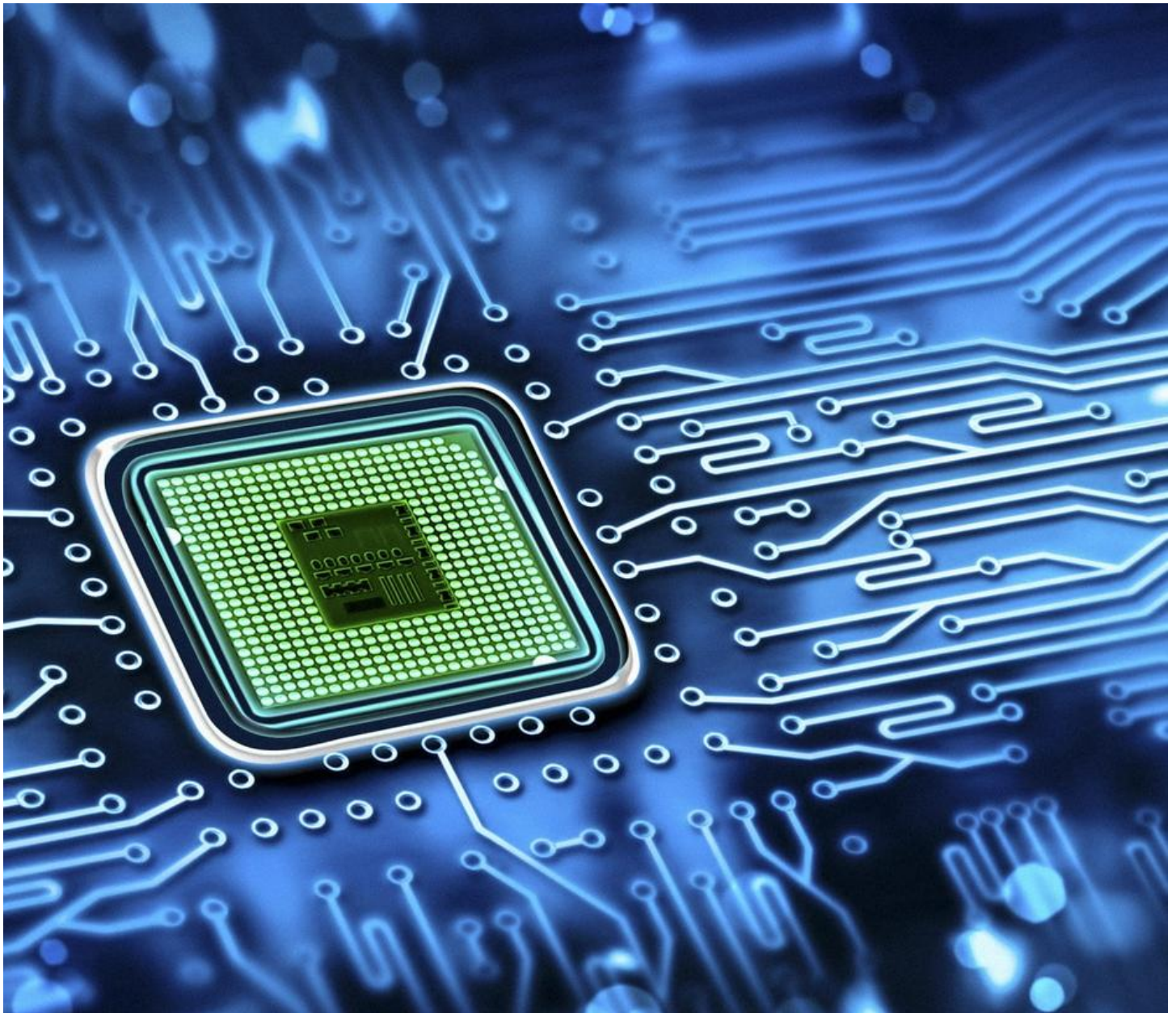




500 Intermediate-Level Embedded Firmware Projects

Aschref Ben Thabet

22 September 2025

Learning Series from scratch
to expert

500 Intermediate-Level Embedded Firmware Projects

Intermediate C Programming for Embedded Systems	16
Project 1: Finite State Machine for Traffic Light Sequence	16
Project 2: Function Pointers for Sensor Data Processing	18
Project 3: Parse 16-Byte Command Packet.....	19
Project 4: Circular Buffer for Sensor Data	21
Project 5: CRC8 Checksum for Data Integrity.....	23
Project 6: 32-Bit Integer Arithmetic with Overflow Detection	24
Project 7: Bit Fields in Struct for Peripheral Registers	25
Project 8: Debounce Algorithm for Push Button	26
Project 9: Lookup Table for Sensor Data Linearization	28
Project 10: Dynamic Array of Sensor Readings with realloc()	29
Project 11: Simple Command-Line Interface over UART	31
Project 12: Struct for Multiple GPIO Configurations	33
Project 13: Floating-Point to Fixed-Point Conversion.....	34
Project 14: Priority-Based Task Scheduler in a Loop	35
Project 15: Multi-Byte Data with Endianness Conversion	37
Project 16: Volatile Variables for Interrupt-Driven Data Access	38
Project 17: Software Timer Using Counter Variable	39
Project 18: Parse JSON String for Configuration Data	41
Project 19: Union for Memory Sharing Between Data Types	42
Project 20: Sliding Window Average for Sensor Data	44
Project 21: Macro for Peripheral Register Settings.....	45
Project 22: Multiple Interrupts with Shared Variables	46
Project 23: Simple Data Compression Algorithm	48
Project 24: Pointer Array for Peripheral Drivers	49
Project 25: Moving Average for ADC Readings.....	51
Project 26: Basic Protocol Encoder for Serial Communication.....	53
Project 27: Static Variable for System State Tracking	54
Project 28: Multi-Level State Machine for Vending Machine.....	55
Project 29: Typedef Struct for Sensor Calibration Data	57
Project 30: Matrix Addition for Sensor Data Processing	58
Project 31: Inline Functions for GPIO Access	59
Project 32: Simple Data Logger with Timestamp	60

Project 33: Bitmask for Controlling Multiple LEDs	61
Project 34: Double-Precision Floating-Point Math	62
Project 35: Basic Error Handler for Peripheral Failures	63
Project 36: Function Pointer Table for Modular Code	64
Project 37: Binary Search on Sorted Sensor Data Array	65
Project 38: Simple Hash Function for Data Lookup	66
Project 39: Volatile Pointer for Shared Memory Access	67
Project 40: Ring Buffer with Head/Tail Pointers	68
Project 41: Basic Packet Parser for Serial Data	70
Project 42: Macro for Peripheral Register Bitfields	72
Project 43: Bubble Sort on Sensor Readings	73
Project 44: Simple Watchdog Reset Mechanism	74
Project 45: Struct for Communication Protocol States	75
Project 46: Multi-Byte Data with Bit Alignment	77
Project 47: Basic FIFO Queue for Sensor Data	78
Project 48: Switch-Case for Protocol Command Handling.....	79
Project 49: CRC16 Checksum for Data Packets	81
Project 50: State Machine with Timeout.....	82
Project 51: Static Array for Peripheral Configuration	84
Project 52: Multi-Threaded Simulation Using Cooperative Tasks	85
Project 53: Basic Data Serializer for UART.....	86
Project 54: Pointer to Struct for Peripheral Control.....	88
Project 55: 64-Bit Integer Arithmetic with Overflow Checks	89
Project 56: Simple Command Parser with Arguments	90
Project 57: Union for Memory-Efficient Data Storage.....	91
Project 58: Linear Interpolation on Sensor Data	92
Project 59: Basic Logging System with Severity Levels.....	93
Project 60: Macro for Interrupt-Safe GPIO Access.....	94
Project 61: Priority Queue for Event Handling.....	95
Project 62: Simple Data Encryption Routine.....	97
Project 63: Volatile Struct for Interrupt-Driven Data	98
Project 64: Checksum Validation for Flash Data	99
Project 65: Event-Driven State Machine	100
Project 66: Function Pointer for Dynamic Callbacks.....	102
Project 67: Circular Buffer with Overflow Detection	103
Project 68: Simple Protocol Decoder for Serial Data	105

Project 69: Bitfield Struct for Memory-Efficient Storage	106
Project 70: Quicksort on Sensor Readings	107
Project 71: Basic Timeout Mechanism for Tasks	109
Project 72: Static Struct for Peripheral State Tracking	110
Project 73: Multi-Byte Protocol with Framing	111
Project 74: Simple Data Validator for Sensor Inputs	113
Project 75: Pointer to Manage a Dynamic Task List.....	114
Microcontroller Configuration and Peripherals	116
Project 76: Configure Multiple GPIO Pins for 4x4 LED Matrix.....	116
Project 77: Read 4x4 Keypad and Display Key Presses on Serial Console	117
Project 78: Control Stepper Motor with Acceleration Control	119
Project 79: Configure ADC to Read Multiple Channels in Scan Mode	121
Project 80: Use DAC to Generate a Sine Wave	122
Project 81: Interface with 16x2 LCD Using GPIO Pins	124
Project 82: Timer for Variable-Frequency PWM Signal	125
Project 83: Timer to Measure Frequency of External Signal.....	126
Project 84: Control DC Motor with PID Using PWM	128
Project 85: Interrupt for Multiple GPIO Button Inputs	129
Project 86: Interface with Temperature Sensor Using ADC	130
Project 87: Timer for Dual PWM Signal for Motor Control.....	132
Project 88: GPIO Pin for Quadrature Encoder with Interrupt	133
Project 89: Control Servo Motor with Precise Angle Control.....	134
Project 90: Read Multi-Axis Accelerometer Using I2C.....	135
Project 91: ADC with Oversampling for Noise Reduction	137
Project 92: DAC to Generate a Triangle Wave.....	138
Project 93: Interface with 7-Segment Display Using Shift Register	139
Project 94: Timer to Generate 1 kHz Interrupt for Periodic Tasks.....	141
Project 95: Control BLDC Motor with Basic Commutation	142
Project 96: Read Pressure Sensor Using SPI	143
Project 97: GPIO Pin for Touch Sensor with Interrupt.....	144
Project 98: Interface with OLED Display Using I2C	145
Project 99: Timer for Precise Delay for Sensor Sampling.....	147
Project 100: ADC to Read Battery Voltage with Calibration	148
Project 101: Control Relay Based on Temperature Sensor Reading.....	149
Project 102: DAC to Generate a Square Wave	150
Project 103: Timer to Trigger ADC Conversion Periodically	151

Project 104: Read Gyroscope Using I2C and Calculate Orientation	153
Project 105: Control Buzzer with Melody Using Timer	154
Project 106: GPIO Pin for Proximity Sensor with Interrupt	155
Project 107: Interface with Color Sensor Using I2C	157
Project 108: Timer to Control Fan with Variable Speed	158
Project 109: ADC to Read Light Sensor with Averaging	159
Project 110: Control 4-Digit 7-Segment Display Using GPIO	160
Project 111: Read Humidity Sensor Using SPI	162
Project 112: Timer for 10 kHz PWM Signal for Motor	163
Project 113: Interface with Real-Time Clock (RTC) Using I2C	164
Project 114: DAC to Generate a Sawtooth Wave	165
Project 115: GPIO Pin for Vibration Sensor with Interrupt	167
Project 116: Control Stepper Motor with Microstepping	168
Project 117: Read Magnetometer Using I2C	169
Project 118: ADC to Read Current Sensor with Scaling	170
Project 119: Interface with Graphic LCD Using SPI	172
Project 120: Timer to Generate a Pulse Train	173
Project 121: GPIO Pin for Reed Switch with Interrupt	174
Project 122: Control Servo Motor with Smooth Motion	175
Project 123: Read Temperature Sensor with Error Checking	177
Project 124: ADC to Read Force Sensor with Calibration	178
Project 125: Interface with Pressure Sensor Using I2C	179
Project 126: Timer to Control PWM-Based Heater	180
Project 127: GPIO Pin for PIR Motion Sensor with Interrupt	181
Project 128: Control 2x16 LCD with Custom Characters	183
Project 129: Read Gas Sensor Using ADC and Trigger Alarm	184
Project 130: Timer for 500 Hz PWM Signal for Buzzer	186
Project 131: Interface with UV Sensor Using I2C	187
Project 132: Control Motor with Forward/Reverse Using Timer	188
Project 133: ADC to Read pH Sensor with Calibration	189
Project 134: Interface with Touch Screen Using SPI	191
Project 135: DAC to Generate a Custom Waveform	192
Project 136: GPIO Pin for Limit Switch with Interrupt	193
Project 137: Control 4x4 LED Matrix with Animation	194
Project 138: Read Flex Sensor Using ADC	196
Project 139: Timer for Periodic Interrupt for Sensor Sampling	197

Project 140: Interface with Color Sensor Using SPI	198
Project 141: Control Fan with Temperature-Based Speed Control	199
Project 142: ADC to Read Strain Gauge with Calibration	201
Project 143: Interface with Humidity Sensor Using I2C.....	202
Project 144: Timer to Generate 1 Hz Signal for RTC	203
Project 145: GPIO Pin for Water Level Sensor with Interrupt	204
Project 146: Control Relay Based on Light Sensor Reading.....	205
Project 147: Read Heart Rate Sensor Using ADC.....	206
Project 148: Timer for 2 kHz PWM Signal for Servo	208
Project 149: Interface with Pressure Sensor Using SPI	209
Project 150: Control Buzzer with Complex Melody Using Timer	210
Communication Protocols	212
Project 151: Configure UART to Send Sensor Data in a Formatted String Every Second.....	212
Project 152: Parse a UART Command with Multiple Arguments	213
Project 153: Use UART to Send JSON-Formatted Sensor Data	214
Project 154: Configure SPI to Read Data from External Flash Memory Chip.....	216
Project 155: Use SPI to Control a Graphic LCD Display	217
Project 156: Interface with SPI-Based Temperature Sensor	218
Project 157: Configure I2C to Read Multiple Registers from Accelerometer	219
Project 158: Write Configuration Data to I2C EEPROM	221
Project 159: Use I2C to Control a Multi-Channel DAC	222
Project 160: Configure UART to Implement a Simple Request-Response Protocol	223
Project 161: Use SPI to Read a Pressure Sensor and Log Data	224
Project 162: Interface with I2C-Based OLED Display.....	226
Project 163: Configure SPI to Control a Shift Register for LED Control.....	227
Project 164: Use I2C to Read a Real-Time Clock and Display Time.....	228
Project 165: Use UART to Send CSV-Formatted Sensor Data	230
Project 166: Configure I2C to Read a Magnetometer and Calculate Heading	231
Project 167: Use SPI to Interface with an SD Card for Data Logging.....	233
Project 168: Use UART to Implement a Basic Bootloader	234
Project 169: Configure I2C to Control a Digital Potentiometer	235
Project 170: Use SPI to Read a Gyroscope and Calculate Rotation	236
Project 171: Use UART to Send a Timestamped Event Log	238
Project 172: Configure SPI to Interface with a High-Speed ADC	239
Project 173: Use I2C to Read a Humidity Sensor and Log Data	240
Project 174: Use UART to Receive and Execute Complex Commands	242

Project 175: Configure I2C to Interface with a Multi-Sensor Module	243
Project 176: Use SPI to Control a Multi-Digit 7-Segment Display.....	244
Project 177: Use UART to Send Sensor Data in a Binary Format	246
Project 178: Configure I2C to Read a Color Sensor and Display RGB Values	247
Project 179: Use SPI to Interface with Flash Memory for Firmware Updates	248
Project 180: Use UART to Implement a Simple Modbus Protocol	250
Project 181: Configure I2C to Control an I2C-Based Motor Driver	251
Project 182: Use SPI to Read a High-Resolution Temperature Sensor	252
Project 183: Use UART to Send a Structured Data Packet	254
Project 184: Configure I2C to Read a Proximity Sensor and Trigger an Action	255
Project 185: Use SPI to Control an RGB LED Strip.....	256
Project 186: Use UART to Implement a Simple Handshake Protocol	258
Project 187: Configure I2C to Interface with a Pressure Sensor and Log Data.....	259
Project 188: Use SPI to Read a Multi-Axis Accelerometer	260
Project 189: Use UART to Send Sensor Data with Error Checking.....	262
Project 190: Configure I2C to Control a Multi-Channel LED Driver	263
Project 191: Use SPI to Interface with a Touch Screen Controller.....	264
Project 192: Use UART to Implement a Basic Terminal Interface.....	265
Project 193: Configure I2C to Read a Gas Sensor and Trigger an Alarm	267
Project 194: Use SPI to Control a High-Speed DAC	268
Project 195: Use UART to Send a Compressed Data Packet.....	269
Project 196: Configure I2C to Interface with a UV Sensor and Log Data.....	271
Project 197: Use SPI to Read a Magnetometer and Calculate Direction	272
Project 198: Use UART to Implement a Simple ACK/NACK Protocol.....	274
Project 199: Configure I2C to Control a Digital-to-Analog Converter for Audio	275
Project 200: Use SPI to Interface with a High-Speed Pressure Sensor	276
Project 201: Use UART to Send Sensor Data with a Timestamp.....	278
Project 202: Configure I2C to Read a Temperature Sensor with Error Handling	279
Project 203: Use SPI to Control a Multi-Channel LED Driver	280
Project 204: Use UART to Implement a Basic Checksum Protocol	282
Project 205: Configure I2C to Interface with a Multi-Sensor Module and Parse Data.....	283
Project 206: Use SPI to Read a High-Resolution ADC	284
Project 207: Use UART to Send Sensor Data in a JSON Format.....	286
Project 208: Configure I2C to Control a Real-Time Clock with Alarm Settings	287
Project 209: Use SPI to Interface with Flash Memory for Data Logging	288
Project 210: Use UART to Implement a Simple Retry Mechanism.....	290

Project 211: Use SPI to Interface with a Gyroscope and Calculate Angular Velocity.....	291
Project 212: Use UART to Send Sensor Data with Priority Levels	293
Project 213: Configure I2C to Read a Light Sensor and Adjust PWM Output	294
Project 214: Use SPI to Control a Stepper Motor Driver	296
Project 215: Use UART to Send Data with a Sequence Number.....	297
Project 216: Configure I2C to Read a Multi-Axis Magnetometer	298
Project 217: Use SPI to Interface with an OLED Display	300
Project 218: Use UART to Implement a Packet-Based Protocol.....	301
Project 219: Configure I2C to Interface with an Accelerometer and Detect Motion.....	302
Project 220: Use SPI to Read a High-Resolution Temperature Sensor with Calibration	304
Project 221: Use UART to Send Data with a Timestamp and Sequence Number	305
Project 222: Configure I2C to Interface with a Barometric Pressure Sensor	307
Project 223: Use SPI to Control a Multi-Channel DAC.....	308
Project 224: Use UART to Implement a Multi-Device Communication Protocol	309
Project 225: Configure I2C to Read a Humidity Sensor with Calibration	310
Timers, Interrupts, and Real-Time Concepts	312
Project 226: Configure a Timer to Trigger an Interrupt Every 10 ms for Sensor Sampling	312
Project 227: Create a Project to Use a Timer to Generate a 1 kHz PWM with Variable Duty Cycle	313
Project 228: Implement a Project to Measure the Period of an External Signal Using a Timer Capture	314
Project 229: Configure a Timer to Generate a 100 Hz Interrupt for Task Scheduling	315
Project 230: Create a Project to Use a Timer to Control a Servo with Smooth Transitions.....	317
Project 231: Implement a Project to Use a Timer Interrupt to Update an LCD Display	318
Project 232: Configure a Timer to Generate a 2 kHz PWM Signal for a Motor.....	319
Project 233: Create a Project to Use a Timer to Measure Pulse Width with High Accuracy	320
Project 234: Implement a Project to Use a Timer Interrupt to Log Sensor Data	322
Project 235: Configure a Timer to Trigger an Interrupt Every 1 ms for Real-Time Tasks	323
Project 236: Create a Project to Use a Timer to Generate a 500 Hz PWM for a Buzzer	324
Project 237: Implement a Project to Use a Timer Capture to Measure Signal Frequency	325
Project 238: Configure a Timer to Generate a Dual PWM Signal for Motor Control	327
Project 239: Create a Project to Use a Timer Interrupt to Control a Sequence of LEDs	328
Project 240: Implement a Project to Use a Timer to Create a 1-Minute Timeout.....	329
Project 241: Configure a Timer to Trigger an Interrupt Every 50 ms for ADC Sampling	330
Project 242: Create a Project to Use a Timer to Generate a 10 kHz PWM for a Heater	332
Project 243: Implement a Project to Use a Timer Interrupt to Toggle a Relay Periodically	333
Project 244: Configure a Timer to Measure the Duration of a Button Press with Accuracy	334
Project 245: Create a Project to Use a Timer to Generate a 1 Hz Signal for an RTC	336

Project 246: Implement a Project to Use a Timer Interrupt to Update a Counter Display	337
Project 247: Configure a Timer to Generate a 25% Duty Cycle PWM Signal for a Fan	338
Project 248: Create a Project to Use a Timer to Control a Stepper Motor with Variable Speed	339
Project 249: Implement a Project to Use a Timer Interrupt to Monitor a Sensor Periodically	341
Project 250: Configure a Timer to Trigger an Interrupt Every 100 ms for Task Scheduling	342
Project 251: Create a Project to Use a Timer to Generate a 5 kHz PWM for a Servo	343
Project 252: Implement a Project to Use a Timer Capture to Measure Signal Pulse Width	344
Project 253: Configure a Timer to Generate a 75% Duty Cycle PWM Signal for a Motor	346
Project 254: Create a Project to Use a Timer Interrupt to Control a Buzzer Melody	347
Project 255: Implement a Project to Use a Timer to Create a Heartbeat Signal for Diagnostics	348
Project 256: Configure a Timer to Trigger an Interrupt Every 20 ms for Sensor Reading	349
Project 257: Create a Project to Use a Timer to Generate a 1 kHz Square Wave	351
Project 258: Implement a Project to Use a Timer Interrupt to Update a Graphic Display	352
Project 259: Configure a Timer to Measure the Frequency of a Sensor Signal	353
Project 260: Create a Project to Use a Timer to Control a PWM-Based Heater with PID	354
Project 261: Implement a Project to Use a Timer Interrupt to Toggle Multiple Relays	356
Project 262: Configure a Timer to Generate a 200 Hz PWM Signal for a Buzzer	357
Project 263: Create a Project to Use a Timer to Create a Multi-Stage LED Pattern	358
Project 264: Implement a Project to Use a Timer Capture to Measure Signal Period	360
Project 265: Configure a Timer to Trigger an Interrupt Every 5 ms for Real-Time Tasks	361
Project 266: Create a Project to Use a Timer to Generate a 10% Duty Cycle PWM Signal	362
Project 267: Implement a Project to Use a Timer Interrupt to Log Temperature Data	363
Project 268: Configure a Timer to Generate a 1 MHz Square Wave for Testing	365
Project 269: Create a Project to Use a Timer to Control a Motor with Acceleration	366
Project 270: Implement a Project to Use a Timer Interrupt to Update a 7-Segment Display	367
Project 271: Configure a Timer to Trigger an Interrupt Every 200 ms for Sensor Sampling	369
Project 272: Create a Project to Use a Timer to Generate a 90% Duty Cycle PWM Signal	370
Project 273: Implement a Project to Use a Timer Capture to Measure Pulse Duration	371
Project 274: Configure a Timer to Generate a 500 Hz PWM Signal for a Servo	372
Project 275: Create a Project to Use a Timer Interrupt to Control a Fan Speed	373
Project 276: Implement a Project to Use a Timer to Create a Periodic Task Scheduler	375
Project 277: Configure a Timer to Trigger an Interrupt Every 50 ms for ADC Conversion	376
Project 278: Create a Project to Use a Timer to Generate a 2 kHz PWM for a Motor	377
Project 279: Implement a Project to Use a Timer Interrupt to Toggle a Buzzer Periodically	378
Project 280: Configure a Timer to Measure the Frequency of an External Signal with Accuracy	380
Project 281: Create a Project to Use a Timer to Control a Servo with Position Feedback	381

Project 282: Implement a Project to Use a Timer Interrupt to Update a Counter Variable	383
Project 283: Configure a Timer to Generate a 100 Hz PWM Signal for a Heater	384
Project 284: Create a Project to Use a Timer to Create a Blinking LED Sequence	385
Project 285: Implement a Project to Use a Timer Capture to Measure Signal Duty Cycle	386
Project 286: Configure a Timer to Trigger an Interrupt Every 10 ms for Task Execution	388
Project 287: Create a Project to Use a Timer to Generate a 1 kHz PWM for a Fan	389
Project 288: Implement a Project to Use a Timer Interrupt to Log Sensor Data with Timestamps	390
Project 289: Configure a Timer to Generate a 5 kHz Square Wave for a Buzzer	391
Project 290: Create a Project to Use a Timer to Control a Motor with Smooth Acceleration	393
Project 291: Implement a Project to Use a Timer Interrupt to Update a Display with Animation	394
Project 292: Configure a Timer to Trigger an Interrupt Every 100 ms for Real-Time Tasks	395
Project 293: Create a Project to Use a Timer to Generate a 25% Duty Cycle PWM for a Servo	397
Project 294: Implement a Project to Use a Timer Capture to Measure Signal Frequency	398
Project 295: Configure a Timer to Generate a 200 Hz PWM Signal for a Motor	399
Project 296: Create a Project to Use a Timer Interrupt to Control a Sequence of Relays	400
Project 297: Implement a Project to Use a Timer to Create a 1-Second Heartbeat Signal	401
Project 298: Configure a Timer to Trigger an Interrupt Every 20 ms for Sensor Sampling	403
Project 299: Create a Project to Use a Timer to Generate a 10 kHz PWM for a Heater	404
Project 300: Implement a Project to Use a Timer Interrupt to Toggle Multiple LEDs with Patterns	405
Introduction to RTOS Concepts	407
Project 301: Create a Project to Use FreeRTOS to Toggle an LED in a Task	407
Project 302: Implement a Project to Create Two FreeRTOS Tasks to Control Two LEDs	408
Project 303: Configure FreeRTOS to Use a Task to Read a Sensor and Print Data	409
Project 304: Create a Project to Use a FreeRTOS Semaphore to Protect Shared Resources	411
Project 305: Implement a Project to Use FreeRTOS to Schedule Three Periodic Tasks	412
Project 306: Configure FreeRTOS to Use a Task to Handle Button Presses	414
Project 307: Create a Project to Use a FreeRTOS Queue to Pass Sensor Data Between Tasks	415
Project 308: Implement a Project to Use FreeRTOS to Toggle a Relay Periodically	417
Project 309: Configure FreeRTOS to Use a Task to Update a Display with Sensor Data	418
Project 310: Create a Project to Use FreeRTOS to Implement a Simple State Machine	419
Project 311: Implement a Project to Use a FreeRTOS Mutex to Protect a Shared Buffer	421
Project 312: Configure FreeRTOS to Use a Task to Control a Motor Speed	422
Project 313: Create a Project to Use FreeRTOS to Log Sensor Data to a Serial Console	423
Project 314: Implement a Project to Use FreeRTOS to Handle Multiple Button Inputs	425
Project 315: Configure FreeRTOS to Use a Task to Generate a PWM Signal	426
Project 316: Create a Project to Use FreeRTOS to Read an ADC and Send Data to a Queue	427

Project 317: Implement a Project to Use FreeRTOS to Toggle LEDs with Different Priorities	429
Project 318: Configure FreeRTOS to Use a Task to Handle a Temperature Sensor	430
Project 319: Create a Project to Use FreeRTOS to Implement a Task with a 1-Second Delay	432
Project 320: Implement a Project to Use FreeRTOS to Manage a Circular Buffer.....	433
Project 321: Configure FreeRTOS to Use a Task to Control a Buzzer with a Melody	435
Project 322: Create a Project to Use FreeRTOS to Handle a Multi-Sensor Module	436
Project 323: Implement a Project to Use FreeRTOS to Toggle a Relay Based on Sensor Data	437
Project 324: Configure FreeRTOS to Use a Task to Update an OLED Display.....	439
Project 325: Create a Project to Use FreeRTOS to Implement a Simple Event Handler	440
Project 326: Implement a Project to Use FreeRTOS to Manage a Task with Timeout	442
Project 327: Configure FreeRTOS to Use a Task to Read a Pressure Sensor	443
Project 328: Create a Project to Use FreeRTOS to Handle a UART Command Parser	445
Project 329: Implement a Project to Use FreeRTOS to Toggle LEDs with a Task Scheduler	446
Project 330: Configure FreeRTOS to Use a Task to Control a Servo Motor.....	448
Project 331: Create a Project to Use FreeRTOS to Log Data with Timestamps	449
Project 332: Implement a Project to Use FreeRTOS to Handle a Multi-Channel ADC	450
Project 333: Configure FreeRTOS to Use a Task to Generate a PWM Signal for a Fan	452
Project 334: Create a Project to Use FreeRTOS to Implement a Simple Watchdog Task.....	453
Project 335: Implement a Project to Use FreeRTOS to Manage a Queue for Sensor Data	454
Project 336: Configure FreeRTOS to Use a Task to Control a 7-Segment Display	456
Project 337: Create a Project to Use FreeRTOS to Handle a Button with Debouncing	457
Project 338: Implement a Project to Use FreeRTOS to Toggle a Buzzer Periodically	459
Project 339: Configure FreeRTOS to Use a Task to Read a Humidity Sensor	460
Project 340: Create a Project to Use FreeRTOS to Implement a Task with Priority Scheduling	461
Project 341: Implement a Project to Use FreeRTOS to Manage a Shared Resource with a Mutex	463
Project 342: Configure FreeRTOS to Use a Task to Control a Stepper Motor	464
Project 343: Create a Project to Use FreeRTOS to Log UART Data to a Queue.....	466
Project 344: Implement a Project to Use FreeRTOS to Handle a Multi-Axis Accelerometer	467
Project 345: Configure FreeRTOS to Use a Task to Generate a 1 kHz PWM Signal	469
Project 346: Create a Project to Use FreeRTOS to Implement a Simple Task Scheduler	470
Project 347: Implement a Project to Use FreeRTOS to Manage a Circular Buffer for UART.....	471
Project 348: Configure FreeRTOS to Use a Task to Control a Relay with Sensor Input	473
Project 349: Create a Project to Use FreeRTOS to Handle a Temperature Sensor with Logging.....	475
Project 350: Implement a Project to Use FreeRTOS to Toggle LEDs with Different Delays	476
Project 351: Configure FreeRTOS to Use a Task to Read a Pressure Sensor with I2C.....	478
Project 352: Create a Project to Use FreeRTOS to Implement a Task with Event Handling.....	479

Project 353: Implement a Project to Use FreeRTOS to Manage a Queue for Button Events	481
Project 354: Configure FreeRTOS to Use a Task to Control a Fan with PWM	482
Project 355: Create a Project to Use FreeRTOS to Log Sensor Data with Error Checking	484
Project 356: Implement a Project to Use FreeRTOS to Handle a Multi-Channel DAC	485
Project 357: Configure FreeRTOS to Use a Task to Update a Graphic Display	486
Project 358: Create a Project to Use FreeRTOS to Implement a Task with Timeout Handling	488
Project 359: Implement a Project to Use FreeRTOS to Manage a Shared Buffer with Semaphore	489
Project 360: Implement a Project to Use FreeRTOS to Manage a Task with Dynamic Priority	491
Project 361: Configure FreeRTOS to Use a Task to Control a Buzzer with Different Frequencies	493
Project 362: Create a Project to Use FreeRTOS to Handle a Temperature Sensor with Alarm	494
Project 363: Implement a Project to Use FreeRTOS to Manage a Queue for ADC Data	495
Project 364: Configure FreeRTOS to Use a Task to Control a Servo with Different Angles	497
Project 365: Create a Project to Use FreeRTOS to Implement a Simple State Machine	498
Project 366: Implement a Project to Use FreeRTOS to Handle a Multi-Sensor Module with Priority	500
Project 367: Configure FreeRTOS to Use a Task to Generate a PWM Signal with Dynamic Duty Cycle	502
Project 368: Create a Project to Use FreeRTOS to Log Data with Multiple Sensors	504
Project 369: Implement a Project to Use FreeRTOS to Handle a UART Command Parser with Queue	505
Project 370: Configure FreeRTOS to Use a Task to Control a Relay with Timeout	507
Project 371: Create a Project to Use FreeRTOS to Implement a Task with Periodic Execution	509
Project 372: Implement a Project to Use FreeRTOS to Handle a Gyroscope Sensor	510
Project 373: Configure FreeRTOS to Use a Task to Control a DC Motor with PWM	511
Project 374: Create a Project to Use FreeRTOS to Implement a Task with Error Handling	513
Project 375: Implement a Project to Use FreeRTOS to Manage a Shared Resource with Priority Inheritance ...	514
Project 360: Implement a Project to Use FreeRTOS to Manage a Task with Dynamic Priority	516
Project 361: Configure FreeRTOS to Use a Task to Control a Buzzer with Different Frequencies	518
Project 362: Create a Project to Use FreeRTOS to Handle a Temperature Sensor with Alarm	519
Project 363: Implement a Project to Use FreeRTOS to Manage a Queue for ADC Data	520
Project 364: Configure FreeRTOS to Use a Task to Control a Servo with Different Angles	522
Project 365: Create a Project to Use FreeRTOS to Implement a Simple State Machine	523
Project 366: Implement a Project to Use FreeRTOS to Handle a Multi-Sensor Module with Priority	525
Project 367: Configure FreeRTOS to Use a Task to Generate a PWM Signal with Dynamic Duty Cycle	527
Project 368: Create a Project to Use FreeRTOS to Log Data with Multiple Sensors	529
Project 369: Implement a Project to Use FreeRTOS to Handle a UART Command Parser with Queue	530
Project 370: Configure FreeRTOS to Use a Task to Control a Relay with Timeout	532
Project 371: Create a Project to Use FreeRTOS to Implement a Task with Periodic Execution	534
Project 372: Implement a Project to Use FreeRTOS to Handle a Gyroscope Sensor	535

Project 373: Configure FreeRTOS to Use a Task to Control a DC Motor with PWM.....	536
Project 374: Create a Project to Use FreeRTOS to Implement a Task with Error Handling.....	538
Project 375: Implement a Project to Use FreeRTOS to Manage a Shared Resource with Priority Inheritance ...	539
Memory Management and Optimization.....	541
Project 393: Configure a Program to Minimize Heap Fragmentation in a Long-Running System	541
Project 394: Create a Project to Implement a Memory-Efficient Queue for Task Communication.....	542
Project 395: Implement a Project to Use a Static Buffer for ADC Data Averaging	544
Project 396: Configure a Program to Optimize Memory Usage for a Command Parser	545
Project 397: Create a Project to Implement a Memory-Efficient State Machine for Control.....	547
Project 398: Implement a Project to Use a Fixed-Size Buffer for SPI Communication.....	548
Project 399: Configure a Program to Minimize Stack Usage for a Multi-Task System.....	550
Project 400: Create a Project to Implement a Memory-Efficient Data Logger for EEPROM.....	552
Project 401: Implement a Project to Use a Static Memory Pool for Sensor Data	553
Project 402: Configure a Program to Optimize Memory Usage for a Multi-Sensor Module	555
Project 403: Create a Project to Implement a Memory-Efficient FIFO Queue for I2C Data	557
Project 404: Implement a Project to Use a Fixed-Size Buffer for UART Command Parsing.....	558
Project 405: Configure a Program to Minimize Heap Usage for a Protocol Stack.....	560
Project 406: Create a Project to Implement a Memory-Efficient Ring Buffer for ADC Data.....	561
Project 407: Implement a Project to Use a Static Array for Peripheral State Tracking	563
Project 408: Configure a Program to Optimize Stack Usage for a State Machine.....	565
Project 409: Create a Project to Implement a Memory-Efficient Circular Buffer for SPI	566
Project 410: Implement a Project to Use a Fixed-Size Buffer for I2C Sensor Data	568
Project 411: Configure a Program to Minimize Stack Usage for Interrupt-Driven Tasks	570
Project 412: Create a Project to Implement a Memory-Efficient Data Logger for Flash Memory.....	572
Project 413: Implement a Project to Use a Static Memory Pool for Communication Buffers.....	573
Project 414: Configure a Program to Optimize Memory Usage for a Multi-Task System.....	575
Project 415: Create a Project to Implement a Memory-Efficient Queue for Sensor Data	577
Project 416: Implement a Project to Use a Fixed-Size Buffer for UART Data Logging	579
Project 417: Configure a Program to Minimize Heap Fragmentation for a Long-Running Task	580
Project 418: Create a Project to Implement a Memory-Efficient State Machine for UART	581
Project 419: Implement a Project to Use a Static Array for ADC Data Storage	583
Project 420: Configure a Program to Optimize Memory Usage for a Protocol Decoder	585
Project 421: Create a Project to Implement a Memory-Efficient FIFO Queue for SPI Data	586
Project 422: Implement a Project to Use a Fixed-Size Buffer for I2C Communication	587
Project 423: Configure a Program to Minimize Stack Usage for a Multi-Sensor System	589
Project 424: Create a Project to Implement a Memory-Efficient Ring Buffer for UART Data	591

Project 425: Implement a Project to Use a Static Memory Pool for Task Communication	593
Project 426: Configure a Program to Optimize Memory Usage for a Command-Line Interface	595
Project 427: Create a Project to Implement a Memory-Efficient Circular Buffer for ADC	596
Project 428: Implement a Project to Use a Fixed-Size Buffer for SPI Sensor Data	598
Project 429: Configure a Program to Minimize Heap Usage for a Data Logger	600
Project 430: Create a Project to Implement a Memory-Efficient Circular Buffer for Sensor Data	602
Project 431: Implement a Project to Use a Static Array for Multi-Peripheral Control	604
Project 432: Configure a Program to Minimize Stack Usage for a Data Processing Task	605
Project 433: Create a Project to Implement a Memory-Efficient Queue for UART Communication	606
Project 434: Implement a Project to Use a Static Buffer for Flash Data Storage	608
Project 435: Configure a Program to Optimize Memory Usage for a State Machine	609
Project 436: Create a Project to Implement a Memory-Efficient FIFO Queue for I2C Data	611
Project 437: Implement a Project to Use a Static Buffer for UART Command Processing	612
Project 438: Configure a Program to Minimize Heap Usage for a Multi-Task System	614
Project 439: Create a Project to Implement a Memory-Efficient Data Logger for EEPROM	615
Project 440: Implement a Project to Use a Static Memory Pool for Sensor Data Processing	617
Project 441: Configure a Program to Optimize Memory Usage for a Multi-Sensor Logger	619
Project 442: Create a Project to Implement a Memory-Efficient Ring Buffer for SPI Data	620
Project 443: Implement a Project to Use a Static Buffer for ADC Data Averaging	622
Project 444: Configure a Program to Minimize Stack Usage for Interrupt-Driven Tasks	624
Project 445: Create a Project to Implement a Memory-Efficient FIFO Queue for ADC Data	625
Project 446: Implement a Project to Use a Static Buffer for UART Data Processing	627
Project 447: Configure a Program to Optimize Memory Usage for a Protocol Stack	628
Project 448: Create a Project to Implement a Memory-Efficient State Machine for SPI Control	629
Project 449: Implement a Project to Use a Fixed-Size Buffer for I2C Sensor Data	631
Project 450: Configure a Program to Minimize Heap Fragmentation for a Long-Running System	633
Power Management and Optimization	634
Project 451: Configure a Microcontroller to Enter Sleep Mode After 10 Seconds of Inactivity	634
Project 452: Create a Project to Use a Low-Power Timer to Wake Up from Sleep Mode	636
Project 453: Implement a Project to Toggle an LED in Low-Power Mode with a Timer	637
Project 454: Configure a Microcontroller to Use Stop Mode for Power Savings	639
Project 455: Create a Project to Wake Up from Stop Mode Using a Button Interrupt	640
Project 456: Implement a Project to Read a Sensor in Low-Power Mode Periodically	642
Project 457: Configure a Microcontroller to Use a Low-Power UART for Communication	643
Project 458: Create a Project to Control a Motor in Low-Power Mode with PWM	645
Project 459: Implement a Project to Log Sensor Data in Low-Power Mode	646

Project 460: Configure a Microcontroller to Enter Deep Sleep Mode with RTC Wake-Up	648
Project 461: Create a Project to Use a Low-Power Timer to Toggle a Relay	649
Project 462: Implement a Project to Read an ADC in Low-Power Mode with Interrupts	651
Project 463: Configure a Microcontroller to Use a Low-Power Clock for Peripherals	652
Project 464: Create a Project to Control a Buzzer in Low-Power Mode with a Timer	654
Project 465: Implement a Project to Wake Up from Sleep Mode Using a PIR Sensor	655
Project 466: Configure a Microcontroller to Minimize Power During ADC Sampling	657
Project 467: Create a Project to Use a Low-Power Timer for Periodic Sensor Reading	658
Project 468: Implement a Project to Toggle an LED in Low-Power Mode with a Button	660
Project 469: Configure a Microcontroller to Enter Standby Mode with GPIO Wake-Up	662
Project 470: Create a Project to Use a Low-Power UART to Send Sensor Data	663
Project 471: Implement a Project to Control a Servo in Low-Power Mode with PWM	665
Project 472: Configure a Microcontroller to Use a Low-Power Clock for Timers	666
Project 473: Create a Project to Log Temperature Data in Low-Power Mode	668
Project 474: Implement a Project to Wake Up from Stop Mode Using a Timer Interrupt	669
Project 475: Configure a Microcontroller to Minimize Power During I2C Communication	671
Project 476: Create a Project to Use a Low-Power Timer to Control a Fan	673
Project 477: Implement a Project to Read a Pressure Sensor in Low-Power Mode	674
Project 478: Configure a Microcontroller to Enter Sleep Mode with RTC Scheduling	676
Project 479: Create a Project to Use a Low-Power UART for Command Parsing	677
Project 480: Implement a Project to Toggle a Relay in Low-Power Mode with a Timer	679
Project 481: Configure a Microcontroller to Minimize Power During SPI Communication	680
Project 482: Create a Project to Use a Low-Power Timer for Periodic LED Toggling	682
Project 483: Implement a Project to Read a Humidity Sensor in Low-Power Mode	684
Project 484: Configure a Microcontroller to Enter Deep Sleep with External Interrupt	685
Project 485: Create a Project to Use a Low-Power Clock for ADC Sampling	687
Project 486: Implement a Project to Control a Motor in Low-Power Mode with Interrupts	688
Project 487: Configure a Microcontroller to Minimize Power During UART Transmission	690
Project 488: Create a Project to Use a Low-Power Timer for Periodic Sensor Logging	692
Project 489: Implement a Project to Wake Up from Standby Mode with a Button Press	693
Project 490: Configure a Microcontroller to Use a Low-Power Clock for I2C Communication	695
Project 491: Create a Project to Use a Low-Power UART to Send Temperature Data	696
Project 492: Implement a Project to Toggle an LED in Low-Power Mode with a Sensor	698
Project 493: Configure a Microcontroller to Enter Sleep Mode with a Timeout	700
Project 494: Create a Project to Use a Low-Power Timer to Control a Buzzer	701
Project 495: Implement a Project to Read an ADC in Low-Power Mode with Calibration	703

Project 496: Configure a Microcontroller to Minimize Power During Timer Operation	705
Project 497: Create a Project to Use a Low-Power UART for Data Logging	706
Project 498: Implement a Project to Wake Up from Stop Mode with a Sensor Interrupt.....	708
Project 499: Configure a Microcontroller to Enter Deep Sleep with Periodic Wake-Up	709
Project 500: Create a Project to Use a Low-Power Timer to Toggle a Relay with Scheduling	711

Intermediate C Programming for Embedded Systems

Project 1: Finite State Machine for Traffic Light Sequence

Description: Design a finite state machine (FSM) to control a traffic light sequence for a North-South (NS) and East-West (EW) intersection.

- The FSM cycles through states: NS_Green, NS_Yellow, NS_Red, EW_Green, EW_Yellow, EW_Red.
- Each state has a predefined duration, and GPIO pins control the LEDs.

Requirements:

- States: NS_Green (30s), NS_Yellow (5s), NS_Red (35s, includes EW_Green + EW_Yellow), EW_Green (30s), EW_Yellow (5s), EW_Red (35s).
- Output: Control GPIO pins for Red, Yellow, Green LEDs.
- Assumptions: 1-second timer interrupt, GPIO pins preconfigured.

Ways to Solve

Algorithm Used

- **Finite State Machine:** Uses a state enum and switch-case to handle state-specific LED settings and transitions.
- A timer interrupt increments a counter to track state duration.
- **State Transition:** Transitions occur when the timer count exceeds the state's duration, resetting the counter and updating the state.

Complexity

- **Time Complexity:** $O(1)$ per state transition (constant-time switch-case and GPIO operations).
- **Space Complexity:** $O(1)$ (fixed state array and variables).

Pseudocode

```
ENUM State {NS_GREEN, NS_YELLOW, NS_RED, EW_GREEN, EW_YELLOW, EW_RED}
GLOBAL state = NS_GREEN, timer_count = 0
CONST durations = [30, 5, 35, 30, 5, 35]
FUNCTION initialize_traffic_light
    Configure GPIO pins
    Set state to NS_GREEN
    Initialize 1s timer interrupt
FUNCTION timer_interrupt
    Increment timer_count
FUNCTION set_leds(state)
    CASE state:
        NS_GREEN: NS Green ON, EW Red ON, others OFF
        NS_YELLOW: NS Yellow ON, EW Red ON, others OFF
        NS_RED: NS Red ON, EW Green ON, others OFF
        EW_GREEN: EW Green ON, NS Red ON, others OFF
        EW_YELLOW: EW Yellow ON, NS Red ON, others OFF
        EW_RED: EW Red ON, NS Green ON, others OFF
FUNCTION get_next_state(state)
    RETURN next state based on state transition table

FUNCTION traffic_light_fsm
    IF timer_count >= durations[state]:
        set_leds(state)
        state = get_next_state(state)
        timer_count = 0
```

```

MAIN
    initialize_traffic_light()
    WHILE true:
        traffic_light_fsm()

```

C Code Implementation

```

#include <stdint.h>

// GPIO pin definitions (example)
#define NS_GREEN_PIN 0
#define NS_YELLOW_PIN 1
#define NS_RED_PIN 2
#define EW_GREEN_PIN 3
#define EW_YELLOW_PIN 4
#define EW_RED_PIN 5

// State enumeration
typedef enum {
    NS_GREEN, NS_YELLOW, NS_RED,
    EW_GREEN, EW_YELLOW, EW_RED
} TrafficState;

// State durations (seconds)
const uint8_t state_durations[] = {30, 5, 35, 30, 5, 35};

// Global variables
volatile uint32_t timer_count = 0;
TrafficState current_state = NS_GREEN;

// Simulated GPIO functions
void gpio_set(uint8_t pin, uint8_t value) { /* Hardware-specific */ }

void timer_init(void) { /* Initialize 1s timer */ }

// Set LEDs based on state
void set_leds(TrafficState state) {
    gpio_set(NS_GREEN_PIN, 0); gpio_set(NS_YELLOW_PIN, 0); gpio_set(NS_RED_PIN, 0);
    gpio_set(EW_GREEN_PIN, 0); gpio_set(EW_YELLOW_PIN, 0); gpio_set(EW_RED_PIN, 0);
    switch (state) {
        case NS_GREEN: gpio_set(NS_GREEN_PIN, 1); gpio_set(EW_RED_PIN, 1); break;
        case NS_YELLOW: gpio_set(NS_YELLOW_PIN, 1); gpio_set(EW_RED_PIN, 1); break;
        case NS_RED: gpio_set(NS_RED_PIN, 1); gpio_set(EW_GREEN_PIN, 1); break;
        case EW_GREEN: gpio_set(EW_GREEN_PIN, 1); gpio_set(NS_RED_PIN, 1); break;
        case EW_YELLOW: gpio_set(EW_YELLOW_PIN, 1); gpio_set(NS_RED_PIN, 1); break;
        case EW_RED: gpio_set(EW_RED_PIN, 1); gpio_set(NS_GREEN_PIN, 1); break;
    }
}

// Get next state
TrafficState get_next_state(TrafficState state) {
    switch (state) {
        case NS_GREEN: return NS_YELLOW;
        case NS_YELLOW: return NS_RED;
        case NS_RED: return EW_GREEN;
        case EW_GREEN: return EW_YELLOW;
        case EW_YELLOW: return EW_RED;
        case EW_RED: return NS_GREEN;
    }
    return NS_GREEN; // Fallback
}

// Timer interrupt handler
void timer_isr(void) {
    timer_count++;
}

```



```
// FSM logic
void traffic_light_fsm(void) {
    if (timer_count >= state_durations[current_state]) {
        set_leds(current_state);
        current_state = get_next_state(current_state);
        timer_count = 0;
    }
}

int main(void) {
    timer_init();
    set_leds(current_state);
    while (1) {
        traffic_light_fsm();
    }
    return 0;
}
```

Expert Tips

- **Error Handling:** Add a default state or reset mechanism for invalid states.
- **Scalability:** Use a state transition table (array of structs) for complex FSMs.
- **Power Efficiency:** Disable unused GPIO pins and use low-power modes.
- **Testing:** Simulate transitions with unit tests before hardware deployment.
- **Hardware:** Replace `gpio_set` with platform-specific HAL functions.

Project 2: Function Pointers for Sensor Data Processing

Description: Create a program to process data from different sensors (e.g., temperature, pressure) using function pointers to select processing routines dynamically.

Requirements:

- Support at least two sensor types.
- Process raw data (e.g., scale, filter).
- Assumptions: Sensor data is `uint16_t`, processed output is `float`.

Ways to Solve

Algorithm Used

- **Function Pointer Array:** Define a function pointer type for processing routines.
- Store pointers in an array indexed by sensor type.
- Call the appropriate function dynamically.
- **Dispatch Logic:** Use an enum to identify sensor types and select the corresponding function.

Complexity

- **Time Complexity:** $O(1)$ for function selection and execution.
- **Space Complexity:** $O(1)$ for fixed-size function pointer array.

Pseudocode

```
ENUM SensorType {TEMP, PRESSURE}
TYPDEF FUNCTION process_fn(uint16_t) RETURNS float
GLOBAL process_fn sensor_funcs[] = {temp_process, pressure_process}
FUNCTION temp_process(raw_data)
    RETURN scaled and filtered temperature
FUNCTION pressure_process(raw_data)
    RETURN scaled and filtered pressure
```

```

FUNCTION process_sensor_data(type, raw_data)
    IF type valid:
        RETURN sensor_funcs[type](raw_data)
    ELSE:
        RETURN error_value
MAIN
    FOR each sensor reading:
        result = process_sensor_data(sensor_type, raw_data)

```

C Code Implementation

```

#include <stdint.h>

typedef enum { TEMP, PRESSURE, SENSOR_COUNT } SensorType;

typedef float (*ProcessFn)(uint16_t);

float temp_process(uint16_t raw) {
    return (raw * 0.1f) - 40.0f; // Example scaling
}

float pressure_process(uint16_t raw) {
    return (raw * 0.05f) + 300.0f; // Example scaling
}

const ProcessFn sensor_funcs[SENSOR_COUNT] = {temp_process, pressure_process};

float process_sensor_data(SensorType type, uint16_t raw_data) {
    if (type >= SENSOR_COUNT) return -1.0f; // Error
    return sensor_funcs[type](raw_data);
}

int main(void) {
    uint16_t temp_data = 500;
    uint16_t pressure_data = 1000;
    float temp = process_sensor_data(TEMP, temp_data);
    float pressure = process_sensor_data(PRESSURE, pressure_data);
    // Use temp and pressure
    return 0;
}

```

Expert Tips

- **Const Storage:** Store `sensor_funcs` in ROM using `const`.
- **Error Handling:** Validate function pointers for NULL.
- **Extensibility:** Use a struct to pair sensor type with metadata (e.g., calibration).
- **Testing:** Unit test each processing function with known inputs.

Project 3: Parse 16-Byte Command Packet

Description: Parse a 16-byte command packet (header, command ID, payload, checksum) and execute corresponding actions.

Requirements:

- Packet: 1-byte header (0xAA), 1-byte command ID, 13-byte payload, 1-byte checksum (sum of bytes).
- Execute actions based on command ID.
- Assumptions: Actions are predefined functions.

Ways to Solve

Algorithm Used

- **Packet Parsing:** Validate header and checksum, then dispatch based on command ID using a switch-case or function pointer table.
- **Checksum:** Sum all bytes (excluding checksum) and compare with received checksum.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size packet parsing and dispatch.
- **Space Complexity:** $O(1)$ for fixed-size packet buffer.

Pseudocode

```
STRUCT Packet { header, cmd_id, payload[13], checksum }
GLOBAL Packet buffer
FUNCTION validate_packet(packet)
    IF packet.header != 0xAA:
        RETURN false
    sum = SUM(packet bytes except checksum)
    RETURN sum == packet.checksum
FUNCTION execute_command(cmd_id, payload)
    CASE cmd_id:
        CMD_1: action_1(payload)
        CMD_2: action_2(payload)
        DEFAULT: handle_error()
FUNCTION parse_packet(data)
    Copy data to buffer
    IF validate_packet(buffer):
        execute_command(buffer.cmd_id, buffer.payload)
    ELSE:
        log_error()
MAIN
    WHILE true:
        receive_data(data, 16)
        parse_packet(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint8_t header;
    uint8_t cmd_id;
    uint8_t payload[13];
    uint8_t checksum;
} Packet;

void action_1(uint8_t* payload) { /* Process payload */ }

void action_2(uint8_t* payload) { /* Process payload */ }

uint8_t validate_packet(Packet* packet) {
    if (packet->header != 0xAA) return 0;
    uint8_t sum = packet->header + packet->cmd_id;
    for (int i = 0; i < 13; i++) sum += packet->payload[i];
    return sum == packet->checksum;
}

void execute_command(uint8_t cmd_id, uint8_t* payload) {
    switch (cmd_id) {
        case 1: action_1(payload); break;
        case 2: action_2(payload); break;
        default: /* Log error */ break;
    }
}
```

```

void parse_packet(uint8_t* data) {
    Packet packet;
    memcpy(&packet, data, sizeof(Packet));
    if (validate_packet(&packet)) {
        execute_command(packet.cmd_id, packet.payload);
    }
}

int main(void) {
    uint8_t data[16] = {0xAA, 1, /* payload */, 0}; // Example data
    parse_packet(data);
    return 0;
}

```

Expert Tips

- **Robustness:** Handle partial packets with a state machine.
- **Memory:** Use a static buffer to avoid stack overflow.
- **Debugging:** Log invalid packets for analysis.
- **Testing:** Test with malformed packets and edge cases.

Project 4: Circular Buffer for Sensor Data

Description: Implement a circular buffer to store sensor data, supporting push and pop operations.

Requirements:

- Buffer size: 16 elements (uint16_t).
- Operations: Push, pop, check full/empty.
- Assumptions: Single-threaded or interrupt-driven access.

Ways to Solve

Algorithm Used

- **Circular Buffer:** Use a fixed-size array with head and tail pointers.
- Increment pointers modulo size.
- Track full/empty states using count or pointer comparison.
- **Access Control:** Use volatile variables for interrupt safety.

Complexity

- **Time Complexity:** $O(1)$ for push and pop.
- **Space Complexity:** $O(n)$, where n is buffer size (16).

Pseudocode

```

STRUCT CircularBuffer { data[16], head, tail, count }
GLOBAL CircularBuffer buffer
FUNCTION init_buffer
    head = 0, tail = 0, count = 0
FUNCTION is_full
    RETURN count == 16
FUNCTION is_empty
    RETURN count == 0
FUNCTION push(value)
    IF not is_full():
        data[head] = value
        head = (head + 1) % 16
        count++
FUNCTION pop(value)
    IF not is_empty():

```

```

        value = data[tail]
        tail = (tail + 1) % 16
        count--
        RETURN true
    RETURN false
MAIN
    init_buffer()
    WHILE true:
        IF new_data:
            push(data)
        IF need_data:
            pop(&value)

```

C Code Implementation

```

#include <stdint.h>

#define BUFFER_SIZE 16

typedef struct {
    uint16_t data[BUFFER_SIZE];
    uint8_t head;
    uint8_t tail;
    uint8_t count;
} CircularBuffer;

CircularBuffer buffer;

void init_buffer(void) {
    buffer.head = 0;
    buffer.tail = 0;
    buffer.count = 0;
}

uint8_t is_full(void) {
    return buffer.count == BUFFER_SIZE;
}

uint8_t is_empty(void) {
    return buffer.count == 0;
}

void push(uint16_t value) {
    if (!is_full()) {
        buffer.data[buffer.head] = value;
        buffer.head = (buffer.head + 1) % BUFFER_SIZE;
        buffer.count++;
    }
}

uint8_t pop(uint16_t* value) {
    if (!is_empty()) {
        *value = buffer.data[buffer.tail];
        buffer.tail = (buffer.tail + 1) % BUFFER_SIZE;
        buffer.count--;
        return 1;
    }
    return 0;
}

int main(void) {
    init_buffer();
    push(100); push(200);
    uint16_t value;
    if (pop(&value)) {
        // Use value
    }
}

```



```
}  
    return 0;  
}
```

Expert Tips

- **Thread Safety:** Use critical sections for interrupt-driven access.
- **Optimization:** Use power-of-2 size (16) for efficient modulo via bitwise AND.
- **Error Handling:** Add overflow/underflow logging.
- **Testing:** Test with rapid push/pop sequences.

Project 5: CRC8 Checksum for Data Integrity

Description: Implement a CRC8 checksum algorithm to verify data integrity.

Requirements:

- Use CRC-8-Dallas/Maxim (polynomial 0x31).
- Input: Arbitrary byte array.
- Assumptions: Lookup table for efficiency.

Ways to Solve

Algorithm Used

- **CRC8 Calculation:** Iterate over bytes, XOR with CRC register, and use a lookup table based on polynomial 0x31.
- **Lookup Table:** Precompute CRC values for all 256 possible bytes to reduce computation time.

Complexity

- **Time Complexity:** $O(n)$ for n bytes (table lookup per byte).
- **Space Complexity:** $O(1)$ (256-byte lookup table).

Pseudocode

```
CONST crc_table[256] = {precomputed CRC values}  
FUNCTION calculate_crc8(data, length)  
    crc = 0  
    FOR each byte in data:  
        index = crc XOR byte  
        crc = crc_table[index]  
    RETURN crc  
MAIN  
    data = get_data()  
    crc = calculate_crc8(data, length)  
    IF crc == expected:  
        process_data()
```

C Code Implementation

```
#include <stdint.h>  
  
const uint8_t crc_table[256] = {  
    // Precomputed CRC-8-Dallas/Maxim values (partial table for brevity)  
    0x00, 0x31, 0x62, 0x53, /* ...  
*/  
};
```

```

uint8_t calculate_crc8(uint8_t* data, uint8_t length) {
    uint8_t crc = 0;
    for (uint8_t i = 0; i < length; i++) {
        uint8_t index = crc ^ data[i];
        crc = crc_table[index];
    }
    return crc;
}

int main(void) {
    uint8_t data[] = {0x01, 0x02, 0x03};
    uint8_t crc = calculate_crc8(data, 3);
    // Verify crc
    return 0;
}

```

Expert Tips

- **Storage:** Store `crc_table` in ROM using `const`.
- **Validation:** Test with standard CRC test vectors.
- **Optimization:** Compute table at compile-time if possible.
- **Robustness:** Handle zero-length input gracefully.

Project 6: 32-Bit Integer Arithmetic with Overflow Detection

Description: Implement 32-bit integer arithmetic (addition, multiplication) with overflow detection.

Requirements:

- Support signed and unsigned operations.
- Detect overflow and return error status.
- Assumptions: Using `int32_t` and `uint32_t`.

Ways to Solve

Algorithm Used

- **Addition:** Check if sum exceeds `INT32_MAX`/`MIN` or wraps around for unsigned.
- **Multiplication:** Use intermediate 64-bit result to detect overflow, then cast back.

Complexity

- **Time Complexity:** $O(1)$ per operation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION add_int32(a, b, result)
    IF a > 0 and b > INT32_MAX - a:
        RETURN overflow
    IF a < 0 and b < INT32_MIN - a:
        RETURN overflow
    *result = a + b
    RETURN success
FUNCTION mul_int32(a, b, result)
    temp = (int64_t)a * b
    IF temp > INT32_MAX or temp < INT32_MIN:
        RETURN overflow
    *result = (int32_t)temp
    RETURN success
MAIN
    IF add_int32(a, b, &sum) == success:

```

```
    use sum
    IF mul_int32(a, b, &product) == success:
        use product
```

C Code Implementation

```
#include <stdint.h>

#define SUCCESS 0
#define OVERFLOW 1

uint8_t add_int32(int32_t a, int32_t b, int32_t* result) {
    if (a > 0 && b > INT32_MAX - a) return OVERFLOW;
    if (a < 0 && b < INT32_MIN - a) return OVERFLOW;
    *result = a + b;
    return SUCCESS;
}

uint8_t mul_int32(int32_t a, int32_t b, int32_t* result) {
    int64_t temp = (int64_t)a * b;
    if (temp > INT32_MAX || temp < INT32_MIN) return OVERFLOW;
    *result = (int32_t)temp;
    return SUCCESS;
}

int main(void) {
    int32_t a = 1000, b = 2000, result;
    if (add_int32(a, b, &result) == SUCCESS) {
        // Use result
    }
    if (mul_int32(a, b, &result) == SUCCESS) {
        // Use result
    }
    return 0;
}
```

Expert Tips

- **Edge Cases:** Test with INT32_MAX, INT32_MIN, and zero.
- **Optimization:** Use compiler intrinsics (e.g., `__builtin_add_overflow`).
- **Unsigned:** Implement similar checks for `uint32_t`.
- **Debugging:** Log overflow events for analysis.

Project 7: Bit Fields in Struct for Peripheral Registers

Description: Use bit fields in a struct to configure peripheral registers (e.g., UART control register).

Requirements:

- Configure a 32-bit register with specific fields (e.g., enable, mode, parity).
- Assumptions: Direct register access.

Ways to Solve

Algorithm Used

- **Bit Fields:** Define a struct with bit fields matching the register layout.
- Assign values to fields and write to the register.
- **Direct Access:** Use a pointer to the register address.

Complexity

- **Time Complexity:** O(1) for configuration.

- **Space Complexity:** $O(1)$.

Pseudocode

```
STRUCT UartConfig {
    enable:1, mode:2, parity:1, reserved:28
}
FUNCTION configure_uart(enable, mode, parity)
    UartConfig config
    config.enable = enable
    config.mode = mode
    config.parity = parity
    UART_REG = *(uint32_t*)&config
MAIN
    configure_uart(1, 2, 1)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint32_t enable : 1;
    uint32_t mode : 2;
    uint32_t parity : 1;
    uint32_t reserved : 28;
} UartConfig;
volatile uint32_t* UART_REG = (uint32_t*)0x40001000; // Example address

void configure_uart(uint8_t enable, uint8_t mode, uint8_t parity) {
    UartConfig config = {0};
    config.enable = enable & 0x1;
    config.mode = mode & 0x3;
    config.parity = parity & 0x1;
    *UART_REG = *(uint32_t*)&config;
}

int main(void) {
    configure_uart(1, 2, 1); // Enable, mode 2, parity on
    return 0;
}
```

Expert Tips

- **Alignment:** Verify bit field order (compiler-dependent).
- **Safety:** Use unions for safer register access.
- **Documentation:** Match bit fields to hardware manual.
- **Testing:** Verify register writes with a debugger.

Project 8: Debounce Algorithm for Push Button

Description: Implement a debounce algorithm for a noisy push-button input.

Requirements:

- Detect stable button states (pressed/released).
- Debounce period: 20ms.
- Assumptions: 10ms timer interrupt, GPIO input.

Ways to Solve

Algorithm Used

- **Debouncing:** Sample button state every 10ms.
- Require 2 consecutive identical readings (20ms) to confirm a state change.
- **State Tracking:** Use a counter to track consecutive readings.

Complexity

- **Time Complexity:** $O(1)$ per sample.
- **Space Complexity:** $O(1)$.

Pseudocode

```
GLOBAL last_state, current_state, counter
FUNCTION init_debounce
    last_state = read_button()
    current_state = last_state
    counter = 0
FUNCTION debounce_button
    state = read_button()
    IF state == last_state:
        counter++
        IF counter >= 2:
            current_state = state
            counter = 2
    ELSE:
        last_state = state
        counter = 0
    RETURN current_state
FUNCTION timer_interrupt
    debounce_button()
MAIN
    init_debounce()
    WHILE true:
        IF current_state changed:
            handle_button(current_state)
```

C Code Implementation

```
#include <stdint.h>
uint8_t read_button(void) { /* Hardware-specific */ return 0; }
volatile uint8_t last_state, current_state, counter;

void init_debounce(void) {
    last_state = read_button();
    current_state = last_state;
    counter = 0;
}

uint8_t debounce_button(void) {
    uint8_t state = read_button();
    if (state == last_state) {
        counter++;
        if (counter >= 2) {
            current_state = state;
            counter = 2;
        }
    } else {
        last_state = state;
        counter = 0;
    }
    return current_state;
}
```

```

void timer_isr(void) {
    debounce_button();
}

int main(void) {
    init_debounce();
    while (1) {
        // Check current_state
    }
    return 0;
}

```

Expert Tips

- **Tuning:** Adjust debounce period (20–50ms) based on button.
- **Interrupts:** Use edge-triggered interrupts if supported.
- **Testing:** Simulate noise with rapid button presses.
- **Power:** Minimize sampling frequency for low power.

Project 9: Lookup Table for Sensor Data Linearization

Description: Use a lookup table to linearize non-linear sensor data (e.g., thermistor).

Requirements:

- Linearize uint16_t raw data to float values.
- Assumptions: Precomputed table with 10 entries.

Ways to Solve

Algorithm Used

- **Lookup Table:** Store precomputed linearized values in a table.
- Use linear interpolation for values between entries.
- **Interpolation:** Compute slope between table points for accuracy.

Complexity

- **Time Complexity:** $O(1)$ for lookup, $O(1)$ for interpolation.
- **Space Complexity:** $O(n)$ for table size ($n=10$).

Pseudocode

```

CONST LookupEntry {raw, value} table[10]
FUNCTION linearize(raw)
    FOR i = 0 to 9:
        IF raw <= table[i].raw:
            IF i == 0:
                RETURN table[0].value
            slope = (table[i].value - table[i-1].value) / (table[i].raw - table[i-1].raw)
            RETURN table[i-1].value + slope * (raw - table[i-1].raw)
    RETURN table[9].value
MAIN
    raw = read_sensor()
    value = linearize(raw)

```

C Code Implementation

```

#include <stdint.h>
typedef struct {
    uint16_t raw;
    float value;
} LookupEntry;

```



```

const LookupEntry table[10] = {    {0, 0.0f}, {100, 10.0f}, {200, 20.0f}, /* ...
*/
};

float linearize(uint16_t raw) {
    for (uint8_t i = 0; i < 10; i++) {
        if (raw <= table[i].raw) {
            if (i == 0) return table[0].value;
            float slope = (table[i].value - table[i-1].value) /
                (table[i].raw - table[i-1].raw);
            return table[i-1].value + slope * (raw - table[i-1].raw);
        }
    }
    return table[9].value;
}

int main(void) {
    uint16_t raw = 150;
    float value = linearize(raw);
    // Use value
    return 0;
}

```

Expert Tips

- **Storage:** Store table in ROM with `const`.
- **Accuracy:** Use polynomial interpolation for non-linear sensors.
- **Testing:** Validate with calibration data.
- **Edge Cases:** Handle out-of-range inputs gracefully.

Project 10: Dynamic Array of Sensor Readings with `realloc()`

Description: Manage a dynamic array of sensor readings using `realloc()` for resizing.

Requirements:

- Store `uint16_t` sensor readings.
- Double array size when full.
- Assumptions: Standard C library available.

Ways to Solve

Algorithm Used

- **Dynamic Array:** Use `malloc()` for initial allocation, `realloc()` to resize.
- Track size and capacity.
- **Resizing Strategy:** Double capacity when full to amortize costs.

Complexity

- **Time Complexity:** $O(1)$ amortized for append, $O(n)$ for `realloc()`.
- **Space Complexity:** $O(n)$ for n elements.

Pseudocode

```

STRUCT DynamicArray { data, size, capacity }
FUNCTION init_array
    Allocate data with capacity 8
    size = 0, capacity = 8
FUNCTION append(value)
    IF size == capacity:
        new_capacity = capacity * 2

```

```

        data = realloc(data, new_capacity)
        capacity = new_capacity
        data[size] = value
        size++
FUNCTION free_array
    free(data)
MAIN
    init_array()
    WHILE true:
        append(read_sensor())

```

C Code Implementation

```

#include <stdint.h>
#include <stdlib.h>

typedef struct {
    uint16_t* data;
    uint32_t size;
    uint32_t capacity;
} DynamicArray;

void init_array(DynamicArray* arr) {
    arr->capacity = 8;
    arr->size = 0;
    arr->data = malloc(arr->capacity * sizeof(uint16_t));
}

void append(DynamicArray* arr, uint16_t value) {
    if (arr->size == arr->capacity) {
        arr->capacity *= 2;
        arr->data = realloc(arr->data, arr->capacity * sizeof(uint16_t));
        if (!arr->data) {
            // Handle allocation failure
        }
    }
    arr->data[arr->size++] = value;
}

void free_array(DynamicArray* arr) {
    free(arr->data);
    arr->data = NULL;
    arr->size = 0;
    arr->capacity = 0;
}

int main(void) {
    DynamicArray arr;
    init_array(&arr);
    append(&arr, 100);
    append(&arr, 200);
    free_array(&arr);
    return 0;
}

```

Expert Tips

- **Error Handling:** Check `realloc()` return for NULL.
- **Optimization:** Use static arrays if size is predictable.
- **Memory:** Free unused memory to prevent leaks.
- **Testing:** Test with frequent resizing and large datasets.

Project 11: Simple Command-Line Interface over UART

Description: Create a program to implement a simple command-line interface (CLI) over UART to process user commands (e.g., "ledon", "ledoff").

- The CLI receives commands, parses them, and executes corresponding actions.

Requirements:

- Support commands: "ledon" (turn LED on), "ledoff" (turn LED off).
- Use UART for input/output.
- Assumptions: UART operates at 9600 baud, commands end with '\n', max command length is 16 bytes.

Ways to Solve

Algorithm Used

- **Command Buffering:** Use a circular buffer to store incoming UART characters until a newline is received.
- **Parsing:** Compare received command with predefined strings using `strcmp`.
- **Dispatch:** Execute actions (e.g., set/clear GPIO) based on parsed command.

Complexity

- **Time Complexity:** $O(n)$ for parsing (string comparison), where n is command length.
- **Space Complexity:** $O(1)$ for fixed-size buffer (16 bytes).

Pseudocode

```
CONST commands = ["ledon", "ledoff"]
GLOBAL buffer[16], buffer_index
FUNCTION init_uart
    Configure UART at 9600 baud
    buffer_index = 0
FUNCTION uart_receive_char
    char = read_uart()
    IF char == '\n':
        buffer[buffer_index] = '\0'
        process_command(buffer)
        buffer_index = 0
    ELSE IF buffer_index < 16:
        buffer[buffer_index++] = char
FUNCTION process_command(cmd)
    IF cmd == "ledon":
        set_led(1)
    ELSE IF cmd == "ledoff":
        set_led(0)
    ELSE:
        send_uart("Invalid command\n")
MAIN
    init_uart()
    WHILE true:
        IF uart_data_available:
            uart_receive_char()
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>
```

```

#define BUFFER_SIZE 16

char buffer[BUFFER_SIZE];
uint8_t buffer_index = 0;

void set_led(uint8_t state) { /* Hardware-specific GPIO */ }

void send_uart(const char* str) { /* Hardware-specific UART send */ }

char read_uart(void) { /* Hardware-specific UART read */ return 0; }

uint8_t uart_data_available(void) { /* Hardware-specific */ return 0; }

void init_uart(void) {
    /* Configure UART at 9600 baud */
    buffer_index = 0;
}

void process_command(const char* cmd) {
    if (strcmp(cmd, "ledon") == 0) {
        set_led(1);
        send_uart("LED ON\n");
    } else if (strcmp(cmd, "ledoff") == 0) {
        set_led(0);
        send_uart("LED OFF\n");
    } else {
        send_uart("Invalid command\n");
    }
}

void uart_receive_char(void) {
    char c = read_uart();
    if (c == '\n') {
        buffer[buffer_index] = '\0';
        process_command(buffer);
        buffer_index = 0;
    } else if (buffer_index < BUFFER_SIZE - 1) {
        buffer[buffer_index++] = c;
    }
}

int main(void) {
    init_uart();
    while (1) {
        if (uart_data_available()) {
            uart_receive_char();
        }
    }
    return 0;
}

```

Expert Tips

- **Robustness:** Add buffer overflow protection and timeout for incomplete commands.
- **Extensibility:** Use a command table with function pointers for scalability.
- **Debugging:** Log received commands to a debug port.
- **Testing:** Simulate UART input with a terminal emulator.
- **Optimization:** Use interrupts for UART reception to reduce polling.

Project 12: Struct for Multiple GPIO Configurations

Description: Use a struct to manage configurations for multiple GPIO pins (e.g., mode, state, direction).

Requirements:

- Configure 4 GPIO pins (e.g., LED1–LED4).
- Fields: pin number, direction (input/output), state (high/low).
- Assumptions: GPIO hardware supports direct register access.

Ways to Solve

Algorithm Used

- **Struct Array:** Define a struct for GPIO configuration (pin, direction, state).
- Store configurations in an array and apply to hardware.
- **Configuration Loop:** Iterate over the array to set each pin's registers.

Complexity

- **Time Complexity:** $O(n)$ for n pins (constant-time per pin).
- **Space Complexity:** $O(n)$ for configuration array.

Pseudocode

```
STRUCT GpioConfig { pin, direction, state }
GLOBAL GpioConfig configs[4]
FUNCTION init_gpio
    FOR each config in configs:
        set_pin_direction(config.pin, config.direction)
        set_pin_state(config.pin, config.state)
FUNCTION set_pin_direction(pin, dir)
    Write dir to GPIO direction register
FUNCTION set_pin_state(pin, state)
    Write state to GPIO output register
MAIN
    configs = [{pin1, OUTPUT, HIGH}, {pin2, OUTPUT, LOW}, ...]
    init_gpio()
```

C Code Implementation

```
#include <stdint.h>

#define NUM_PINS 4

typedef enum { INPUT, OUTPUT } Direction;

typedef enum { LOW, HIGH } State;

typedef struct {
    uint8_t pin;
    Direction direction;
    State state;
} GpioConfig;

GpioConfig configs[NUM_PINS] = {
    {0, OUTPUT, HIGH},
    {1, OUTPUT, LOW},
    {2, INPUT, LOW},
    {3, OUTPUT, HIGH}
};
```

```

void set_pin_direction(uint8_t pin, Direction dir) { /* Hardware-specific */ }

void set_pin_state(uint8_t pin, State state) { /* Hardware-specific */ }

void init_gpio(void) {
    for (uint8_t i = 0; i < NUM_PINS; i++) {
        set_pin_direction(configs[i].pin, configs[i].direction);
        if (configs[i].direction == OUTPUT) {
            set_pin_state(configs[i].pin, configs[i].state);
        }
    }
}

int main(void) {
    init_gpio();
    return 0;
}

```

Expert Tips

- **Const Storage:** Use `const` for static configurations to save RAM.
- **Validation:** Check pin numbers and states against hardware limits.
- **Abstraction:** Use a HAL for portability across platforms.
- **Testing:** Verify configurations with a logic analyzer.
- **Scalability:** Support dynamic reconfiguration if needed.

Project 13: Floating-Point to Fixed-Point Conversion

Description: Convert a floating-point value to fixed-point for math operations in an embedded system without an FPU.

Requirements:

- Convert float to Q16.16 fixed-point (16-bit integer, 16-bit fractional).
- Support arithmetic operations (e.g., addition).
- Assumptions: 32-bit integers available.

Ways to Solve

Algorithm Used

- **Conversion:** Multiply float by 2^{16} (65536) and cast to `int32_t`.
- **Arithmetic:** Perform operations in fixed-point, adjusting for scaling.
- **Inverse Conversion:** Divide by 2^{16} to convert back to float.

Complexity

- **Time Complexity:** $O(1)$ per conversion/operation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

CONST SCALE = 65536
FUNCTION float_to_fixed(float_val)
    RETURN (int32_t)(float_val * SCALE)
FUNCTION fixed_to_float(fixed_val)
    RETURN (float)fixed_val / SCALE
FUNCTION fixed_add(a, b)
    RETURN a + b
MAIN
    float_val = read_float()
    fixed_val = float_to_fixed(float_val)

```

```
result = fixed_add(fixed_val, fixed_val)
float_result = fixed_to_float(result)
```

C Code Implementation

```
#include <stdint.h>

#define SCALE 65536

int32_t float_to_fixed(float val) {
    return (int32_t)(val * SCALE);
}

float fixed_to_float(int32_t val) {
    return (float)val / SCALE;
}

int32_t fixed_add(int32_t a, int32_t b) {
    return a + b; // Check overflow if needed
}

int main(void) {
    float val1 = 1.5f, val2 = 2.3f;
    int32_t fixed1 = float_to_fixed(val1);
    int32_t fixed2 = float_to_fixed(val2);
    int32_t sum = fixed_add(fixed1, fixed2);
    float result = fixed_to_float(sum);
    // Use result
    return 0;
}
```

Expert Tips

- **Overflow:** Check for overflow in fixed-point operations.
- **Precision:** Choose scale (e.g., Q16.16) based on range and precision needs.
- **Optimization:** Use shift operations for multiplication/division by 2^{16} .
- **Testing:** Verify conversions with edge cases (e.g., max/min values).
- **Documentation:** Use Q-format notation in comments.

Project 14: Priority-Based Task Scheduler in a Loop

Description: Implement a priority-based task scheduler in a main loop for embedded systems.

Requirements:

- Support 3 tasks with priorities (0–2, lower is higher priority).
- Tasks run periodically (e.g., 10ms, 20ms, 50ms).
- Assumptions: Non-preemptive, timer interrupt for scheduling.

Ways to Solve

Algorithm Used

- **Task Struct:** Define a struct for tasks (function pointer, priority, period, counter).
- **Scheduler Loop:** Check each task's counter against its period; execute highest-priority ready task.
- **Timer:** Increment counters in a 1ms interrupt.

Complexity

- **Time Complexity:** $O(n)$ per loop iteration, where n is number of tasks.
- **Space Complexity:** $O(n)$ for task array.

Pseudocode

```
STRUCT Task { fn, priority, period, counter }
GLOBAL Task tasks[3]
FUNCTION init_scheduler
    tasks = [{task1, 0, 10, 0}, {task2, 1, 20, 0}, {task3, 2, 50, 0}]
FUNCTION timer_interrupt
    FOR each task in tasks:
        task.counter++
FUNCTION schedule
    min_priority = MAX_INT
    selected_task = NULL
    FOR each task in tasks:
        IF task.counter >= task.period AND task.priority < min_priority:
            min_priority = task.priority
            selected_task = task
    IF selected_task:
        selected_task.fn()
        selected_task.counter = 0
MAIN
    init_scheduler()
    WHILE true:
        schedule()
```

C Code Implementation

```
#include <stdint.h>

#define NUM_TASKS 3

typedef void (*TaskFn)(void);

typedef struct {
    TaskFn fn;
    uint8_t priority;
    uint16_t period;
    uint16_t counter;
} Task;

void task1(void) { /* Task 1 */ }
void task2(void) { /* Task 2 */ }
void task3(void) { /* Task 3 */ }

Task tasks[NUM_TASKS] = {
    {task1, 0, 10, 0},
    {task2, 1, 20, 0},
    {task3, 2, 50, 0}
};

void timer_isr(void) {
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        tasks[i].counter++;
    }
}

void schedule(void) {
    uint8_t min_priority = 255;
    Task* selected = NULL;
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        if (tasks[i].counter >= tasks[i].period && tasks[i].priority < min_priority) {
            min_priority = tasks[i].priority;
            selected = &tasks[i];
        }
    }
}
```

```

        if (selected) {
            selected->fn();
            selected->counter = 0;
        }
    }

int main(void) {
    while (1) {
        schedule();
    }
    return 0;
}

```

Expert Tips

- **Starvation:** Ensure low-priority tasks run by adjusting periods.
- **Timing:** Use a high-resolution timer for accuracy.
- **Testing:** Simulate task execution with logging.
- **Optimization:** Sort tasks by priority for faster selection.
- **Extensibility:** Support dynamic task addition/removal.

Project 15: Multi-Byte Data with Endianness Conversion

Description: Handle multi-byte data with endianness conversion (e.g., big-endian to little-endian for 32-bit integers).

Requirements:

- Convert 32-bit integers between big-endian and little-endian.
- Assumptions: System is little-endian, data received in big-endian.

Ways to Solve

Algorithm Used

- **Byte Swapping:** Use shift and bitwise operations to reorder bytes (e.g., for 32-bit: swap bytes 0 and 3, 1 and 2).
- **Conditional Conversion:** Detect system endianness if needed, but assume little-endian here.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size data.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION to_little_endian(value)
    RETURN ((value >> 24) & 0xFF) |
           ((value >> 8) & 0xFF00) |
           ((value << 8) & 0xFF0000) |
           ((value << 24) & 0xFF000000)

MAIN
    big_endian = receive_data()
    little_endian = to_little_endian(big_endian)

```

C Code Implementation

```

#include <stdint.h>

uint32_t to_little_endian(uint32_t value) {
    return ((value >> 24) & 0xFF) | ((value >> 8) & 0xFF00) | ((value << 8) & 0xFF0000) |
           ((value << 24) & 0xFF000000);
}

```

```

int main(void) {
    uint32_t big_endian = 0x12345678; // Example: 12 34 56 78
    uint32_t little_endian = to_little_endian(big_endian); // 78 56 34 12
    // Use little_endian
    return 0;
}

```

Expert Tips

- **Portability:** Use standard functions like `htonl()` if available.
- **Validation:** Test with both endianness directions.
- **Optimization:** Use inline functions for performance.
- **Testing:** Verify with network data (often big-endian).
- **Robustness:** Handle 16-bit and 64-bit conversions if needed.

Project 16: Volatile Variables for Interrupt-Driven Data Access

Description: Use volatile variables to ensure safe data access in interrupt handlers (e.g., for sensor data).

Requirements:

- Share a `uint16_t` sensor value between main loop and interrupt.
- Assumptions: Timer interrupt updates sensor value.

Ways to Solve

Algorithm Used

- **Volatile Variables:** Declare shared variables as `volatile` to prevent compiler optimizations.
- **Atomic Access:** Use single-byte or aligned accesses to avoid race conditions.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$.

Pseudocode

```

GLOBAL volatile sensor_value
FUNCTION timer_interrupt
    sensor_value = read_sensor()
FUNCTION process_sensor
    local_value = sensor_value
    process(local_value)
MAIN
    WHILE true:
        process_sensor()

```

C Code Implementation

```

#include <stdint.h>
volatile uint16_t sensor_value = 0;

uint16_t read_sensor(void) { /* Hardware-specific */ return 0; }

void process(uint16_t value) { /* Process value */ }

void timer_isr(void) {
    sensor_value = read_sensor();
}

```

```

void process_sensor(void) {
    uint16_t local_value = sensor_value; // Atomic read
    process(local_value);
}

int main(void) {
    while (1) {
        process_sensor();
    }
    return 0;
}

```

Expert Tips

- **Atomicity:** Use critical sections for multi-byte data.
- **Safety:** Minimize interrupt latency during access.
- **Testing:** Simulate high-frequency interrupts.
- **Optimization:** Use memory barriers if needed.
- **Debugging:** Log access conflicts.

Project 17: Software Timer Using Counter Variable

Description: Implement a software timer using a counter variable for timed events.

Requirements:

- Support multiple timers (e.g., 3 timers).
- Resolution: 1ms.
- Assumptions: 1ms timer interrupt.

Ways to Solve

Algorithm Used

- **Timer Struct:** Define a struct for timers (counter, period, callback).
- **Interrupt:** Increment counters in a 1ms interrupt; trigger callbacks when period reached.
- **Management:** Allow starting/stopping timers.

Complexity

- **Time Complexity:** $O(n)$ per interrupt for n timers.
- **Space Complexity:** $O(n)$ for timer array.

Pseudocode

```

STRUCT Timer { counter, period, callback, active }
GLOBAL Timer timers[3]
FUNCTION init_timers
    FOR each timer:
        timer.active = false

FUNCTION start_timer(index, period, callback)
    timers[index].period = period
    timers[index].callback = callback
    timers[index].counter = 0
    timers[index].active = true

```

```

FUNCTION timer_interrupt
  FOR each timer:
    IF timer.active:
      timer.counter++
      IF timer.counter >= timer.period:
        timer.callback()
        timer.counter = 0
MAIN
  init_timers()
  start_timer(0, 100, callback1)
  WHILE true:
    /* Idle */

```

C Code Implementation

```

#include <stdint.h>

#define NUM_TIMERS 3
typedef void (*TimerCallback)(void);
typedef struct {
    uint16_t counter;
    uint16_t period;
    TimerCallback callback;
    uint8_t active;
} Timer;
Timer timers[NUM_TIMERS];

void callback1(void) { /* Timer action */ }

void init_timers(void) {
    for (uint8_t i = 0; i < NUM_TIMERS; i++) {
        timers[i].active = 0;
    }
}

void start_timer(uint8_t index, uint16_t period, TimerCallback callback) {
    if (index < NUM_TIMERS) {
        timers[index].period = period;
        timers[index].callback = callback;
        timers[index].counter = 0;
        timers[index].active = 1;
    }
}

void timer_isr(void) {
    for (uint8_t i = 0; i < NUM_TIMERS; i++) {
        if (timers[i].active) {
            timers[i].counter++;
            if (timers[i].counter >= timers[i].period) {
                timers[i].callback();
                timers[i].counter = 0;
            }
        }
    }
}

int main(void) {
    init_timers();
    start_timer(0, 100, callback1);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Scalability:** Use a linked list for dynamic timers.
- **Overflow:** Handle counter overflow for long periods.
- **Testing:** Verify timing accuracy with a logic analyzer.
- **Optimization:** Disable inactive timers to save CPU.
- **Robustness:** Add timer stop function.

Project 18: Parse JSON String for Configuration Data

Description: Parse a JSON string to extract configuration data (e.g., key-value pairs for sensor settings).

Requirements:

- Parse simple JSON (e.g., `{"sensor_id":1,"gain":2.5}`).
- Assumptions: Use a lightweight parser, max JSON length 64 bytes.

Ways to Solve

Algorithm Used

- **State Machine Parser:** Parse JSON character by character, tracking state (e.g., key, value, object).
- **Key-Value Extraction:** Store parsed values in a struct.
- **Simplified Parsing:** Assume well-formed JSON for simplicity.

Complexity

- **Time Complexity:** $O(n)$ for n -character JSON string.
- **Space Complexity:** $O(1)$ for fixed-size output struct.

Pseudocode

```
STRUCT Config { sensor_id, gain }
GLOBAL Config config
FUNCTION parse_json(json)
    state = START
    key = ""
    FOR each char in json:
        IF state == START and char == '{':
            state = KEY
        ELSE IF state == KEY and char == '"':
            collect key until '"'
            state = VALUE
        ELSE IF state == VALUE and char == ':':
            IF key == "sensor_id":
                parse integer
            ELSE IF key == "gain":
                parse float
        ELSE IF char == '}':
            state = DONE
    RETURN config
MAIN
    json = receive_json()
    config = parse_json(json)
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

typedef struct {
    uint8_t sensor_id;
    float gain;
} Config;

Config parse_json(const char* json) {
    Config config = {0};
    char key[16] = {0};
    uint8_t key_index = 0;
    uint8_t state = 0; // 0: start, 1: key, 2: value
    const char* p = json;
    while (*p && *p != '}' ) {
        if (state == 0 && *p == '{') {
            state = 1;
        } else if (state == 1 && *p != '"') {
            key[key_index++] = *p;
        } else if (state == 1 && *p == '"') {
            key[key_index] = '\\0';
            state = 2;
            key_index = 0;
        } else if (state == 2 && *p == ':') {
            p++;
            if (strcmp(key, "sensor_id") == 0) {
                config.sensor_id = atoi(p);
            } else if (strcmp(key, "gain") == 0) {
                config.gain = atof(p);
            }
            state = 0;
        }
        p++;
    }
    return config;
}

int main(void) {
    const char* json = "{\"sensor_id\":1,\"gain\":2.5}";
    Config config = parse_json(json);
    // Use config
    return 0;
}
```

Expert Tips

- **Robustness:** Handle malformed JSON with error codes.
- **Library:** Use a lightweight JSON library (e.g., cJSON) for complex JSON.
- **Testing:** Test with invalid and edge-case JSON strings.
- **Memory:** Use static buffers to avoid dynamic allocation.
- **Validation:** Verify parsed values against expected ranges.

Project 19: Union for Memory Sharing Between Data Types

Description: Use a union to share memory between different data types (e.g., uint16_t, int16_t, two uint8_t).

Requirements:

- Share memory for a 16-bit value (uint16_t, int16_t, or two uint8_t).
- Assumptions: Used for sensor data conversion.

Ways to Solve

Algorithm Used

- **Union Definition:** Define a union with multiple data types sharing the same memory.
- **Access Control:** Access the appropriate member based on context.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$ (fixed-size union).

Pseudocode

```
UNION Data { uint16_t u16, int16_t i16, uint8_t bytes[2] }
FUNCTION process_data(data, type)
    UNION Data d
    IF type == UINT16:
        d.u16 = data
        RETURN d.u16
    ELSE IF type == INT16:
        d.i16 = data
        RETURN d.i16
    ELSE IF type == BYTES:
        d.bytes[0] = data & 0xFF
        d.bytes[1] = data >> 8
        RETURN d.bytes
MAIN
    data = read_sensor()
    result = process_data(data, type)
```

C Code Implementation

```
#include <stdint.h>

typedef union {
    uint16_t u16;
    int16_t i16;
    uint8_t bytes[2];
} Data;

typedef enum { UINT16, INT16, BYTES } DataType;

void process_data(uint16_t input, DataType type, Data* result) {
    Data d;
    switch (type) {
        case UINT16:
            d.u16 = input;
            *result = d;
            break;
        case INT16:
            d.i16 = (int16_t)input;
            *result = d;
            break;
        case BYTES:
            d.bytes[0] = input & 0xFF;
            d.bytes[1] = input >> 8;
            *result = d;
            break;
    }
}

int main(void) {
    Data result;
    process_data(0x1234, UINT16, &result);
}
```

```

    // Use result.u16
    process_data(0x1234, BYTES, &result);
    // Use result.bytes
    return 0;
}

```

Expert Tips

- **Alignment:** Ensure union members are properly aligned.
- **Type Safety:** Avoid type punning issues (use explicit casts).
- **Testing:** Verify all member accesses with known data.
- **Documentation:** Clearly document the active member.
- **Use Case:** Combine with structs for complex data.

Project 20: Sliding Window Average for Sensor Data

Description: Implement a sliding window average for sensor data to smooth noisy readings.

Requirements:

- Window size: 8 samples (uint16_t).
- Compute average incrementally.
- Assumptions: Continuous sensor data stream.

Ways to Solve

Algorithm Used

- **Circular Buffer:** Store samples in a fixed-size array with a head pointer.
- **Incremental Average:** Maintain a running sum, subtract oldest sample, add newest sample.

Complexity

- **Time Complexity:** $O(1)$ per update (incremental sum).
- **Space Complexity:** $O(n)$ for window size ($n=8$).

Pseudocode

```

STRUCT SlidingWindow { data[8], head, sum }
GLOBAL SlidingWindow window
FUNCTION init_window
    head = 0
    sum = 0
    FOR i = 0 to 7:
        data[i] = 0
FUNCTION update_average(new_sample)
    sum -= data[head]
    data[head] = new_sample
    sum += new_sample
    head = (head + 1) % 8
    RETURN sum / 8
MAIN
    init_window()
    WHILE true:
        sample = read_sensor()
        avg = update_average(sample)

```

C Code Implementation

```
#include <stdint.h>

#define WINDOW_SIZE 8

typedef struct {
    uint16_t data[WINDOW_SIZE];
    uint8_t head;
    uint32_t sum;
} SlidingWindow;

SlidingWindow window;

uint16_t read_sensor(void) { /* Hardware-specific */ return 0; }

void init_window(void) {
    window.head = 0;
    window.sum = 0;
    for (uint8_t i = 0; i < WINDOW_SIZE; i++) {
        window.data[i] = 0;
    }
}

float update_average(uint16_t new_sample) {
    window.sum -= window.data[window.head];
    window.data[window.head] = new_sample;
    window.sum += new_sample;
    window.head = (window.head + 1) % WINDOW_SIZE;
    return (float)window.sum / WINDOW_SIZE;
}

int main(void) {
    init_window();
    while (1) {
        uint16_t sample = read_sensor();
        float avg = update_average(sample);
        // Use avg
    }
    return 0;
}
```

Expert Tips

- **Initialization:** Handle initial window fill (e.g., return partial average).
- **Precision:** Use fixed-point for division if FPU is unavailable.
- **Testing:** Simulate noisy sensor data.
- **Optimization:** Use power-of-2 window size for efficient modulo.
- **Robustness:** Add overflow checks for sum.

Project 21: Macro for Peripheral Register Settings

Description: Use a macro to configure peripheral register settings (e.g., for a timer or GPIO peripheral).

Requirements:

- Define macros to set/clear specific bits in a 32-bit timer control register (e.g., enable, mode).
- Assumptions: Direct register access, timer has enable (bit 0) and mode (bits 1–2).

Ways to Solve

Algorithm Used

- **Macros:** Define macros to manipulate specific bits using bitwise operations.
- **Configuration:** Apply macros to set register values atomically.

Complexity

- **Time Complexity:** $O(1)$ per macro invocation.
- **Space Complexity:** $O(1)$ (no additional storage).

Pseudocode

```
DEFINE MACRO SET_ENABLE(reg, val) reg = (reg & ~1) | (val & 1)
DEFINE MACRO SET_MODE(reg, val) reg = (reg & ~0x6) | ((val & 0x3) << 1)
FUNCTION configure_timer(enable, mode)
    SET_ENABLE(TIMER_REG, enable)
    SET_MODE(TIMER_REG, mode)
MAIN
    configure_timer(1, 2)
```

C Code Implementation

```
#include <stdint.h>

#define TIMER_REG (*(volatile uint32_t*)0x40002000) // Example address
#define SET_ENABLE(reg, val) ((reg) = ((reg) & ~0x1) | ((val) & 0x1))
#define SET_MODE(reg, val) ((reg) = ((reg) & ~0x6) | (((val) & 0x3) << 1))

void configure_timer(uint8_t enable, uint8_t mode) {
    SET_ENABLE(TIMER_REG, enable);
    SET_MODE(TIMER_REG, mode);
}

int main(void) {
    configure_timer(1, 2); // Enable timer, set mode to 2
    return 0;
}
```

Expert Tips

- **Readability:** Use descriptive macro names and document bit fields.
- **Safety:** Avoid side effects in macros (e.g., multiple evaluations).
- **Testing:** Verify register values with a debugger.
- **Alternative:** Use inline functions for complex operations.
- **Portability:** Ensure macros match hardware documentation.

Project 22: Multiple Interrupts with Shared Variables

Description: Handle multiple interrupts accessing shared variables (e.g., sensor data updated by two interrupts).

Requirements:

- Share a `uint16_t` variable between two interrupts (e.g., timer and ADC).
- Ensure thread-safe access.
- Assumptions: Interrupts have different priorities, 16-bit aligned access.

Ways to Solve

Algorithm Used

- **Volatile Variables:** Declare shared variables as volatile.
- **Critical Sections:** Disable/enable interrupts for atomic access to shared data.
- **Priority Handling:** Ensure higher-priority interrupt isn't blocked excessively.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$.

Pseudocode

```
GLOBAL volatile sensor_data
FUNCTION disable_interrupts
    /* Platform-specific disable */
FUNCTION enable_interrupts
    /* Platform-specific enable */
FUNCTION timer_interrupt
    disable_interrupts()
    sensor_data = read_timer_data()
    enable_interrupts()
FUNCTION adc_interrupt
    disable_interrupts()
    sensor_data = read_adc_data()
    enable_interrupts()
FUNCTION read_sensor_data
    disable_interrupts()
    local_data = sensor_data
    enable_interrupts()
    RETURN local_data
MAIN
    WHILE true:
        data = read_sensor_data()
        process(data)
```

C Code Implementation

```
#include <stdint.h>

volatile uint16_t sensor_data = 0;

void disable_interrupts(void) { /* Platform-specific */ }

void enable_interrupts(void) { /* Platform-specific */ }

uint16_t read_timer_data(void) { return 100; } // Stub

uint16_t read_adc_data(void) { return 200; } // Stub

void process(uint16_t data) { /* Process data */ }

void timer_isr(void) {
    disable_interrupts();
    sensor_data = read_timer_data();
    enable_interrupts();
}

void adc_isr(void) {
    disable_interrupts();
    sensor_data = read_adc_data();
    enable_interrupts();
}
```

```

uint16_t read_sensor_data(void) {
    uint16_t local_data;
    disable_interrupts();
    local_data = sensor_data;
    enable_interrupts();
    return local_data;
}

int main(void) {
    while (1) {
        uint16_t data = read_sensor_data();
        process(data);
    }
    return 0;
}

```

Expert Tips

- **Atomicity:** Use 16-bit variables for atomic access on 16/32-bit systems.
- **Latency:** Minimize critical section duration to avoid blocking high-priority interrupts.
- **Testing:** Simulate concurrent interrupts with a debugger.
- **Optimization:** Use atomic operations (e.g., LDREX/STREX on ARM) if available.
- **Debugging:** Log access conflicts for analysis.

Project 23: Simple Data Compression Algorithm

Description: Implement a simple data compression algorithm (e.g., run-length encoding) for sensor data.

Requirements:

- Compress a `uint8_t` array using run-length encoding (RLE).
- Output format: {value, count} pairs.
- Assumptions: Max count per run is 255, max input size 64 bytes.

Ways to Solve

Algorithm Used

- **Run-Length Encoding:** Iterate through input, counting consecutive identical values.
- Store each run as a {value, count} pair.
- **Decompression:** Reverse the process to reconstruct the original data.

Complexity

- **Time Complexity:** $O(n)$ for compression/decompression, where n is input length.
- **Space Complexity:** $O(n)$ for output buffer (worst case).

Pseudocode

```

STRUCT RleEntry { value, count }
GLOBAL RleEntry output[64]
FUNCTION compress_rle(input, length, output, out_length)
    out_index = 0
    count = 1
    FOR i = 1 to length:
        IF input[i] == input[i-1] AND count < 255:
            count++
        ELSE:
            output[out_index++] = {input[i-1], count}
            count = 1
    output[out_index++] = {input[length-1], count}
    *out_length = out_index

```

```

    RETURN output
MAIN
    input = read_data()
    compress_rle(input, length, output, &out_length)

```

C Code Implementation

```

#include <stdint.h>

#define MAX_INPUT 64

typedef struct {
    uint8_t value;
    uint8_t count;
} RleEntry;

void compress_rle(uint8_t* input, uint8_t length, RleEntry* output, uint8_t* out_length) {
    if (length == 0) {
        *out_length = 0;
        return;
    }
    uint8_t out_index = 0;
    uint8_t count = 1;
    for (uint8_t i = 1; i < length; i++) {
        if (input[i] == input[i-1] && count < 255) {
            count++;
        } else {
            output[out_index++] = (RleEntry){input[i-1], count};
            count = 1;
        }
    }
    output[out_index++] = (RleEntry){input[length-1], count};
    *out_length = out_index;
}

int main(void) {
    uint8_t input[] = {1, 1, 1, 2, 2, 3};
    RleEntry output[MAX_INPUT];
    uint8_t out_length;
    compress_rle(input, 6, output, &out_length);
    // Use output
    return 0;
}

```

Expert Tips

- **Efficiency:** Test compression ratio with real sensor data.
- **Robustness:** Handle edge cases (e.g., single value, max count).
- **Testing:** Verify decompression restores original data.
- **Optimization:** Use variable-length encoding for counts > 255.
- **Storage:** Store compressed data in non-volatile memory.

Project 24: Pointer Array for Peripheral Drivers

Description: Use a pointer array to manage peripheral drivers (e.g., UART, I2C, SPI).

Requirements:

- Support 3 peripherals with common interface (init, read, write).
- Assumptions: Function pointer-based driver interface.

Ways to Solve

Algorithm Used

- **Function Pointer Array:** Define a driver interface with function pointers for init, read, and write.
- Store drivers in an array indexed by peripheral ID.
- **Dispatch:** Call driver functions based on peripheral ID.

Complexity

- **Time Complexity:** $O(1)$ for driver function access.
- **Space Complexity:** $O(n)$ for n drivers.

Pseudocode

```
STRUCT Driver { init_fn, read_fn, write_fn }
GLOBAL Driver drivers[3]
FUNCTION init_peripheral(id)
    IF id valid:
        drivers[id].init_fn()
FUNCTION read_peripheral(id, data)
    IF id valid:
        RETURN drivers[id].read_fn(data)
FUNCTION write_peripheral(id, data)
    IF id valid:
        drivers[id].write_fn(data)
MAIN
    drivers = [{uart_init, uart_read, uart_write}, ...]
    init_peripheral(0)
    write_peripheral(0, data)
```

C Code Implementation

```
#include <stdint.h>

typedef void (*InitFn)(void);

typedef uint8_t (*ReadFn)(uint8_t*);

typedef void (*WriteFn)(uint8_t);

typedef struct {
    InitFn init;
    ReadFn read;
    WriteFn write;
} Driver;

void uart_init(void) { /* UART init */ }

uint8_t uart_read(uint8_t* data) { return 0; } // Stub

void uart_write(uint8_t data) { /* UART write */ }

void i2c_init(void) { /* I2C init */ }

uint8_t i2c_read(uint8_t* data) { return 0; } // Stub

void i2c_write(uint8_t data) { /* I2C write */ }

Driver drivers[] = {
    {uart_init, uart_read, uart_write},
    {i2c_init, i2c_read, i2c_write}
};
```



```

void init_peripheral(uint8_t id) {
    if (id < 2) drivers[id].init();
}

uint8_t read_peripheral(uint8_t id, uint8_t* data) {
    if (id < 2) return drivers[id].read(data);
    return 0;
}

void write_peripheral(uint8_t id, uint8_t data) {
    if (id < 2) drivers[id].write(data);
}

int main(void) {
    init_peripheral(0);
    uint8_t data;
    write_peripheral(0, 0xAA);
    read_peripheral(0, &data);
    return 0;
}

```

Expert Tips

- **Validation:** Check for NULL pointers in driver table.
- **Storage:** Use const for driver array to save RAM.
- **Testing:** Unit test each driver independently.
- **Extensibility:** Support dynamic driver registration.
- **Debugging:** Log driver calls for debugging.

Project 25: Moving Average for ADC Readings

Description: Calculate a moving average for ADC readings to smooth noisy data.

Requirements:

- Window size: 8 samples (uint16_t).
- Compute average incrementally.
- Assumptions: Continuous ADC data stream.

Ways to Solve

Algorithm Used

- **Circular Buffer:** Store samples in a fixed-size array with a head pointer.
- **Incremental Average:** Maintain a running sum, subtract oldest sample, add newest.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(n)$ for window size ($n=8$).

Pseudocode

```

STRUCT MovingAvg { data[8], head, sum }
GLOBAL MovingAvg avg
FUNCTION init_moving_avg
    head = 0, sum = 0
    FOR i = 0 to 7:
        data[i] = 0
FUNCTION update_moving_avg(new_sample)
    sum -= data[head]
    data[head] = new_sample

```

```

    sum += new_sample
    head = (head + 1) % 8
    RETURN sum / 8
MAIN
    init_moving_avg()
    WHILE true:
        sample = read_adc()
        avg = update_moving_avg(sample)

```

C Code Implementation

```

#include <stdint.h>

#define WINDOW_SIZE 8

typedef struct {
    uint16_t data[WINDOW_SIZE];
    uint8_t head;
    uint32_t sum;
} MovingAvg;

MovingAvg avg;

uint16_t read_adc(void) { return 0; } // Stub

void init_moving_avg(void) {
    avg.head = 0;
    avg.sum = 0;
    for (uint8_t i = 0; i < WINDOW_SIZE; i++) {
        avg.data[i] = 0;
    }
}

float update_moving_avg(uint16_t new_sample) {
    avg.sum -= avg.data[avg.head];
    avg.data[avg.head] = new_sample;
    avg.sum += new_sample;
    avg.head = (avg.head + 1) % WINDOW_SIZE;
    return (float)avg.sum / WINDOW_SIZE;
}

int main(void) {
    init_moving_avg();
    while (1) {
        uint16_t sample = read_adc();
        float result = update_moving_avg(sample);
        // Use result
    }
    return 0;
}

```

Expert Tips

- **Initialization:** Handle partial window during startup.
- **Precision:** Use fixed-point for division if no FPU.
- **Testing:** Simulate noisy ADC data.
- **Optimization:** Use power-of-2 window size for efficient modulo.
- **Overflow:** Check sum for overflow.

Project 26: Basic Protocol Encoder for Serial Communication

Description: Implement a protocol encoder for serial communication (e.g., start byte, data, checksum, end byte).

Requirements:

- Frame format: start (0xAA), 8-byte data, 1-byte checksum (sum), end (0xBB).
- Assumptions: UART transmission.

Ways to Solve

Algorithm Used

- **Framing:** Construct frame with start byte, data, checksum, and end byte.
- **Checksum:** Sum all data bytes modulo 256.

Complexity

- **Time Complexity:** $O(n)$ for n -byte data.
- **Space Complexity:** $O(n)$ for frame buffer.

Pseudocode

```
FUNCTION encode_frame(data, length, frame)
    frame[0] = 0xAA
    FOR i = 0 to length-1:
        frame[i+1] = data[i]
    checksum = SUM(data) % 256
    frame[length+1] = checksum
    frame[length+2] = 0xBB
    RETURN frame
MAIN
    data = get_data()
    frame = encode_frame(data, 8, frame_buffer)
    send_frame(frame)
```

C Code Implementation

```
#include <stdint.h>

#define DATA_SIZE 8
void send_frame(uint8_t* frame, uint8_t length) { /* UART send */ }

void encode_frame(uint8_t* data, uint8_t length, uint8_t* frame) {
    frame[0] = 0xAA;
    uint8_t checksum = 0;
    for (uint8_t i = 0; i < length; i++) {
        frame[i + 1] = data[i];
        checksum += data[i];
    }
    frame[length + 1] = checksum;
    frame[length + 2] = 0xBB;
}

int main(void) {
    uint8_t data[DATA_SIZE] = {1, 2, 3, 4, 5, 6, 7, 8};
    uint8_t frame[DATA_SIZE + 3];
    encode_frame(data, DATA_SIZE, frame);
    send_frame(frame, DATA_SIZE + 3);
    return 0;
}
```

Expert Tips

- **Robustness:** Add error detection for frame boundaries.
- **Testing:** Simulate transmission errors (e.g., dropped bytes).
- **Optimization:** Preallocate frame buffer to avoid dynamic allocation.
- **Extensibility:** Support variable-length data.
- **Debugging:** Log frames for verification.

Project 27: Static Variable for System State Tracking

Description: Use a static variable to track system state (e.g., power mode: normal, low-power).

Requirements:

- Track state with values: NORMAL, LOW_POWER, SLEEP.
- Assumptions: State accessed by multiple functions.

Ways to Solve

Algorithm Used

- **Static Variable:** Declare a static variable in a function or module to maintain state.
- **State Access:** Provide getter/setter functions for controlled access.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$.

Pseudocode

```
ENUM SystemState { NORMAL, LOW_POWER, SLEEP }
FUNCTION get_system_state
    STATIC state = NORMAL
    RETURN state
FUNCTION set_system_state(new_state)
    STATIC state = NORMAL
    state = new_state
MAIN
    set_system_state(LOW_POWER)
    state = get_system_state()
```

C Code Implementation

```
#include <stdint.h>

typedef enum { NORMAL, LOW_POWER, SLEEP } SystemState;

SystemState get_system_state(void) {
    static SystemState state = NORMAL;
    return state;
}

void set_system_state(SystemState new_state) {
    static SystemState state = NORMAL;
    state = new_state;
}

int main(void) {
    set_system_state(LOW_POWER);
    SystemState state = get_system_state();
    // Use state
    return 0; }
```

Expert Tips

- **Thread Safety:** Use critical sections if accessed in interrupts.
- **Initialization:** Ensure proper initial state.
- **Testing:** Verify state transitions with unit tests.
- **Debugging:** Log state changes for analysis.
- **Extensibility:** Use enums for clear state definitions.

Project 28: Multi-Level State Machine for Vending Machine

Description: Implement a multi-level state machine for a vending machine (e.g., idle, selecting, dispensing).

Requirements:

- States: IDLE, SELECTING, DISPENSING; sub-states in SELECTING (e.g., ITEM1, ITEM2).
- Assumptions: Input via buttons, output via actuators.

Ways to Solve

Algorithm Used

- **Hierarchical FSM:** Use a main state machine with sub-states for SELECTING.
- **Transition Logic:** Handle transitions based on button inputs and timers.

Complexity

- **Time Complexity:** $O(1)$ per transition.
- **Space Complexity:** $O(1)$.

Pseudocode

```
ENUM MainState { IDLE, SELECTING, DISPENSING }
ENUM SelectState { NONE, ITEM1, ITEM2 }
GLOBAL main_state, select_state
FUNCTION vending_machine_fsm
    CASE main_state:
        IDLE:
            IF button_pressed:
                main_state = SELECTING
                select_state = NONE
        SELECTING:
            CASE select_state:
                NONE:
                    IF button1: select_state = ITEM1
                    IF button2: select_state = ITEM2
                ITEM1, ITEM2:
                    IF confirm_button:
                        main_state = DISPENSING
        DISPENSING:
            dispense_item(select_state)
            main_state = IDLE
MAIN
    main_state = IDLE
    WHILE true:
        vending_machine_fsm()
```

C Code Implementation

```
#include <stdint.h>

typedef enum { IDLE, SELECTING, DISPENSING } MainState;

typedef enum { NONE, ITEM1, ITEM2 } SelectState;

MainState main_state = IDLE;

SelectState select_state = NONE;

uint8_t button_pressed(void) { return 0; } // Stub

uint8_t button1(void) { return 0; } // Stub

uint8_t button2(void) { return 0; } // Stub

uint8_t confirm_button(void) { return 0; } // Stub

void dispense_item(SelectState item) { /* Actuate */ }

void vending_machine_fsm(void) {
    switch (main_state) {
        case IDLE:
            if (button_pressed()) {
                main_state = SELECTING;
                select_state = NONE;
            }
            break;
        case SELECTING:
            switch (select_state) {
                case NONE:
                    if (button1()) select_state = ITEM1;
                    else if (button2()) select_state = ITEM2;
                    break;
                case ITEM1:
                case ITEM2:
                    if (confirm_button()) main_state = DISPENSING;
                    break;
            }
            break;
        case DISPENSING:
            dispense_item(select_state);
            main_state = IDLE;
            break;
    }
}

int main(void) {
    while (1) {
        vending_machine_fsm();
    }
    return 0;
}
```

Expert Tips

- **Robustness:** Add timeout for SELECTING state.
- **Testing:** Simulate all state transitions.
- **Scalability:** Use state transition table for complex FSMs.
- **Debugging:** Log state changes for analysis.
- **Extensibility:** Support additional items or states.

Project 29: Typedef Struct for Sensor Calibration Data

Description: Use a typedef struct to manage sensor calibration data (e.g., offset, gain).

Requirements:

- Store calibration for a sensor (uint16_t offset, float gain).
- Apply calibration to raw data.
- Assumptions: Sensor data is uint16_t.

Ways to Solve

Algorithm Used

- **Struct Definition:** Define a struct for offset and gain.
- **Calibration:** Apply formula: $\text{calibrated} = (\text{raw} + \text{offset}) * \text{gain}$.

Complexity

- **Time Complexity:** $O(1)$ per calibration.
- **Space Complexity:** $O(1)$.

Pseudocode

```
STRUCT SensorCalibration { offset, gain }
GLOBAL SensorCalibration calib
FUNCTION apply_calibration(raw)
    RETURN (raw + calib.offset) * calib.gain
MAIN
    calib = {offset: 10, gain: 1.5}
    raw = read_sensor()
    value = apply_calibration(raw)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t offset;
    float gain;
} SensorCalibration;

SensorCalibration calib = {10, 1.5f};

uint16_t read_sensor(void) { return 0; } // Stub
float apply_calibration(uint16_t raw) {
    return (raw + calib.offset) * calib.gain;
}

int main(void) {
    uint16_t raw = read_sensor();
    float value = apply_calibration(raw);
    // Use value
    return 0;
}
```

Expert Tips

- **Storage:** Store calibration in non-volatile memory.
- **Validation:** Check calibration parameters for valid ranges.
- **Testing:** Verify with known sensor inputs.
- **Optimization:** Use fixed-point for gain if no FPU.

- **Extensibility:** Support multiple sensors with an array.

Project 30: Matrix Addition for Sensor Data Processing

Description: Perform matrix addition for sensor data processing (e.g., combining 2D sensor arrays).

Requirements:

- Add two 3x3 matrices of `uint16_t`.
- Assumptions: Fixed-size matrices.

Ways to Solve

Algorithm Used

- **Matrix Addition:** Iterate over elements, adding corresponding values from two matrices.
- **Storage:** Use 2D arrays for matrices.

Complexity

- **Time Complexity:** $O(m \times n)$ for $m \times n$ matrix (here, 3x3, so $O(9)$).
- **Space Complexity:** $O(m \times n)$ for result matrix.

Pseudocode

```
FUNCTION matrix_add(matrix1, matrix2, result, rows, cols)
    FOR i = 0 to rows-1:
        FOR j = 0 to cols-1:
            result[i][j] = matrix1[i][j] + matrix2[i][j]
MAIN
    matrix1 = {{1,2,3}, ...}
    matrix2 = {{4,5,6}, ...}
    matrix_add(matrix1, matrix2, result, 3, 3)
```

C Code Implementation

```
#include <stdint.h>

#define ROWS 3
#define COLS 3

void matrix_add(uint16_t m1[ROWS][COLS], uint16_t m2[ROWS][COLS], uint16_t result[ROWS][COLS]) {
    for (uint8_t i = 0; i < ROWS; i++) {
        for (uint8_t j = 0; j < COLS; j++) {
            result[i][j] = m1[i][j] + m2[i][j];
        }
    }
}

int main(void) {
    uint16_t m1[ROWS][COLS] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    uint16_t m2[ROWS][COLS] = {{1, 1, 1}, {2, 2, 2}, {3, 3, 3}};
    uint16_t result[ROWS][COLS];
    matrix_add(m1, m2, result);
    // Use result
    return 0;
}
```

Expert Tips

- **Validation:** Check matrix dimensions for compatibility.
- **Optimization:** Use pointer arithmetic for faster access.
- **Testing:** Verify with edge cases (e.g., zero matrices).

- **Overflow:** Check for uint16_t overflow in addition.
- **Extensibility:** Support dynamic matrix sizes.

Project 31: Inline Functions for GPIO Access

Description: Use inline functions to optimize GPIO access (e.g., set/clear pins).

Requirements:

- Define inline functions for setting/clearing GPIO pins.
- Assumptions: 32-bit GPIO register, direct access.

Ways to Solve

Algorithm Used

- **Inline Functions:** Define static inline functions to set/clear bits in a GPIO register.
- **Bit Manipulation:** Use bitwise operations for pin control.

Complexity

- **Time Complexity:** O(1) per access.
- **Space Complexity:** O(1).

Pseudocode

```
FUNCTION inline set_pin(pin)
    GPIO_REG |= (1 << pin)
FUNCTION inline clear_pin(pin)
    GPIO_REG &= ~(1 << pin)
MAIN
    set_pin(0)
    clear_pin(1)
```

C Code Implementation

```
#include <stdint.h>

#define GPIO_REG (*(volatile uint32_t*)0x40003000) // Example address

static inline void set_pin(uint8_t pin) {
    GPIO_REG |= (1 << pin);
}

static inline void clear_pin(uint8_t pin) {
    GPIO_REG &= ~(1 << pin);
}

int main(void) {
    set_pin(0);
    clear_pin(1);
    return 0;
}
```

Expert Tips

- **Inlining:** Ensure compiler supports inline (e.g., -O2).
- **Validation:** Check pin number validity.
- **Testing:** Verify with hardware debugger.
- **Portability:** Use HAL for cross-platform code.
- **Performance:** Benchmark against direct writes.

Project 32: Simple Data Logger with Timestamp

Description: Implement a data logger with timestamps for sensor data.

Requirements:

- Log `uint16_t` sensor data with 32-bit timestamp (ms).
- Store in a fixed-size buffer (16 entries).
- Assumptions: RTC provides timestamp.

Ways to Solve

Algorithm Used

- **Log Struct:** Define a struct for data and timestamp.
- **Circular Buffer:** Store log entries in a fixed-size array.
- **Logging:** Append new entries with current timestamp.

Complexity

- **Time Complexity:** $O(1)$ per log entry.
- **Space Complexity:** $O(n)$ for n entries ($n=16$).

Pseudocode

```
STRUCT LogEntry { data, timestamp }
GLOBAL LogEntry log[16], index
FUNCTION init_logger
    index = 0
FUNCTION log_data(data)
    timestamp = get_timestamp()
    log[index] = {data, timestamp}
    index = (index + 1) % 16
MAIN
    init_logger()
    WHILE true:
        data = read_sensor()
        log_data(data)
```

C Code Implementation

```
#include <stdint.h>

#define LOG_SIZE 16

typedef struct {
    uint16_t data;
    uint32_t timestamp;
} LogEntry;

LogEntry log[LOG_SIZE];
uint8_t index = 0;
uint16_t read_sensor(void) { return 0; } // Stub

uint32_t get_timestamp(void) { return 0; } // Stub

void init_logger(void) {    index = 0;}

void log_data(uint16_t data) {
    log[index].data = data;
    log[index].timestamp = get_timestamp();
    index = (index + 1) % LOG_SIZE;
}
```

```

int main(void) {
    init_logger();
    while (1) {
        uint16_t data = read_sensor();
        log_data(data);
    }
    return 0;
}

```

Expert Tips

- **Storage:** Save logs to non-volatile memory periodically.
- **Testing:** Verify timestamp accuracy with RTC.
- **Overflow:** Handle timestamp overflow.
- **Optimization:** Compress data before logging.
- **Extensibility:** Add log retrieval function.

Project 33: Bitmask for Controlling Multiple LEDs

Description: Use a bitmask to control multiple LEDs (e.g., 4 LEDs).

Requirements:

- Control 4 LEDs using a single uint8_t bitmask.
- Assumptions: LEDs mapped to GPIO bits 0–3.

Ways to Solve

Algorithm Used

- **Bitmask:** Use a uint8_t to represent LED states (1=on, 0=off).
- **GPIO Update:** Write bitmask to GPIO register.

Complexity

- **Time Complexity:** O(1) per update.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION set_leds(mask)
    GPIO_REG = (GPIO_REG & ~0xF) | (mask & 0xF)
MAIN
    set_leds(0b1010) // LED1, LED3 on

```

C Code Implementation

```

#include <stdint.h>

#define LED_MASK 0x0F
#define GPIO_REG (*(volatile uint32_t*)0x40004000) // Example address
void set_leds(uint8_t mask) {
    GPIO_REG = (GPIO_REG & ~LED_MASK) | (mask & LED_MASK);
}

int main(void) {
    set_leds(0b1010); // LED1, LED3 on
    return 0;
}

```

Expert Tips

- **Validation:** Ensure mask aligns with LED bits.
- **Testing:** Verify with hardware LEDs.
- **Optimization:** Use bit-banding if available (e.g., ARM Cortex-M).
- **Debugging:** Log mask changes.
- **Extensibility:** Support more LEDs with wider bitmask.

Project 34: Double-Precision Floating-Point Math

Description: Handle double-precision floating-point math in firmware (e.g., multiply two doubles).

Requirements:

- Perform double-precision multiplication.
- Assumptions: FPU available or software floating-point.

Ways to Solve

Algorithm Used

- **Double-Precision:** Use double type for calculations.
- **Validation:** Check for overflow/underflow.

Complexity

- **Time Complexity:** $O(1)$ per operation (hardware-dependent).
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION multiply_double(a, b)
    result = a * b
    IF result is inf or nan:
        RETURN error
    RETURN result
MAIN
    result = multiply_double(a, b)
```

C Code Implementation

```
#include <stdint.h>
#include <math.h>

#define ERROR -1.0

double multiply_double(double a, double b) {
    double result = a * b;
    if (isinf(result) || isnan(result)) return ERROR;
    return result;
}

int main(void) {
    double a = 1.5, b = 2.3;
    double result = multiply_double(a, b);
    // Use result
    return 0;
}
```

Expert Tips

- **FPU Check:** Verify FPU availability; use fixed-point if absent.

- **Testing:** Test with edge cases (e.g., max/min doubles).
- **Optimization:** Minimize floating-point use in tight loops.
- **Precision:** Document precision requirements.
- **Robustness:** Handle edge cases explicitly.

Project 35: Basic Error Handler for Peripheral Failures

Description: Implement an error handler for peripheral failures (e.g., UART timeout).

Requirements:

- Detect and handle UART transmission timeout.
- Assumptions: UART status register indicates errors.

Ways to Solve

Algorithm Used

- **Error Detection:** Check UART status register for timeout flag.
- **Recovery:** Reset UART and log error.

Complexity

- **Time Complexity:** $O(1)$ per check.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION check_uart_error
    IF UART_STATUS & TIMEOUT_FLAG:
        log_error("UART timeout")
        reset_uart()
        RETURN error
    RETURN success
MAIN
    WHILE true:
        IF transmit_data():
            check_uart_error()
  
```

C Code Implementation

```

#include <stdint.h>

#define TIMEOUT_FLAG 0x1
#define UART_STATUS (*(volatile uint32_t*)0x40005000) // Example address

void log_error(const char* msg) { /* Log error */ }

void reset_uart(void) { /* Reset UART */ }

uint8_t transmit_data(void) { return 0; } // Stub

uint8_t check_uart_error(void) {
    if (UART_STATUS & TIMEOUT_FLAG) {
        log_error("UART timeout");
        reset_uart();
        return 1;
    }
    return 0;
}
  
```

```

int main(void) {
    while (1) {
        if (transmit_data()) {
            check_uart_error();
        }
    }
    return 0;
}

```

Expert Tips

- **Recovery:** Implement retry logic for transient errors.
- **Testing:** Simulate peripheral failures.
- **Logging:** Store errors in non-volatile memory.
- **Robustness:** Prioritize critical errors.
- **Extensibility:** Support multiple peripherals.

Project 36: Function Pointer Table for Modular Code

Description: Use a function pointer table for modular code organization (e.g., for different modes).

Requirements:

- Support 3 modes with distinct processing functions.
- Assumptions: Modes selected dynamically.

Ways to Solve

Algorithm Used

- **Function Pointer Table:** Store function pointers in an array indexed by mode.
- **Dispatch:** Call function based on mode ID.

Complexity

- **Time Complexity:** $O(1)$ per call.
- **Space Complexity:** $O(n)$ for n modes.

Pseudocode

```

GLOBAL process_fn table[3]
FUNCTION mode1_process
    /* Mode 1 logic */
FUNCTION mode2_process
    /* Mode 2 logic */
FUNCTION execute_mode(mode)
    IF mode valid:
        table[mode]()
    ELSE:
        handle_error()
MAIN
    table = [mode1_process, mode2_process, mode3_process]
    execute_mode(selected_mode)

```

C Code Implementation

```

#include <stdint.h>

typedef void (*ProcessFn)(void);

void mode1_process(void) { /* Mode 1 */ }

void mode2_process(void) { /* Mode 2 */ }

```

```

void mode3_process(void) { /* Mode 3 */ }

const ProcessFn table[] = {mode1_process, mode2_process, mode3_process};

void execute_mode(uint8_t mode) {
    if (mode < 3) {
        table[mode]();
    }
}

int main(void) {
    execute_mode(1);
    return 0;
}

```

Expert Tips

- **Storage:** Use const for table to save RAM.
- **Validation:** Check for NULL pointers.
- **Testing:** Unit test each mode function.
- **Extensibility:** Support dynamic table updates.
- **Debugging:** Log mode transitions.

Project 37: Binary Search on Sorted Sensor Data Array

Description: Perform a binary search on a sorted sensor data array (uint16_t).

Requirements:

- Search for a value in a sorted array of 16 elements.
- Assumptions: Array is pre-sorted.

Ways to Solve

Algorithm Used

- **Binary Search:** Divide search space in half iteratively until value is found or not present.
- **Iterative Approach:** Avoid recursion for stack efficiency.

Complexity

- **Time Complexity:** $O(\log n)$ for n elements ($n=16$).
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION binary_search(array, size, target)
    left = 0, right = size - 1
    WHILE left <= right:
        mid = (left + right) / 2
        IF array[mid] == target:
            RETURN mid
        IF array[mid] < target:
            left = mid + 1
        ELSE:
            right = mid - 1
    RETURN -1
MAIN
array = sorted_sensor_data()
index = binary_search(array, 16, target)

```

C Code Implementation

```
#include <stdint.h>

int8_t binary_search(uint16_t* array, uint8_t size, uint16_t target) {
    int8_t left = 0, right = size - 1;
    while (left <= right) {
        int8_t mid = (left + right) / 2;
        if (array[mid] == target) return mid;
        if (array[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

int main(void) {
    uint16_t array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    int8_t index = binary_search(array, 16, 10);
    // Use index
    return 0;
}
```

Expert Tips

- **Validation:** Ensure array is sorted before searching.
- **Testing:** Test with edge cases (first, last, missing).
- **Optimization:** Use unsigned arithmetic for mid calculation.
- **Robustness:** Handle empty array.
- **Extensibility:** Support dynamic array sizes.

Project 38: Simple Hash Function for Data Lookup

Description: Implement a simple hash function for data lookup (e.g., sensor IDs).

Requirements:

- Hash uint16_t sensor IDs to indices (0–15).
- Assumptions: Use multiplicative hash, handle collisions with linear probing.

Ways to Solve

Algorithm Used

- **Hash Function:** Use multiplicative hash with a constant (e.g., 0x9E37).
- **Collision Resolution:** Use linear probing to find next free slot.

Complexity

- **Time Complexity:** $O(1)$ average, $O(n)$ worst case for collisions.
- **Space Complexity:** $O(n)$ for hash table ($n=16$).

Pseudocode

```
CONST HASH_SIZE = 16
FUNCTION hash(id)
    RETURN (id * 0x9E37) % HASH_SIZE
FUNCTION insert(id, table)
    index = hash(id)
    WHILE table[index] != EMPTY:
        index = (index + 1) % HASH_SIZE
    table[index] = id
```



```
MAIN
    table = {EMPTY}
    insert(sensor_id, table)
```

C Code Implementation

```
#include <stdint.h>

#define HASH_SIZE 16
#define EMPTY 0xFFFF

uint8_t hash(uint16_t id) {
    return (id * 0x9E37) % HASH_SIZE;
}

void insert(uint16_t id, uint16_t* table) {
    uint8_t index = hash(id);
    while (table[index] != EMPTY) {
        index = (index + 1) % HASH_SIZE;
    }
    table[index] = id;
}

int main(void) {
    uint16_t table[HASH_SIZE] = {[0 ...
HASH_SIZE-1] = EMPTY};
    insert(123, table);
    // Use table
    return 0;
}
```

Expert Tips

- **Distribution:** Test hash function for uniform distribution.
- **Collisions:** Minimize collisions with prime table sizes.
- **Testing:** Verify with large datasets.
- **Optimization:** Use faster hash functions if needed.
- **Robustness:** Handle full table gracefully.

Project 39: Volatile Pointer for Shared Memory Access

Description: Use a volatile pointer for shared memory access in interrupts.

Requirements:

- Share a uint16_t buffer via a volatile pointer.
- Assumptions: Buffer updated by interrupt, read by main loop.

Ways to Solve

Algorithm Used

- **Volatile Pointer:** Declare a pointer to shared memory as volatile.
- **Access Control:** Use atomic reads or critical sections.

Complexity

- **Time Complexity:** O(1) per access.
- **Space Complexity:** O(n) for buffer.

Pseudocode

```
GLOBAL volatile uint16_t* buffer
FUNCTION interrupt_handler
    *buffer = read_data()
FUNCTION read_shared_data
    local_data = *buffer
    RETURN local_data
MAIN
    buffer = allocate_buffer()
    WHILE true:
        data = read_shared_data()
```

C Code Implementation

```
#include <stdint.h>

volatile uint16_t* buffer;

uint16_t read_data(void) { return 0; } // Stub

void process(uint16_t data) { /* Process data */ }

void interrupt_handler(void) {
    *buffer = read_data();
}

uint16_t read_shared_data(void) {
    return *buffer; // Atomic for 16-bit
}

int main(void) {
    uint16_t local_buffer = 0;
    buffer = &local_buffer;
    while (1) {
        uint16_t data = read_shared_data();
        process(data);
    }
    return 0;
}
```

Expert Tips

- **Atomicity:** Ensure aligned access for atomicity.
- **Safety:** Use critical sections for multi-byte data.
- **Testing:** Simulate interrupt conflicts.
- **Optimization:** Use memory barriers if needed.
- **Debugging:** Log access issues.

Project 40: Ring Buffer with Head/Tail Pointers

Description: Implement a ring buffer with head/tail pointers for data storage.

Requirements:

- Buffer size: 16 elements (uint16_t).
- Support push/pop operations.
- Assumptions: Interrupt-driven access.

Ways to Solve

Algorithm Used

- **Ring Buffer:** Use a fixed-size array with head (write) and tail (read) pointers.
- **State Tracking:** Track full/empty using pointer comparison.

Complexity

- **Time Complexity:** $O(1)$ for push/pop.
- **Space Complexity:** $O(n)$ for buffer ($n=16$).

Pseudocode

```
STRUCT RingBuffer { data[16], head, tail }
GLOBAL RingBuffer buffer
FUNCTION init_buffer
    head = 0, tail = 0
FUNCTION is_full
    RETURN (head + 1) % 16 == tail
FUNCTION is_empty
    RETURN head == tail
FUNCTION push(value)
    IF not is_full():
        data[head] = value
        head = (head + 1) % 16
FUNCTION pop(value)
    IF not is_empty():
        *value = data[tail]
        tail = (tail + 1) % 16
        RETURN true
    RETURN false
MAIN
    init_buffer()
    WHILE true:
        IF data_available:
            push(data)
        IF need_data:
            pop(&value)
```

C Code Implementation

```
#include <stdint.h>

#define BUFFER_SIZE 16

typedef struct {
    uint16_t data[BUFFER_SIZE];
    uint8_t head;
    uint8_t tail;
} RingBuffer;

RingBuffer buffer;

void init_buffer(void) {
    buffer.head = 0;
    buffer.tail = 0;
}

uint8_t is_full(void) {
    return (buffer.head + 1) % BUFFER_SIZE == buffer.tail;
}

uint8_t is_empty(void) {
    return buffer.head == buffer.tail;
}
```

```

void push(uint16_t value) {
    if (!is_full()) {
        buffer.data[buffer.head] = value;
        buffer.head = (buffer.head + 1) % BUFFER_SIZE;
    }
}

uint8_t pop(uint16_t* value) {
    if (!is_empty()) {
        *value = buffer.data[buffer.tail];
        buffer.tail = (buffer.tail + 1) % BUFFER_SIZE;
        return 1;
    }
    return 0;
}

int main(void) {
    init_buffer();
    push(100);
    uint16_t value;
    if (pop(&value)) {
        // Use value
    }
    return 0;
}

```

Expert Tips

- **Thread Safety:** Use critical sections for interrupt access.
- **Optimization:** Use power-of-2 size for efficient modulo.
- **Testing:** Simulate rapid push/pop sequences.
- **Robustness:** Add overflow/underflow logging.
- **Extensibility:** Support variable buffer sizes.

Project 41: Basic Packet Parser for Serial Data

Description: Create a program to parse serial data packets with a defined format (e.g., header, length, payload, checksum).

Requirements:

- Packet format: 1-byte header (0xAA), 1-byte length (≤ 32), payload, 1-byte checksum (sum modulo 256).
- Parse and validate packets, extract payload.
- Assumptions: UART input, max payload 32 bytes.

Ways to Solve

Algorithm Used

- **State Machine:** Parse incoming bytes in states: header, length, payload, checksum.
- **Validation:** Check header and checksum; store payload if valid.

Complexity

- **Time Complexity:** $O(n)$ for n -byte packet.
- **Space Complexity:** $O(n)$ for payload buffer.

Pseudocode

```

ENUM State { HEADER, LENGTH, PAYLOAD, CHECKSUM }
GLOBAL buffer[32], payload[32], index, length, state

```

```

FUNCTION parse_packet(byte)
    CASE state:
        HEADER:
            IF byte == 0xAA:
                state = LENGTH
                index = 0
        LENGTH:
            length = byte
            state = PAYLOAD
        PAYLOAD:
            IF index < length:
                payload[index++] = byte
            IF index == length:
                state = CHECKSUM
        CHECKSUM:
            sum = SUM(payload[0..length-1]) + 0xAA + length
            IF byte == sum % 256:
                process_payload(payload, length)
            state = HEADER
MAIN
    WHILE true:
        byte = read_uart()
        parse_packet(byte)

```

C Code Implementation

```

#include <stdint.h>

#define MAX_PAYLOAD 32

typedef enum { HEADER, LENGTH, PAYLOAD, CHECKSUM } State;

uint8_t payload[MAX_PAYLOAD], index = 0, length = 0;

State state = HEADER;

uint8_t read_uart(void) { return 0; } // Stub

void process_payload(uint8_t* data, uint8_t len) { /* Process payload */ }

void parse_packet(uint8_t byte) {
    static uint8_t sum = 0;
    switch (state) {
        case HEADER:
            if (byte == 0xAA) {
                state = LENGTH;
                sum = 0xAA;
                index = 0;
            }
            break;
        case LENGTH:
            length = byte;
            sum += byte;
            state = PAYLOAD;
            break;
        case PAYLOAD:
            if (index < length && index < MAX_PAYLOAD) {
                payload[index++] = byte;
                sum += byte;
            }
            if (index == length) state = CHECKSUM;
            break;
        case CHECKSUM:
            if (byte == (sum % 256)) process_payload(payload, length);
            state = HEADER;
            break;
    }
}

```

```

}

int main(void) {
    while (1) {
        uint8_t byte = read_uart();
        parse_packet(byte);
    }
    return 0;
}

```

Expert Tips

- **Robustness:** Handle malformed packets (e.g., invalid length).
- **Testing:** Simulate partial or corrupted packets.
- **Optimization:** Use a circular buffer for input.
- **Debugging:** Log invalid packets for analysis.
- **Extensibility:** Support variable header values.

Project 42: Macro for Peripheral Register Bitfields

Description: Use a macro to handle peripheral register bitfields (e.g., for SPI control register).

Requirements:

- Define macros to set/clear bits for enable (bit 0) and mode (bits 1–2).
- Assumptions: 32-bit register, direct access.

Ways to Solve

Algorithm Used

- **Macros:** Define macros for bitfield manipulation using bitwise operations.
- **Atomic Updates:** Ensure macros update specific bits without affecting others.

Complexity

- **Time Complexity:** $O(1)$ per macro invocation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

DEFINE MACRO SET_SPI_ENABLE(reg, val) reg = (reg & ~0x1) | (val & 0x1)
DEFINE MACRO SET_SPI_MODE(reg, val) reg = (reg & ~0x6) | ((val & 0x3) << 1)
FUNCTION configure_spi(enable, mode)
    SET_SPI_ENABLE(SPI_REG, enable)
    SET_SPI_MODE(SPI_REG, mode)
MAIN
    configure_spi(1, 2)

```

C Code Implementation

```

#include <stdint.h>

#define SPI_REG (*(volatile uint32_t*)0x40006000) // Example address

#define SET_SPI_ENABLE(reg, val) ((reg) = ((reg) & ~0x1) | ((val) & 0x1))
#define SET_SPI_MODE(reg, val) ((reg) = ((reg) & ~0x6) | (((val) & 0x3) << 1))

void configure_spi(uint8_t enable, uint8_t mode) {
    SET_SPI_ENABLE(SPI_REG, enable);
    SET_SPI_MODE(SPI_REG, mode);
}

```

```
int main(void) {
    configure_spi(1, 2); // Enable SPI, set mode 2
    return 0;
}
```

Expert Tips

- **Readability:** Use clear macro names and document bitfields.
- **Safety:** Avoid side effects in macros.
- **Testing:** Verify register values with a debugger.
- **Alternative:** Use inline functions for complex bitfields.
- **Portability:** Align with hardware manual.

Project 43: Bubble Sort on Sensor Readings

Description: Perform a bubble sort on an array of sensor readings (uint16_t).

Requirements:

- Sort an array of 10 sensor readings in ascending order.
- Assumptions: In-place sorting.

Ways to Solve

Algorithm Used

- **Bubble Sort:** Repeatedly compare adjacent elements and swap if out of order until sorted.
- **Optimization:** Stop if no swaps occur in a pass.

Complexity

- **Time Complexity:** $O(n^2)$ for n elements ($n=10$).
- **Space Complexity:** $O(1)$ (in-place).

Pseudocode

```
FUNCTION bubble_sort(array, size)
    FOR i = 0 to size-1:
        swapped = false
        FOR j = 0 to size-i-2:
            IF array[j] > array[j+1]:
                swap(array[j], array[j+1])
                swapped = true
        IF not swapped:
            BREAK
MAIN
    array = read_sensors()
    bubble_sort(array, 10)
```

C Code Implementation

```
#include <stdint.h>
void bubble_sort(uint16_t* array, uint8_t size) {
    for (uint8_t i = 0; i < size - 1; i++) {
        uint8_t swapped = 0;
        for (uint8_t j = 0; j < size - i - 1; j++) {
            if (array[j] > array[j + 1]) {
                uint16_t temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
                swapped = 1;
            }
        }
    }
}
```

```

        if (!swapped) break;
    }
}

int main(void) {
    uint16_t array[] = {5, 2, 8, 1, 9, 3, 7, 4, 6, 10};
    bubble_sort(array, 10);
    // Use sorted array
    return 0;
}

```

Expert Tips

- **Optimization:** Use early termination for nearly sorted arrays.
- **Testing:** Verify with random and edge-case arrays.
- **Alternative:** Use insertion sort for small arrays.
- **Robustness:** Check array size validity.
- **Performance:** Avoid bubble sort for large datasets.

Project 44: Simple Watchdog Reset Mechanism

Description: Implement a simple watchdog reset mechanism to prevent system hangs.

Requirements:

- Reset watchdog every 100ms.
- Trigger system reset if watchdog expires.
- Assumptions: Hardware watchdog with kick function.

Ways to Solve

Algorithm Used

- **Timer-Based Kick:** Use a timer interrupt to kick the watchdog periodically.
- **Initialization:** Enable watchdog with timeout (e.g., 500ms).

Complexity

- **Time Complexity:** $O(1)$ per kick.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_watchdog
    Configure watchdog with 500ms timeout
    Start watchdog
FUNCTION kick_watchdog
    /* Hardware-specific kick */
FUNCTION timer_interrupt
    EVERY 100ms:
        kick_watchdog()
MAIN
    init_watchdog()
    WHILE true:
        /* Main loop */

```


C Code Implementation

```
#include <stdint.h>

void init_watchdog(void) { /* Configure 500ms timeout */ }

void kick_watchdog(void) { /* Hardware-specific kick */ }

void timer_isr(void) {
    static uint8_t count = 0;
    if (++count >= 10) { // 100ms at 10ms interrupt
        kick_watchdog();
        count = 0;
    }
}

int main(void) {
    init_watchdog();
    while (1) {
        /* Main loop */
    }
    return 0;
}
```

Expert Tips

- **Timing:** Ensure kick frequency is less than watchdog timeout.
- **Testing:** Simulate hangs to verify reset.
- **Robustness:** Add watchdog disable for debugging.
- **Safety:** Log watchdog resets if possible.
- **Hardware:** Align with specific watchdog API.

Project 45: Struct for Communication Protocol States

Description: Use a struct to manage communication protocol states (e.g., idle, sending, receiving).

Requirements:

- States: IDLE, SENDING, RECEIVING.
- Store state and associated data (e.g., bytes sent).
- Assumptions: UART-based protocol.

Ways to Solve

Algorithm Used

- **State Struct:** Define a struct with state enum and metadata (e.g., bytes sent).
- **State Transitions:** Update struct based on protocol events.

Complexity

- **Time Complexity:** $O(1)$ per state update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
STRUCT Protocol { state, bytes_sent }
GLOBAL Protocol proto
FUNCTION init_protocol
    proto.state = IDLE
    proto.bytes_sent = 0
```

```

FUNCTION update_protocol(event)
    CASE proto.state:
        IDLE:
            IF event == START_SEND:
                proto.state = SENDING
                proto.bytes_sent = 0
        SENDING:
            IF event == BYTE_SENT:
                proto.bytes_sent++
            IF all_bytes_sent:
                proto.state = IDLE
        RECEIVING:
            IF event == DATA_RECEIVED:
                proto.state = IDLE
MAIN
    init_protocol()
    WHILE true:
        event = get_event()
        update_protocol(event)

```

C Code Implementation

```

#include <stdint.h>

typedef enum { IDLE, SENDING, RECEIVING } ProtoState;

typedef enum { START_SEND, BYTE_SENT, DATA_RECEIVED } Event;

typedef struct {
    ProtoState state;
    uint8_t bytes_sent;
} Protocol;

Protocol proto;

void init_protocol(void) {
    proto.state = IDLE;
    proto.bytes_sent = 0;
}

void update_protocol(Event event) {
    switch (proto.state) {
        case IDLE:
            if (event == START_SEND) {
                proto.state = SENDING;
                proto.bytes_sent = 0;
            }
            break;
        case SENDING:
            if (event == BYTE_SENT) {
                proto.bytes_sent++;
                if (proto.bytes_sent >= 8) proto.state = IDLE;
            }
            break;
        case RECEIVING:
            if (event == DATA_RECEIVED) proto.state = IDLE;
            break;
    }
}

Event get_event(void) { return START_SEND; } // Stub

int main(void) {
    init_protocol();
    while (1) {
        Event event = get_event();
        update_protocol(event);
    }
}

```

```
}  
    return 0;  
}
```

Expert Tips

- **Extensibility:** Add more states or metadata as needed.
- **Testing:** Simulate all state transitions.
- **Debugging:** Log state changes.
- **Robustness:** Handle invalid events.
- **Optimization:** Use bitfields for small state variables.

Project 46: Multi-Byte Data with Bit Alignment

Description: Handle multi-byte data with bit alignment (e.g., extract 10-bit fields from a byte stream).

Requirements:

- Extract two 10-bit fields from a 3-byte buffer.
- Assumptions: Big-endian byte order.

Ways to Solve

Algorithm Used

- **Bit Manipulation:** Use shifts and masks to extract 10-bit fields from 3 bytes.
- **Alignment:** Handle bit offsets within bytes.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size data.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION extract_fields(buffer, field1, field2)  
    field1 = (buffer[0] << 2) | (buffer[1] >> 6)  
    field2 = ((buffer[1] & 0x3F) << 4) | (buffer[2] >> 4)  
MAIN  
    buffer = read_data()  
    extract_fields(buffer, &field1, &field2)
```

C Code Implementation

```
#include <stdint.h>  
void extract_fields(uint8_t* buffer, uint16_t* field1, uint16_t* field2) {  
    *field1 = ((uint16_t)buffer[0] << 2) | (buffer[1] >> 6); // Bits 0-9  
    *field2 = ((buffer[1] & 0x3F) << 4) | (buffer[2] >> 4); // Bits 10-19  
}  
  
int main(void) {  
    uint8_t buffer[] = {0x12, 0x34, 0x56};  
    uint16_t field1, field2;  
    extract_fields(buffer, &field1, &field2);  
    // Use field1, field2  
    return 0;  
}
```

Expert Tips

- **Validation:** Check buffer size and alignment.

- **Testing:** Verify with known bit patterns.
- **Optimization:** Use lookup tables for complex alignments.
- **Portability:** Handle endianness variations.
- **Documentation:** Document bitfield layout.

Project 47: Basic FIFO Queue for Sensor Data

Description: Implement a basic FIFO queue for sensor data (uint16_t).

Requirements:

- Queue size: 16 elements.
- Support enqueue/dequeue operations.
- Assumptions: Single-threaded access.

Ways to Solve

Algorithm Used

- **FIFO Queue:** Use a fixed-size array with head (write) and tail (read) pointers.
- **State Tracking:** Track full/empty using count or pointer comparison.

Complexity

- **Time Complexity:** $O(1)$ for enqueue/dequeue.
- **Space Complexity:** $O(n)$ for queue size ($n=16$).

Pseudocode

```

STRUCT Queue { data[16], head, tail, count }
GLOBAL Queue queue
FUNCTION init_queue
    head = 0, tail = 0, count = 0
FUNCTION is_full
    RETURN count == 16
FUNCTION is_empty
    RETURN count == 0
FUNCTION enqueue(value)
    IF not is_full():
        data[head] = value
        head = (head + 1) % 16
        count++
FUNCTION dequeue(value)
    IF not is_empty():
        *value = data[tail]
        tail = (tail + 1) % 16
        count--
    RETURN true
RETURN false
MAIN
    init_queue()
    enqueue(data)
    dequeue(&value)

```

C Code Implementation

```

#include <stdint.h>

#define QUEUE_SIZE 16
typedef struct {
    uint16_t data[QUEUE_SIZE];
    uint8_t head, tail, count;
} Queue;

```

```

Queue queue;

void init_queue(void) {
    queue.head = 0;
    queue.tail = 0;
    queue.count = 0;
}

uint8_t is_full(void) {
    return queue.count == QUEUE_SIZE;
}

uint8_t is_empty(void) {
    return queue.count == 0;
}

void enqueue(uint16_t value) {
    if (!is_full()) {
        queue.data[queue.head] = value;
        queue.head = (queue.head + 1) % QUEUE_SIZE;
        queue.count++;
    }
}

uint8_t dequeue(uint16_t* value) {
    if (!is_empty()) {
        *value = queue.data[queue.tail];
        queue.tail = (queue.tail + 1) % QUEUE_SIZE;
        queue.count--;
        return 1;
    }
    return 0;
}

int main(void) {
    init_queue();
    enqueue(100);
    uint16_t value;
    dequeue(&value);
    // Use value
    return 0;
}

```

Expert Tips

- **Thread Safety:** Add critical sections for interrupt access.
- **Optimization:** Use power-of-2 size for efficient modulo.
- **Testing:** Simulate rapid enqueue/dequeue.
- **Robustness:** Log overflow/underflow errors.
- **Extensibility:** Support dynamic queue sizes.

Project 48: Switch-Case for Protocol Command Handling

Description: Use a switch-case to handle protocol commands (e.g., read, write, status).

Requirements:

- Support 3 commands: READ (0x01), WRITE (0x02), STATUS (0x03).
- Assumptions: Commands received via UART.

Ways to Solve

Algorithm Used

- **Switch-Case:** Map command IDs to actions using a switch-case.
- **Command Processing:** Execute predefined functions for each command.

Complexity

- **Time Complexity:** $O(1)$ per command.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION handle_command(cmd_id, data)
    CASE cmd_id:
        0x01: read_command(data)
        0x02: write_command(data)
        0x03: status_command(data)
        DEFAULT: log_error()
MAIN
    WHILE true:
        cmd_id = read_uart()
        handle_command(cmd_id, data)
```

C Code Implementation

```
#include <stdint.h>

void read_command(uint8_t* data) { /* Read action */ }

void write_command(uint8_t* data) { /* Write action */ }

void status_command(uint8_t* data) { /* Status action */ }

void log_error(void) { /* Log error */ }

uint8_t read_uart(void) { return 0; } // Stub

void handle_command(uint8_t cmd_id, uint8_t* data) {
    switch (cmd_id) {
        case 0x01:
            read_command(data);
            break;
        case 0x02:
            write_command(data);
            break;
        case 0x03:
            status_command(data);
            break;
        default:
            log_error();
            break;
    }
}

int main(void) {
    uint8_t data[16];
    while (1) {
        uint8_t cmd_id = read_uart();
        handle_command(cmd_id, data);
    }
    return 0;
}
```

Expert Tips

- **Scalability:** Use function pointer table for many commands.
- **Testing:** Simulate all command types.
- **Robustness:** Handle invalid command IDs.
- **Debugging:** Log command execution.
- **Optimization:** Inline small command handlers.

Project 49: CRC16 Checksum for Data Packets

Description: Perform a CRC16 checksum for data packets.

Requirements:

- Use CRC-16-IBM (polynomial 0x8005).
- Input: Arbitrary byte array.
- Assumptions: Lookup table for efficiency.

Ways to Solve

Algorithm Used

- **CRC16 Calculation:** Iterate over bytes, updating CRC with XOR and shifts using a lookup table.
- **Table-Based:** Precompute CRC values for all 256 bytes.

Complexity

- **Time Complexity:** $O(n)$ for n bytes.
- **Space Complexity:** $O(1)$ (256-word table).

Pseudocode

```
CONST crc_table[256] = {precomputed CRC16 values}
FUNCTION calculate_crc16(data, length)
    crc = 0
    FOR each byte in data:
        index = (crc >> 8) ^ byte
        crc = (crc << 8) ^ crc_table[index]
    RETURN crc
MAIN
    data = get_data()
    crc = calculate_crc16(data, length)
```

C Code Implementation

```
#include <stdint.h>

const uint16_t crc_table[256] = {
    0x0000, 0xC0C1, 0xC181, 0x0140, /* Partial table */
    // Full table omitted for brevity
};

uint16_t calculate_crc16(uint8_t* data, uint8_t length) {
    uint16_t crc = 0;
    for (uint8_t i = 0; i < length; i++) {
        uint8_t index = (crc >> 8) ^ data[i];
        crc = (crc << 8) ^ crc_table[index];
    }
    return crc;
}
```

```

int main(void) {
    uint8_t data[] = {0x01, 0x02, 0x03};
    uint16_t crc = calculate_crc16(data, 3);
    // Use crc
    return 0;
}

```

Expert Tips

- **Storage:** Store `crc_table` in ROM with `const`.
- **Testing:** Verify with standard CRC16 test vectors.
- **Optimization:** Compute table at compile-time.
- **Robustness:** Handle zero-length input.
- **Extensibility:** Support other CRC polynomials.

Project 50: State Machine with Timeout

Description: Implement a state machine with timeout for a communication protocol.

Requirements:

- States: IDLE, WAITING, PROCESSING.
- Timeout: 100ms in WAITING state.
- Assumptions: 1ms timer interrupt.

Ways to Solve

Algorithm Used

- **State Machine:** Use an enum for states and a timer for timeouts.
- **Timeout Handling:** Reset to IDLE if timeout expires in WAITING.

Complexity

- **Time Complexity:** $O(1)$ per transition.
- **Space Complexity:** $O(1)$.

Pseudocode

```

ENUM State { IDLE, WAITING, PROCESSING }
GLOBAL state, timer
FUNCTION init_state_machine
    state = IDLE
    timer = 0
FUNCTION timer_interrupt
    timer++
FUNCTION state_machine(event)
    CASE state:
        IDLE:
            IF event == START:
                state = WAITING
                timer = 0
        WAITING:
            IF event == DATA_RECEIVED:
                state = PROCESSING
            ELSE IF timer >= 100:
                state = IDLE
        PROCESSING:
            IF event == DONE:
                state = IDLE
MAIN
    init_state_machine()

```



```
WHILE true:
    event = get_event()
    state_machine(event)
```

C Code Implementation

```
#include <stdint.h>

typedef enum { IDLE, WAITING, PROCESSING } State;

typedef enum { START, DATA_RECEIVED, DONE } Event;

State state = IDLE;

uint16_t timer = 0;

Event get_event(void) { return START; } // Stub

void init_state_machine(void) {
    state = IDLE;
    timer = 0;
}

void timer_isr(void) {
    timer++;
}

void state_machine(Event event) {
    switch (state) {
        case IDLE:
            if (event == START) {
                state = WAITING;
                timer = 0;
            }
            break;

        case WAITING:
            if (event == DATA_RECEIVED) {
                state = PROCESSING;
            } else if (timer >= 100) {
                state = IDLE;
            }
            break;
        case PROCESSING:
            if (event == DONE) state = IDLE;
            break;
    }
}

int main(void) {
    init_state_machine();
    while (1) {
        Event event = get_event();
        state_machine(event);
    }
    return 0;
}
```

Expert Tips

- **Robustness:** Handle unexpected events.
- **Testing:** Simulate timeouts and state transitions.
- **Debugging:** Log state changes and timeouts.
- **Optimization:** Use a single timer for multiple states.

- **Extensibility:** Add more states or timeouts.

Project 51: Static Array for Peripheral Configuration

Description: Use a static array to manage peripheral configurations (e.g., for 3 UARTs).

Requirements:

- Configure 3 UARTs (baud rate, parity).
- Assumptions: Fixed configurations.

Ways to Solve

Algorithm Used

- **Static Array:** Store configurations in a `const` array of structs.
- **Initialization:** Iterate over array to configure peripherals.

Complexity

- **Time Complexity:** $O(n)$ for n peripherals.
- **Space Complexity:** $O(n)$ for configuration array.

Pseudocode

```
STRUCT UartConfig { baud, parity }

const UartConfig configs[3]
FUNCTION init_uarts
    FOR each config in configs:
        set_uart_baud(config.baud)
        set_uart_parity(config.parity)

MAIN
    configs = [{9600, 0}, {115200, 1}, {38400, 0}]
    init_uarts()
```

C Code Implementation

```
#include <stdint.h>

#define NUM_UARTS 3

typedef struct {
    uint32_t baud;
    uint8_t parity;
} UartConfig;

const UartConfig configs[NUM_UARTS] = {
    {9600, 0},
    {115200, 1},
    {38400, 0}
};

void set_uart_baud(uint8_t id, uint32_t baud) { /* UART config */ }

void set_uart_parity(uint8_t id, uint8_t parity) { /* UART config */ }

void init_uarts(void) {
    for (uint8_t i = 0; i < NUM_UARTS; i++) {
        set_uart_baud(i, configs[i].baud);
        set_uart_parity(i, configs[i].parity);
    }
}
```

```
int main(void) {
    init_uarts();
    return 0;
}
```

Expert Tips

- **Storage:** Use `const` to save RAM.
- **Validation:** Check configuration values.
- **Testing:** Verify with hardware debugger.
- **Extensibility:** Support dynamic configurations.
- **Portability:** Use HAL for UART access.

Project 52: Multi-Threaded Simulation Using Cooperative Tasks

Description: Handle a multi-threaded simulation using cooperative tasks (e.g., 3 tasks).

Requirements:

- Tasks: sensor read, data process, LED toggle.
- Run cooperatively in a main loop.
- Assumptions: Non-preemptive scheduling.

Ways to Solve

Algorithm Used

- **Task Struct:** Define tasks with function pointers and periods.
- **Cooperative Scheduling:** Execute tasks in a loop based on their periods.

Complexity

- **Time Complexity:** $O(n)$ per loop for n tasks.
- **Space Complexity:** $O(n)$ for task array.

Pseudocode

```
STRUCT Task { fn, period, counter }
GLOBAL Task tasks[3]
FUNCTION init_tasks
    tasks = [{sensor_read, 10}, {data_process, 20}, {led_toggle, 50}]
FUNCTION schedule
    FOR each task:
        IF task.counter >= task.period:
            task.fn()
            task.counter = 0
            task.counter++
MAIN
    init_tasks()
    WHILE true:
        schedule()
```

C Code Implementation

```
#include <stdint.h>

#define NUM_TASKS 3

typedef void (*TaskFn)(void);
```

```

typedef struct {
    TaskFn fn;
    uint16_t period;
    uint16_t counter;
} Task;

void sensor_read(void) { /* Read sensor */ }

void data_process(void) { /* Process data */ }

void led_toggle(void) { /* Toggle LED */ }

Task tasks[NUM_TASKS] = {
    {sensor_read, 10, 0},
    {data_process, 20, 0},
    {led_toggle, 50, 0}
};

void init_tasks(void) {
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        tasks[i].counter = 0;
    }
}

void schedule(void) {
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        if (tasks[i].counter >= tasks[i].period) {
            tasks[i].fn();
            tasks[i].counter = 0;
        }
        tasks[i].counter++;
    }
}

int main(void) {
    init_tasks();
    while (1) {
        schedule();
    }
    return 0;
}

```

Expert Tips

- **Timing:** Use a timer interrupt for accurate scheduling.
- **Testing:** Simulate task execution with logging.
- **Optimization:** Prioritize tasks if needed.
- **Robustness:** Handle task overruns.
- **Extensibility:** Support dynamic task addition.

Project 53: Basic Data Serializer for UART

Description: Implement a basic data serializer for UART (e.g., convert struct to byte stream).

Requirements:

- Serialize a struct with uint16_t and uint8_t fields.
- Assumptions: UART output, fixed struct format.

Ways to Solve

Algorithm Used

- **Serialization:** Copy struct fields to a byte array in defined order.
- **Transmission:** Send byte array via UART.

Complexity

- **Time Complexity:** $O(n)$ for n -byte struct.
- **Space Complexity:** $O(n)$ for output buffer.

Pseudocode

```
STRUCT Data { id, value }
FUNCTION serialize(data, buffer)
    buffer[0] = data.id >> 8
    buffer[1] = data.id & 0xFF
    buffer[2] = data.value
    RETURN buffer
MAIN
    data = {id: 123, value: 45}
    buffer = serialize(data, buffer)
    send_uart(buffer, 3)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t id;
    uint8_t value;
} Data;

void send_uart(uint8_t* buffer, uint8_t len) { /* UART send */ }

void serialize(Data* data, uint8_t* buffer) {
    buffer[0] = data->id >> 8;
    buffer[1] = data->id & 0xFF;
    buffer[2] = data->value;
}

int main(void) {
    Data data = {123, 45};
    uint8_t buffer[3];
    serialize(&data, buffer);
    send_uart(buffer, 3);
    return 0;
}
```

Expert Tips

- **Endianness:** Define byte order explicitly.
- **Testing:** Verify deserialization on receiver.
- **Robustness:** Add checksum to serialized data.
- **Optimization:** Minimize buffer copies.
- **Extensibility:** Support variable-length structs.

Project 54: Pointer to Struct for Peripheral Control

Description: Use a pointer to a struct to manage peripheral control (e.g., I2C settings).

Requirements:

- Struct for I2C: address, speed, mode.
- Use pointer to configure peripheral.
- Assumptions: Single I2C peripheral.

Ways to Solve

Algorithm Used

- **Struct Pointer:** Define a struct for settings and use a pointer to access it.
- **Configuration:** Apply settings to hardware via pointer.

Complexity

- **Time Complexity:** $O(1)$ per configuration.
- **Space Complexity:** $O(1)$.

Pseudocode

```
STRUCT I2cConfig { address, speed, mode }
FUNCTION configure_i2c(config_ptr)
    set_i2c_address(config_ptr->address)
    set_i2c_speed(config_ptr->speed)
    set_i2c_mode(config_ptr->mode)
MAIN
    config = {address: 0x50, speed: 100000, mode: 0}
    configure_i2c(&config)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint8_t address;
    uint32_t speed;
    uint8_t mode;
} I2cConfig;

void set_i2c_address(uint8_t addr) { /* I2C config */ }

void set_i2c_speed(uint32_t speed) { /* I2C config */ }

void set_i2c_mode(uint8_t mode) { /* I2C config */ }

void configure_i2c(I2cConfig* config) {
    set_i2c_address(config->address);
    set_i2c_speed(config->speed);
    set_i2c_mode(config->mode);
}

int main(void) {
    I2cConfig config = {0x50, 100000, 0};
    configure_i2c(&config);
    return 0;
}
```

Expert Tips

- **Validation:** Check config values for validity.

- **Testing:** Verify with hardware debugger.
- **Storage:** Use `const` for fixed configs.
- **Extensibility:** Support multiple peripherals.
- **Robustness:** Handle NULL pointers.

Project 55: 64-Bit Integer Arithmetic with Overflow Checks

Description: Handle 64-bit integer arithmetic with overflow checks (e.g., addition).

Requirements:

- Support `int64_t` addition with overflow detection.
- Assumptions: Standard C library available.

Ways to Solve

Algorithm Used

- **Addition:** Check for overflow using conditional checks.
- **Validation:** Return error if overflow occurs.

Complexity

- **Time Complexity:** $O(1)$ per operation.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION add_int64(a, b, result)
    IF a > 0 and b > INT64_MAX - a:
        RETURN overflow
    IF a < 0 and b < INT64_MIN - a:
        RETURN overflow
    *result = a + b
    RETURN success
MAIN
    result = add_int64(a, b, &sum)
```

C Code Implementation

```
#include <stdint.h>

#define SUCCESS 0
#define OVERFLOW 1

uint8_t add_int64(int64_t a, int64_t b, int64_t* result) {
    if (a > 0 && b > INT64_MAX - a) return OVERFLOW;
    if (a < 0 && b < INT64_MIN - a) return OVERFLOW;
    *result = a + b;
    return SUCCESS;
}

int main(void) {
    int64_t a = 1000, b = 2000, result;
    if (add_int64(a, b, &result) == SUCCESS) {
        // Use result
    }
    return 0;
}
```

Expert Tips

- **Testing:** Verify with `INT64_MAX/MIN`.

- **Optimization:** Use compiler intrinsics if available.
- **Robustness:** Handle edge cases explicitly.
- **Debugging:** Log overflow events.
- **Extensibility:** Support other operations (e.g., multiplication).

Project 56: Simple Command Parser with Arguments

Description: Implement a simple command parser with arguments (e.g., "set led 1").

Requirements:

- Parse commands: "set led " (value: 0 or 1).
- Assumptions: UART input, max command length 16 bytes.

Ways to Solve

Algorithm Used

- **Tokenization:** Split input string into command and arguments.
- **Parsing:** Match command and extract argument using string functions.

Complexity

- **Time Complexity:** $O(n)$ for n-character input.
- **Space Complexity:** $O(n)$ for buffer.

Pseudocode

```
FUNCTION parse_command(input)
    tokens = split(input, " ")
    IF tokens[0] == "set" AND tokens[1] == "led":
        value = atoi(tokens[2])
        set_led(value)

MAIN
    WHILE true:
        input = read_uart()
        parse_command(input)
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

void set_led(uint8_t value) { /* Set LED */ }

char* read_uart(void) { return ""; } // Stub

void parse_command(const char* input) {
    char buffer[16];
    strcpy(buffer, input);
    char* tokens[3];
    uint8_t i = 0;
    tokens[i] = strtok(buffer, " ");
    while (tokens[i] && i < 3) tokens[++i] = strtok(NULL, " ");
    if (i >= 3 && strcmp(tokens[0], "set") == 0 && strcmp(tokens[1], "led") == 0) {
        uint8_t value = atoi(tokens[2]);
        if (value <= 1) set_led(value);
    }
}
```



```

int main(void) {
    while (1) {
        char* input = read_uart();
        parse_command(input);
    }
    return 0;
}

```

Expert Tips

- **Robustness:** Handle invalid arguments and syntax.
- **Testing:** Simulate various command inputs.
- **Optimization:** Use static buffers to avoid allocation.
- **Extensibility:** Support more commands/arguments.
- **Debugging:** Log parsed commands.

Project 57: Union for Memory-Efficient Data Storage

Description: Use a union for memory-efficient storage of different data types (e.g., uint16_t, float).

Requirements:

- Store either a uint16_t or float in the same memory.
- Assumptions: Used for sensor data.

Ways to Solve

Algorithm Used

- **Union:** Define a union with uint16_t and float members.
- **Access Control:** Use a type flag to track active member.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$ (size of largest member).

Pseudocode

```

UNION Data { u16, f }
STRUCT Storage { type, data }
FUNCTION store_data(value, type, storage)
    storage.type = type
    IF type == UINT16:
        storage.data.u16 = value
    ELSE:
        storage.data.f = value
MAIN
    storage = store_data(123, UINT16)

```

C Code Implementation

```

#include <stdint.h>

typedef enum { UINT16, FLOAT } DataType;

typedef union {
    uint16_t u16;
    float f;
} Data;

```

```

typedef struct {
    DataType type;
    Data data;
} Storage;

void store_data(void* value, DataType type, Storage* storage) {
    storage->type = type;
    if (type == UINT16) storage->data.u16 = *(uint16_t*)value;
    else storage->data.f = *(float*)value;
}

int main(void) {
    Storage storage;
    uint16_t u16 = 123;
    float f = 1.5f;
    store_data(&u16, UINT16, &storage);
    store_data(&f, FLOAT, &storage);
    // Use storage
    return 0;
}

```

Expert Tips

- **Safety:** Use type flag to avoid incorrect access.
- **Testing:** Verify all member accesses.
- **Alignment:** Ensure proper alignment for all types.
- **Debugging:** Log type and value changes.
- **Extensibility:** Add more types to union.

Project 58: Linear Interpolation on Sensor Data

Description: Perform linear interpolation on sensor data to estimate values between samples.

Requirements:

- Interpolate between two uint16_t sensor samples given a fractional position (0.0 to 1.0).
- Assumptions: Input data is valid, float available.

Ways to Solve

Algorithm Used

- **Linear Interpolation:** Use formula: $y = y_1 + (x - x_1) * (y_2 - y_1) / (x_2 - x_1)$.
- **Simplified:** For two points at $x=0$ and $x=1$, $y = y_1 + t * (y_2 - y_1)$, where t is the fractional position.

Complexity

- **Time Complexity:** $O(1)$ per interpolation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION interpolate(y1, y2, t)
    RETURN y1 + t * (y2 - y1)
MAIN
    y1 = read_sensor(0)
    y2 = read_sensor(1)
    t = get_fraction()
    result = interpolate(y1, y2, t)

```

C Code Implementation

```
#include <stdint.h>
float interpolate(uint16_t y1, uint16_t y2, float t) {
    return y1 + t * (y2 - y1);
}

uint16_t read_sensor(uint8_t index) { return 0; } // Stub
float get_fraction(void) { return 0.5f; } // Stub
int main(void) {
    uint16_t y1 = read_sensor(0);
    uint16_t y2 = read_sensor(1);
    float t = get_fraction();
    float result = interpolate(y1, y2, t);
    // Use result
    return 0;
}
```

Expert Tips

- **Validation:** Ensure $0.0 \leq t \leq 1.0$.
- **Testing:** Verify with known data points.
- **Optimization:** Use fixed-point for systems without FPU.
- **Robustness:** Handle edge cases ($y1 == y2$).
- **Extensibility:** Support multi-point interpolation.

Project 59: Basic Logging System with Severity Levels

Description: Implement a basic logging system with severity levels (e.g., INFO, WARNING, ERROR).

Requirements:

- Support 3 severity levels: INFO, WARNING, ERROR.
- Log messages with timestamp to UART.
- Assumptions: 32-bit timestamp (ms), UART output.

Ways to Solve

Algorithm Used

- **Log Struct:** Store severity, timestamp, and message.
- **Logging:** Format and send log entry via UART.
- **Severity Filter:** Optionally filter logs by level.

Complexity

- **Time Complexity:** $O(n)$ for n-character message.
- **Space Complexity:** $O(n)$ for message buffer.

Pseudocode

```
ENUM Severity { INFO, WARNING, ERROR }
FUNCTION log_message(severity, message)
    timestamp = get_timestamp()
    format = "[severity][timestamp]: message"
    send_uart(format)
MAIN
    log_message(WARNING, "Sensor error")
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

typedef enum { INFO, WARNING, ERROR } Severity;

const char* severity_str[] = {"INFO", "WARNING", "ERROR"};

void send_uart(const char* msg) { /* UART send */ }

uint32_t get_timestamp(void) { return 0; } // Stub

void log_message(Severity severity, const char* message) {
    char buffer[64];
    uint32_t timestamp = get_timestamp();
    snprintf(buffer, sizeof(buffer), "[%s][%u]: %s\n", severity_str[severity], timestamp, message);
    send_uart(buffer);
}

int main(void) {
    log_message(WARNING, "Sensor error");
    return 0;
}
```

Expert Tips

- **Storage:** Buffer logs for non-volatile storage.
- **Testing:** Simulate all severity levels.
- **Optimization:** Use static buffers to avoid allocation.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add log filtering by severity.

Project 60: Macro for Interrupt-Safe GPIO Access

Description: Use a macro to ensure interrupt-safe GPIO access (e.g., set/clear pins).

Requirements:

- Define macros to set/clear GPIO pins atomically.
- Assumptions: 32-bit GPIO register, interrupts disabled during access.

Ways to Solve

Algorithm Used

- **Macros with Critical Sections:** Disable interrupts, update GPIO, re-enable interrupts.
- **Bit Manipulation:** Use bitwise operations for pin control.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$.

Pseudocode

```
DEFINE MACRO SET_GPIO(pin) \
    disable_interrupts() \
    GPIO_REG |= (1 << pin) \
    enable_interrupts()
DEFINE MACRO CLEAR_GPIO(pin) \
    disable_interrupts() \
```

```

    GPIO_REG &= ~(1 << pin) \
    enable_interrupts()
MAIN
    SET_GPIO(0)
    CLEAR_GPIO(1)

```

C Code Implementation

```

#include <stdint.h>

#define GPIO_REG (*(volatile uint32_t*)0x40007000) // Example address

#define SET_GPIO(pin) do { \
    disable_interrupts(); \
    GPIO_REG |= (1 << (pin)); \
    enable_interrupts(); \
} while (0)

#define CLEAR_GPIO(pin) do { \
    disable_interrupts(); \
    GPIO_REG &= ~(1 << (pin)); \
    enable_interrupts(); \
} while (0)

void disable_interrupts(void) { /* Platform-specific */ }

void enable_interrupts(void) { /* Platform-specific */ }

int main(void) {
    SET_GPIO(0);
    CLEAR_GPIO(1);
    return 0;
}

```

Expert Tips

- **Safety:** Minimize critical section duration.
- **Testing:** Verify atomicity with interrupts.
- **Optimization:** Use bit-banding if available (e.g., ARM).
- **Portability:** Abstract with HAL.
- **Debugging:** Log GPIO changes.

Project 61: Priority Queue for Event Handling

Description: Implement a priority queue for event handling (e.g., high/low priority events).

Requirements:

- Queue size: 8 events (event ID, priority).
- Higher priority (lower number) dequeued first.
- Assumptions: Single-threaded access.

Ways to Solve

Algorithm Used

- **Heap-Based Priority Queue:** Use a min-heap to store events, ensuring highest priority is dequeued first.
- **Heap Operations:** Insert with up-heap, extract with down-heap.

Complexity

- **Time Complexity:** $O(\log n)$ for insert/dequeue, $O(1)$ for peek.
- **Space Complexity:** $O(n)$ for queue ($n=8$).

Pseudocode

```
STRUCT Event { id, priority }
GLOBAL Event queue[8], size
FUNCTION insert(event)
    IF size < 8:
        queue[size] = event
        up_heap(size)
        size++
FUNCTION dequeue(event)
    IF size > 0:
        *event = queue[0]
        queue[0] = queue[size-1]
        size--
        down_heap(0)
        RETURN true
    RETURN false
MAIN
    insert({id: 1, priority: 2})
    dequeue(&event)
```

C Code Implementation

```
#include <stdint.h>

#define QUEUE_SIZE 8

typedef struct {
    uint8_t id;
    uint8_t priority;
} Event;

Event queue[QUEUE_SIZE];

uint8_t size = 0;

void swap(Event* a, Event* b) {
    Event temp = *a;
    *a = *b;
    *b = temp;
}

void up_heap(uint8_t index) {
    while (index > 0) {
        uint8_t parent = (index - 1) / 2;
        if (queue[index].priority >= queue[parent].priority) break;
        swap(&queue[index], &queue[parent]);
        index = parent;
    }
}

void down_heap(uint8_t index) {
    while (index < size) {
        uint8_t left = 2 * index + 1;
        uint8_t right = 2 * index + 2;
        uint8_t smallest = index;
        if (left < size && queue[left].priority < queue[smallest].priority) smallest = left;
        if (right < size && queue[right].priority < queue[smallest].priority) smallest = right;
        if (smallest == index) break;
        swap(&queue[index], &queue[smallest]);
        index = smallest;
    }
}
```

```

void insert(Event event) {
    if (size < QUEUE_SIZE) {
        queue[size] = event;
        up_heap(size);
        size++;
    }
}

uint8_t dequeue(Event* event) {
    if (size == 0) return 0;
    *event = queue[0];
    queue[0] = queue[size - 1];
    size--;
    down_heap(0);
    return 1;
}

int main(void) {
    insert((Event){1, 2});
    Event event;
    dequeue(&event);
    // Use event
    return 0;
}

```

Expert Tips

- **Thread Safety:** Add critical sections for interrupt access.
- **Testing:** Simulate high/low priority events.
- **Optimization:** Use fixed-size heap for efficiency.
- **Robustness:** Handle queue overflow.
- **Extensibility:** Support dynamic priorities.

Project 62: Simple Data Encryption Routine

Description: Implement a simple data encryption routine (e.g., XOR-based) for a data buffer.

Requirements:

- Encrypt/decrypt a uint8_t buffer with a 4-byte key.
- Assumptions: XOR encryption, fixed key.

Ways to Solve

Algorithm Used

- **XOR Encryption:** XOR each byte with a repeating key.
- **Reversible:** Same function for encryption/decryption.

Complexity

- **Time Complexity:** $O(n)$ for n-byte buffer.
- **Space Complexity:** $O(n)$ for output buffer.

Pseudocode

```

FUNCTION encrypt(buffer, length, key)
    FOR i = 0 to length-1:
        buffer[i] ^= key[i % 4]
MAIN
    data = get_data()
    encrypt(data, length, key)

```

C Code Implementation

```
#include <stdint.h>

void encrypt(uint8_t* buffer, uint8_t length, const uint8_t* key) {
    for (uint8_t i = 0; i < length; i++) {
        buffer[i] ^= key[i % 4];
    }
}

int main(void) {
    uint8_t data[] = {1, 2, 3, 4, 5};
    const uint8_t key[] = {0xAA, 0xBB, 0xCC, 0xDD};
    encrypt(data, 5, key);
    // Use encrypted data
    return 0;
}
```

Expert Tips

- **Security:** Use stronger encryption (e.g., AES) for production.
- **Testing:** Verify encrypt/decrypt reversibility.
- **Optimization:** Unroll loop for small buffers.
- **Robustness:** Validate buffer and key lengths.
- **Extensibility:** Support variable key lengths.

Project 63: Volatile Struct for Interrupt-Driven Data

Description: Use a volatile struct for interrupt-driven data (e.g., sensor readings).

Requirements:

- Struct with uint16_t value and uint8_t status.
- Updated by interrupt, read by main loop.
- Assumptions: Atomic access for uint16_t.

Ways to Solve

Algorithm Used

- **Volatile Struct:** Declare struct as `volatile` to ensure compiler doesn't optimize access.
- **Atomic Access:** Use single reads for atomicity.

Complexity

- **Time Complexity:** O(1) per access.
- **Space Complexity:** O(1).

Pseudocode

```
STRUCT Sensor { value, status }
GLOBAL volatile Sensor sensor
FUNCTION interrupt_handler
    sensor.value = read_sensor()
    sensor.status = get_status()
FUNCTION read_sensor_data
    local = sensor
    RETURN local
MAIN
    WHILE true:
        data = read_sensor_data()
```


C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t value;
    uint8_t status;
} Sensor;

volatile Sensor sensor;

uint16_t read_sensor(void) { return 0; } // Stub

uint8_t get_status(void) { return 0; } // Stub

void interrupt_handler(void) {
    sensor.value = read_sensor();
    sensor.status = get_status();
}

Sensor read_sensor_data(void) {
    return sensor; // Atomic for aligned struct
}

int main(void) {
    while (1) {
        Sensor data = read_sensor_data();
        // Use data
    }
    return 0;
}
```

Expert Tips

- **Atomicity:** Ensure struct alignment for atomic access.
- **Testing:** Simulate interrupt updates.
- **Safety:** Use critical sections for non-atomic structs.
- **Optimization:** Minimize interrupt latency.
- **Debugging:** Log data changes.

Project 64: Checksum Validation for Flash Data

Description: Perform checksum validation for flash data to ensure integrity.

Requirements:

- Compute and verify 16-bit sum for a 64-byte block.
- Assumptions: Flash read function available.

Ways to Solve

Algorithm Used

- **Checksum Calculation:** Sum all bytes modulo 65536.
- **Validation:** Compare computed checksum with stored value.

Complexity

- **Time Complexity:** $O(n)$ for n -byte block.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION validate_checksum(data, length, stored_checksum)
    sum = 0
    FOR each byte in data:
        sum += byte
    RETURN sum == stored_checksum
MAIN
    data = read_flash()
    stored_checksum = read_flash_checksum()
    valid = validate_checksum(data, 64, stored_checksum)
```

C Code Implementation

```
#include <stdint.h>

uint8_t read_flash(uint8_t index) { return 0; } // Stub

uint16_t read_flash_checksum(void) { return 0; } // Stub

uint8_t validate_checksum(uint8_t* data, uint8_t length, uint16_t stored_checksum) {
    uint16_t sum = 0;
    for (uint8_t i = 0; i < length; i++) {
        sum += data[i];
    }
    return sum == stored_checksum;
}

int main(void) {
    uint8_t data[64];
    for (uint8_t i = 0; i < 64; i++) data[i] = read_flash(i);
    uint16_t stored_checksum = read_flash_checksum();
    uint8_t valid = validate_checksum(data, 64, stored_checksum);
    // Use valid
    return 0;
}
```

Expert Tips

- **Robustness:** Handle read errors from flash.
- **Testing:** Simulate corrupted data.
- **Optimization:** Use hardware CRC if available.
- **Storage:** Store checksum in reliable flash region.
- **Extensibility:** Support different checksum algorithms.

Project 65: Event-Driven State Machine

Description: Implement a basic event-driven state machine for a system (e.g., button press events).

Requirements:

- States: OFF, ON; Events: BUTTON_PRESS, TIMEOUT.
- Assumptions: 100ms timeout in ON state, 1ms timer interrupt.

Ways to Solve

Algorithm Used

- **State Machine:** Use enum for states and events, transition based on events.
- **Timeout:** Track time in ON state using a counter.

Complexity

- **Time Complexity:** $O(1)$ per transition.
- **Space Complexity:** $O(1)$.

Pseudocode

```
ENUM State { OFF, ON }
ENUM Event { BUTTON_PRESS, TIMEOUT }
GLOBAL state, timer
FUNCTION timer_interrupt
    timer++
FUNCTION state_machine(event)
    CASE state:
        OFF:
            IF event == BUTTON_PRESS:
                state = ON
                timer = 0
        ON:
            IF event == BUTTON_PRESS:
                state = OFF
            ELSE IF timer >= 100:
                state = OFF
MAIN
    state = OFF
    WHILE true:
        event = get_event()
        state_machine(event)
```

C Code Implementation

```
#include <stdint.h>

typedef enum { OFF, ON } State;

typedef enum { BUTTON_PRESS, TIMEOUT } Event;

State state = OFF;
uint16_t timer = 0;

Event get_event(void) { return BUTTON_PRESS; } // Stub

void timer_isr(void) {
    timer++;
}

void state_machine(Event event) {
    switch (state) {
        case OFF:
            if (event == BUTTON_PRESS) {
                state = ON;
                timer = 0;
            }
            break;
        case ON:
            if (event == BUTTON_PRESS || timer >= 100) {
                state = OFF;
            }
            break;
    }
}

int main(void) {
    while (1) {
        Event event = get_event();
        state_machine(event);
    }
}
```

```
}  
    return 0;  
}
```

Expert Tips

- **Robustness:** Handle unexpected events.
- **Testing:** Simulate all transitions.
- **Debugging:** Log state changes.
- **Optimization:** Use table-driven FSM for scalability.
- **Extensibility:** Add more states/events.

Project 66: Function Pointer for Dynamic Callbacks

Description: Use a function pointer for dynamic callbacks (e.g., sensor data handlers).

Requirements:

- Support two callback functions for sensor data processing.
- Assumptions: Callbacks set dynamically.

Ways to Solve

Algorithm Used

- **Function Pointer:** Store callback in a variable or struct.
- **Dynamic Assignment:** Set callback based on condition.

Complexity

- **Time Complexity:** $O(1)$ per callback invocation.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION process_data(data)  
    callback = get_callback()  
    IF callback:  
        callback(data)  
MAIN  
    set_callback(handler1)  
    data = read_sensor()  
    process_data(data)
```

C Code Implementation

```
#include <stdint.h>  
  
typedef void (*Callback)(uint16_t);  
  
void handler1(uint16_t data) { /* Handler 1 */ }  
void handler2(uint16_t data) { /* Handler 2 */ }  
  
Callback callback = NULL;  
  
uint16_t read_sensor(void) { return 0; } // Stub  
  
void set_callback(Callback cb) {  
    callback = cb;  
}
```

```

void process_data(uint16_t data) {
    if (callback) callback(data);
}

int main(void) {
    set_callback(handler1);
    uint16_t data = read_sensor();
    process_data(data);
    return 0;
}

```

Expert Tips

- **Safety:** Check for NULL callbacks.
- **Testing:** Verify callback execution.
- **Optimization:** Inline small callbacks.
- **Extensibility:** Support multiple callbacks.
- **Debugging:** Log callback invocations.

Project 67: Circular Buffer with Overflow Detection

Description: Implement a circular buffer with overflow detection for data storage.

Requirements:

- Buffer size: 16 uint16_t elements.
- Detect and handle overflow.
- Assumptions: Interrupt-driven writes.

Ways to Solve

Algorithm Used

- **Circular Buffer:** Use head/tail pointers and count for state.
- **Overflow Detection:** Check if buffer is full before writing.

Complexity

- **Time Complexity:** $O(1)$ for push/pop.
- **Space Complexity:** $O(n)$ for buffer ($n=16$).

Pseudocode

```

STRUCT Buffer { data[16], head, tail, count }
GLOBAL Buffer buffer
FUNCTION init_buffer
    head = 0, tail = 0, count = 0
FUNCTION push(value)
    IF count == 16:
        RETURN overflow
    data[head] = value
    head = (head + 1) % 16
    count++
    RETURN success
FUNCTION pop(value)
    IF count == 0:
        RETURN underflow
    *value = data[tail]
    tail = (tail + 1) % 16
    count--
    RETURN success
MAIN
    init_buffer()

```

```
push(data)
pop(&value)
```

C Code Implementation

```
#include <stdint.h>

#define BUFFER_SIZE 16
#define SUCCESS 0
#define OVERFLOW 1
#define UNDERFLOW 2

typedef struct {
    uint16_t data[BUFFER_SIZE];
    uint8_t head, tail, count;
} Buffer;

Buffer buffer;

void init_buffer(void) {
    buffer.head = 0;
    buffer.tail = 0;
    buffer.count = 0;
}

uint8_t push(uint16_t value) {
    if (buffer.count == BUFFER_SIZE) return OVERFLOW;
    buffer.data[buffer.head] = value;
    buffer.head = (buffer.head + 1) % BUFFER_SIZE;
    buffer.count++;
    return SUCCESS;
}

uint8_t pop(uint16_t* value) {
    if (buffer.count == 0) return UNDERFLOW;
    *value = buffer.data[buffer.tail];
    buffer.tail = (buffer.tail + 1) % BUFFER_SIZE;
    buffer.count--;
    return SUCCESS;
}

int main(void) {
    init_buffer();
    push(100);
    uint16_t value;
    pop(&value);
    // Use value
    return 0;
}
```

Expert Tips

- **Thread Safety:** Use critical sections for interrupt access.
- **Testing:** Simulate overflow/underflow.
- **Optimization:** Use power-of-2 size for modulo.
- **Robustness:** Log overflow errors.
- **Extensibility:** Support dynamic buffer sizes.

Project 68: Simple Protocol Decoder for Serial Data

Description: Implement a protocol decoder for serial data (e.g., header, payload, footer).

Requirements:

- Format: header (0xBB), 8-byte payload, footer (0xCC).
- Assumptions: UART input, fixed payload size.

Ways to Solve

Algorithm Used

- **State Machine:** Parse bytes in states: header, payload, footer.
- **Validation:** Check header/footer, process payload if valid.

Complexity

- **Time Complexity:** $O(n)$ for n -byte packet.
- **Space Complexity:** $O(n)$ for payload.

Pseudocode

```
ENUM State { HEADER, PAYLOAD, FOOTER }
GLOBAL buffer[8], index, state
FUNCTION decode_packet(byte)
    CASE state:
        HEADER:
            IF byte == 0xBB:
                state = PAYLOAD
                index = 0
        PAYLOAD:
            IF index < 8:
                buffer[index++] = byte
            IF index == 8:
                state = FOOTER
        FOOTER:
            IF byte == 0xCC:
                process_payload(buffer, 8)
                state = HEADER
MAIN
    WHILE true:
        byte = read_uart()
        decode_packet(byte)
```

C Code Implementation

```
#include <stdint.h>

#define PAYLOAD_SIZE 8

typedef enum { HEADER, PAYLOAD, FOOTER } State;

uint8_t buffer[PAYLOAD_SIZE], index = 0;

State state = HEADER;

uint8_t read_uart(void) { return 0; } // Stub

void process_payload(uint8_t* data, uint8_t len) { /* Process payload */ }
```

```

void decode_packet(uint8_t byte) {
    switch (state) {
        case HEADER:
            if (byte == 0xBB) {
                state = PAYLOAD;
                index = 0;
            }
            break;
        case PAYLOAD:
            if (index < PAYLOAD_SIZE) {
                buffer[index++] = byte;
            }
            if (index == PAYLOAD_SIZE) state = FOOTER;
            break;
        case FOOTER:
            if (byte == 0xCC) process_payload(buffer, PAYLOAD_SIZE);
            state = HEADER;
            break;
    }
}

int main(void) {
    while (1) {
        uint8_t byte = read_uart();
        decode_packet(byte);
    }
    return 0;
}

```

Expert Tips

- **Robustness:** Handle malformed packets.
- **Testing:** Simulate invalid headers/footers.
- **Optimization:** Use circular buffer for input.
- **Debugging:** Log packet errors.
- **Extensibility:** Support variable payload sizes.

Project 69: Bitfield Struct for Memory-Efficient Storage

Description: Use a bitfield struct for memory-efficient storage (e.g., sensor flags).

Requirements:

- Store 4 boolean flags and a 4-bit value in a single byte.
- Assumptions: Used for configuration.

Ways to Solve

Algorithm Used

- **Bitfield Struct:** Define a struct with bitfields for flags and value.
- **Access:** Use struct members for readable access.

Complexity

- **Time Complexity:** O(1) per access.
- **Space Complexity:** O(1) (1 byte).

Pseudocode

```

STRUCT Config { flag1:1, flag2:1, flag3:1, flag4:1, value:4 }
GLOBAL Config config

```



```

FUNCTION set_config(f1, f2, f3, f4, val)
    config.flag1 = f1
    config.flag2 = f2
    config.flag3 = f3
    config.flag4 = f4
    config.value = val
MAIN
    set_config(1, 0, 1, 0, 5)

```

C Code Implementation

```

#include <stdint.h>

typedef struct {
    uint8_t flag1 : 1;
    uint8_t flag2 : 1;
    uint8_t flag3 : 1;
    uint8_t flag4 : 1;
    uint8_t value : 4;
} Config;

Config config;

void set_config(uint8_t f1, uint8_t f2, uint8_t f3, uint8_t f4, uint8_t val) {
    config.flag1 = f1;
    config.flag2 = f2;
    config.flag3 = f3;
    config.flag4 = f4;
    config.value = val & 0x0F;
}

int main(void) {
    set_config(1, 0, 1, 0, 5);
    // Use config
    return 0;
}

```

Expert Tips

- **Alignment:** Ensure bitfield fits in target size.
- **Testing:** Verify bitfield access with debugger.
- **Portability:** Avoid relying on bitfield ordering.
- **Robustness:** Validate input values.
- **Extensibility:** Add more fields if needed.

Project 70: Quicksort on Sensor Readings

Description: Perform a quicksort on an array of sensor readings (uint16_t).

Requirements:

- Sort an array of 10 sensor readings in ascending order.
- Assumptions: In-place sorting.

Ways to Solve

Algorithm Used

- **Quicksort:** Choose a pivot, partition array, recursively sort subarrays.
- **Optimization:** Use median-of-three pivot for better performance.

Complexity

- **Time Complexity:** $O(n \log n)$ average, $O(n^2)$ worst case.
- **Space Complexity:** $O(\log n)$ for recursion stack.

Pseudocode

```
FUNCTION quicksort(array, low, high)
    IF low < high:
        pivot_index = partition(array, low, high)
        quicksort(array, low, pivot_index - 1)
        quicksort(array, pivot_index + 1, high)
FUNCTION partition(array, low, high)
    pivot = array[high]
    i = low - 1
    FOR j = low to high-1:
        IF array[j] <= pivot:
            i++
            swap(array[i], array[j])
    swap(array[i+1], array[high])
    RETURN i+1
MAIN
    array = read_sensors()
    quicksort(array, 0, 9)
```

C Code Implementation

```
#include <stdint.h>

void swap(uint16_t* a, uint16_t* b) {
    uint16_t temp = *a;
    *a = *b;
    *b = temp;
}

int partition(uint16_t* array, int low, int high) {
    uint16_t pivot = array[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (array[j] <= pivot) {
            i++;
            swap(&array[i], &array[j]);
        }
    }
    swap(&array[i + 1], &array[high]);
    return i + 1;
}

void quicksort(uint16_t* array, int low, int high) {
    if (low < high) {
        int pi = partition(array, low, high);
        quicksort(array, low, pi - 1);
        quicksort(array, pi + 1, high);
    }
}

int main(void) {
    uint16_t array[] = {5, 2, 8, 1, 9, 3, 7, 4, 6, 10};
    quicksort(array, 0, 9);
    // Use sorted array
    return 0;
}
```

Expert Tips

- **Optimization:** Use insertion sort for small subarrays.

- **Testing:** Verify with random and edge-case arrays.
- **Robustness:** Handle empty or single-element arrays.
- **Stack Usage:** Use iterative quicksort for low memory.
- **Performance:** Benchmark against other sorts.

Project 71: Basic Timeout Mechanism for Tasks

Description: Implement a basic timeout mechanism for tasks (e.g., wait for sensor data).

Requirements:

- Timeout after 100ms if task doesn't complete.
- Assumptions: 1ms timer interrupt.

Ways to Solve

Algorithm Used

- **Timer Counter:** Increment counter in interrupt, check against timeout.
- **Task Control:** Reset counter on task start, check for timeout.

Complexity

- **Time Complexity:** $O(1)$ per check.
- **Space Complexity:** $O(1)$.

Pseudocode

```
GLOBAL timer, task_active
FUNCTION timer_interrupt
    timer++
FUNCTION start_task
    timer = 0
    task_active = true
FUNCTION check_timeout
    IF task_active AND timer >= 100:
        task_active = false
        RETURN timeout
    RETURN no_timeout
MAIN
    start_task()
    WHILE true:
        IF check_timeout():
            handle_timeout()
```

C Code Implementation

```
#include <stdint.h>

#define TIMEOUT_MS 100

uint16_t timer = 0;

uint8_t task_active = 0;

void timer_isr(void) {
    timer++;
}

void start_task(void) {
    timer = 0;
    task_active = 1;
}
```

```

uint8_t check_timeout(void) {
    if (task_active && timer >= TIMEOUT_MS) {
        task_active = 0;
        return 1;
    }
    return 0;
}

void handle_timeout(void) { /* Handle timeout */ }

int main(void) {
    start_task();
    while (1) {
        if (check_timeout()) handle_timeout();
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Ensure timer interrupt is precise.
- **Testing:** Simulate task delays.
- **Robustness:** Handle multiple tasks with separate timers.
- **Optimization:** Use hardware timers if available.
- **Extensibility:** Support variable timeouts.

Project 72: Static Struct for Peripheral State Tracking

Description: Use a static struct to track peripheral state (e.g., UART status).

Requirements:

- Track UART state: enabled, baud rate, error count.
- Assumptions: Single UART peripheral.

Ways to Solve

Algorithm Used

- **Static Struct:** Define a static struct for state variables.
- **Access Functions:** Provide getters/setters for controlled access.

Complexity

- **Time Complexity:** $O(1)$ per access.
- **Space Complexity:** $O(1)$.

Pseudocode

```

STRUCT UartState { enabled, baud, errors }
STATIC UartState state
FUNCTION init_uart_state
    state = {enabled: 0, baud: 9600, errors: 0}
FUNCTION set_enabled(val)
    state.enabled = val
FUNCTION get_errors
    RETURN state.errors
MAIN
    init_uart_state()
    set_enabled(1)

```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint8_t enabled;
    uint32_t baud;
    uint16_t errors;
} UartState;

static UartState state;

void init_uart_state(void) {
    state.enabled = 0;
    state.baud = 9600;
    state.errors = 0;
}

void set_enabled(uint8_t val) {
    state.enabled = val;
}

uint16_t get_errors(void) {
    return state.errors;
}

int main(void) {
    init_uart_state();
    set_enabled(1);
    // Use state
    return 0;
}
```

Expert Tips

- **Thread Safety:** Use critical sections for interrupt access.
- **Testing:** Verify state transitions.
- **Storage:** Consider non-volatile storage for state.
- **Robustness:** Validate input values.
- **Extensibility:** Support multiple peripherals.

Project 73: Multi-Byte Protocol with Framing

Description: Handle a multi-byte protocol with framing (e.g., start, length, data, end).

Requirements:

- Format: start (0x7E), 1-byte length (≤ 32), data, end (0x7F).
- Assumptions: UART input, max data 32 bytes.

Ways to Solve

Algorithm Used

- **State Machine:** Parse bytes in states: start, length, data, end.
- **Validation:** Check start/end bytes, process data if valid.

Complexity

- **Time Complexity:** $O(n)$ for n-byte packet.
- **Space Complexity:** $O(n)$ for data buffer.

Pseudocode

```
ENUM State { START, LENGTH, DATA, END }
GLOBAL buffer[32], index, length, state
FUNCTION parse_protocol(byte)
    CASE state:
        START:
            IF byte == 0x7E:
                state = LENGTH
                index = 0
        LENGTH:
            length = byte
            state = DATA
        DATA:
            IF index < length:
                buffer[index++] = byte
            IF index == length:
                state = END
        END:
            IF byte == 0x7F:
                process_data(buffer, length)
                state = START
MAIN
    WHILE true:
        byte = read_uart()
        parse_protocol(byte)
```

C Code Implementation

```
#include <stdint.h>

#define MAX_DATA 32

typedef enum { START, LENGTH, DATA, END } State;

uint8_t buffer[MAX_DATA], index = 0, length = 0;

State state = START;

uint8_t read_uart(void) { return 0; } // Stub

void process_data(uint8_t* data, uint8_t len) { /* Process data */ }

void parse_protocol(uint8_t byte) {
    switch (state) {
        case START:
            if (byte == 0x7E) {
                state = LENGTH;
                index = 0;
            }
            break;
        case LENGTH:
            length = byte;
            state = DATA;
            break;
        case DATA:
            if (index < length && index < MAX_DATA) {
                buffer[index++] = byte;
            }
            if (index == length) state = END;
            break;
        case END:
            if (byte == 0x7F) process_data(buffer, length);
            state = START;
            break;
    }
}
```

```
int main(void) {
    while (1) {
        uint8_t byte = read_uart();
        parse_protocol(byte);
    }
    return 0;
}
```

Expert Tips

- **Robustness:** Handle malformed packets.
- **Testing:** Simulate invalid frames.
- **Optimization:** Use circular buffer for input.
- **Debugging:** Log packet errors.
- **Extensibility:** Support escape sequences for special bytes.

Project 74: Simple Data Validator for Sensor Inputs

Description: Implement a simple data validator for sensor inputs (e.g., range check).

Requirements:

- Validate uint16_t sensor data (range: 100–1000).
- Assumptions: Invalid data logged.

Ways to Solve

Algorithm Used

- **Range Check:** Verify data is within specified range.
- **Error Handling:** Log invalid data and return status.

Complexity

- **Time Complexity:** $O(1)$ per validation.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION validate_sensor(data)
    IF data >= 100 AND data <= 1000:
        RETURN valid
    log_error("Out of range")
    RETURN invalid
MAIN
    data = read_sensor()
    valid = validate_sensor(data)
```

C Code Implementation

```
#include <stdint.h>

#define MIN_VALUE 100
#define MAX_VALUE 1000

void log_error(const char* msg) { /* Log error */ }

uint16_t read_sensor(void) { return 0; } // Stub

uint8_t validate_sensor(uint16_t data) {
    if (data >= MIN_VALUE && data <= MAX_VALUE) return 1;
    log_error("Out of range");
    return 0;
}
```

```

int main(void) {
    uint16_t data = read_sensor();
    uint8_t valid = validate_sensor(data);
    // Use valid
    return 0;
}

```

Expert Tips

- **Robustness:** Add checks for outliers or noise.
- **Testing:** Simulate invalid inputs.
- **Optimization:** Inline validation for speed.
- **Extensibility:** Support multiple sensors.
- **Debugging:** Log detailed error info.

Project 75: Pointer to Manage a Dynamic Task List

Description: Use a pointer to manage a dynamic task list (e.g., task ID, function pointer).

Requirements:

- Support up to 4 tasks with ID and callback.
- Assumptions: Dynamic addition/removal, fixed max size.

Ways to Solve

Algorithm Used

- **Task Struct:** Define a struct for task ID and callback.
- **Dynamic Array:** Use a pointer to an array of tasks, track size.

Complexity

- **Time Complexity:** $O(n)$ for add/remove, $O(1)$ for execution.
- **Space Complexity:** $O(n)$ for task list ($n=4$).

Pseudocode

```

STRUCT Task { id, callback }
GLOBAL Task* tasks, size
FUNCTION init_task_list
    tasks = allocate(4 * sizeof(Task))
    size = 0
FUNCTION add_task(id, callback)
    IF size < 4:
        tasks[size] = {id, callback}
        size++
FUNCTION remove_task(id)
    FOR i = 0 to size-1:
        IF tasks[i].id == id:
            tasks[i] = tasks[size-1]
            size--
            BREAK
FUNCTION execute_tasks
    FOR i = 0 to size-1:
        tasks[i].callback()
MAIN
    init_task_list()
    add_task(1, task1)
    execute_tasks()

```


C Code Implementation

```
#include <stdint.h>

#define MAX_TASKS 4

typedef void (*Callback)(void);

typedef struct {
    uint8_t id;
    Callback callback;
} Task;

Task* tasks = NULL;

uint8_t size = 0;

void task1(void) { /* Task 1 */ }

void* allocate(size_t s) { return NULL; } // Stub

void init_task_list(void) {
    tasks = allocate(MAX_TASKS * sizeof(Task));
    size = 0;
}

void add_task(uint8_t id, Callback callback) {
    if (size < MAX_TASKS) {
        tasks[size].id = id;
        tasks[size].callback = callback;
        size++;
    }
}

void remove_task(uint8_t id) {
    for (uint8_t i = 0; i < size; i++) {
        if (tasks[i].id == id) {
            tasks[i] = tasks[size - 1];
            size--;
            break;
        }
    }
}

void execute_tasks(void) {
    for (uint8_t i = 0; i < size; i++) {
        tasks[i].callback();
    }
}

int main(void) {
    init_task_list();
    add_task(1, task1);
    execute_tasks();
    return 0;
}
```

Expert Tips

- **Memory:** Replace `allocate` with platform-specific allocator.
- **Testing:** Simulate task addition/removal.
- **Robustness:** Handle NULL pointer and full list.
- **Optimization:** Use static array for fixed tasks.
- **Extensibility:** Support task priorities.

Microcontroller Configuration and Peripherals

Project 76: Configure Multiple GPIO Pins for 4x4 LED Matrix

Description: Configure multiple GPIO pins to control a 4x4 LED matrix using row-column scanning.

Requirements:

- Control 16 LEDs (4 rows, 4 columns).
- Assumptions: Common-anode matrix, rows as outputs (active high), columns as outputs (active low).

Ways to Solve

Algorithm Used

- **Row-Column Scanning:** Activate one row at a time, set column states to light specific LEDs.
- **GPIO Configuration:** Set row pins as outputs (push-pull), column pins as outputs.

Complexity

- **Time Complexity:** $O(1)$ per update (single row scan).
- **Space Complexity:** $O(1)$ (fixed-size pattern).

Pseudocode

```
FUNCTION init_gpio
    Configure row pins as output
    Configure column pins as output
FUNCTION display_pattern(pattern)
    FOR each row:
        Set all rows low
        Set current row high
        Set columns based on pattern[row]
        Delay for scan period
MAIN
    init_gpio()
    pattern = {0b1010, 0b0101, 0b1111, 0b0000}
    WHILE true:
        display_pattern(pattern)
```

C Code Implementation

```
#include <stdint.h>

#define ROW_COUNT 4
#define COL_COUNT 4

void set_row(uint8_t row, uint8_t state) { /* Set row pin */ }

void set_column(uint8_t col, uint8_t state) { /* Set column pin */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_gpio(void) {
    // Configure rows and columns as outputs
    for (uint8_t i = 0; i < ROW_COUNT; i++) set_row(i, 0);
    for (uint8_t i = 0; i < COL_COUNT; i++) set_column(i, 1); // Active low
}
```

```

void display_pattern(uint8_t pattern[ROW_COUNT]) {
    for (uint8_t row = 0; row < ROW_COUNT; row++) {
        // Disable all rows
        for (uint8_t i = 0; i < ROW_COUNT; i++) set_row(i, 0);
        // Set current row high
        set_row(row, 1);
        // Set columns (active low)
        for (uint8_t col = 0; col < COL_COUNT; col++) {
            set_column(col, !(pattern[row] & (1 << col)));
        }
        delay_ms(5); // Scan period
    }
}

int main(void) {
    uint8_t pattern[ROW_COUNT] = {0b1010, 0b0101, 0b1111, 0b0000};
    init_gpio();
    while (1) {
        display_pattern(pattern);
    }
    return 0;
}

```

Expert Tips

- **Multiplexing:** Use timer interrupts for precise scan timing.
- **Testing:** Verify each LED with test patterns.
- **Optimization:** Adjust scan rate to reduce flicker.
- **Robustness:** Handle GPIO initialization failures.
- **Extensibility:** Support larger matrices.

Project 77: Read 4x4 Keypad and Display Key Presses on Serial Console

Description: Create a project to read a 4x4 keypad and send key presses to a serial console.

Requirements:

- Detect key presses in a 4x4 keypad (rows as outputs, columns as inputs with pull-ups).
- Send key value (e.g., '0'-'9', 'A'-'F') via UART.
- Assumptions: Debouncing via software delay.

Ways to Solve

Algorithm Used

- **Row Scanning:** Set one row low, read columns to detect presses.
- **Key Mapping:** Map row-column pairs to characters.
- **UART Output:** Send key character via UART.

Complexity

- **Time Complexity:** $O(1)$ per scan (fixed 4x4).
- **Space Complexity:** $O(1)$ (fixed key map).

Pseudocode

```

FUNCTION init_keypad
    Configure rows as outputs
    Configure columns as inputs with pull-ups

```

```

FUNCTION read_keypad
  FOR each row:
    Set row low
    Read columns
    IF column low:
      Debounce
      RETURN key_map[row][col]
  RETURN no_key
MAIN
  init_keypad()
  WHILE true:
    key = read_keypad()
    IF key != no_key:
      send_uart(key)

```

C Code Implementation

```

#include <stdint.h>

#define ROW_COUNT 4
#define COL_COUNT 4

const char key_map[ROW_COUNT][COL_COUNT] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

void set_row(uint8_t row, uint8_t state) { /* Set row pin */ }

uint8_t read_column(uint8_t col) { return 1; } // Stub

void send_uart(char key) { /* UART send */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_keypad(void) {
  for (uint8_t i = 0; i < ROW_COUNT; i++) set_row(i, 1); // Active low
  // Columns configured as inputs with pull-ups
}

char read_keypad(void) {
  for (uint8_t row = 0; row < ROW_COUNT; row++) {
    set_row(row, 0); // Active low
    for (uint8_t col = 0; col < COL_COUNT; col++) {
      if (!read_column(col)) {
        delay_ms(20); // Debounce
        if (!read_column(col)) return key_map[row][col];
      }
    }
    set_row(row, 1);
  }
  return 0; // No key
}

int main(void) {
  init_keypad();
  while (1) {
    char key = read_keypad();
    if (key) send_uart(key);
  }
  return 0;
}

```

Expert Tips

- **Debouncing:** Use hardware debouncing or timer-based checks.
- **Testing:** Simulate multiple key presses.
- **Optimization:** Cache last key to reduce UART traffic.
- **Robustness:** Handle multiple simultaneous presses.
- **Extensibility:** Support different keypad layouts.

Project 78: Control Stepper Motor with Acceleration Control

Description: Implement a project to control a stepper motor with acceleration to smooth motion.

Requirements:

- Control a unipolar stepper motor (4 phases) with ramp-up/ramp-down.
- Assumptions: GPIO outputs, 200 steps/rev, timer for stepping.

Ways to Solve

Algorithm Used

- **Acceleration Profile:** Linearly increase/decrease step delay from max to min.
- **State Machine:** Handle states: idle, accelerating, running, decelerating.

Complexity

- **Time Complexity:** $O(n)$ for n steps in profile.
- **Space Complexity:** $O(1)$.

Pseudocode

```
ENUM State { IDLE, ACCEL, RUN, DECEL }
GLOBAL state, step_count, delay
FUNCTION init_stepper
    Configure GPIO for phases
    Configure timer for stepping
FUNCTION step_motor
    CASE state:
        IDLE:
            IF start:
                state = ACCEL
                delay = max_delay
        ACCEL:
            IF step_count < accel_steps:
                Set phase pins
                step_count++
                delay -= delta
            ELSE:
                state = RUN
        RUN:
            IF step_count < total_steps - accel_steps:
                Set phase pins
                step_count++
            ELSE:
                state = DECEL
        DECEL:
            IF step_count < total_steps:
                Set phase pins
                step_count++
                delay += delta
            ELSE:
                state = IDLE
```

```
MAIN
    init_stepper()
    WHILE true:
        step_motor()
```

C Code Implementation

```
#include <stdint.h>

#define MAX_DELAY 1000 // us
#define MIN_DELAY 200 // us
#define ACCEL_STEPS 50
#define TOTAL_STEPS 200

typedef enum { IDLE, ACCEL, RUN, DECEL } State;

State state = IDLE;

uint16_t step_count = 0;

uint16_t delay = MAX_DELAY;

void set_phase(uint8_t phase) { /* Set GPIO for phase */ }

void timer_delay(uint16_t us) { /* Timer delay */ }

void init_stepper(void) {
    // Configure GPIO for 4 phases
    state = IDLE;
    step_count = 0;
    delay = MAX_DELAY;
}

void step_motor(void) {
    static const uint8_t phases[] = {0b1000, 0b0100, 0b0010, 0b0001};
    static uint8_t phase_idx = 0;
    switch (state) {
        case IDLE:
            if (/* start condition */) {
                state = ACCEL;
                delay = MAX_DELAY;
                step_count = 0;
            }
            break;
        case ACCEL:
            if (step_count < ACCEL_STEPS) {
                set_phase(phases[phase_idx]);
                phase_idx = (phase_idx + 1) % 4;
                timer_delay(delay);
                step_count++;
                delay -= (MAX_DELAY - MIN_DELAY) / ACCEL_STEPS;
            } else {
                state = RUN;
            }
            break;
        case RUN:
            if (step_count < TOTAL_STEPS - ACCEL_STEPS) {
                set_phase(phases[phase_idx]);
                phase_idx = (phase_idx + 1) % 4;
                timer_delay(delay);
                step_count++;
            } else {
                state = DECEL;
            }
            break;
    }
}
```

```

        case DECEL:
            if (step_count < TOTAL_STEPS) {
                set_phase(phases[phase_idx]);
                phase_idx = (phase_idx + 1) % 4;
                timer_delay(delay);
                step_count++;
                delay += (MAX_DELAY - MIN_DELAY) / ACCEL_STEPS;
            } else {
                state = IDLE;
            }
            break;
    }
}

int main(void) {
    init_stepper();
    while (1) {
        step_motor();
    }
    return 0;
}

```

Expert Tips

- **Precision:** Use timer interrupts for accurate delays.
- **Testing:** Verify acceleration with a scope.
- **Optimization:** Precompute delay steps.
- **Robustness:** Add stall detection.
- **Extensibility:** Support variable step counts.

Project 79: Configure ADC to Read Multiple Channels in Scan Mode

Description: Configure an ADC to read multiple channels in scan mode for sensor data.

Requirements:

- Read 4 channels (uint16_t) sequentially.
- Assumptions: 12-bit ADC, continuous scan mode.

Ways to Solve

Algorithm Used

- **ADC Configuration:** Enable scan mode, set channel sequence.
- **DMA or Interrupt:** Store results in a buffer using DMA or interrupt.

Complexity

- **Time Complexity:** $O(1)$ per conversion cycle.
- **Space Complexity:** $O(n)$ for n channels.

Pseudocode

```

FUNCTION init_adc
    Configure ADC for scan mode
    Set channels 0-3
    Enable ADC interrupt
FUNCTION adc_interrupt
    FOR each channel:
        buffer[channel] = read_adc_result()

```

```
MAIN
    init_adc()
    WHILE true:
        process_buffer(buffer)
```

C Code Implementation

```
#include <stdint.h>

#define NUM_CHANNELS 4

uint16_t adc_buffer[NUM_CHANNELS];

void configure_adc(void) { /* Set scan mode, channels 0-3 */ }

uint16_t read_adc_result(void) { return 0; } // Stub

void process_buffer(uint16_t* buffer) { /* Process data */ }

void init_adc(void) {
    configure_adc();
    // Enable ADC interrupt
}

void adc_isr(void) {
    for (uint8_t i = 0; i < NUM_CHANNELS; i++) {
        adc_buffer[i] = read_adc_result();
    }
}

int main(void) {
    init_adc();
    while (1) {
        process_buffer(adc_buffer);
    }
    return 0;
}
```

Expert Tips

- **DMA:** Use DMA for efficient data transfer.
- **Testing:** Verify channel order and values.
- **Calibration:** Apply offset/gain corrections.
- **Robustness:** Handle ADC overruns.
- **Extensibility:** Support dynamic channel selection.

Project 80: Use DAC to Generate a Sine Wave

Description: Create a project to generate a sine wave using a DAC.

Requirements:

- Output 12-bit sine wave at 1 kHz.
- Assumptions: Timer-driven updates, lookup table.

Ways to Solve

Algorithm Used

- **Lookup Table:** Precompute sine values for one cycle.
- **Timer Trigger:** Update DAC at regular intervals.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(n)$ for lookup table.

Pseudocode

```
CONST sine_table[32] = {precomputed sine values}
FUNCTION init_dac
    Configure DAC
    Configure timer for 32 kHz (1 kHz * 32 samples)
FUNCTION timer_interrupt
    index = (index + 1) % 32
    write_dac(sine_table[index])
MAIN
    init_dac()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define TABLE_SIZE 32

const uint16_t sine_table[TABLE_SIZE] = {
    2048, 2832, 3496, 3996, 4287, 4392, 4307, 4032,
    3584, 2992, 2312, 1592, 896, 352, 88, 32,
    88, 352, 896, 1592, 2312, 2992, 3584, 4032,
    4307, 4392, 4287, 3996, 3496, 2832, 2048, 0
};

void configure_dac(void) { /* DAC config */ }

void configure_timer(void) { /* Timer config for 32 kHz */ }

void write_dac(uint16_t value) { /* Write to DAC */ }

void init_dac(void) {
    configure_dac();
    configure_timer();
}

void timer_isr(void) {
    static uint8_t index = 0;
    write_dac(sine_table[index]);
    index = (index + 1) % TABLE_SIZE;
}

int main(void) {
    init_dac();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}
```

Expert Tips

- **Precision:** Increase table size for smoother wave.
- **Testing:** Verify waveform with an oscilloscope.
- **Optimization:** Use DMA for DAC updates.
- **Robustness:** Handle DAC initialization errors.
- **Extensibility:** Support variable frequencies.

Project 81: Interface with 16x2 LCD Using GPIO Pins

Description: Implement a project to interface with a 16x2 LCD using GPIO pins in 4-bit mode.

Requirements:

- Display text on a 16x2 LCD (HD44780-compatible).
- Assumptions: 4-bit mode, GPIO for data/control.

Ways to Solve

Algorithm Used

- **Initialization:** Send LCD initialization commands.
- **Data Write:** Use 4-bit mode to send commands/data via GPIO.

Complexity

- **Time Complexity:** $O(n)$ for n-character string.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_lcd
    Configure GPIO for RS, EN, D4-D7
    Send init commands (function set, display on, clear)
FUNCTION write_char(char)
    Set RS high
    Send high nibble
    Pulse EN
    Send low nibble
    Pulse EN
FUNCTION write_string(str)
    FOR each char in str:
        write_char(char)
MAIN
    init_lcd()
    write_string("Hello World")
```

C Code Implementation

```
#include <stdint.h>

void set_gpio(uint8_t pin, uint8_t state) { /* Set GPIO */ }

void delay_us(uint16_t us) { /* Delay */ }

void init_lcd(void) {
    // Configure GPIO for RS, EN, D4-D7
    delay_us(50000); // Wait for LCD power-up
    set_gpio(0, 0); // RS low (command)
    // Send init sequence: function set, display on, clear
    uint8_t cmds[] = {0x28, 0x0C, 0x01};
    for (uint8_t i = 0; i < 3; i++) {
        set_gpio(1, 1); // EN high
        for (uint8_t j = 0; j < 4; j++) set_gpio(4 + j, (cmds[i] >> (4 + j)) & 1);
        delay_us(1);
        set_gpio(1, 0); // EN low
        delay_us(1);
        set_gpio(1, 1);
        for (uint8_t j = 0; j < 4; j++) set_gpio(4 + j, (cmds[i] >> j) & 1);
        delay_us(1);
        set_gpio(1, 0);
    }
}
```

```

        delay_us(2000);
    }
}

void write_char(char c) {
    set_gpio(0, 1); // RS high (data)
    set_gpio(1, 1); // EN high
    for (uint8_t i = 0; i < 4; i++) set_gpio(4 + i, (c >> (4 + i)) & 1);
    delay_us(1);
    set_gpio(1, 0); // EN low
    delay_us(1);
    set_gpio(1, 1);
    for (uint8_t i = 0; i < 4; i++) set_gpio(4 + i, (c >> i) & 1);
    delay_us(1);
    set_gpio(1, 0);
    delay_us(40);
}

void write_string(const char* str) {
    while (*str) write_char(*str++);
}

int main(void) {
    init_lcd();
    write_string("Hello World");
    return 0;
}

```

Expert Tips

- **Timing:** Adhere to LCD datasheet timings.
- **Testing:** Verify display with test strings.
- **Optimization:** Use 8-bit mode for faster writes.
- **Robustness:** Handle initialization failures.
- **Extensibility:** Support custom characters.

Project 82: Timer for Variable-Frequency PWM Signal

Description: Configure a timer to generate a variable-frequency PWM signal.

Requirements:

- Generate PWM with frequency range 1–10 kHz, 50% duty cycle.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer period and compare value for PWM.
- **Frequency Adjustment:** Update period register dynamically.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_pwm
    Configure timer for PWM mode
    Set duty cycle to 50%

```

```

FUNCTION set_pwm_freq(freq)
    period = clock / freq
    Set timer period to period
    Set compare to period / 2
MAIN
    init_pwm()
    set_pwm_freq(5000)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz

void configure_timer(uint16_t period, uint16_t compare) { /* Timer config */ }

void init_pwm(void) {
    configure_timer(1000, 500); // Default 1 kHz, 50% duty
}

void set_pwm_freq(uint16_t freq) {
    if (freq < 1000 || freq > 10000) return;
    uint16_t period = CLOCK_FREQ / freq;
    configure_timer(period, period / 2);
}

int main(void) {
    init_pwm();
    set_pwm_freq(5000); // 5 kHz
    while (1) {
        /* Run PWM */
    }
    return 0;
}

```

Expert Tips

- **Validation:** Check frequency range.
- **Testing:** Verify PWM with an oscilloscope.
- **Optimization:** Use hardware PWM channels.
- **Robustness:** Handle invalid frequencies.
- **Extensibility:** Support variable duty cycles.

Project 83: Timer to Measure Frequency of External Signal

Description: Use a timer to measure the frequency of an external signal.

Requirements:

- Measure frequency of a square wave (1–10 kHz).
- Assumptions: Input capture mode, 1 MHz timer clock.

Ways to Solve

Algorithm Used

- **Input Capture:** Capture timer value on rising edges.
- **Frequency Calculation:** Compute period from consecutive captures, $\text{freq} = 1/\text{period}$.

Complexity

- **Time Complexity:** $O(1)$ per measurement.

- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_timer
    Configure timer for input capture
    Enable capture interrupt
FUNCTION capture_interrupt
    prev_time = current_time
    current_time = get_capture_value()
    period = current_time - prev_time
    freq = clock / period
MAIN
    init_timer()
    WHILE true:
        display_freq(freq)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz

uint32_t prev_time = 0, current_time = 0;

uint32_t freq = 0;

void configure_timer(void) { /* Input capture config */ }

uint32_t get_capture_value(void) { return 0; } // Stub

void display_freq(uint32_t freq) { /* Display */ }

void init_timer(void) {
    configure_timer();
}

void capture_isr(void) {
    prev_time = current_time;
    current_time = get_capture_value();
    if (current_time > prev_time) {
        uint32_t period = current_time - prev_time;
        freq = CLOCK_FREQ / period;
    }
}

int main(void) {
    init_timer();
    while (1) {
        display_freq(freq);
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Use high-resolution timer.
- **Testing:** Verify with known signal frequencies.
- **Robustness:** Handle timer overflow.
- **Optimization:** Average multiple periods.
- **Extensibility:** Support pulse width measurement.

Project 84: Control DC Motor with PID Using PWM

Description: Implement a project to control a DC motor with PID using PWM for speed control.

Requirements:

- Maintain target speed using PID ($K_p=1$, $K_i=0.1$, $K_d=0.01$).
- Assumptions: Encoder feedback, 10-bit PWM.

Ways to Solve

Algorithm Used

- **PID Controller:** Compute error, update PWM based on proportional, integral, derivative terms.
- **Timer Interrupt:** Update PID periodically (e.g., 10ms).

Complexity

- **Time Complexity:** $O(1)$ per PID update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
GLOBAL error_sum, prev_error
FUNCTION init_pid
    Configure PWM timer
    Configure encoder interrupt
FUNCTION pid_update(target, actual)
    error = target - actual
    error_sum += error
    derivative = error - prev_error
    output =  $K_p * error + K_i * error\_sum + K_d * derivative$ 
    Set PWM to output
    prev_error = error
MAIN
    init_pid()
    WHILE true:
        speed = read_encoder()
        pid_update(target_speed, speed)
```

C Code Implementation

```
#include <stdint.h>

#define KP 1.0f
#define KI 0.1f
#define KD 0.01f

float error_sum = 0, prev_error = 0;

void configure_pwm(uint16_t value) { /* Set PWM */ }

uint16_t read_encoder(void) { return 0; } // Stub

void init_pid(void) {
    configure_pwm(0);
    error_sum = 0;
    prev_error = 0;
}

void pid_update(uint16_t target, uint16_t actual) {
    float error = target - actual;
    error_sum += error;
    float derivative = error - prev_error;
    float output =  $KP * error + KI * error\_sum + KD * derivative$ ;
```

```

        if (output > 1023) output = 1023;
        if (output < 0) output = 0;
        configure_pwm((uint16_t)output);
        prev_error = error;
    }

    int main(void) {
        init_pid();
        uint16_t target_speed = 500;
        while (1) {
            uint16_t speed = read_encoder();
            pid_update(target_speed, speed);
        }
        return 0;
    }

```

Expert Tips

- **Tuning:** Adjust PID constants experimentally.
- **Testing:** Verify with known speeds.
- **Optimization:** Use fixed-point for PID math.
- **Robustness:** Limit integral windup.
- **Extensibility:** Support direction control.

Project 85: Interrupt for Multiple GPIO Button Inputs

Description: Configure an interrupt to handle multiple GPIO pins for button inputs.

Requirements:

- Handle 4 buttons with interrupts (active low).
- Assumptions: External interrupts, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupts on GPIO pins (falling edge).
- **Debouncing:** Use software delay to filter bounces.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_buttons
    Configure GPIO pins as inputs with pull-ups
    Enable interrupts on falling edge
FUNCTION button_interrupt
    FOR each button:
        IF pin low:
            Debounce
            IF still low:
                process_button(button)
MAIN
    init_buttons()
    WHILE true:
        /* Wait for interrupts */

```

C Code Implementation

```
#include <stdint.h>

#define BUTTON_COUNT 4

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_button(uint8_t id) { /* Process button */ }

void init_buttons(void) {
    for (uint8_t i = 0; i < BUTTON_COUNT; i++) {
        configure_gpio(i, 1); // Input with pull-up
    }
}

void button_isr(void) {
    for (uint8_t i = 0; i < BUTTON_COUNT; i++) {
        if (!read_pin(i)) {
            delay_ms(20); // Debounce
            if (!read_pin(i)) process_button(i);
        }
    }
}

int main(void) {
    init_buttons();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate multiple button presses.
- **Optimization:** Use single interrupt for all pins.
- **Robustness:** Handle simultaneous presses.
- **Extensibility:** Support more buttons.

Project 86: Interface with Temperature Sensor Using ADC

Description: Create a project to read a temperature sensor (e.g., NTC thermistor) using ADC and display results.

Requirements:

- Read ADC (12-bit), convert to temperature (°C), send to UART.
- Assumptions: Linear approximation for thermistor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for thermistor voltage.
- **Conversion:** Use linear formula: $\text{temp} = m * \text{ADC} + b$.
- **UART Output:** Send temperature string.

Complexity

- **Time Complexity:** $O(1)$ per reading.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_temperature
    adc_value = read_adc()
    temp = m * adc_value + b
    RETURN temp
FUNCTION display_temperature(temp)
    Format temp as string
    Send to UART
MAIN
    init_adc()
    WHILE true:
        temp = read_temperature()
        display_temperature(temp)
```

C Code Implementation

```
#include <stdint.h>

#define M 0.0244f // Slope
#define B -10.0f // Offset

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void send_uart(const char* str) { /* UART send */ }

void init_adc(void) {
    configure_adc();
}

float read_temperature(void) {
    uint16_t adc_value = read_adc();
    return M * adc_value + B;
}

void display_temperature(float temp) {
    char buffer[16];
    sprintf(buffer, sizeof(buffer), "Temp: %.1f C\n", temp);
    send_uart(buffer);
}

int main(void) {
    init_adc();
    while (1) {
        float temp = read_temperature();
        display_temperature(temp);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Use thermistor datasheet for accurate conversion.
- **Testing:** Verify with known temperatures.
- **Optimization:** Use lookup table for non-linear sensors.
- **Robustness:** Handle ADC errors.

- **Extensibility:** Support multiple sensors.

Project 87: Timer for Dual PWM Signal for Motor Control

Description: Use a timer to generate dual PWM signals for motor control (e.g., H-bridge).

Requirements:

- Generate two 1 kHz PWM signals with adjustable duty cycles.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Use two compare channels for PWM.
- **Duty Cycle Update:** Adjust compare values dynamically.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_pwm
    Configure timer for two PWM channels
    Set period for 1 kHz
    Set initial duty cycles
FUNCTION set_pwm_duty(channel, duty)
    Set compare register to duty * period / 100
MAIN
    init_pwm()
    set_pwm_duty(0, 50)
    set_pwm_duty(1, 75)
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 1000      // 1 kHz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t duty1, uint16_t duty2) { /* PWM config */ }

void init_pwm(void) {
    configure_pwm(PERIOD, PERIOD / 2, PERIOD / 2); // 50% duty
}

void set_pwm_duty(uint8_t channel, uint8_t duty) {
    if (duty > 100) duty = 100;
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, channel == 0 ? compare : PERIOD / 2, channel == 1 ? compare : PERIOD / 2);
}

int main(void) {
    init_pwm();
    set_pwm_duty(0, 50); // 50% duty
    set_pwm_duty(1, 75); // 75% duty
    while (1) {
        /* Run PWM */
    }
}
```

```
    return 0;
}
```

Expert Tips

- **Validation:** Check duty cycle range.
- **Testing:** Verify PWM with oscilloscope.
- **Optimization:** Use hardware PWM channels.
- **Robustness:** Handle timer initialization errors.
- **Extensibility:** Support phase shift.

Project 88: GPIO Pin for Quadrature Encoder with Interrupt

Description: Configure a GPIO pin to handle a quadrature encoder with interrupt.

Requirements:

- Track encoder position (int32_t) using A/B signals.
- Assumptions: External interrupts on both channels.

Ways to Solve

Algorithm Used

- **Interrupt Handling:** Detect edges on A/B signals, update position based on state.
- **Quadrature Decoding:** Use A/B transitions to determine direction.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
GLOBAL position
FUNCTION init_encoder
    Configure GPIO A/B as inputs
    Enable interrupts on both edges
FUNCTION encoder_interrupt
    a = read_pin(A)
    b = read_pin(B)
    IF a changed:
        IF a == b:
            position++
        ELSE:
            position--
MAIN
    init_encoder()
    WHILE true:
        use_position(position)
```

C Code Implementation

```
#include <stdint.h>

int32_t position = 0;
uint8_t prev_a = 0;

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 0; } // Stub

void use_position(int32_t pos) { /* Use position */ }
```

```

void init_encoder(void) {
    configure_gpio(0, 1); // A input
    configure_gpio(1, 1); // B input
    prev_a = read_pin(0);
}

void encoder_isr(void) {
    uint8_t a = read_pin(0);
    uint8_t b = read_pin(1);
    if (a != prev_a) {
        if (a == b) position++;
        else position--;
        prev_a = a;
    }
}

int main(void) {
    init_encoder();
    while (1) {
        use_position(position);
    }
    return 0;
}

```

Expert Tips

- **Debouncing:** Filter noise with hardware or software.
- **Testing:** Verify with known rotations.
- **Optimization:** Use timer input capture for high-speed.
- **Robustness:** Handle overflow in position.
- **Extensibility:** Support multiple encoders.

Project 89: Control Servo Motor with Precise Angle Control

Description: Create a project to control a servo motor with precise angle control.

Requirements:

- Control servo angle (0–180°) using PWM (50 Hz, 1–2ms pulse).
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer for 50 Hz PWM, adjust pulse width for angle.
- **Angle Mapping:** Map 0–180° to 1–2ms pulse (1000–2000us).

Complexity

- **Time Complexity:** O(1) per update.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_servo
    Configure timer for 50 Hz PWM
    Set initial pulse width
FUNCTION set_servo_angle(angle)
    pulse_us = 1000 + (angle * 1000 / 180)
    Set PWM compare to pulse_us
MAIN
    init_servo()

```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 50        // 50 Hz
#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t pulse) { /* PWM config */ }

void init_servo(void) {
    configure_pwm(PERIOD, 1500 * (CLOCK_FREQ / 1000000)); // 1.5ms
}

void set_servo_angle(uint8_t angle) {
    if (angle > 180) angle = 180;
    uint16_t pulse_us = 1000 + (angle * 1000 / 180);
    configure_pwm(PERIOD, pulse_us * (CLOCK_FREQ / 1000000));
}

int main(void) {
    init_servo();
    set_servo_angle(90);
    while (1) {
        /* Run servo */
    }
    return 0;
}
```

Expert Tips

- **Precision:** Use high-resolution timer.
- **Testing:** Verify angles with a protractor.
- **Optimization:** Precompute pulse widths.
- **Robustness:** Validate angle range.
- **Extensibility:** Support multiple servos.

Project 90: Read Multi-Axis Accelerometer Using I2C

Description: Implement a project to read a multi-axis accelerometer (e.g., MPU6050) using I2C.

Requirements:

- Read 3-axis data (x, y, z as int16_t).
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read 6 bytes from accelerometer registers.
- **Data Parsing:** Combine bytes into int16_t for each axis.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```
FUNCTION init_accelerometer
    Configure I2C
    Write accelerometer config (e.g., enable, range)
FUNCTION read_accelerometer(data)
    Read 6 bytes from register
    data.x = (bytes[0] << 8) | bytes[1]
    data.y = (bytes[2] << 8) | bytes[3]
    data.z = (bytes[4] << 8) | bytes[5]
MAIN
    init_accelerometer()
    WHILE true:
        read_accelerometer(&data)
        process_data(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    int16_t x, y, z;
} AccelData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(AccelData* data) { /* Process data */ }

void init_accelerometer(void) {
    configure_i2c();
    i2c_write(0x68, 0x6B, 0x00); // Wake up MPU6050
}

void read_accelerometer(AccelData* data) {
    uint8_t buffer[6];
    i2c_read(0x68, 0x3B, buffer, 6); // Read accel registers
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_accelerometer();
    AccelData data;
    while (1) {
        read_accelerometer(&data);
        process_data(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply offset corrections.
- **Testing:** Verify with known orientations.
- **Optimization:** Use burst reads for efficiency.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add gyroscope support.

Project 91: ADC with Oversampling for Noise Reduction

Description: Configure an ADC to read a sensor with oversampling to reduce noise.

Requirements:

- Perform 16x oversampling on a single channel (12-bit ADC).
- Assumptions: Average results for output.

Ways to Solve

Algorithm Used

- **Oversampling:** Take 16 samples, average to reduce noise.
- **ADC Configuration:** Continuous mode for single channel.

Complexity

- **Time Complexity:** $O(n)$ for n samples ($n=16$).
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for continuous mode
FUNCTION read_oversampled
    sum = 0
    FOR i = 0 to 15:
        sum += read_adc()
    RETURN sum / 16
MAIN
    init_adc()
    WHILE true:
        value = read_oversampled()
        process_value(value)
```

C Code Implementation

```
#include <stdint.h>

#define OVERSAMPLE_COUNT 16

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_value(uint16_t value) { /* Process value */ }

void init_adc(void) {
    configure_adc();
}

uint16_t read_oversampled(void) {
    uint32_t sum = 0;
    for (uint8_t i = 0; i < OVERSAMPLE_COUNT; i++) {
        sum += read_adc();
    }
    return sum / OVERSAMPLE_COUNT;
}
```

```

int main(void) {
    init_adc();
    while (1) {
        uint16_t value = read_oversampled();
        process_value(value);
    }
    return 0;
}

```

Expert Tips

- **Noise Reduction:** Increase oversampling for better results.
- **Testing:** Verify with noisy input.
- **Optimization:** Use DMA for sampling.
- **Robustness:** Check for ADC errors.
- **Extensibility:** Support multiple channels.

Project 92: DAC to Generate a Triangle Wave

Description: Use a DAC to generate a triangle wave.

Requirements:

- Output 12-bit triangle wave at 1 kHz.
- Assumptions: Timer-driven updates, 32 samples/cycle.

Ways to Solve

Algorithm Used

- **Lookup Table:** Precompute triangle wave values.
- **Timer Trigger:** Update DAC at 32 kHz for 1 kHz wave.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(n)$ for lookup table.

Pseudocode

```

CONST triangle_table[32] = {precomputed values}
FUNCTION init_dac
    Configure DAC
    Configure timer for 32 kHz
FUNCTION timer_interrupt
    index = (index + 1) % 32
    write_dac(triangle_table[index])
MAIN
    init_dac()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>

#define TABLE_SIZE 32

const uint16_t triangle_table[TABLE_SIZE] = {
    0, 256, 512, 768, 1024, 1280, 1536, 1792, 2048, 2304, 2560, 2816, 3072, 3328, 3584, 3840,
    4095, 3840, 3584, 3328, 3072, 2816, 2560, 2304, 2048, 1792, 1536, 1280, 1024, 768, 512, 256
};

```



```

void configure_dac(void) { /* DAC config */ }

void configure_timer(void) { /* Timer config for 32 kHz */ }

void write_dac(uint16_t value) { /* Write to DAC */ }

void init_dac(void) {
    configure_dac();
    configure_timer();
}

void timer_isr(void) {
    static uint8_t index = 0;
    write_dac(triangle_table[index]);
    index = (index + 1) % TABLE_SIZE;
}

int main(void) {
    init_dac();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Smoothness:** Increase table size for better resolution.
- **Testing:** Verify waveform with oscilloscope.
- **Optimization:** Use DMA for DAC updates.
- **Robustness:** Handle DAC errors.
- **Extensibility:** Support variable frequencies.

Project 93: Interface with 7-Segment Display Using Shift Register

Description: Implement a project to interface with a 7-segment display using a shift register (e.g., 74HC595).

Requirements:

- Display digits 0–9 on a single 7-segment display.
- Assumptions: SPI-like GPIO bit-banging.

Ways to Solve

Algorithm Used

- **Shift Register:** Shift out 8-bit pattern for 7-segment segments.
- **Digit Mapping:** Use lookup table for digit-to-segment mapping.

Complexity

- **Time Complexity:** $O(1)$ per digit update.
- **Space Complexity:** $O(1)$ (fixed table).

Pseudocode

```

CONST segment_table[10] = {patterns for 0-9}
FUNCTION init_shift_register
    Configure GPIO for data, clock, latch
FUNCTION write_digit(digit)
    pattern = segment_table[digit]

```

```

    FOR each bit in pattern:
        Set data pin
        Pulse clock
        Pulse latch
MAIN
    init_shift_register()
    WHILE true:
        FOR digit = 0 to 9:
            write_digit(digit)
            Delay

```

C Code Implementation

```

#include <stdint.h>

const uint8_t segment_table[10] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F // 0-9
};

void set_data(uint8_t state) { /* Set data pin */ }

void set_clock(uint8_t state) { /* Set clock pin */ }

void set_latch(uint8_t state) { /* Set latch pin */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_shift_register(void) {
    set_data(0);
    set_clock(0);
    set_latch(0);
}

void write_digit(uint8_t digit) {
    if (digit > 9) return;
    uint8_t pattern = segment_table[digit];
    for (uint8_t i = 0; i < 8; i++) {
        set_data((pattern >> (7 - i)) & 1);
        set_clock(1);
        set_clock(0);
    }
    set_latch(1);
    set_latch(0);
}

int main(void) {
    init_shift_register();
    while (1) {
        for (uint8_t digit = 0; digit <= 9; digit++) {
            write_digit(digit);
            delay_ms(1000);
        }
    }
    return 0;
}

```

Expert Tips

- **Timing:** Ensure proper shift register timing.
- **Testing:** Verify each digit display.
- **Optimization:** Use hardware SPI if available.
- **Robustness:** Handle invalid digits.
- **Extensibility:** Support multiple digits.

Project 94: Timer to Generate 1 kHz Interrupt for Periodic Tasks

Description: Configure a timer to generate a 1 kHz interrupt for periodic tasks.

Requirements:

- Generate interrupt every 1ms (1 kHz).
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer prescaler and period to achieve 1ms ticks.
- **Interrupt Handling:** Execute tasks in interrupt service routine (ISR).

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_timer
    Set prescaler to achieve 1 MHz
    Set period to 1000 (1ms)
    Enable timer interrupt
FUNCTION timer_interrupt
    Execute periodic tasks
MAIN
    init_timer()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define TARGET_FREQ 1000 // 1 kHz

void configure_timer(uint16_t period) { /* Timer config */ }

void periodic_task(void) { /* Task code */ }

void init_timer(void) {
    configure_timer(CLOCK_FREQ / TARGET_FREQ); // 1000 ticks = 1ms
}

void timer_isr(void) {
    periodic_task();
}

int main(void) {
    init_timer();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify timer clock source.

- **Testing:** Measure interrupt frequency with a scope.
- **Optimization:** Minimize ISR execution time.
- **Robustness:** Handle missed interrupts.
- **Extensibility:** Support multiple task scheduling.

Project 95: Control BLDC Motor with Basic Commutation

Description: Create a project to control a brushless DC (BLDC) motor with basic commutation.

Requirements:

- Drive a 3-phase BLDC motor using 6-step commutation.
- Assumptions: Hall sensor feedback, GPIO outputs.

Ways to Solve

Algorithm Used

- **6-Step Commutation:** Sequence through 6 states based on Hall sensor inputs.
- **GPIO Control:** Set high/low side drivers for each phase.

Complexity

- **Time Complexity:** $O(1)$ per commutation step.
- **Space Complexity:** $O(1)$ (fixed commutation table).

Pseudocode

```
CONST commutation_table[6] = {phase configurations}
FUNCTION init_bldc
    Configure GPIO for phases
    Configure Hall sensor inputs
FUNCTION commutate
    hall_state = read_hall_sensors()
    Set phase pins based on commutation_table[hall_state]
MAIN
    init_bldc()
    WHILE true:
        commutate()
```

C Code Implementation

```
#include <stdint.h>

const uint8_t commutation_table[6][3] = {
    {1, 0, 2}, // Phase A high, B low, C float
    {1, 2, 0}, // Phase A high, C low, B float
    {2, 1, 0}, // Phase B high, C low, A float
    {0, 1, 2}, // Phase B high, A low, C float
    {0, 2, 1}, // Phase C high, A low, B float
    {2, 0, 1}  // Phase C high, B low, A float
};

void set_phase(uint8_t phase, uint8_t state) { /* Set phase state */ }

uint8_t read_hall_sensors(void) { return 0; } // Stub

void init_bldc(void) {
    // Configure GPIO for phases and Hall inputs
}
```

```

void commutate(void) {
    uint8_t hall_state = read_hall_sensors();
    if (hall_state < 6) {
        for (uint8_t i = 0; i < 3; i++) {
            set_phase(i, commutation_table[hall_state][i]);
        }
    }
}

int main(void) {
    init_bldc();
    while (1) {
        commutate();
    }
    return 0;
}

```

Expert Tips

- **Validation:** Verify Hall sensor states.
- **Testing:** Use a scope to check phase signals.
- **Optimization:** Use PWM for speed control.
- **Robustness:** Handle invalid Hall states.
- **Extensibility:** Add speed control loop.

Project 96: Read Pressure Sensor Using SPI

Description: Implement a project to read a pressure sensor (e.g., BMP280) using SPI.

Requirements:

- Read pressure data as `uint32_t`.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read sensor registers.
- **Data Parsing:** Combine bytes into `uint32_t` pressure.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_spi
    Configure SPI (4-wire, master)
FUNCTION read_pressure
    Send read command
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    RETURN pressure
MAIN
    init_spi()
    WHILE true:
        pressure = read_pressure()
        process_pressure(pressure)

```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_pressure(uint32_t pressure) { /* Process data */ }

void init_spi(void) {
    configure_spi();
    spi_write(0xF4, 0x27); // Configure BMP280
}

uint32_t read_pressure(void) {
    uint8_t buffer[3];
    spi_read(0xF7, buffer, 3); // Read pressure registers
    return ((uint32_t)buffer[0] << 16) | ((uint32_t)buffer[1] << 8) | buffer[2];
}

int main(void) {
    init_spi();
    while (1) {
        uint32_t pressure = read_pressure();
        process_pressure(pressure);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply sensor-specific compensation.
- **Testing:** Verify with known pressures.
- **Optimization:** Use burst reads for efficiency.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add temperature reading.

Project 97: GPIO Pin for Touch Sensor with Interrupt

Description: Configure a GPIO pin to handle a touch sensor with interrupt.

Requirements:

- Detect touch events (active low) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on falling edge.
- **Debouncing:** Use software delay to filter noise.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_touch
    Configure GPIO as input with pull-up
    Enable interrupt on falling edge
FUNCTION touch_interrupt
    IF pin low:
        Debounce
        IF still low:
            process_touch()
MAIN
    init_touch()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_touch(void) { /* Process touch */ }

void init_touch(void) {
    configure_gpio(0, 1); // Input with pull-up
}

void touch_isr(void) {
    if (!read_pin(0)) {
        delay_ms(20); // Debounce
        if (!read_pin(0)) process_touch();
    }
}

int main(void) {
    init_touch();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate touch events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple sensors.

Project 98: Interface with OLED Display Using I2C

Description: Create a project to interface with an OLED display (e.g., SSD1306) using I2C.

Requirements:

- Display text on a 128x64 OLED.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, send initialization and text data.
- **Display Update:** Write pixel data to display buffer.

Complexity

- **Time Complexity:** $O(n)$ for n-pixel update.
- **Space Complexity:** $O(n)$ for display buffer.

Pseudocode

```
FUNCTION init_oled
    Configure I2C
    Send OLED init commands
FUNCTION write_text(str, x, y)
    Convert str to pixel data
    Send data to OLED via I2C

MAIN
    init_oled()
    write_text("Hello", 0, 0)
```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

void init_oled(void) {
    configure_i2c();
    uint8_t init_cmds[] = {0xAE, 0xA8, 0x3F, 0xD3, 0x00, 0x40, 0xA1, 0xC8, 0xDA, 0x12, 0x81, 0x7F,
0xA4, 0xA6, 0xD5, 0x80, 0x8D, 0x14, 0xAF};
    i2c_write(0x3C, init_cmds, sizeof(init_cmds));
}

void write_text(const char* str, uint8_t x, uint8_t y) {
    uint8_t buffer[128];
    // Simplified: assumes font table and pixel mapping
    for (uint8_t i = 0; str[i] && i < 128; i++) {
        buffer[i] = str[i]; // Placeholder for font conversion
    }
    i2c_write(0x3C, buffer, strlen(str));
}

int main(void) {
    init_oled();
    write_text("Hello", 0, 0);
    while (1) {
        /* Keep display active */
    }
    return 0;
}
```

Expert Tips

- **Font:** Use a font table for character rendering.
- **Testing:** Verify display with test patterns.
- **Optimization:** Buffer updates locally.
- **Robustness:** Handle I2C errors.

- **Extensibility:** Support graphics drawing.

Project 99: Timer for Precise Delay for Sensor Sampling

Description: Use a timer to create a precise delay for sensor sampling.

Requirements:

- Sample sensor every 10ms.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer for 10ms interrupts.
- **Sampling:** Read sensor in ISR.

Complexity

- **Time Complexity:** $O(1)$ per sample.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_timer
    Set timer period for 10ms
    Enable timer interrupt
FUNCTION timer_interrupt
    value = read_sensor()
    process_value(value)
MAIN
    init_timer()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define SAMPLE_PERIOD 10000 // 10ms in us

void configure_timer(uint32_t period) { /* Timer config */ }

uint16_t read_sensor(void) { return 0; } // Stub
void process_value(uint16_t value) { /* Process value */ }

void init_timer(void) {
    configure_timer(SAMPLE_PERIOD * (CLOCK_FREQ / 1000000));
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    process_value(value);
}

int main(void) {
    init_timer();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Use high-priority interrupt.
- **Testing:** Verify sampling rate.
- **Optimization:** Use DMA for sensor reads.
- **Robustness:** Handle missed interrupts.
- **Extensibility:** Support variable periods.

Project 100: ADC to Read Battery Voltage with Calibration

Description: Configure an ADC to read a battery voltage with calibration.

Requirements:

- Read 12-bit ADC, convert to voltage (0–3.3V).
- Apply calibration offset/gain.
- Assumptions: Voltage divider used.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for battery voltage.
- **Calibration:** Apply gain and offset: $V = \text{ADC} * \text{gain} + \text{offset}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_battery
    adc_value = read_adc()
    voltage = adc_value * gain + offset
    RETURN voltage
MAIN
    init_adc()
    WHILE true:
        voltage = read_battery()
        process_voltage(voltage)
```

C Code Implementation

```
#include <stdint.h>

#define GAIN 0.000805664f // 3.3V / 4095

#define OFFSET 0.0f // Calibration offset

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_voltage(float voltage) { /* Process voltage */ }

void init_adc(void) {
    configure_adc();
}
```

```

float read_battery(void) {
    uint16_t adc_value = read_adc();
    return adc_value * GAIN + OFFSET;
}

int main(void) {
    init_adc();
    while (1) {
        float voltage = read_battery();
        process_voltage(voltage);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Measure gain/offset with a multimeter.
- **Testing:** Verify with known voltages.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple channels.

Project 101: Control Relay Based on Temperature Sensor Reading

Description: Create a project to control a relay based on a temperature sensor reading.

Requirements:

- Turn relay on if temperature > 30°C, off if < 28°C.
- Assumptions: ADC-based sensor, GPIO for relay.

Ways to Solve

Algorithm Used

- **Temperature Reading:** Read ADC, convert to temperature.
- **Hysteresis Control:** Use thresholds to control relay.

Complexity

- **Time Complexity:** O(1) per check.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_system
    Configure ADC
    Configure GPIO for relay
FUNCTION control_relay
    temp = read_temperature()
    IF temp > 30:
        Set relay on
    ELSE IF temp < 28:
        Set relay off
MAIN
    init_system()
    WHILE true:
        control_relay()

```

C Code Implementation

```
#include <stdint.h>

#define TEMP_ON 30.0f
#define TEMP_OFF 28.0f

void configure_adc(void) { /* ADC config */ }

void set_relay(uint8_t state) { /* Set relay */ }

float read_temperature(void) { return 0.0f; } // Stub

void init_system(void) {
    configure_adc();
    set_relay(0);
}

void control_relay(void) {
    float temp = read_temperature();
    if (temp > TEMP_ON) set_relay(1);
    else if (temp < TEMP_OFF) set_relay(0);
}

int main(void) {
    init_system();
    while (1) {
        control_relay();
    }
    return 0;
}
```

Expert Tips

- **Hysteresis:** Prevents relay chatter.
- **Testing:** Simulate temperature changes.
- **Optimization:** Use timer for periodic checks.
- **Robustness:** Handle sensor errors.
- **Extensibility:** Add adjustable thresholds.

Project 102: DAC to Generate a Square Wave

Description: Use a DAC to generate a square wave.

Requirements:

- Output 12-bit square wave at 1 kHz.
- Assumptions: Timer-driven updates.

Ways to Solve

Algorithm Used

- **State Toggle:** Alternate between high (4095) and low (0) values.
- **Timer Trigger:** Update DAC every 0.5ms for 1 kHz.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_dac
    Configure DAC
    Configure timer for 2 kHz (toggle every 0.5ms)
FUNCTION timer_interrupt
    state = !state
    write_dac(state ? 4095 : 0)
MAIN
    init_dac()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

void configure_dac(void) { /* DAC config */ }

void configure_timer(void) { /* Timer config for 2 kHz */ }

void write_dac(uint16_t value) { /* Write to DAC */ }

void init_dac(void) {
    configure_dac();
    configure_timer();
}

void timer_isr(void) {
    static uint8_t state = 0;
    state = !state;
    write_dac(state ? 4095 : 0);
}

int main(void) {
    init_dac();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}
```

Expert Tips

- **Testing:** Verify waveform with oscilloscope.
- **Optimization:** Use DMA for updates.
- **Robustness:** Handle DAC errors.
- **Extensibility:** Support variable frequency.
- **Accuracy:** Ensure timer precision.

Project 103: Timer to Trigger ADC Conversion Periodically

Description: Configure a timer to trigger an ADC conversion periodically.

Requirements:

- Trigger ADC every 10ms.
- Assumptions: 12-bit ADC, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Trigger:** Configure timer to trigger ADC at 100 Hz.
- **ADC Handling:** Store conversion result in ISR.

Complexity

- **Time Complexity:** $O(1)$ per conversion.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure timer for 100 Hz
    Configure ADC with timer trigger
FUNCTION adc_interrupt
    value = read_adc()
    process_value(value)
MAIN
    init_system()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define TRIGGER_FREQ 100 // 100 Hz

void configure_timer(uint32_t period) { /* Timer config */ }

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_value(uint16_t value) { /* Process value */ }

void init_system(void) {
    configure_timer(CLOCK_FREQ / TRIGGER_FREQ);
    configure_adc();
}

void adc_isr(void) {
    uint16_t value = read_adc();
    process_value(value);
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify trigger frequency.
- **Testing:** Check ADC values with known inputs.
- **Optimization:** Use DMA for conversions.
- **Robustness:** Handle ADC overruns.

- **Extensibility:** Support multiple channels.

Project 104: Read Gyroscope Using I2C and Calculate Orientation

Description: Implement a project to read a gyroscope (e.g., MPU6050) using I2C and calculate orientation.

Requirements:

- Read 3-axis gyro data (int16_t), compute angle via integration.
- Assumptions: I2C interface, 100 Hz sampling.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read 6 bytes for gyro data.
- **Integration:** Sum angular rates to estimate angle (simple Euler integration).

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_gyro
    Configure I2C
    Write gyro config
FUNCTION read_gyro(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
FUNCTION update_angle(data)
    angle += data.z * dt
MAIN
    init_gyro()
    WHILE true:
        read_gyro(&data)
        update_angle(data)

```

C Code Implementation

```

#include <stdint.h>

#define DT 0.01f // 10ms
typedef struct {
    int16_t x, y, z;
} GyroData;

float angle = 0;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_angle(float angle) { /* Process angle */ }

void init_gyro(void) {
    configure_i2c();
    i2c_write(0x68, 0x6B, 0x00); // Wake up MPU6050
}

```

```

void read_gyro(GyroData* data) {
    uint8_t buffer[6];
    i2c_read(0x68, 0x43, buffer, 6); // Read gyro registers
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

void update_angle(GyroData* data) {
    angle += data->z * DT; // Simple integration
}

int main(void) {
    init_gyro();
    GyroData data;
    while (1) {
        read_gyro(&data);
        update_angle(&data);
        process_angle(angle);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Remove gyro bias.
- **Testing:** Verify with known rotations.
- **Optimization:** Use complementary filter with accelerometer.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add accelerometer fusion.

Project 105: Control Buzzer with Melody Using Timer

Description: Implement a project to control a buzzer with a melody using a timer.

Requirements:

- Play a simple melody (e.g., 3 notes).
- Assumptions: PWM-driven buzzer, 16-bit timer.

Ways to Solve

Algorithm Used

- **PWM Control:** Use timer PWM to generate note frequencies.
- **Melody Playback:** Sequence through note array with durations.

Complexity

- **Time Complexity:** $O(1)$ per note change.
- **Space Complexity:** $O(n)$ for melody array.

Pseudocode

```

CONST melody[] = {{freq, duration}, ...}
FUNCTION init_buzzer
    Configure timer for PWM
FUNCTION play_note(freq, duration)
    Set PWM frequency to freq
    Delay for duration
MAIN
    init_buzzer()
    FOR each note in melody:

```


C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t freq; // Hz
    uint16_t duration; // ms
} Note;

const Note melody[] = {{261, 500}, {329, 500}, {392, 500}}; // C4, E4, G4

#define MELODY_SIZE 3

void configure_pwm(uint16_t freq) { /* PWM config */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_buzzer(void) {
    configure_pwm(0);
}

void play_note(uint16_t freq, uint16_t duration) {
    configure_pwm(freq);
    delay_ms(duration);
    configure_pwm(0); // Silence
}

int main(void) {
    init_buzzer();
    while (1) {
        for (uint8_t i = 0; i < MELODY_SIZE; i++) {
            play_note(melody[i].freq, melody[i].duration);
            delay_ms(50); // Gap between notes
        }
    }
    return 0;
}
```

Expert Tips

- **Precision:** Use high-resolution timer.
- **Testing:** Verify notes with an audio analyzer.
- **Optimization:** Precompute PWM settings.
- **Robustness:** Handle invalid frequencies.
- **Extensibility:** Support complex melodies.

Project 106: GPIO Pin for Proximity Sensor with Interrupt

Description: Configure a GPIO pin to handle a proximity sensor with interrupt.

Requirements:

- Detect proximity events (active low) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on falling edge.
- **Debouncing:** Use software delay for noise filtering.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_proximity
    Configure GPIO as input with pull-up
    Enable interrupt on falling edge
FUNCTION proximity_interrupt
    IF pin low:
        Debounce
        IF still low:
            process_proximity()
MAIN
    init_proximity()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_proximity(void) { /* Process proximity */ }

void init_proximity(void) {
    configure_gpio(0, 1); // Input with pull-up
}

void proximity_isr(void) {
    if (!read_pin(0)) {
        delay_ms(20); // Debounce
        if (!read_pin(0)) process_proximity();
    }
}

int main(void) {
    init_proximity();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate proximity events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple sensors.

Project 107: Interface with Color Sensor Using I2C

Description: Create a project to interface with a color sensor (e.g., TCS34725) using I2C.

Requirements:

- Read RGB data (uint16_t for each channel).
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read RGB registers.
- **Data Parsing:** Combine bytes into uint16_t values.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_color_sensor
    Configure I2C
    Write sensor config
FUNCTION read_color(data)
    Read 6 bytes for RGB
    data.r = (byte0 << 8) | byte1
    data.g = (byte2 << 8) | byte3
    data.b = (byte4 << 8) | byte5
MAIN
    init_color_sensor()
    WHILE true:
        read_color(&data)
        process_color(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t r, g, b;
} ColorData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_color(ColorData* data) { /* Process color */ }

void init_color_sensor(void) {
    configure_i2c();
    i2c_write(0x29, 0x00, 0x03); // Enable TCS34725
}

void read_color(ColorData* data) {
    uint8_t buffer[6];
    i2c_read(0x29, 0x14, buffer, 6); // Read RGB registers
    data->r = (buffer[0] << 8) | buffer[1];
    data->g = (buffer[2] << 8) | buffer[3];
    data->b = (buffer[4] << 8) | buffer[5];
}
```

```

int main(void) {
    init_color_sensor();
    ColorData data;
    while (1) {
        read_color(&data);
        process_color(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Adjust for ambient light.
- **Testing:** Verify with known colors.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add clear channel support.

Project 108: Timer to Control Fan with Variable Speed

Description: Use a timer to control a fan with variable speed using PWM.

Requirements:

- Generate 25 kHz PWM with 0–100% duty cycle.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer for 25 kHz PWM.
- **Duty Cycle Update:** Adjust compare value for speed.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_fan
    Configure timer for 25 kHz PWM
    Set initial duty cycle
FUNCTION set_fan_speed(duty)
    Set PWM compare to duty * period / 100
MAIN
    init_fan()
    set_fan_speed(50)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 25000     // 25 kHz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

```

```

void init_fan(void) {
    configure_pwm(PERIOD, 0); // 0% duty
}

void set_fan_speed(uint8_t duty) {
    if (duty > 100) duty = 100;
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_fan();
    set_fan_speed(50); // 50% speed
    while (1) {
        /* Run fan */
    }
    return 0;
}

```

Expert Tips

- **Validation:** Check duty cycle range.
- **Testing:** Verify fan speed with tachometer.
- **Optimization:** Use hardware PWM.
- **Robustness:** Handle timer errors.
- **Extensibility:** Add feedback control.

Project 109: ADC to Read Light Sensor with Averaging

Description: Configure an ADC to read a light sensor with averaging to reduce noise.

Requirements:

- Read 12-bit ADC, average 8 samples.
- Assumptions: Single channel, continuous mode.

Ways to Solve

Algorithm Used

- **Averaging:** Sum 8 ADC samples, divide by 8.
- **ADC Configuration:** Continuous mode for light sensor.

Complexity

- **Time Complexity:** $O(n)$ for n samples ($n=8$).
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_adc
    Configure ADC for continuous mode
FUNCTION read_light
    sum = 0
    FOR i = 0 to 7:
        sum += read_adc()
    RETURN sum / 8
MAIN
    init_adc()
    WHILE true:
        value = read_light()
        process_value(value)

```

C Code Implementation

```
#include <stdint.h>

#define SAMPLE_COUNT 8

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_value(uint16_t value) { /* Process value */ }

void init_adc(void) {
    configure_adc();
}

uint16_t read_light(void) {
    uint32_t sum = 0;
    for (uint8_t i = 0; i < SAMPLE_COUNT; i++) {
        sum += read_adc();
    }
    return sum / SAMPLE_COUNT;
}

int main(void) {
    init_adc();
    while (1) {
        uint16_t value = read_light();
        process_value(value);
    }
    return 0;
}
```

Expert Tips

- **Noise Reduction:** Increase sample count for better averaging.
- **Testing:** Verify with known light levels.
- **Optimization:** Use DMA for sampling.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple channels.

Project 110: Control 4-Digit 7-Segment Display Using GPIO

Description: Create a project to control a 4-digit 7-segment display using GPIO.

Requirements:

- Display numbers 0–9999.
- Assumptions: Common-anode, multiplexed display.

Ways to Solve

Algorithm Used

- **Multiplexing:** Activate one digit at a time, set segment patterns.
- **GPIO Control:** Use GPIO for segments and digit select.

Complexity

- **Time Complexity:** O(1) per update.
- **Space Complexity:** O(1) (fixed segment table).

Pseudocode

```
CONST segment_table[10] = {patterns for 0-9}
FUNCTION init_display
    Configure GPIO for segments and digits
FUNCTION display_number(num)
    digits = split num into 4 digits
    FOR each digit:
        Set all digits off
        Set segment pins for digit
        Set current digit on
        Delay for scan period
MAIN
    init_display()
    WHILE true:
        display_number(1234)
```

C Code Implementation

```
#include <stdint.h>

#define DIGIT_COUNT 4

const uint8_t segment_table[10] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F // 0-9
};

void set_segment(uint8_t pattern) { /* Set segment pins */ }

void set_digit(uint8_t digit, uint8_t state) { /* Set digit pin */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_display(void) {
    // Configure GPIO for segments and digits
    for (uint8_t i = 0; i < DIGIT_COUNT; i++) set_digit(i, 0); // Off
}

void display_number(uint16_t num) {
    uint8_t digits[4];
    digits[0] = num / 1000;
    digits[1] = (num / 100) % 10;
    digits[2] = (num / 10) % 10;
    digits[3] = num % 10;
    for (uint8_t i = 0; i < DIGIT_COUNT; i++) {
        for (uint8_t j = 0; j < DIGIT_COUNT; j++) set_digit(j, 0); // All off
        set_segment(segment_table[digits[i]]);
        set_digit(i, 1); // Active high
        delay_ms(5); // Scan period
    }
}

int main(void) {
    init_display();
    while (1) {
        display_number(1234);
    }
    return 0;
}
```

Expert Tips

- **Multiplexing:** Use timer for precise scan timing.
- **Testing:** Verify digits with test numbers.
- **Optimization:** Reduce scan time to minimize flicker.

- **Robustness:** Handle invalid numbers.
- **Extensibility:** Support decimal points.

Project 111: Read Humidity Sensor Using SPI

Description: Implement a project to read a humidity sensor (e.g., HDC1080) using SPI.

Requirements:

- Read humidity data as `uint16_t`.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read humidity register.
- **Data Parsing:** Combine bytes into `uint16_t` humidity value.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_spi
    Configure SPI (4-wire, master)
FUNCTION read_humidity
    Send read command
    Read 2 bytes
    humidity = (byte0 << 8) | byte1
    RETURN humidity
MAIN
    init_spi()
    WHILE true:
        humidity = read_humidity()
        process_humidity(humidity)
  
```

C Code Implementation

```

#include <stdint.h>
void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_humidity(uint16_t humidity) { /* Process data */ }

void init_spi(void) {
    configure_spi();
    spi_write(0x02, 0x10); // Configure HDC1080
}

uint16_t read_humidity(void) {
    uint8_t buffer[2];
    spi_read(0x00, buffer, 2); // Read humidity register
    return (buffer[0] << 8) | buffer[1];
}
  
```



```

int main(void) {
    init_spi();
    while (1) {
        uint16_t humidity = read_humidity();
        process_humidity(humidity);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Apply sensor-specific scaling.
- **Testing:** Verify with known humidity levels.
- **Optimization:** Use burst reads for efficiency.
- **Robustness:** Handle SPI communication errors.
- **Extensibility:** Add temperature reading.

Project 112: Timer for 10 kHz PWM Signal for Motor

Description: Configure a timer to generate a 10 kHz PWM signal for motor control.

Requirements:

- Generate 10 kHz PWM with adjustable duty cycle (0–100%).
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer period for 10 kHz, adjust compare value for duty cycle.
- **Dynamic Update:** Update compare register for duty cycle changes.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_pwm
    Configure timer for 10 kHz PWM
    Set initial duty cycle
FUNCTION set_pwm_duty(duty)
    Set compare register to duty * period / 100
MAIN
    init_pwm()
    set_pwm_duty(50)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 10000     // 10 kHz
#define PERIOD (CLOCK_FREQ / PWM_FREQ)
void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

void init_pwm(void) {
    configure_pwm(PERIOD, PERIOD / 2); // 50% duty
}

```

```

void set_pwm_duty(uint8_t duty) {
    if (duty > 100) duty = 100;
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_pwm();
    set_pwm_duty(50); // 50% duty
    while (1) {
        /* Run PWM */
    }
    return 0;
}

```

Expert Tips

- **Validation:** Ensure duty cycle is within range.
- **Testing:** Verify PWM with an oscilloscope.
- **Optimization:** Use hardware PWM channels.
- **Robustness:** Handle timer initialization errors.
- **Extensibility:** Support multiple PWM channels.

Project 113: Interface with Real-Time Clock (RTC) Using I2C

Description: Create a project to interface with a real-time clock (e.g., DS3231) using I2C.

Requirements:

- Read date/time as a struct (year, month, day, hour, minute, second).
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read RTC registers.
- **Data Parsing:** Convert BCD data to decimal for date/time.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

STRUCT Time { year, month, day, hour, minute, second }
FUNCTION init_rtc
    Configure I2C
    Write RTC config
FUNCTION read_rtc(time)
    Read 7 bytes from RTC
    Convert BCD to decimal for each field
    Store in time struct
MAIN
    init_rtc()
    WHILE true:
        read_rtc(&time)
        process_time(time)

```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint8_t second, minute, hour, day, month, year;
} Time;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_time(Time* time) { /* Process time */ }

uint8_t bcd_to_dec(uint8_t bcd) {
    return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

void init_rtc(void) {
    configure_i2c();
    i2c_write(0x68, 0x0E, 0x00); // Enable DS3231
}

void read_rtc(Time* time) {
    uint8_t buffer[7];
    i2c_read(0x68, 0x00, buffer, 7); // Read time registers
    time->second = bcd_to_dec(buffer[0]);
    time->minute = bcd_to_dec(buffer[1]);
    time->hour = bcd_to_dec(buffer[2]);
    time->day = bcd_to_dec(buffer[4]);
    time->month = bcd_to_dec(buffer[5]);
    time->year = bcd_to_dec(buffer[6]);
}

int main(void) {
    init_rtc();
    Time time;
    while (1) {
        read_rtc(&time);
        process_time(&time);
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify RTC clock drift.
- **Testing:** Check with known time values.
- **Optimization:** Cache time for efficiency.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support alarm functions.

Project 114: DAC to Generate a Sawtooth Wave

Description: Use a DAC to generate a sawtooth wave.

Requirements:

- Output 12-bit sawtooth wave at 1 kHz.
- Assumptions: Timer-driven updates, 32 samples/cycle.

Ways to Solve

Algorithm Used

- **Linear Increment:** Increment DAC value linearly, reset at max.
- **Timer Trigger:** Update DAC at 32 kHz for 1 kHz wave.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_dac
    Configure DAC
    Configure timer for 32 kHz
FUNCTION timer_interrupt
    value += 4095 / 32
    IF value >= 4095:
        value = 0
    write_dac(value)
MAIN
    init_dac()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define SAMPLES_PER_CYCLE 32

void configure_dac(void) { /* DAC config */ }

void configure_timer(void) { /* Timer config for 32 kHz */ }

void write_dac(uint16_t value) { /* Write to DAC */ }

void init_dac(void) {
    configure_dac();
    configure_timer();
}

void timer_isr(void) {
    static uint16_t value = 0;
    value += 4095 / SAMPLES_PER_CYCLE;
    if (value >= 4095) value = 0;
    write_dac(value);
}

int main(void) {
    init_dac();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}
```

Expert Tips

- **Smoothness:** Increase samples for better resolution.
- **Testing:** Verify waveform with oscilloscope.
- **Optimization:** Use DMA for DAC updates.
- **Robustness:** Handle DAC errors.

- **Extensibility:** Support variable frequencies.

Project 115: GPIO Pin for Vibration Sensor with Interrupt

Description: Configure a GPIO pin to handle a vibration sensor with interrupt.

Requirements:

- Detect vibration events (active low) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on falling edge.
- **Debouncing:** Use software delay to filter noise.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_vibration
    Configure GPIO as input with pull-up
    Enable interrupt on falling edge
FUNCTION vibration_interrupt
    IF pin low:
        Debounce
        IF still low:
            process_vibration()
MAIN
    init_vibration()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_vibration(void) { /* Process vibration */ }

void init_vibration(void) {
    configure_gpio(0, 1); // Input with pull-up
}

void vibration_isr(void) {
    if (!read_pin(0)) {
        delay_ms(20); // Debounce
        if (!read_pin(0)) process_vibration();
    }
}
```

```
int main(void) {
    init_vibration();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate vibration events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple sensors.

Project 116: Control Stepper Motor with Microstepping

Description: Create a project to control a stepper motor with microstepping for smoother motion.

Requirements:

- Implement 1/4 microstepping for a 4-phase stepper.
- Assumptions: PWM-driven phases, 200 steps/rev.

Ways to Solve

Algorithm Used

- **Microstepping:** Use PWM to control phase currents for fractional steps.
- **State Machine:** Sequence through microstep table.

Complexity

- **Time Complexity:** $O(1)$ per microstep.
- **Space Complexity:** $O(1)$ (fixed microstep table).

Pseudocode

```
CONST microstep_table[] = {current values for phases}
FUNCTION init_stepper
    Configure PWM for phases
    Configure timer for stepping
FUNCTION step_motor
    index = (index + 1) % table_size
    Set PWM duties from microstep_table[index]
MAIN
    init_stepper()
    WHILE true:
        step_motor()
```

C Code Implementation

```
#include <stdint.h>

#define MICROSTEPS 4

#define TABLE_SIZE (4 * MICROSTEPS)

const uint8_t microstep_table[TABLE_SIZE][4] = {
    {255, 0, 0, 0}, {180, 180, 0, 0}, {0, 255, 0, 0}, {0, 180, 180, 0},
    {0, 0, 255, 0}, {0, 0, 180, 180}, {0, 0, 0, 255}, {180, 0, 0, 180}
};
```

```

void configure_pwm(uint8_t phase, uint8_t duty) { /* PWM config */ }

void init_stepper(void) {
    for (uint8_t i = 0; i < 4; i++) configure_pwm(i, 0);
}

void step_motor(void) {
    static uint8_t index = 0;
    for (uint8_t i = 0; i < 4; i++) {
        configure_pwm(i, microstep_table[index][i]);
    }
    index = (index + 1) % TABLE_SIZE;
}

int main(void) {
    init_stepper();
    while (1) {
        step_motor();
    }
    return 0;
}

```

Expert Tips

- **Precision:** Use high-resolution PWM.
- **Testing:** Verify smoothness with encoder.
- **Optimization:** Precompute table values.
- **Robustness:** Handle motor stall.
- **Extensibility:** Support variable microstepping.

Project 117: Read Magnetometer Using I2C

Description: Implement a project to read a magnetometer (e.g., HMC5883L) using I2C.

Requirements:

- Read 3-axis data (x, y, z as int16_t).
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read magnetometer registers.
- **Data Parsing:** Combine bytes into int16_t for each axis.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_magnetometer
    Configure I2C
    Write magnetometer config
FUNCTION read_magnetometer(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
MAIN
    init_magnetometer()

```

```
WHILE true:
    read_magnetometer(&data)
    process_data(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    int16_t x, y, z;
} MagData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(MagData* data) { /* Process data */ }

void init_magnetometer(void) {
    configure_i2c();
    i2c_write(0x1E, 0x00, 0x70); // Configure HMC5883L
}

void read_magnetometer(MagData* data) {
    uint8_t buffer[6];
    i2c_read(0x1E, 0x03, buffer, 6); // Read mag registers
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_magnetometer();
    MagData data;
    while (1) {
        read_magnetometer(&data);
        process_data(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply hard/soft iron corrections.
- **Testing:** Verify with known magnetic fields.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add heading calculation.

Project 118: ADC to Read Current Sensor with Scaling

Description: Configure an ADC to read a current sensor with scaling.

Requirements:

- Read 12-bit ADC, scale to current (0–10A).
- Assumptions: Linear scaling, voltage-based sensor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for sensor voltage.
- **Scaling:** Apply formula: $I = \text{ADC} * \text{scale_factor}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_current
    adc_value = read_adc()
    current = adc_value * scale_factor
    RETURN current
MAIN
    init_adc()
    WHILE true:
        current = read_current()
        process_current(current)
```

C Code Implementation

```
#include <stdint.h>

#define SCALE_FACTOR (10.0f / 4095.0f) // 0-10A for 12-bit ADC

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_current(float current) { /* Process current */ }

void init_adc(void) {
    configure_adc();
}

float read_current(void) {
    uint16_t adc_value = read_adc();
    return adc_value * SCALE_FACTOR;
}

int main(void) {
    init_adc();
    while (1) {
        float current = read_current();
        process_current(current);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Measure scale factor with a multimeter.
- **Testing:** Verify with known currents.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple sensors.

Project 119: Interface with Graphic LCD Using SPI

Description: Create a project to interface with a graphic LCD (e.g., ST7735) using SPI.

Requirements:

- Display text on a 128x160 LCD.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, send initialization and text data.
- **Display Update:** Write pixel data to LCD buffer.

Complexity

- **Time Complexity:** $O(n)$ for n-pixel update.
- **Space Complexity:** $O(n)$ for display buffer.

Pseudocode

```
FUNCTION init_lcd
    Configure SPI
    Send LCD init commands
FUNCTION write_text(str, x, y)
    Convert str to pixel data
    Send data via SPI

MAIN
    init_lcd()
    write_text("Hello", 0, 0)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void set_dc(uint8_t state) { /* Set DC pin */ }

void init_lcd(void) {
    configure_spi();
    uint8_t init_cmds[] = {0x01, 0x11, 0x29}; // Simplified init
    set_dc(0); // Command mode
    spi_write(init_cmds, sizeof(init_cmds));
}

void write_text(const char* str, uint8_t x, uint8_t y) {
    uint8_t buffer[128];
    // Simplified: assumes font table
    for (uint8_t i = 0; str[i] && i < 128; i++) {
        buffer[i] = str[i]; // Placeholder for font conversion
    }
    set_dc(1); // Data mode
    spi_write(buffer, strlen(str));
}

int main(void) {
    init_lcd();
    write_text("Hello", 0, 0);
}
```

```

while (1) {
    /* Keep display active */
}
return 0;
}

```

Expert Tips

- **Font:** Use a font table for rendering.
- **Testing:** Verify with test patterns.
- **Optimization:** Buffer updates locally.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support graphics drawing.

Project 120: Timer to Generate a Pulse Train

Description: Use a timer to generate a pulse train.

Requirements:

- Generate 10 pulses at 1 kHz, 50% duty cycle.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer for 1 kHz PWM, toggle for pulses.
- **Pulse Counting:** Stop after 10 pulses.

Complexity

- **Time Complexity:** $O(1)$ per pulse.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_timer
    Configure timer for 1 kHz PWM
    Set pulse count to 0
FUNCTION timer_interrupt
    IF pulse_count < 10:
        Toggle output
        pulse_count++
    ELSE:
        Stop timer
MAIN
    init_timer()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PULSE_FREQ 1000    // 1 kHz
#define PULSE_COUNT 10

void configure_timer(uint16_t period) { /* Timer config */ }

void toggle_output(void) { /* Toggle GPIO */ }

```

```

void stop_timer(void) { /* Stop timer */ }

void init_timer(void) {
    configure_timer(CLOCK_FREQ / PULSE_FREQ);
}

void timer_isr(void) {
    static uint8_t pulse_count = 0;
    if (pulse_count < PULSE_COUNT) {
        toggle_output();
        pulse_count++;
    } else {
        stop_timer();
    }
}

int main(void) {
    init_timer();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Verify pulse timing.
- **Testing:** Count pulses with a scope.
- **Optimization:** Use hardware toggle mode.
- **Robustness:** Handle timer errors.
- **Extensibility:** Support variable pulse counts.

Project 121: GPIO Pin for Reed Switch with Interrupt

Description: Configure a GPIO pin to handle a reed switch with interrupt.

Requirements:

- Detect reed switch events (active low) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on falling edge.
- **Debouncing:** Use software delay to filter noise.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_reed
    Configure GPIO as input with pull-up
    Enable interrupt on falling edge
FUNCTION reed_interrupt
    IF pin low:
        Debounce
        IF still low:
            process_reed()

```

```
MAIN
    init_reed()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_reed(void) { /* Process reed switch */ }

void init_reed(void) {
    configure_gpio(0, 1); // Input with pull-up
}

void reed_isr(void) {
    if (!read_pin(0)) {
        delay_ms(20); // Debounce
        if (!read_pin(0)) process_reed();
    }
}

int main(void) {
    init_reed();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate switch events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple switches.

Project 122: Control Servo Motor with Smooth Motion

Description: Create a project to control a servo motor with smooth motion.

Requirements:

- Move servo from 0–180° in 1° steps over 1s.
- Assumptions: 50 Hz PWM, 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set 50 Hz PWM, adjust pulse width for angle.
- **Smooth Transition:** Increment angle in small steps with delay.

Complexity

- **Time Complexity:** O(n) for n steps (180 steps).

- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_servo
    Configure timer for 50 Hz PWM
FUNCTION move_servo(start, end)
    FOR angle = start to end:
        pulse_us = 1000 + (angle * 1000 / 180)
        Set PWM compare to pulse_us
        Delay 5ms
MAIN
    init_servo()
    move_servo(0, 180)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 50 // 50 Hz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t pulse) { /* PWM config */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_servo(void) {
    configure_pwm(PERIOD, 1500 * (CLOCK_FREQ / 1000000)); // 1.5ms
}

void move_servo(uint8_t start, uint8_t end) {
    int8_t step = (end >= start) ? 1 : -1;
    for (uint8_t angle = start; angle != end + step; angle += step) {
        uint16_t pulse_us = 1000 + (angle * 1000 / 180);
        configure_pwm(PERIOD, pulse_us * (CLOCK_FREQ / 1000000));
        delay_ms(5); // ~180 steps over 1s
    }
}

int main(void) {
    init_servo();
    move_servo(0, 180);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Smoothness:** Adjust step size and delay.
- **Testing:** Verify motion with a protractor.
- **Optimization:** Use timer for step timing.
- **Robustness:** Validate angle range.
- **Extensibility:** Support multiple servos.

Project 123: Read Temperature Sensor with Error Checking

Description: Implement a project to read a temperature sensor with error checking.

Requirements:

- Read ADC-based sensor, check for out-of-range values.
- Assumptions: 12-bit ADC, temperature range 0–100°C.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for temperature.
- **Error Checking:** Validate against range, return error if invalid.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_temperature
    adc_value = read_adc()
    IF adc_value out of range:
        RETURN error
    temp = adc_value * scale
    RETURN temp
MAIN
    init_adc()
    WHILE true:
        temp = read_temperature()
        IF no error:
            process_temperature(temp)
```

C Code Implementation

```
#include <stdint.h>

#define SCALE 0.0244f // 100°C / 4095
#define MIN_ADC 0
#define MAX_ADC 4095

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_temperature(float temp) { /* Process temp */ }

void init_adc(void) {
    configure_adc();
}

int8_t read_temperature(float* temp) {
    uint16_t adc_value = read_adc();
    if (adc_value < MIN_ADC || adc_value > MAX_ADC) return -1;
    *temp = adc_value * SCALE;
    return 0;
}
```

```

int main(void) {
    init_adc();
    float temp;
    while (1) {
        if (read_temperature(&temp) == 0) {
            process_temperature(temp);
        }
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Adjust scale per sensor.
- **Testing:** Verify with known temperatures.
- **Optimization:** Use lookup table for non-linear sensors.
- **Robustness:** Log error conditions.
- **Extensibility:** Support multiple sensors.

Project 124: ADC to Read Force Sensor with Calibration

Description: Configure an ADC to read a force sensor with calibration.

Requirements:

- Read 12-bit ADC, scale to force (0–100N).
- Assumptions: Linear scaling, voltage-based sensor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for sensor voltage.
- **Calibration:** Apply scale factor: $F = \text{ADC} * \text{scale_factor}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_force
    adc_value = read_adc()
    force = adc_value * scale_factor
    RETURN force
MAIN
    init_adc()
    WHILE true:
        force = read_force()
        process_force(force)

```

C Code Implementation

```

#include <stdint.h>

#define SCALE_FACTOR (100.0f / 4095.0f) // 0-100N for 12-bit ADC

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

```



```

void process_force(float force) { /* Process force */ }

void init_adc(void) {
    configure_adc();
}

float read_force(void) {
    uint16_t adc_value = read_adc();
    return adc_value * SCALE_FACTOR;
}

int main(void) {
    init_adc();
    while (1) {
        float force = read_force();
        process_force(force);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Measure scale factor with known forces.
- **Testing:** Verify with known weights.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple sensors.

Project 125: Interface with Pressure Sensor Using I2C

Description: Create a project to interface with a pressure sensor (e.g., BMP280) using I2C.

Requirements:

- Read pressure data as uint32_t.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read pressure registers.
- **Data Parsing:** Combine bytes into uint32_t pressure.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_pressure_sensor
    Configure I2C
    Write sensor config
FUNCTION read_pressure
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    RETURN pressure

MAIN
    init_pressure_sensor()
    WHILE true:
        pressure = read_pressure()

```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_pressure(uint32_t pressure) { /* Process pressure */ }

void init_pressure_sensor(void) {
    configure_i2c();
    i2c_write(0x76, 0xF4, 0x27); // Configure BMP280
}

uint32_t read_pressure(void) {
    uint8_t buffer[3];
    i2c_read(0x76, 0xF7, buffer, 3); // Read pressure registers
    return ((uint32_t)buffer[0] << 16) | ((uint32_t)buffer[1] << 8) | buffer[2];
}

int main(void) {
    init_pressure_sensor();
    while (1) {
        uint32_t pressure = read_pressure();
        process_pressure(pressure);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply sensor-specific compensation.
- **Testing:** Verify with known pressures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add temperature reading.

Project 126: Timer to Control PWM-Based Heater

Description: Use a timer to control a PWM-based heater.

Requirements:

- Generate 100 Hz PWM with adjustable duty cycle (0–100%).
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer for 100 Hz PWM.
- **Duty Cycle Update:** Adjust compare value for heat control.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_heater
    Configure timer for 100 Hz PWM
    Set initial duty cycle
FUNCTION set_heater_duty(duty)
    Set compare register to duty * period / 100
MAIN
    init_heater()
    set_heater_duty(50)
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 100      // 100 Hz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

void init_heater(void) {
    configure_pwm(PERIOD, PERIOD / 2); // 50% duty
}

void set_heater_duty(uint8_t duty) {
    if (duty > 100) duty = 100;
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_heater();
    set_heater_duty(50); // 50% duty
    while (1) {
        /* Run heater */
    }
    return 0;
}
```

Expert Tips

- **Validation:** Check duty cycle range.
- **Testing:** Verify heat output.
- **Optimization:** Use hardware PWM.
- **Robustness:** Handle timer errors.
- **Extensibility:** Add temperature feedback.

Project 127: GPIO Pin for PIR Motion Sensor with Interrupt

Description: Configure a GPIO pin to handle a PIR motion sensor with interrupt.

Requirements:

- Detect motion events (active high) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on rising edge.

- **Debouncing:** Use software delay to filter noise.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_pir
    Configure GPIO as input
    Enable interrupt on rising edge
FUNCTION pir_interrupt
    IF pin high:
        Debounce
        IF still high:
            process_motion()
MAIN
    init_pir()
    WHILE true:
        /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 0; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_motion(void) { /* Process motion */ }

void init_pir(void) {
    configure_gpio(0, 0); // Input
}

void pir_isr(void) {
    if (read_pin(0)) {
        delay_ms(20); // Debounce
        if (read_pin(0)) process_motion();
    }
}

int main(void) {
    init_pir();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate motion events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple sensors.

Project 128: Control 2x16 LCD with Custom Characters

Description: Create a project to control a 2x16 LCD with custom characters.

Requirements:

- Display text and 2 custom characters on HD44780-compatible LCD.
- Assumptions: 4-bit mode, GPIO control.

Ways to Solve

Algorithm Used

- **Initialization:** Send LCD init commands and custom character data.
- **Data Write:** Use 4-bit mode to display text and custom characters.

Complexity

- **Time Complexity:** $O(n)$ for n -character string.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_lcd
    Configure GPIO for RS, EN, D4-D7
    Send init commands
    Define custom characters
FUNCTION write_char(char)
    Set RS high
    Send high nibble
    Pulse EN
    Send low nibble
    Pulse EN
FUNCTION write_string(str)
    FOR each char in str:
        write_char(char)
MAIN
    init_lcd()
    write_string("Hello")
    write_char(custom_char)
```

C Code Implementation

```
#include <stdint.h>

void set_gpio(uint8_t pin, uint8_t state) { /* Set GPIO */ }

void delay_us(uint16_t us) { /* Delay */ }

void write_nibble(uint8_t nibble, uint8_t rs) {
    set_gpio(0, rs); // RS
    set_gpio(1, 1); // EN high
    for (uint8_t i = 0; i < 4; i++) set_gpio(4 + i, (nibble >> i) & 1);
    delay_us(1);
    set_gpio(1, 0); // EN low
    delay_us(1);
}

void init_lcd(void) {
    delay_us(50000); // Wait for LCD power-up
    uint8_t init_cmds[] = {0x28, 0x0C, 0x01}; // Function set, display on, clear
    for (uint8_t i = 0; i < sizeof(init_cmds); i++) {
        write_nibble(init_cmds[i] >> 4, 0);
        write_nibble(init_cmds[i], 0);
    }
}
```

```

        delay_us(2000);
    }
    // Define custom characters (e.g., smiley, heart)
    uint8_t custom_chars[] = {0x40, 0x00, 0x0A, 0x00, 0x11, 0x0E, 0x00, 0x0A, 0x00, // Smiley
                              0x00, 0x0A, 0x1F, 0x1F, 0x0E, 0x04, 0x00, 0x00}; // Heart
    for (uint8_t i = 0; i < sizeof(custom_chars); i++) {
        write_nibble(custom_chars[i] >> 4, 0);
        write_nibble(custom_chars[i], 0);
        delay_us(40);
    }
}

void write_char(char c) {
    write_nibble(c >> 4, 1);
    write_nibble(c, 1);
    delay_us(40);
}

void write_string(const char* str) {
    while (*str) write_char(*str++);
}

int main(void) {
    init_lcd();
    write_string("Hello ");
    write_char(0); // Smiley
    write_char(1); // Heart
    return 0;
}

```

Expert Tips

- **Timing:** Adhere to LCD datasheet timings.
- **Testing:** Verify custom characters.
- **Optimization:** Use 8-bit mode for speed.
- **Robustness:** Handle initialization failures.
- **Extensibility:** Support more custom characters.

Project 129: Read Gas Sensor Using ADC and Trigger Alarm

Description: Implement a project to read a gas sensor using ADC and trigger an alarm if the reading exceeds a threshold.

Requirements:

- Read 12-bit ADC, trigger alarm if gas level > threshold (e.g., 3000).
- Assumptions: Voltage-based sensor, GPIO for alarm.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for gas sensor voltage.
- **Threshold Check:** Compare reading to threshold, set alarm GPIO if exceeded.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure ADC
    Configure GPIO for alarm
FUNCTION read_gas
    adc_value = read_adc()
    IF adc_value > threshold:
        Set alarm on
    ELSE:
        Set alarm off
    RETURN adc_value
MAIN
    init_system()
    WHILE true:
        gas_level = read_gas()
        process_gas(gas_level)
```

C Code Implementation

```
#include <stdint.h>

#define THRESHOLD 3000

void configure_adc(void) { /* ADC config */ }

void set_alarm(uint8_t state) { /* Set alarm GPIO */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_gas(uint16_t gas_level) { /* Process gas level */ }

void init_system(void) {
    configure_adc();
    set_alarm(0);
}

uint16_t read_gas(void) {
    uint16_t adc_value = read_adc();
    set_alarm(adc_value > THRESHOLD ? 1 : 0);
    return adc_value;
}

int main(void) {
    init_system();
    while (1) {
        uint16_t gas_level = read_gas();
        process_gas(gas_level);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Adjust threshold based on sensor datasheet.
- **Testing:** Verify with known gas levels.
- **Optimization:** Use timer for periodic sampling.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Add hysteresis to alarm.

Project 130: Timer for 500 Hz PWM Signal for Buzzer

Description: Configure a timer to generate a 500 Hz PWM signal for a buzzer.

Requirements:

- Generate 500 Hz PWM with adjustable duty cycle (0–100%).
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer period for 500 Hz, adjust compare value for duty cycle.
- **Dynamic Update:** Update compare register for duty cycle changes.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_pwm
    Configure timer for 500 Hz PWM
    Set initial duty cycle
FUNCTION set_pwm_duty(duty)
    Set compare register to duty * period / 100
MAIN
    init_pwm()
    set_pwm_duty(50)
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 500      // 500 Hz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

void init_pwm(void) {
    configure_pwm(PERIOD, PERIOD / 2); // 50% duty
}

void set_pwm_duty(uint8_t duty) {
    if (duty > 100) duty = 100;
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_pwm();
    set_pwm_duty(50); // 50% duty
    while (1) {
        /* Run PWM */
    }
    return 0;
}
```


Expert Tips

- **Validation:** Check duty cycle range.
- **Testing:** Verify tone with audio analyzer.
- **Optimization:** Use hardware PWM.
- **Robustness:** Handle timer errors.
- **Extensibility:** Support variable frequencies.

Project 131: Interface with UV Sensor Using I2C

Description: Create a project to interface with a UV sensor (e.g., VEML6070) using I2C.

Requirements:

- Read UV index as uint8_t.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read UV register.
- **Data Parsing:** Read single byte for UV index.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```
FUNCTION init_uv_sensor
    Configure I2C
    Write sensor config

FUNCTION read_uv
    Read 1 byte
    RETURN uv_index

MAIN
    init_uv_sensor()
    WHILE true:
        uv_index = read_uv()
        process_uv(uv_index)
```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_uv(uint8_t uv_index) { /* Process UV index */ }

void init_uv_sensor(void) {
    configure_i2c();
    i2c_write(0x38, 0x00, 0x02); // Configure VEML6070
}
```

```

uint8_t read_uv(void) {
    uint8_t uv_index;
    i2c_read(0x39, 0x00, &uv_index, 1); // Read UV register
    return uv_index;
}

int main(void) {
    init_uv_sensor();
    while (1) {
        uint8_t uv_index = read_uv();
        process_uv(uv_index);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Adjust per sensor datasheet.
- **Testing:** Verify with known UV sources.
- **Optimization:** Use burst reads if supported.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add integration time adjustment.

Project 132: Control Motor with Forward/Reverse Using Timer

Description: Implement a project to control a motor with forward/reverse using a timer.

Requirements:

- Control DC motor direction using PWM (1 kHz).
- Assumptions: H-bridge, 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer for 1 kHz PWM, control H-bridge pins.
- **Direction Control:** Set GPIO states for forward/reverse.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_motor
    Configure timer for 1 kHz PWM
    Configure GPIO for H-bridge
FUNCTION set_motor(direction, speed)
    IF direction = forward:
        Set H-bridge for forward
    ELSE:
        Set H-bridge for reverse
    Set PWM duty to speed
MAIN
    init_motor()
    set_motor(FORWARD, 50)

```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 1000      // 1 kHz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

#define FORWARD 1
#define REVERSE 0

void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

void set_hbridge(uint8_t direction) { /* Set H-bridge pins */ }

void init_motor(void) {
    configure_pwm(PERIOD, 0);
    set_hbridge(FORWARD);
}

void set_motor(uint8_t direction, uint8_t speed) {
    if (speed > 100) speed = 100;
    uint16_t compare = (uint32_t)PERIOD * speed / 100;
    set_hbridge(direction);
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_motor();
    set_motor(FORWARD, 50); // 50% speed forward
    while (1) {
        /* Run motor */
    }
    return 0;
}
```

Expert Tips

- **Safety:** Ensure H-bridge shoot-through protection.
- **Testing:** Verify direction and speed.
- **Optimization:** Use hardware PWM.
- **Robustness:** Handle invalid inputs.
- **Extensibility:** Add speed feedback.

Project 133: ADC to Read pH Sensor with Calibration

Description: Configure an ADC to read a pH sensor with calibration.

Requirements:

- Read 12-bit ADC, scale to pH (0–14).
- Assumptions: Linear scaling, voltage-based sensor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for sensor voltage.
- **Calibration:** Apply scale factor: $\text{pH} = \text{ADC} * \text{scale_factor} + \text{offset}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_ph
    adc_value = read_adc()
    ph = adc_value * scale_factor + offset
    RETURN ph
MAIN
    init_adc()
    WHILE true:
        ph = read_ph()
        process_ph(ph)
```

C Code Implementation

```
#include <stdint.h>

#define SCALE_FACTOR (14.0f / 4095.0f) // 0-14 pH

#define OFFSET 0.0f

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_ph(float ph) { /* Process pH */ }

void init_adc(void) {
    configure_adc();
}

float read_ph(void) {
    uint16_t adc_value = read_adc();
    return adc_value * SCALE_FACTOR + OFFSET;
}

int main(void) {
    init_adc();
    while (1) {
        float ph = read_ph();
        process_ph(ph);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Use buffer solutions for accurate scaling.
- **Testing:** Verify with known pH values.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support temperature compensation.

Project 134: Interface with Touch Screen Using SPI

Description: Create a project to interface with a touch screen (e.g., XPT2046) using SPI.

Requirements:

- Read touch coordinates (x, y as uint16_t).
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read touch coordinates.
- **Data Parsing:** Convert raw ADC values to screen coordinates.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_touch
    Configure SPI
    Write touch controller config
FUNCTION read_touch(x, y)
    Send X read command, read 2 bytes
    x = convert raw to screen coordinate
    Send Y read command, read 2 bytes
    y = convert raw to screen coordinate
MAIN
    init_touch()
    WHILE true:
        read_touch(&x, &y)
        process_touch(x, y)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t x, y;
} TouchData;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_touch(TouchData* data) { /* Process touch */ }

void init_touch(void) {
    configure_spi();
    spi_write(0x80, 0x00); // Configure XPT2046
}

void read_touch(TouchData* data) {
    uint8_t buffer[2];
    spi_read(0xD0, buffer, 2); // Read X
    data->x = ((buffer[0] << 8) | buffer[1]) >> 3; // 12-bit
    spi_read(0x90, buffer, 2); // Read Y
    data->y = ((buffer[0] << 8) | buffer[1]) >> 3; }
```

```

int main(void) {
    init_touch();
    TouchData data;
    while (1) {
        read_touch(&data);
        process_touch(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Map raw coordinates to screen.
- **Testing:** Verify with known touch points.
- **Optimization:** Filter noisy readings.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add pressure sensing.

Project 135: DAC to Generate a Custom Waveform

Description: Use a DAC to generate a custom waveform.

Requirements:

- Output 12-bit custom waveform at 1 kHz.
- Assumptions: Timer-driven updates, lookup table.

Ways to Solve

Algorithm Used

- **Lookup Table:** Define custom waveform values.
- **Timer Trigger:** Update DAC at 32 kHz for 1 kHz wave.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(n)$ for lookup table.

Pseudocode

```

CONST custom_table[32] = {custom waveform values}
FUNCTION init_dac
    Configure DAC
    Configure timer for 32 kHz
FUNCTION timer_interrupt
    index = (index + 1) % 32
    write_dac(custom_table[index])
MAIN
    init_dac()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>

#define TABLE_SIZE 32

const uint16_t custom_table[TABLE_SIZE] = {
    0, 512, 1024, 1536, 2048, 2560, 3072, 3584, 4095, 3584, 3072, 2560, 2048, 1536, 1024, 512,
    0, 512, 1024, 1536, 2048, 2560, 3072, 3584, 4095, 3584, 3072, 2560, 2048, 1536, 1024, 512
};

```

```

void configure_dac(void) { /* DAC config */ }

void configure_timer(void) { /* Timer config for 32 kHz */ }

void write_dac(uint16_t value) { /* Write to DAC */ }

void init_dac(void) {
    configure_dac();
    configure_timer();
}

void timer_isr(void) {
    static uint8_t index = 0;
    write_dac(custom_table[index]);
    index = (index + 1) % TABLE_SIZE;
}

int main(void) {
    init_dac();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Flexibility:** Allow runtime waveform updates.
- **Testing:** Verify waveform with oscilloscope.
- **Optimization:** Use DMA for DAC updates.
- **Robustness:** Handle DAC errors.
- **Extensibility:** Support dynamic waveforms.

Project 136: GPIO Pin for Limit Switch with Interrupt

Description: Configure a GPIO pin to handle a limit switch with interrupt.

Requirements:

- Detect limit switch events (active low) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on falling edge.
- **Debouncing:** Use software delay to filter noise.

Complexity

- **Time Complexity:** $O(1)$ per interrupt.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_limit
    Configure GPIO as input with pull-up
    Enable interrupt on falling edge
FUNCTION limit_interrupt
    IF pin low:
        Debounce
        IF still low:

```

```

        process_limit()
MAIN
    init_limit()
    WHILE true:
        /* Wait for interrupts */

```

C Code Implementation

```

#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 1; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_limit(void) { /* Process limit switch */ }

void init_limit(void) {
    configure_gpio(0, 1); // Input with pull-up
}

void limit_isr(void) {
    if (!read_pin(0)) {
        delay_ms(20); // Debounce
        if (!read_pin(0)) process_limit();
    }
}

int main(void) {
    init_limit();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}

```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate switch events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple switches.

Project 137: Control 4x4 LED Matrix with Animation

Description: Create a project to control a 4x4 LED matrix with animation.

Requirements:

- Display a moving pattern (e.g., shifting row).
- Assumptions: Common-anode matrix, row-column scanning.

Ways to Solve

Algorithm Used

- **Row-Column Scanning:** Activate one row, set column states.
- **Animation:** Shift pattern array periodically.

Complexity

- **Time Complexity:** $O(1)$ per frame.
- **Space Complexity:** $O(1)$ (fixed pattern array).

Pseudocode

```
FUNCTION init_matrix
    Configure rows/columns as outputs
FUNCTION display_frame(pattern)
    FOR each row:
        Set all rows low
        Set current row high
        Set columns based on pattern[row]
        Delay for scan period
FUNCTION update_animation(pattern)
    Shift pattern left
MAIN
    init_matrix()
    pattern = initial_pattern
    WHILE true:
        display_frame(pattern)
        update_animation(pattern)
```

C Code Implementation

```
#include <stdint.h>

#define ROW_COUNT 4
#define COL_COUNT 4

void set_row(uint8_t row, uint8_t state) { /* Set row pin */ }

void set_column(uint8_t col, uint8_t state) { /* Set column pin */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_matrix(void) {
    for (uint8_t i = 0; i < ROW_COUNT; i++) set_row(i, 0);
    for (uint8_t i = 0; i < COL_COUNT; i++) set_column(i, 1);
}

void display_frame(uint8_t pattern[ROW_COUNT]) {
    for (uint8_t row = 0; row < ROW_COUNT; row++) {
        for (uint8_t i = 0; i < ROW_COUNT; i++) set_row(i, 0);
        set_row(row, 1);
        for (uint8_t col = 0; col < COL_COUNT; col++) {
            set_column(col, !(pattern[row] & (1 << col)));
        }
        delay_ms(5);
    }
}

void update_animation(uint8_t pattern[ROW_COUNT]) {
    uint8_t temp = pattern[0];
    for (uint8_t i = 0; i < ROW_COUNT - 1; i++) {
        pattern[i] = pattern[i + 1];
    }
    pattern[ROW_COUNT - 1] = temp;
}

int main(void) {
    uint8_t pattern[ROW_COUNT] = {0b1000, 0b0100, 0b0010, 0b0001};
    init_matrix();
    while (1) {
        for (uint8_t i = 0; i < 4; i++) {
            display_frame(pattern);
```

```

        delay_ms(100);
        update_animation(pattern);
    }
}
return 0;
}

```

Expert Tips

- **Multiplexing:** Use timer for scan timing.
- **Testing:** Verify animation patterns.
- **Optimization:** Reduce scan time to avoid flicker.
- **Robustness:** Handle GPIO failures.
- **Extensibility:** Support complex animations.

Project 138: Read Flex Sensor Using ADC

Description: Implement a project to read a flex sensor using ADC.

Requirements:

- Read 12-bit ADC, convert to flex angle (0–90°).
- Assumptions: Linear scaling, voltage-based sensor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for sensor voltage.
- **Scaling:** Apply formula: $\text{angle} = \text{ADC} * \text{scale_factor}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_flex
    adc_value = read_adc()
    angle = adc_value * scale_factor
    RETURN angle
MAIN
    init_adc()
    WHILE true:
        angle = read_flex()
        process_angle(angle)

```

C Code Implementation

```

#include <stdint.h>

#define SCALE_FACTOR (90.0f / 4095.0f) // 0-90°
void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub
void process_angle(float angle) { /* Process angle */ }

void init_adc(void) {
    configure_adc();
}

```

```

float read_flex(void) {
    uint16_t adc_value = read_adc();
    return adc_value * SCALE_FACTOR;
}

int main(void) {
    init_adc();
    while (1) {
        float angle = read_flex();
        process_angle(angle);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Measure scale factor with known angles.
- **Testing:** Verify with known flex positions.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple sensors.

Project 139: Timer for Periodic Interrupt for Sensor Sampling

Description: Configure a timer to trigger a periodic interrupt for sensor sampling.

Requirements:

- Trigger interrupt every 5ms for sensor reads.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer for 5ms interrupts.
- **Sampling:** Read sensor in ISR.

Complexity

- **Time Complexity:** $O(1)$ per sample.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_timer
    Set timer period for 5ms
    Enable timer interrupt
FUNCTION timer_interrupt
    value = read_sensor()
    process_value(value)
MAIN
    init_timer()
    WHILE true:
        /* Wait for interrupts */

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define SAMPLE_PERIOD 5000 // 5ms in us

```

```

void configure_timer(uint32_t period) { /* Timer config */ }

uint16_t read_sensor(void) { return 0; } // Stub

void process_value(uint16_t value) { /* Process value */ }

void init_timer(void) {
    configure_timer(SAMPLE_PERIOD * (CLOCK_FREQ / 1000000));
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    process_value(value);
}

int main(void) {
    init_timer();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Use high-priority interrupt.
- **Testing:** Verify sampling rate.
- **Optimization:** Use DMA for sensor reads.
- **Robustness:** Handle missed interrupts.
- **Extensibility:** Support variable periods.

Project 140: Interface with Color Sensor Using SPI

Description: Create a project to interface with a color sensor (e.g., TCS34725) using SPI.

Requirements:

- Read RGB data (uint16_t for each channel).
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read RGB registers.
- **Data Parsing:** Combine bytes into uint16_t values.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_color_sensor
    Configure SPI
    Write sensor config
FUNCTION read_color(data)
    Read 6 bytes for RGB
    data.r = (byte0 << 8) | byte1
    data.g = (byte2 << 8) | byte3
    data.b = (byte4 << 8) | byte5

```

```

MAIN
    init_color_sensor()
    WHILE true:
        read_color(&data)
        process_color(data)

```

C Code Implementation

```

#include <stdint.h>

typedef struct {
    uint16_t r, g, b;
} ColorData;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_color(ColorData* data) { /* Process color */ }

void init_color_sensor(void) {
    configure_spi();
    spi_write(0x00, 0x03); // Enable TCS34725
}

void read_color(ColorData* data) {
    uint8_t buffer[6];
    spi_read(0x14, buffer, 6); // Read RGB registers
    data->r = (buffer[0] << 8) | buffer[1];
    data->g = (buffer[2] << 8) | buffer[3];
    data->b = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_color_sensor();
    ColorData data;
    while (1) {
        read_color(&data);
        process_color(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Adjust for ambient light.
- **Testing:** Verify with known colors.
- **Optimization:** Use burst reads.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add clear channel support.

Project 141: Control Fan with Temperature-Based Speed Control

Description: Implement a project to control a fan with temperature-based speed control.

Requirements:

- Adjust fan PWM (0–100%) based on temperature (e.g., 0% at 20°C, 100% at 40°C).
- Assumptions: ADC-based sensor, 25 kHz PWM.

Ways to Solve

Algorithm Used

- **Temperature Reading:** Read ADC, convert to temperature.
- **Linear Scaling:** Map temperature to PWM duty cycle.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure ADC
    Configure timer for 25 kHz PWM
FUNCTION control_fan
    temp = read_temperature()
    duty = map temp to 0-100% (20°C -> 0%, 40°C -> 100%)
    Set PWM duty to duty
MAIN
    init_system()
    WHILE true:
        control_fan()
```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 25000     // 25 kHz

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

#define TEMP_MIN 20.0f
#define TEMP_MAX 40.0f

void configure_adc(void) { /* ADC config */ }

void configure_pwm(uint16_t period, uint16_t duty) { /* PWM config */ }

float read_temperature(void) { return 0.0f; } // Stub

void init_system(void) {
    configure_adc();
    configure_pwm(PERIOD, 0);
}

void control_fan(void) {
    float temp = read_temperature();
    uint8_t duty = 0;
    if (temp >= TEMP_MAX) duty = 100;
    else if (temp > TEMP_MIN) {
        duty = (uint8_t)((temp - TEMP_MIN) * 100.0f / (TEMP_MAX - TEMP_MIN));
    }
    uint16_t compare = (uint32_t)PERIOD * duty / 100;
    configure_pwm(PERIOD, compare);
}

int main(void) {
    init_system();
    while (1) {
        control_fan();
    }
    return 0;
}
```

Expert Tips

- **Hysteresis:** Add to prevent rapid PWM changes.
- **Testing:** Verify with known temperatures.
- **Optimization:** Use lookup table for mapping.
- **Robustness:** Handle sensor errors.
- **Extensibility:** Support adjustable thresholds.

Project 142: ADC to Read Strain Gauge with Calibration

Description: Configure an ADC to read a strain gauge with calibration.

Requirements:

- Read 12-bit ADC, scale to strain (0–1000 $\mu\epsilon$).
- Assumptions: Linear scaling, voltage-based sensor.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for strain gauge voltage.
- **Calibration:** Apply scale factor: $\text{strain} = \text{ADC} * \text{scale_factor}$.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure ADC for single channel
FUNCTION read_strain
    adc_value = read_adc()
    strain = adc_value * scale_factor
    RETURN strain
MAIN
    init_adc()
    WHILE true:
        strain = read_strain()
        process_strain(strain)
```

C Code Implementation

```
#include <stdint.h>

#define SCALE_FACTOR (1000.0f / 4095.0f) // 0-1000  $\mu\epsilon$ 

void configure_adc(void) { /* ADC config */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_strain(float strain) { /* Process strain */ }

void init_adc(void) {
    configure_adc();
}

float read_strain(void) {
    uint16_t adc_value = read_adc();
    return adc_value * SCALE_FACTOR; }
```

```

int main(void) {
    init_adc();
    while (1) {
        float strain = read_strain();
        process_strain(strain);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Use known loads for scaling.
- **Testing:** Verify with known strains.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle ADC errors.
- **Extensibility:** Support multiple gauges.

Project 143: Interface with Humidity Sensor Using I2C

Description: Create a project to interface with a humidity sensor (e.g., HDC1080) using I2C.

Requirements:

- Read humidity data as uint16_t.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read humidity register.
- **Data Parsing:** Combine bytes into uint16_t humidity.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_humidity_sensor
    Configure I2C
    Write sensor config
FUNCTION read_humidity
    Read 2 bytes
    humidity = (byte0 << 8) | byte1
    RETURN humidity
MAIN
    init_humidity_sensor()
    WHILE true:
        humidity = read_humidity()
        process_humidity(humidity)

```

C Code Implementation

```

#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

```



```

void process_humidity(uint16_t humidity) { /* Process humidity */ }

void init_humidity_sensor(void) {
    configure_i2c();
    i2c_write(0x40, 0x02, 0x10); // Configure HDC1080
}

uint16_t read_humidity(void) {
    uint8_t buffer[2];
    i2c_read(0x40, 0x00, buffer, 2); // Read humidity register
    return (buffer[0] << 8) | buffer[1];
}

int main(void) {
    init_humidity_sensor();
    while (1) {
        uint16_t humidity = read_humidity();
        process_humidity(humidity);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Apply sensor-specific scaling.
- **Testing:** Verify with known humidity levels.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add temperature reading.

Project 144: Timer to Generate 1 Hz Signal for RTC

Description: Use a timer to generate a 1 Hz signal for an RTC.

Requirements:

- Generate 1 Hz square wave.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **Timer Configuration:** Set timer for 1 Hz toggle.
- **Output Toggle:** Flip GPIO in ISR.

Complexity

- **Time Complexity:** $O(1)$ per toggle.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_timer
    Configure timer for 1 Hz
    Enable timer interrupt
FUNCTION timer_interrupt
    Toggle output pin
MAIN
    init_timer()
    WHILE true:
        /* Wait for interrupts */

```

C Code Implementation

```
#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define SIGNAL_FREQ 1      // 1 Hz

void configure_timer(uint32_t period) { /* Timer config */ }

void toggle_output(void) { /* Toggle GPIO */ }

void init_timer(void) {
    configure_timer(CLOCK_FREQ / SIGNAL_FREQ);
}

void timer_isr(void) {
    toggle_output();
}

int main(void) {
    init_timer();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Use precise clock source.
- **Testing:** Verify signal with oscilloscope.
- **Optimization:** Use hardware toggle mode.
- **Robustness:** Handle timer errors.
- **Extensibility:** Support variable frequencies.

Project 145: GPIO Pin for Water Level Sensor with Interrupt

Description: Configure a GPIO pin to handle a water level sensor with interrupt.

Requirements:

- Detect water level events (active high) with interrupt.
- Assumptions: External interrupt, debouncing.

Ways to Solve

Algorithm Used

- **Interrupt Configuration:** Enable interrupt on rising edge.
- **Debouncing:** Use software delay to filter noise.

Pseudocode

```
FUNCTION init_water_level
    Configure GPIO as input
    Enable interrupt on rising edge
FUNCTION water_level_interrupt
    IF pin high:
        Debounce
        IF still high:
            process_water_level()
MAIN
    init_water_level()
```

```
WHILE true:
    /* Wait for interrupts */
```

C Code Implementation

```
#include <stdint.h>

void configure_gpio(uint8_t pin, uint8_t mode) { /* GPIO config */ }

uint8_t read_pin(uint8_t pin) { return 0; } // Stub

void delay_ms(uint16_t ms) { /* Delay */ }

void process_water_level(void) { /* Process water level */ }

void init_water_level(void) {
    configure_gpio(0, 0); // Input
}

void water_level_isr(void) {
    if (read_pin(0)) {
        delay_ms(20); // Debounce
        if (read_pin(0)) process_water_level();
    }
}

int main(void) {
    init_water_level();
    while (1) {
        /* Wait for interrupts */
    }
    return 0;
}
```

Expert Tips

- **Debouncing:** Use timer-based debouncing.
- **Testing:** Simulate water level events.
- **Optimization:** Minimize ISR latency.
- **Robustness:** Handle noise spikes.
- **Extensibility:** Support multiple sensors.

Project 146: Control Relay Based on Light Sensor Reading

Description: Create a project to control a relay based on a light sensor reading.

Requirements:

- Turn relay on if light < 1000, off if > 1200.
- Assumptions: ADC-based sensor, GPIO for relay.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC for light sensor.
- **Hysteresis Control:** Use thresholds to control relay.

Complexity

- **Time Complexity:** O(1) per check.
- **Space Complexity:** O(1).

Pseudocode

```
FUNCTION init_system
    Configure ADC
    Configure GPIO for relay
FUNCTION control_relay
    light = read_light()
    IF light < 1000:
        Set relay on
    ELSE IF light > 1200:
        Set relay off
MAIN
    init_system()
    WHILE true:
        control_relay()
```

C Code Implementation

```
#include <stdint.h>

#define LIGHT_ON 1000
#define LIGHT_OFF 1200

void configure_adc(void) { /* ADC config */ }

void set_relay(uint8_t state) { /* Set relay */ }

uint16_t read_light(void) { return 0; } // Stub

void init_system(void) {
    configure_adc();
    set_relay(0);
}

void control_relay(void) {
    uint16_t light = read_light();
    if (light < LIGHT_ON) set_relay(1);
    else if (light > LIGHT_OFF) set_relay(0);
}

int main(void) {
    init_system();
    while (1) {
        control_relay();
    }
    return 0;
}
```

Expert Tips

- **Hysteresis:** Prevents relay chatter.
- **Testing:** Verify with known light levels.
- **Optimization:** Use timer for periodic checks.
- **Robustness:** Handle sensor errors.
- **Extensibility:** Add adjustable thresholds.

Project 147: Read Heart Rate Sensor Using ADC

Description: Implement a project to read a heart rate sensor using ADC.

Requirements:

- Read 12-bit ADC, process to detect heart rate (beats per minute).

- Assumptions: Pulse sensor, basic peak detection.

Ways to Solve

Algorithm Used

- **ADC Reading:** Sample ADC at 100 Hz.
- **Peak Detection:** Detect peaks in signal to calculate BPM.

Complexity

- **Time Complexity:** $O(n)$ for n samples in BPM calculation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_adc
    Configure ADC
    Configure timer for 100 Hz sampling
FUNCTION read_heart_rate
    adc_value = read_adc()
    IF adc_value > threshold AND prev_value < threshold:
        Record peak time
        Calculate BPM from peak intervals
    prev_value = adc_value
    RETURN bpm
MAIN
    init_adc()
    WHILE true:
        bpm = read_heart_rate()
        process_bpm(bpm)

```

C Code Implementation

```

#include <stdint.h>

#define THRESHOLD 2000
#define SAMPLE_FREQ 100 // Hz

void configure_adc(void) { /* ADC config */ }

void configure_timer(void) { /* Timer config for 100 Hz */ }

uint16_t read_adc(void) { return 0; } // Stub

void process_bpm(uint16_t bpm) { /* Process BPM */ }

void init_adc(void) {
    configure_adc();
    configure_timer();
}

uint16_t read_heart_rate(void) {
    static uint16_t prev_value = 0;
    static uint32_t last_peak = 0;
    static uint32_t tick = 0;
    uint16_t adc_value = read_adc();
    uint16_t bpm = 0;
    if (adc_value > THRESHOLD && prev_value <= THRESHOLD) {
        if (last_peak != 0) {
            uint32_t interval = tick - last_peak;
            bpm = 60000 / (interval * 1000 / SAMPLE_FREQ); // ms to BPM
        }
        last_peak = tick;
    }
    prev_value = adc_value;
}

```

```

        tick++;
        return bpm;
    }

    void timer_isr(void) {
        uint16_t bpm = read_heart_rate();
        if (bpm) process_bpm(bpm);
    }

    int main(void) {
        init_adc();
        while (1) {
            /* Wait for timer */
        }
        return 0;
    }

```

Expert Tips

- **Signal Processing:** Use filtering for noise reduction.
- **Testing:** Verify with known heart rates.
- **Optimization:** Use DMA for sampling.
- **Robustness:** Handle missed peaks.
- **Extensibility:** Add signal quality check.

Project 148: Timer for 2 kHz PWM Signal for Servo

Description: Configure a timer to generate a 2 kHz PWM signal for a servo motor.

Requirements:

- Generate 2 kHz PWM with pulse width 0.5–2.5ms.
- Assumptions: 16-bit timer, 1 MHz clock.

Ways to Solve

Algorithm Used

- **PWM Configuration:** Set timer for 2 kHz, adjust pulse width for angle.
- **Angle Mapping:** Map 0–180° to 0.5–2.5ms pulse.

Complexity

- **Time Complexity:** O(1) per update.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_servo
    Configure timer for 2 kHz PWM
FUNCTION set_servo_angle(angle)
    pulse_us = 500 + (angle * 2000 / 180)
    Set PWM compare to pulse_us
MAIN
    init_servo()
    set_servo_angle(90)

```

C Code Implementation

```

#include <stdint.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define PWM_FREQ 2000      // 2 kHz

```

```

#define PERIOD (CLOCK_FREQ / PWM_FREQ)

void configure_pwm(uint16_t period, uint16_t pulse) { /* PWM config */ }

void init_servo(void) {
    configure_pwm(PERIOD, 1500 * (CLOCK_FREQ / 1000000)); // 1.5ms
}

void set_servo_angle(uint8_t angle) {
    if (angle > 180) angle = 180;
    uint16_t pulse_us = 500 + (angle * 2000 / 180);
    configure_pwm(PERIOD, pulse_us * (CLOCK_FREQ / 1000000));
}

int main(void) {
    init_servo();
    set_servo_angle(90);
    while (1) {
        /* Run servo */
    }
    return 0;
}

```

Expert Tips

- **Precision:** Use high-resolution timer.
- **Testing:** Verify angles with a protractor.
- **Optimization:** Precompute pulse widths.
- **Robustness:** Validate angle range.
- **Extensibility:** Support multiple servos.

Project 149: Interface with Pressure Sensor Using SPI

Description: Create a project to interface with a pressure sensor (e.g., BMP280) using SPI.

Requirements:

- Read pressure data as `uint32_t`.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read pressure registers.
- **Data Parsing:** Combine bytes into `uint32_t` pressure.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_pressure_sensor
    Configure SPI
    Write sensor config
FUNCTION read_pressure
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    RETURN pressure
MAIN
    init_pressure_sensor()

```

```
WHILE true:
    pressure = read_pressure()
    process_pressure(pressure)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_pressure(uint32_t pressure) { /* Process pressure */ }

void init_pressure_sensor(void) {
    configure_spi();
    spi_write(0xF4, 0x27); // Configure BMP280
}

uint32_t read_pressure(void) {
    uint8_t buffer[3];
    spi_read(0xF7, buffer, 3); // Read pressure registers
    return ((uint32_t)buffer[0] << 16) | ((uint32_t)buffer[1] << 8) | buffer[2];
}

int main(void) {
    init_pressure_sensor();
    while (1) {
        uint32_t pressure = read_pressure();
        process_pressure(pressure);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply sensor-specific compensation.
- **Testing:** Verify with known pressures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add temperature reading.

Project 150: Control Buzzer with Complex Melody Using Timer

Description: Implement a project to control a buzzer with a complex melody using a timer.

Requirements:

- Play a melody with 5 notes of varying frequencies and durations.
- Assumptions: PWM-driven buzzer, 16-bit timer.

Ways to Solve

Algorithm Used

- **PWM Control:** Use timer PWM for note frequencies.
- **Melody Playback:** Sequence through note array with durations.

Complexity

- **Time Complexity:** O(1) per note change.

- **Space Complexity:** $O(n)$ for melody array.

Pseudocode

```
CONST melody[] = {{freq, duration}, ...}
FUNCTION init_buzzer
    Configure timer for PWM
FUNCTION play_note(freq, duration)
    Set PWM frequency to freq
    Delay for duration
MAIN
    init_buzzer()
    FOR each note in melody:
        play_note(note.freq, note.duration)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    uint16_t freq; // Hz
    uint16_t duration; // ms
} Note;

const Note melody[] = {
    {261, 500}, // C4
    {329, 400}, // E4
    {392, 300}, // G4
    {440, 600}, // A4
    {523, 500}  // C5
};

#define MELODY_SIZE 5

void configure_pwm(uint16_t freq) { /* PWM config */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_buzzer(void) {
    configure_pwm(0);
}

void play_note(uint16_t freq, uint16_t duration) {
    configure_pwm(freq);
    delay_ms(duration);
    configure_pwm(0); // Silence
}

int main(void) {
    init_buzzer();
    while (1) {
        for (uint8_t i = 0; i < MELODY_SIZE; i++) {
            play_note(melody[i].freq, melody[i].duration);
            delay_ms(50); // Gap between notes
        }
    }
    return 0;
}
```

Expert Tips

- **Precision:** Use high-resolution timer.
- **Testing:** Verify notes with audio analyzer.
- **Optimization:** Precompute PWM settings.
- **Robustness:** Handle invalid frequencies.
- **Extensibility:** Support dynamic melodies.

Communication Protocols

Project 151: Configure UART to Send Sensor Data in a Formatted String Every Second

Description: Configure a UART to send sensor data in a formatted string every second.

Requirements:

- Send string like "Temp: XX.X C" at 9600 baud.
- Assumptions: 1 MHz clock, timer for 1s intervals, ADC-based sensor.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud, 8N1.
- **Timer Trigger:** Use timer to trigger data send every 1s.
- **String Formatting:** Convert ADC reading to string.

Complexity

- **Time Complexity:** $O(1)$ per send (fixed string length).
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    temp = read_sensor()
    Format string "Temp: XX.X C"
    Send string via UART
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define BAUD_RATE 9600

#define TEMP_SCALE (100.0f / 4095.0f) // 0-100°C

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

uint16_t read_sensor(void) { return 0; } // Stub
void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}
```

```

void timer_isr(void) {
    char buffer[16];
    float temp = read_sensor() * TEMP_SCALE;
    snprintf(buffer, sizeof(buffer), "Temp: %.1f C\r\n", temp);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Verify baud rate calculation.
- **Testing:** Check string with a terminal.
- **Optimization:** Use DMA for UART transmission.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Support multiple sensors.

Project 152: Parse a UART Command with Multiple Arguments

Description: Create a project to parse a UART command with multiple arguments (e.g., "SET:10,20").

Requirements:

- Parse command into command type and two integer arguments.
- Assumptions: 9600 baud, commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Reception:** Receive string until '\r'.
- **Parsing:** Split string into command and arguments using delimiters.

Complexity

- **Time Complexity:** $O(n)$ for n-character string.
- **Space Complexity:** $O(n)$ for input buffer.

Pseudocode

```

FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION parse_command(str)
    Split str at ":" for command
    Split args at "," for arg1, arg2
    Convert args to integers
    Execute command with args
FUNCTION receive_command
    Read UART until '\r'
    parse_command(buffer)
MAIN
    init_uart()
    WHILE true:
        receive_command()

```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void execute_command(const char* cmd, int arg1, int arg2) { /* Process command */ }

void init_uart(void) {
    configure_uart(9600);
}

void parse_command(const char* str) {
    char cmd[16];
    int arg1, arg2;
    if (sscanf(str, "%[^:]:%d,%d", cmd, &arg1, &arg2) == 3) {
        execute_command(cmd, arg1, arg2);
    }
}

void receive_command(void) {
    char buffer[32];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
            parse_command(buffer);
            break;
        }
        buffer[i++] = c;
    }
}

int main(void) {
    init_uart();
    while (1) {
        receive_command();
    }
    return 0;
}
```

Expert Tips

- **Validation:** Check for valid command format.
- **Testing:** Test with known commands.
- **Optimization:** Use state machine for parsing.
- **Robustness:** Handle malformed inputs.
- **Extensibility:** Support more arguments.

Project 153: Use UART to Send JSON-Formatted Sensor Data

Description: Implement a project to send JSON-formatted sensor data via UART.

Requirements:

- Send data like {"temp":XX.X,"hum":XX} at 9600 baud.
- Assumptions: ADC-based sensors, timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Timer Trigger:** Send data every 1s.
- **JSON Formatting:** Create JSON string from sensor readings.

Complexity

- **Time Complexity:** $O(1)$ per send (fixed string length).
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    temp = read_temp()
    hum = read_hum()
    Format JSON string {"temp":XX.X,"hum":XX}
    Send string via UART
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define BAUD_RATE 9600

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

float read_temp(void) { return 0.0f; } // Stub

uint16_t read_hum(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    char buffer[32];
    float temp = read_temp();
    uint16_t hum = read_hum();
    snprintf(buffer, sizeof(buffer), "{\"temp\":%.1f,\"hum\":%u}\r\n", temp, hum);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}
```

Expert Tips

- **Formatting:** Ensure valid JSON syntax.
- **Testing:** Verify with a JSON parser.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add more sensor fields.

Project 154: Configure SPI to Read Data from External Flash Memory Chip

Description: Configure SPI to read data from an external flash memory chip (e.g., W25Q64).

Requirements:

- Read 16 bytes from a specified address.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send read command and address, receive data.
- **Data Handling:** Store 16 bytes in a buffer.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size read.
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```
FUNCTION init_spi
    Configure SPI (4-wire, master)
FUNCTION read_flash(addr, buffer)
    Send read command (0x03)
    Send 24-bit address
    Read 16 bytes into buffer
MAIN
    init_spi()
    WHILE true:
        read_flash(address, buffer)
        process_data(buffer)
```

C Code Implementation

```
#include <uint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void spi_read(uint8_t* data, uint8_t len) { /* SPI read */ }

void process_data(uint8_t* buffer) { /* Process data */ }

void init_spi(void) {
    configure_spi();
}
```

```

void read_flash(uint32_t addr, uint8_t* buffer) {
    uint8_t cmd[4] = {0x03, (addr >> 16) & 0xFF, (addr >> 8) & 0xFF, addr & 0xFF};
    spi_write(cmd, 4);
    spi_read(buffer, 16);
}

int main(void) {
    init_spi();
    uint8_t buffer[16];
    while (1) {
        read_flash(0x000000, buffer);
        process_data(buffer);
    }
    return 0;
}

```

Expert Tips

- **Verification:** Check flash ID before reading.
- **Testing:** Verify data with known patterns.
- **Optimization:** Use DMA for large reads.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support write operations.

Project 155: Use SPI to Control a Graphic LCD Display

Description: Create a project to use SPI to control a graphic LCD display (e.g., ST7735).

Requirements:

- Display text on a 128x160 LCD.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, send initialization and text data.
- **Display Update:** Write pixel data to LCD buffer.

Complexity

- **Time Complexity:** $O(n)$ for n-pixel update.
- **Space Complexity:** $O(n)$ for display buffer.

Pseudocode

```

FUNCTION init_lcd
    Configure SPI
    Send LCD init commands
FUNCTION write_text(str, x, y)
    Convert str to pixel data
    Send data via SPI
MAIN
    init_lcd()
    write_text("Hello", 0, 0)

```

C Code Implementation

```

#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

```

```

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void set_dc(uint8_t state) { /* Set DC pin */ }

void init_lcd(void) {
    configure_spi();
    uint8_t init_cmds[] = {0x01, 0x11, 0x29}; // Simplified init
    set_dc(0); // Command mode
    spi_write(init_cmds, sizeof(init_cmds));
}

void write_text(const char* str, uint8_t x, uint8_t y) {
    uint8_t buffer[128];
    for (uint8_t i = 0; str[i] && i < 128; i++) {
        buffer[i] = str[i]; // Placeholder for font conversion
    }
    set_dc(1); // Data mode
    spi_write(buffer, strlen(str));
}

int main(void) {
    init_lcd();
    write_text("Hello", 0, 0);
    while (1) {
        /* Keep display active */
    }
    return 0;
}

```

Expert Tips

- **Font:** Use a font table for rendering.
- **Testing:** Verify with test patterns.
- **Optimization:** Buffer updates locally.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support graphics drawing.

Project 156: Interface with SPI-Based Temperature Sensor

Description: Implement a project to interface with an SPI-based temperature sensor (e.g., MAX31855).

Requirements:

- Read temperature as `int16_t`.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send read command, receive 4 bytes.
- **Data Parsing:** Extract temperature from 14-bit data.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_spi
    Configure SPI (4-wire, master)
FUNCTION read_temperature

```



```

    Read 4 bytes
    temp = (byte0 << 8 | byte1) >> 2
    RETURN temp
MAIN
    init_spi()
    WHILE true:
        temp = read_temperature()
        process_temperature(temp)

```

C Code Implementation

```

#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_read(uint8_t* data, uint8_t len) { /* SPI read */ }

void process_temperature(int16_t temp) { /* Process temp */ }

void init_spi(void) {
    configure_spi();
}

int16_t read_temperature(void) {
    uint8_t buffer[4];
    spi_read(buffer, 4);
    return ((buffer[0] << 8) | buffer[1]) >> 2; // 14-bit temp
}

int main(void) {
    init_spi();
    while (1) {
        int16_t temp = read_temperature();
        process_temperature(temp);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Check sensor-specific scaling.
- **Testing:** Verify with known temperatures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle fault bits in data.
- **Extensibility:** Add thermocouple type support.

Project 157: Configure I2C to Read Multiple Registers from Accelerometer

Description: Configure I2C to read multiple registers from an accelerometer (e.g., MPU6050).

Requirements:

- Read 6 bytes for 3-axis data (x, y, z as int16_t).
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, read 6 bytes from accelerometer.
- **Data Parsing:** Combine bytes into int16_t for each axis.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_accelerometer
    Configure I2C
    Write accelerometer config
FUNCTION read_accelerometer(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
MAIN
    init_accelerometer()
    WHILE true:
        read_accelerometer(&data)
        process_data(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    int16_t x, y, z;
} AccelData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(AccelData* data) { /* Process data */ }

void init_accelerometer(void) {
    configure_i2c();
    i2c_write(0x68, 0x6B, 0x00); // Wake up MPU6050
}

void read_accelerometer(AccelData* data) {
    uint8_t buffer[6];
    i2c_read(0x68, 0x3B, buffer, 6); // Read accel registers
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_accelerometer();
    AccelData data;
    while (1) {
        read_accelerometer(&data);
        process_data(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Remove accelerometer bias.
- **Testing:** Verify with known movements.
- **Optimization:** Use burst reads.

- **Robustness:** Handle I2C errors.
- **Extensibility:** Add gyro data reading.

Project 158: Write Configuration Data to I2C EEPROM

Description: Create a project to write configuration data to an I2C EEPROM (e.g., 24LC256).

Requirements:

- Write 16 bytes to a specified address.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send write command with address and data.
- **Page Write:** Write up to 16 bytes in a single transaction.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size write.
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```
FUNCTION init_eeprom
    Configure I2C
FUNCTION write_eeprom(addr, data)
    Send write command
    Send 16-bit address
    Send 16 bytes of data
    Wait for write completion
MAIN
    init_eeprom()
    data = {configuration data}
    write_eeprom(address, data)
```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void init_eeprom(void) {
    configure_i2c();
}

void write_eeprom(uint16_t addr, uint8_t* data) {
    uint8_t buffer[18];
    buffer[0] = addr >> 8; // High byte
    buffer[1] = addr & 0xFF; // Low byte
    for (uint8_t i = 0; i < 16; i++) {
        buffer[i + 2] = data[i];
    }
    i2c_write(0x50, buffer, 18);
    delay_ms(5); // Write cycle time
}
```

```

int main(void) {
    init_eeprom();
    uint8_t data[16] = {0x01, 0x02, /* ...
*/ 0x10}; // Config data
    write_eeprom(0x0000, data);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Verification:** Read back data to confirm.
- **Testing:** Verify with known data patterns.
- **Optimization:** Use page writes for efficiency.
- **Robustness:** Handle write timeouts.
- **Extensibility:** Support read operations.

Project 159: Use I2C to Control a Multi-Channel DAC

Description: Implement a project to control a multi-channel DAC (e.g., MCP4728) using I2C.

Requirements:

- Set 4 channels to specified 12-bit values.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send channel data to DAC.
- **Data Formatting:** Format 12-bit values per channel.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_dac
    Configure I2C
    Write DAC config
FUNCTION set_dac_channels(values)
    FOR each channel:
        Send channel address and 12-bit value
MAIN
    init_dac()
    values = {val1, val2, val3, val4}
    set_dac_channels(values)

```

C Code Implementation

```

#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

```

```

void init_dac(void) {
    configure_i2c();
    uint8_t config[] = {0x40, 0x00}; // Fast write mode
    i2c_write(0x60, config, 2);
}

void set_dac_channels(uint16_t* values) {
    uint8_t buffer[8];
    for (uint8_t i = 0; i < 4; i++) {
        buffer[i * 2] = (values[i] >> 8) & 0x0F; // High byte
        buffer[i * 2 + 1] = values[i] & 0xFF; // Low byte
    }
    i2c_write(0x60, buffer, 8);
}

int main(void) {
    init_dac();
    uint16_t values[4] = {1000, 2000, 3000, 4000};
    set_dac_channels(values);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Verify DAC output with a multimeter.
- **Testing:** Test with known values.
- **Optimization:** Use fast write mode.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support dynamic updates.

Project 160: Configure UART to Implement a Simple Request-Response Protocol

Description: Configure UART to implement a simple request-response protocol.

Requirements:

- Respond to "PING" with "PONG" at 9600 baud.
- Assumptions: Commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Command Parsing:** Check for "PING", respond with "PONG".

Complexity

- **Time Complexity:** $O(n)$ for n-character command.
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```

FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION handle_request
    Read UART until '\r'
    IF buffer = "PING":

```

```
        Send "PONG\r"
MAIN
    init_uart()
    WHILE true:
        handle_request()
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

void init_uart(void) {
    configure_uart(9600);
}

void handle_request(void) {
    char buffer[16];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
            if (strcmp(buffer, "PING") == 0) {
                uart_send("PONG\r");
            }
            break;
        }
        buffer[i++] = c;
    }
}

int main(void) {
    init_uart();
    while (1) {
        handle_request();
    }
    return 0;
}
```

Expert Tips

- **Validation:** Check for valid commands.
- **Testing:** Verify with a terminal.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle malformed inputs.
- **Extensibility:** Add more commands.

Project 161: Use SPI to Read a Pressure Sensor and Log Data

Description: Create a project to use SPI to read a pressure sensor (e.g., BMP280) and log data.

Requirements:

- Read pressure as `uint32_t`, store in buffer.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read pressure registers.
- **Data Logging:** Store readings in a circular buffer.

Complexity

- **Time Complexity:** $O(1)$ per read/log.
- **Space Complexity:** $O(n)$ for buffer size.

Pseudocode

```
FUNCTION init_system
    Configure SPI
    Initialize circular buffer
FUNCTION read_pressure
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    Store pressure in buffer
MAIN
    init_system()
    WHILE true:
        read_pressure()
```

C Code Implementation

```
#include <stdint.h>

#define BUFFER_SIZE 10

void configure_spi(void) { /* SPI config */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

uint32_t buffer[BUFFER_SIZE];

uint8_t buffer_index = 0;

void init_system(void) {
    configure_spi();
    for (uint8_t i = 0; i < BUFFER_SIZE; i++) buffer[i] = 0;
}

void read_pressure(void) {
    uint8_t data[3];
    spi_read(0xF7, data, 3); // Read BMP280 pressure
    uint32_t pressure = ((uint32_t)data[0] << 16) | ((uint32_t)data[1] << 8) | data[2];
    buffer[buffer_index] = pressure;
    buffer_index = (buffer_index + 1) % BUFFER_SIZE;
}

int main(void) {
    init_system();
    while (1) {
        read_pressure();
    }
    return 0;
}
```

Expert Tips

- **Logging:** Use flash for persistent storage.
- **Testing:** Verify pressure data.

- **Optimization:** Use DMA for SPI reads.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add timestamp logging.

Project 162: Interface with I2C-Based OLED Display

Description: Implement a project to interface with an I2C-based OLED display (e.g., SSD1306).

Requirements:

- Display text on a 128x64 OLED.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Initialize I2C, send initialization and text data.
- **Display Update:** Write pixel data to display buffer.

Complexity

- **Time Complexity:** $O(n)$ for n-pixel update.
- **Space Complexity:** $O(n)$ for display buffer.

Pseudocode

```
FUNCTION init_oled
    Configure I2C
    Send OLED init commands
FUNCTION write_text(str, x, y)
    Convert str to pixel data
    Send data to OLED via I2C
MAIN
    init_oled()
    write_text("Hello", 0, 0)
```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

void init_oled(void) {
    configure_i2c();
    uint8_t init_cmds[] = {0xAE, 0xA8, 0x3F, 0xD3, 0x00, 0x40, 0xA1, 0xC8, 0xDA, 0x12, 0x81, 0x7F,
0xA4, 0xA6, 0xD5, 0x80, 0x8D, 0x14, 0xAF};
    i2c_write(0x3C, init_cmds, sizeof(init_cmds));
}

void write_text(const char* str, uint8_t x, uint8_t y) {
    uint8_t buffer[128];
    for (uint8_t i = 0; str[i] && i < 128; i++) {
        buffer[i] = str[i]; // Placeholder for font conversion
    }
    i2c_write(0x3C, buffer, strlen(str));
}

int main(void) {
    init_oled();
    write_text("Hello", 0, 0);
}
```



```
while (1) {
    /* Keep display active */
}
return 0;
}
```

Expert Tips

- **Font:** Use a font table for rendering.
- **Testing:** Verify with test patterns.
- **Optimization:** Buffer updates locally.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support graphics drawing.

Project 163: Configure SPI to Control a Shift Register for LED Control

Description: Configure SPI to control a shift register (e.g., 74HC595) for LED control.

Requirements:

- Control 8 LEDs with a pattern.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send 8-bit pattern to shift register.
- **Output Update:** Latch data to update LEDs.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_spi
    Configure SPI
    Configure latch pin
FUNCTION update_leds(pattern)
    Send pattern via SPI
    Pulse latch pin
MAIN
    init_spi()
    WHILE true:
        update_leds(pattern)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void set_latch(uint8_t state) { /* Set latch pin */ }

void init_spi(void) {
    configure_spi();
    set_latch(0); }
}
```

```

void update_leds(uint8_t pattern) {
    spi_write(&pattern, 1);
    set_latch(1);
    set_latch(0);
}

int main(void) {
    init_spi();
    while (1) {
        update_leds(0xAA); // Alternating pattern
    }
    return 0;
}

```

Expert Tips

- **Timing:** Ensure proper latch timing.
- **Testing:** Verify LED patterns.
- **Optimization:** Use SPI DMA for multiple shift registers.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support cascading registers.

Project 164: Use I2C to Read a Real-Time Clock and Display Time

Description: Create a project to use I2C to read a real-time clock (e.g., DS3231) and display time.

Requirements:

- Read time (hour, minute, second) and display as "HH:MM:SS".
- Assumptions: I2C interface, 100 kHz, UART for display.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read RTC registers.
- **Time Formatting:** Convert BCD to decimal, format as string.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Configure UART
    Write RTC config
FUNCTION read_time(time)
    Read 3 bytes for hour, minute, second
    Convert BCD to decimal
    Format as "HH:MM:SS"
MAIN
    init_system()
    WHILE true:
        read_time(&time)
        display_time(time)

```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

typedef struct {
    uint8_t hour, minute, second;
} Time;

void configure_i2c(void) { /* I2C config */ }

void configure_uart(uint32_t baud) { /* UART config */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void uart_send(const char* str) { /* Send string */ }

uint8_t bcd_to_dec(uint8_t bcd) {
    return ((bcd >> 4) * 10) + (bcd & 0x0F);
}

void init_system(void) {
    configure_i2c();
    configure_uart(9600);
}

void read_time(Time* time) {
    uint8_t buffer[3];
    i2c_read(0x68, 0x00, buffer, 3); // Read DS3231 time
    time->second = bcd_to_dec(buffer[0]);
    time->minute = bcd_to_dec(buffer[1]);
    time->hour = bcd_to_dec(buffer[2]);
}

void display_time(Time* time) {
    char buffer[16];
    snprintf(buffer, sizeof(buffer), "%02u:%02u:%02u\r\n", time->hour, time->minute, time->second);
    uart_send(buffer);
}

int main(void) {
    init_system();
    Time time;
    while (1) {
        read_time(&time);
        display_time(&time);
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify RTC drift.
- **Testing:** Check with known times.
- **Optimization:** Cache time for efficiency.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add date display.

Project 165: Use UART to Send CSV-Formatted Sensor Data

Description: Implement a project to use UART to send CSV-formatted sensor data.

Requirements:

- Send data like "temp,hum" at 9600 baud.
- Assumptions: ADC-based sensors, timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Timer Trigger:** Send data every 1s.
- **CSV Formatting:** Format sensor readings as CSV string.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    temp = read_temp()
    hum = read_hum()
    Format CSV string "temp,hum"
    Send string via UART
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define CLOCK_FREQ 1000000 // 1 MHz
#define BAUD_RATE 9600

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

float read_temp(void) { return 0.0f; } // Stub

uint16_t read_hum(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}
```

```

void timer_isr(void) {
    char buffer[32];
    float temp = read_temp();
    uint16_t hum = read_hum();
    snprintf(buffer, sizeof(buffer), "%.1f,%u\r\n", temp, hum);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Formatting:** Ensure valid CSV syntax.
- **Testing:** Verify with a CSV parser.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add more sensor fields.

Project 166: Configure I2C to Read a Magnetometer and Calculate Heading

Description: Configure I2C to read a magnetometer (e.g., HMC5883L) and calculate heading.

Requirements:

- Read 3-axis data, compute heading in degrees.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read 6 bytes for magnetometer data.
- **Heading Calculation:** Use $\text{atan2}(y, x)$ for heading.

Complexity

- **Time Complexity:** $O(1)$ per read/calculation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_magnetometer
    Configure I2C
    Write magnetometer config
FUNCTION read_magnetometer(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
FUNCTION calculate_heading(data)
    heading = atan2(data.y, data.x) * 180 / PI
    RETURN heading
MAIN
    init_magnetometer()
    WHILE true:

```

```
read_magnetometer(&data)
heading = calculate_heading(data)
process_heading(heading)
```

C Code Implementation

```
#include <stdint.h>
#include <math.h>

typedef struct {
    int16_t x, y, z;
} MagData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_heading(float heading) { /* Process heading */ }

void init_magnetometer(void) {
    configure_i2c();
    i2c_write(0x1E, 0x00, 0x70); // Configure HMC5883L
}

void read_magnetometer(MagData* data) {
    uint8_t buffer[6];
    i2c_read(0x1E, 0x03, buffer, 6); // Read mag registers
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

float calculate_heading(MagData* data) {
    return atan2(data->y, data->x) * 180.0f / M_PI;
}

int main(void) {
    init_magnetometer();
    MagData data;
    while (1) {
        read_magnetometer(&data);
        float heading = calculate_heading(&data);
        process_heading(heading);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply hard/soft iron corrections.
- **Testing:** Verify with known orientations.
- **Optimization:** Use fixed-point math for atan2.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add tilt compensation.

Project 167: Use SPI to Interface with an SD Card for Data Logging

Description: Create a project to use SPI to interface with an SD card for data logging.

Requirements:

- Log sensor data as CSV to SD card.
- Assumptions: SPI interface, FAT filesystem.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SD card, write data.
- **File Handling:** Append data to CSV file.

Complexity

- **Time Complexity:** $O(n)$ for n-byte write.
- **Space Complexity:** $O(1)$ (fixed buffer).

Pseudocode

```
FUNCTION init_system
    Configure SPI
    Initialize SD card
    Open CSV file
FUNCTION log_data(value)
    Format data as CSV string
    Write string to file
    Flush file
MAIN
    init_system()
    WHILE true:
        value = read_sensor()
        log_data(value)
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

void configure_spi(void) { /* SPI config */ }

void sd_init(void) { /* SD card init */ }

void sd_write(const char* data) { /* Write to SD */ }

uint16_t read_sensor(void) { return 0; } // Stub

void init_system(void) {
    configure_spi();
    sd_init();
}

void log_data(uint16_t value) {
    char buffer[32];
    snprintf(buffer, sizeof(buffer), "%u\r\n", value);
    sd_write(buffer);
}
```

```

int main(void) {
    init_system();
    while (1) {
        uint16_t value = read_sensor();
        log_data(value);
    }
    return 0;
}

```

Expert Tips

- **Filesystem:** Use FATFS library for SD card.
- **Testing:** Verify file contents.
- **Optimization:** Buffer multiple writes.
- **Robustness:** Handle SD card errors.
- **Extensibility:** Add timestamp to logs.

Project 168: Use UART to Implement a Basic Bootloader

Description: Implement a project to use UART to implement a basic bootloader.

Requirements:

- Receive and program 256 bytes of data at 9600 baud.
- Assumptions: Flash write function available.

Ways to Solve

Algorithm Used

- **UART Reception:** Receive data packet with address and payload.
- **Flash Programming:** Write data to flash memory.

Complexity

- **Time Complexity:** $O(n)$ for n-byte packet.
- **Space Complexity:** $O(n)$ for buffer.

Pseudocode

```

FUNCTION init_bootloader
    Configure UART (9600 baud, 8N1)
FUNCTION receive_packet
    Read address (4 bytes)
    Read 256 bytes of data
    Write data to flash at address
MAIN
    init_bootloader()
    WHILE true:
        receive_packet()

```

C Code Implementation

```

#include <stdint.h>

#define PACKET_SIZE 256

void configure_uart(uint32_t baud) { /* UART config */ }

void flash_write(uint32_t addr, uint8_t* data, uint16_t len) { /* Flash write */ }

uint8_t uart_read(void) { return 0; } // Stub

```



```

void init_bootloader(void) {
    configure_uart(9600);
}

void receive_packet(void) {
    uint8_t addr_bytes[4];
    for (uint8_t i = 0; i < 4; i++) addr_bytes[i] = uart_read();
    uint32_t addr = (addr_bytes[0] << 24) | (addr_bytes[1] << 16) | (addr_bytes[2] << 8) |
    addr_bytes[3];
    uint8_t data[PACKET_SIZE];
    for (uint16_t i = 0; i < PACKET_SIZE; i++) data[i] = uart_read();
    flash_write(addr, data, PACKET_SIZE);
}

int main(void) {
    init_bootloader();
    while (1) {
        receive_packet();
    }
    return 0;
}

```

Expert Tips

- **Validation:** Add checksum to packets.
- **Testing:** Verify flash writes.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle flash write errors.
- **Extensibility:** Add erase command.

Project 169: Configure I2C to Control a Digital Potentiometer

Description: Configure I2C to control a digital potentiometer (e.g., MCP4561).

Requirements:

- Set wiper to a specified 8-bit value.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send wiper value to potentiometer.
- **Command Formatting:** Format command per datasheet.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_pot
    Configure I2C
FUNCTION set_wiper(value)
    Send write command
    Send 8-bit value
MAIN
    init_pot()
    set_wiper(value)

```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

void init_pot(void) {
    configure_i2c();
}

void set_wiper(uint8_t value) {
    uint8_t data[2] = {0x00, value}; // Write wiper register
    i2c_write(0x2F, data, 2);
}

int main(void) {
    init_pot();
    set_wiper(128); // Mid-scale
    while (1) {
        /* Idle */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify resistance with a multimeter.
- **Testing:** Test with known values.
- **Optimization:** Minimize I2C transactions.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support multiple channels.

Project 170: Use SPI to Read a Gyroscope and Calculate Rotation

Description: Create a project to use SPI to read a gyroscope (e.g., L3GD20) and calculate rotation.

Requirements:

- Read 3-axis data, compute angular position via integration.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read 6 bytes for gyro data.
- **Integration:** Sum angular rates to estimate angle.

Complexity

- **Time Complexity:** $O(1)$ per read/calculation.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_gyro
    Configure SPI
    Write gyro config
```

```

FUNCTION read_gyro(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
FUNCTION calculate_rotation(data)
    angle += data.z * dt
    RETURN angle
MAIN
    init_gyro()
    WHILE true:
        read_gyro(&data)
        angle = calculate_rotation(data)
        process_angle(angle)

```

C Code Implementation

```

#include <stdint.h>

#define DT 0.01f // 10ms

typedef struct {
    int16_t x, y, z;
} GyroData;

float angle = 0;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_angle(float angle) { /* Process angle */ }

void init_gyro(void) {
    configure_spi();
    spi_write(0x20, 0x0F); // Enable L3GD20
}

void read_gyro(GyroData* data) {
    uint8_t buffer[6];
    spi_read(0x28 | 0x80, buffer, 6); // Auto-increment read
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

float calculate_rotation(GyroData* data) {
    angle += data->z * DT; // Simple integration
    return angle;
}

int main(void) {
    init_gyro();
    GyroData data;
    while (1) {
        read_gyro(&data);
        float angle = calculate_rotation(&data);
        process_angle(angle);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Remove gyro bias.

- **Testing:** Verify with known rotations.
- **Optimization:** Use fixed-point math.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add accelerometer fusion.

Project 171: Use UART to Send a Timestamped Event Log

Description: Implement a project to use UART to send a timestamped event log.

Requirements:

- Send events like "Event: X, Time: HH:MM:SS" at 9600 baud.
- Assumptions: Timer-based timestamp, UART for output.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud, 8N1.
- **Timestamp Generation:** Use system timer for HH:MM:SS.
- **Event Logging:** Format event and timestamp as string.

Complexity

- **Time Complexity:** $O(1)$ per log (fixed string length).
- **Space Complexity:** $O(1)$ (fixed buffer size).

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for timestamp
FUNCTION log_event(event)
    time = get_timestamp()
    Format string "Event: X, Time: HH:MM:SS"
    Send string via UART
MAIN
    init_system()
    WHILE true:
        IF event_detected:
            log_event(event)
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define BAUD_RATE 9600
typedef struct {
    uint8_t hour, minute, second;
} Time;

void configure_uart(uint32_t baud) { /* UART config */ }

void uart_send(const char* str) { /* Send string */ }

void get_timestamp(Time* time) { /* Get time (stub) */ time->hour = time->minute = time->second = 0; }

uint8_t event_detected(void) { return 0; } // Stub
uint8_t get_event(void) { return 0; } // Stub
void init_system(void) {
    configure_uart(BAUD_RATE);
}
```

```

void log_event(uint8_t event) {
    Time time;
    get_timestamp(&time);
    char buffer[32];
    sprintf(buffer, sizeof(buffer), "Event: %u, Time: %02u:%02u:%02u\r\n",
        event, time.hour, time.minute, time.second);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        if (event_detected()) {
            uint8_t event = get_event();
            log_event(event);
        }
    }
    return 0;
}

```

Expert Tips

- **Timestamp Source:** Use RTC for accurate time.
- **Testing:** Verify with a terminal.
- **Optimization:** Use DMA for UART transmission.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add event categories.

Project 172: Configure SPI to Interface with a High-Speed ADC

Description: Configure SPI to interface with a high-speed ADC (e.g., ADS7886).

Requirements:

- Read 12-bit ADC data at 1 MHz.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Initialize SPI, read 2 bytes for ADC data.
- **Data Parsing:** Extract 12-bit value from received bytes.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_adc
    Configure SPI (1 MHz, master)
FUNCTION read_adc
    Send dummy bytes to trigger conversion
    Read 2 bytes
    adc_value = (byte0 << 4) | (byte1 >> 4)
    RETURN adc_value
MAIN
    init_adc()
    WHILE true:
        value = read_adc()
        process_value(value)

```

C Code Implementation

```
#include <stdint.h>

void configure_spi(uint32_t freq) { /* SPI config */ }

void spi_write_read(uint8_t* tx, uint8_t* rx, uint8_t len) { /* SPI write/read */ }

void process_value(uint16_t value) { /* Process ADC value */ }

void init_adc(void) {
    configure_spi(1000000); // 1 MHz
}

uint16_t read_adc(void) {
    uint8_t tx[2] = {0, 0};
    uint8_t rx[2];
    spi_write_read(tx, rx, 2);
    return ((rx[0] << 4) | (rx[1] >> 4)) & 0xFFF; // 12-bit
}

int main(void) {
    init_adc();
    while (1) {
        uint16_t value = read_adc();
        process_value(value);
    }
    return 0;
}
```

Expert Tips

- **Timing:** Ensure SPI clock matches ADC spec.
- **Testing:** Verify with known voltages.
- **Optimization:** Use DMA for continuous sampling.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support multiple channels.

Project 173: Use I2C to Read a Humidity Sensor and Log Data

Description: Create a project to use I2C to read a humidity sensor (e.g., HDC1080) and log data.

Requirements:

- Read humidity as uint16_t, store in buffer.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read humidity register.
- **Data Logging:** Store readings in a circular buffer.

Complexity

- **Time Complexity:** $O(1)$ per read/log.
- **Space Complexity:** $O(n)$ for buffer size.

Pseudocode

```
FUNCTION init_system
    Configure I2C
```

```

    Initialize circular buffer
FUNCTION read_humidity
    Read 2 bytes
    humidity = (byte0 << 8) | byte1
    Store humidity in buffer
MAIN
    init_system()
    WHILE true:
        read_humidity()

```

C Code Implementation

```

#include <stdint.h>

#define BUFFER_SIZE 10

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

uint16_t buffer[BUFFER_SIZE];

uint8_t buffer_index = 0;

void init_system(void) {
    configure_i2c();
    i2c_write(0x40, 0x02, 0x10); // Configure HDC1080
    for (uint8_t i = 0; i < BUFFER_SIZE; i++) buffer[i] = 0;
}

void read_humidity(void) {
    uint8_t data[2];
    i2c_read(0x40, 0x00, data, 2); // Read humidity
    uint16_t humidity = (data[0] << 8) | data[1];
    buffer[buffer_index] = humidity;
    buffer_index = (buffer_index + 1) % BUFFER_SIZE;
}

int main(void) {
    init_system();
    while (1) {
        read_humidity();
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Apply sensor-specific scaling.
- **Testing:** Verify with known humidity levels.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add temperature logging.

Project 174: Use UART to Receive and Execute Complex Commands

Description: Implement a project to use UART to receive and execute complex commands.

Requirements:

- Parse commands like "CMD:ARG1,ARG2" and execute (e.g., set LED states).
- Assumptions: 9600 baud, commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Reception:** Receive string until '\r'.
- **Command Parsing:** Split into command and arguments, execute based on command type.

Complexity

- **Time Complexity:** $O(n)$ for n-character string.
- **Space Complexity:** $O(n)$ for input buffer.

Pseudocode

```
FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION parse_command(str)
    Split str at ":" for command
    Split args at "," for arg1, arg2
    Execute command with args
FUNCTION receive_command
    Read UART until '\r'
    parse_command(buffer)
MAIN
    init_uart()
    WHILE true:
        receive_command()
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void execute_command(const char* cmd, int arg1, int arg2) { /* Execute command */ }

void init_uart(void) {
    configure_uart(9600);
}

void parse_command(const char* str) {
    char cmd[16];
    int arg1, arg2;
    if (sscanf(str, "%[^:]:%d,%d", cmd, &arg1, &arg2) == 3) {
        execute_command(cmd, arg1, arg2);
    }
}
```



```

void receive_command(void) {
    char buffer[32];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
            parse_command(buffer);
            break;
        }
        buffer[i++] = c;
    }
}

int main(void) {
    init_uart();
    while (1) {
        receive_command();
    }
    return 0;
}

```

Expert Tips

- **Validation:** Check command validity.
- **Testing:** Test with known commands.
- **Optimization:** Use state machine for parsing.
- **Robustness:** Handle malformed inputs.
- **Extensibility:** Support more command types.

Project 175: Configure I2C to Interface with a Multi-Sensor Module

Description: Configure I2C to interface with a multi-sensor module (e.g., BME280 for temperature and pressure).

Requirements:

- Read temperature and pressure as float values.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read temperature and pressure registers.
- **Data Parsing:** Apply sensor-specific compensation formulas.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_sensor
    Configure I2C
    Write sensor config
FUNCTION read_sensors(data)
    Read 6 bytes for temp and pressure
    Apply compensation formulas

```

```

    data.temp = calculated temp
    data.pressure = calculated pressure
MAIN
    init_sensor()
    WHILE true:
        read_sensors(&data)
        process_data(data)

```

C Code Implementation

```

#include <stdint.h>

typedef struct {
    float temp, pressure;
} SensorData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(SensorData* data) { /* Process data */ }

void init_sensor(void) {
    configure_i2c();
    i2c_write(0x76, 0xF4, 0x27); // Configure BME280
}

void read_sensors(SensorData* data) {
    uint8_t buffer[6];
    i2c_read(0x76, 0xF7, buffer, 6); // Read temp and pressure
    uint32_t raw_press = ((uint32_t)buffer[0] << 12) | ((uint32_t)buffer[1] << 4) | (buffer[2] >> 4);
    uint32_t raw_temp = ((uint32_t)buffer[3] << 12) | ((uint32_t)buffer[4] << 4) | (buffer[5] >> 4);
    data->temp = raw_temp / 16384.0f; // Simplified
    data->pressure = raw_press / 256.0f; // Simplified
}

int main(void) {
    init_sensor();
    SensorData data;
    while (1) {
        read_sensors(&data);
        process_data(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Use datasheet compensation formulas.
- **Testing:** Verify with known conditions.
- **Optimization:** Cache calibration data.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add humidity reading.

Project 176: Use SPI to Control a Multi-Digit 7-Segment Display

Description: Create a project to use SPI to control a multi-digit 7-segment display (e.g., MAX7219).

Requirements:

- Display a 4-digit number.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send digit data to MAX7219.
- **Display Update:** Convert number to digit patterns.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_display
    Configure SPI
    Send MAX7219 init commands
FUNCTION display_number(num)
    Convert num to 4 digits
    Send digit data via SPI
MAIN
    init_display()
    WHILE true:
        display_number(number)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void init_display(void) {
    configure_spi();
    uint8_t init_cmds[] = {0x09, 0xFF, 0x0A, 0x0F, 0x0B, 0x03, 0x0C, 0x01}; // Decode mode, intensity,
    scan limit, display on
    spi_write(init_cmds, sizeof(init_cmds));
}

void display_number(uint16_t num) {
    uint8_t digits[4] = {num / 1000, (num / 100) % 10, (num / 10) % 10, num % 10};
    for (uint8_t i = 0; i < 4; i++) {
        uint8_t data[2] = {4 - i, digits[i]}; // Digit address, value
        spi_write(data, 2);
    }
}

int main(void) {
    init_display();
    while (1) {
        display_number(1234);
    }
    return 0;
}
```

Expert Tips

- **Brightness:** Adjust intensity for visibility.
- **Testing:** Verify with known numbers.
- **Optimization:** Buffer updates locally.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support decimal points.

Project 177: Use UART to Send Sensor Data in a Binary Format

Description: Implement a project to use UART to send sensor data in a binary format.

Requirements:

- Send 2-byte sensor data at 9600 baud.
- Assumptions: Timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Timer Trigger:** Send data every 1s.
- **Binary Formatting:** Send raw bytes.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    value = read_sensor()
    Send value as 2 bytes
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send_bytes(uint8_t* data, uint8_t len) { /* Send bytes */ }

uint16_t read_sensor(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    uint8_t data[2] = {value >> 8, value & 0xFF};
    uart_send_bytes(data, 2);
}
```

```

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Protocol:** Add header for framing.
- **Testing:** Verify with a binary receiver.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle transmission errors.
- **Extensibility:** Add multiple sensors.

Project 178: Configure I2C to Read a Color Sensor and Display RGB Values

Description: Configure I2C to read a color sensor (e.g., TCS34725) and display RGB values.

Requirements:

- Read RGB as `uint16_t`, display via UART.
- Assumptions: I2C interface, 100 kHz, 9600 baud UART.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read RGB registers.
- **Data Display:** Format and send RGB values via UART.

Complexity

- **Time Complexity:** $O(1)$ per read/display.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Configure UART (9600 baud, 8N1)
    Write sensor config
FUNCTION read_color(data)
    Read 6 bytes for RGB
    data.r = (byte0 << 8) | byte1
    data.g = (byte2 << 8) | byte3
    data.b = (byte4 << 8) | byte5
FUNCTION display_rgb(data)
    Format string "R:XX,G:XX,B:XX"
    Send via UART
MAIN
    init_system()
    WHILE true:
        read_color(&data)
        display_rgb(data)

```

C Code Implementation

```

#include <stdint.h>
#include <stdio.h>

```

```

typedef struct {
    uint16_t r, g, b;
} ColorData;

void configure_i2c(void) { /* I2C config */ }

void configure_uart(uint32_t baud) { /* UART config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void uart_send(const char* str) { /* Send string */ }

void init_system(void) {
    configure_i2c();
    configure_uart(9600);
    i2c_write(0x29, 0x00, 0x03); // Enable TCS34725
}

void read_color(ColorData* data) {
    uint8_t buffer[6];
    i2c_read(0x29, 0x14, buffer, 6); // Read RGB
    data->r = (buffer[0] << 8) | buffer[1];
    data->g = (buffer[2] << 8) | buffer[3];
    data->b = (buffer[4] << 8) | buffer[5];
}

void display_rgb(ColorData* data) {
    char buffer[32];
    sprintf(buffer, sizeof(buffer), "R:%u,G:%u,B:%u\r\n", data->r, data->g, data->b);
    uart_send(buffer);
}

int main(void) {
    init_system();
    ColorData data;
    while (1) {
        read_color(&data);
        display_rgb(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Adjust for ambient light.
- **Testing:** Verify with known colors.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add clear channel support.

Project 179: Use SPI to Interface with Flash Memory for Firmware Updates

Description: Create a project to use SPI to interface with flash memory (e.g., W25Q64) for firmware updates.

Requirements:

- Write 256 bytes to a specified address.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send write command, address, and data.
- **Flash Programming:** Handle page write and wait for completion.

Complexity

- **Time Complexity:** $O(1)$ for fixed-size write.
- **Space Complexity:** $O(n)$ for data buffer.

Pseudocode

```
FUNCTION init_flash
    Configure SPI
FUNCTION write_flash(addr, data)
    Send write enable
    Send write command
    Send 24-bit address
    Send 256 bytes
    Wait for write completion
MAIN
    init_flash()
    data = {firmware data}
    write_flash(address, data)
```

C Code Implementation

```
#include <stdint.h>

#define PAGE_SIZE 256

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void delay_ms(uint16_t ms) { /* Delay */ }

uint8_t read_status(void) { return 0; } // Stub
void init_flash(void) {
    configure_spi();
}

void write_flash(uint32_t addr, uint8_t* data) {
    uint8_t write_en = 0x06;
    spi_write(&write_en, 1); // Write enable
    uint8_t cmd[4 + PAGE_SIZE] = {0x02, (addr >> 16) & 0xFF, (addr >> 8) & 0xFF, addr & 0xFF};
    for (uint16_t i = 0; i < PAGE_SIZE; i++) cmd[i + 4] = data[i];
    spi_write(cmd, 4 + PAGE_SIZE);
    while (read_status() & 0x01) delay_ms(1); // Wait for write
}

int main(void) {
    init_flash();
    uint8_t data[PAGE_SIZE] = {0x00}; // Firmware data
    write_flash(0x000000, data);
    while (1) {
        /* Idle */
    }
    return 0;
}
```

Expert Tips

- **Verification:** Read back data to confirm.

- **Testing:** Verify with known patterns.
- **Optimization:** Use page writes efficiently.
- **Robustness:** Handle write timeouts.
- **Extensibility:** Add erase commands.

Project 180: Use UART to Implement a Simple Modbus Protocol

Description: Implement a project to use UART to implement a simple Modbus protocol.

Requirements:

- Respond to read register command (function code 03) at 9600 baud.
- Assumptions: ASCII mode, simple register read.

Ways to Solve

Algorithm Used

- **UART Reception:** Receive Modbus ASCII frame.
- **Command Parsing:** Validate frame, respond with register value.

Complexity

- **Time Complexity:** $O(n)$ for n-byte frame.
- **Space Complexity:** $O(n)$ for buffer.

Pseudocode

```

FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION handle_modbus
    Read frame until '\r\n'
    IF function_code = 03:
        Read register address
        Send response with register value
MAIN
    init_uart()
    WHILE true:
        handle_modbus()
  
```

C Code Implementation

```

#include <stdint.h>
#include <string.h>

#define BAUD_RATE 9600

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

uint16_t read_register(uint8_t addr) { return 0; } // Stub

void init_uart(void) {
    configure_uart(BAUD_RATE);
}
  
```



```

void handle_modbus(void) {
    char buffer[32];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        buffer[i++] = c;
        if (i >= 2 && buffer[i-2] == '\r' && buffer[i-1] == '\n') {
            buffer[i-2] = '\0';
            if (buffer[3] == '0' && buffer[4] == '3') { // Function code 03
                uint8_t reg_addr = (buffer[5] - '0') * 16 + (buffer[6] - '0');
                uint16_t value = read_register(reg_addr);
                char response[16];
                snprintf(response, sizeof(response), ":03%02X%04X\r\n", reg_addr, value);
                uart_send(response);
            }
            break;
        }
    }
}

int main(void) {
    init_uart();
    while (1) {
        handle_modbus();
    }
    return 0;
}

```

Expert Tips

- **Validation:** Check LRC for frame integrity.
- **Testing:** Use a Modbus master tool.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle invalid frames.
- **Extensibility:** Support more function codes.

Project 181: Configure I2C to Control an I2C-Based Motor Driver

Description: Configure I2C to control an I2C-based motor driver (e.g., DRV8830).

Requirements:

- Set motor speed and direction.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send speed and direction commands.
- **Command Formatting:** Format per driver datasheet.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_driver
    Configure I2C
FUNCTION set_motor(speed, direction)
    Send control register with speed and direction
MAIN
    init_driver()
    set_motor(speed, direction)
```

C Code Implementation

```
#include <stdint.h>

#define FORWARD 0x01
#define REVERSE 0x02

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

void init_driver(void) {
    configure_i2c();
}

void set_motor(uint8_t speed, uint8_t direction) {
    uint8_t data[2] = {0x00, (speed & 0x3F) | (direction << 6)};
    i2c_write(0x60, data, 2); // DRV8830 control
}

int main(void) {
    init_driver();
    set_motor(50, FORWARD); // 50% speed, forward
    while (1) {
        /* Idle */
    }
    return 0;
}
```

Expert Tips

- **Validation:** Check motor response.
- **Testing:** Verify with known speeds.
- **Optimization:** Minimize I2C transactions.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support multiple motors.

Project 182: Use SPI to Read a High-Resolution Temperature Sensor

Description: Create a project to use SPI to read a high-resolution temperature sensor (e.g., MAX31865).

Requirements:

- Read temperature as float.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read temperature register.

- **Data Conversion:** Convert raw data to temperature using sensor formula.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_sensor
    Configure SPI
    Write sensor config
FUNCTION read_temperature
    Read 2 bytes
    temp = convert raw to temperature
    RETURN temp
MAIN
    init_sensor()
    WHILE true:
        temp = read_temperature()
        process_temperature(temp)

```

C Code Implementation

```

#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_temperature(float temp) { /* Process temp */ }

void init_sensor(void) {
    configure_spi();
    spi_write(0x00, 0xA0); // Configure MAX31865 (3-wire, 50Hz)
}

float read_temperature(void) {
    uint8_t buffer[2];
    spi_read(0x01, buffer, 2); // Read RTD
    uint16_t rtd = ((buffer[0] << 8) | buffer[1]) >> 1; // 15-bit
    return (rtd / 32.0f) - 256.0f; // Simplified conversion
}

int main(void) {
    init_sensor();
    while (1) {
        float temp = read_temperature();
        process_temperature(temp);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Use datasheet formula for accuracy.
- **Testing:** Verify with known temperatures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle fault bits.
- **Extensibility:** Support multiple sensors.

Project 183: Use UART to Send a Structured Data Packet

Description: Implement a project to use UART to send a structured data packet.

Requirements:

- Send packet with ID, sensor data, and timestamp at 9600 baud.
- Assumptions: Timer-based timestamp.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Packet Formatting:** Create structured packet with ID, data, timestamp.
- **Timer Trigger:** Send packet periodically.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for periodic send
FUNCTION timer_interrupt
    packet.id = get_id()
    packet.data = read_sensor()
    packet.time = get_timestamp()
    Send packet bytes
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000
typedef struct {
    uint8_t id;
    uint16_t data;
    uint32_t time;
} Packet;

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send_bytes(uint8_t* data, uint8_t len) { /* Send bytes */ }

uint8_t get_id(void) { return 1; } // Stub
uint16_t read_sensor(void) { return 0; } // Stub
uint32_t get_timestamp(void) { return 0; } // Stub
void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}
```

```

void timer_isr(void) {
    Packet packet = {get_id(), read_sensor(), get_timestamp()};
    uart_send_bytes((uint8_t*)&packet, sizeof(Packet));
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Protocol:** Add header/checksum for framing.
- **Testing:** Verify with a packet parser.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle transmission errors.
- **Extensibility:** Add more fields.

Project 184: Configure I2C to Read a Proximity Sensor and Trigger an Action

Description: Configure I2C to read a proximity sensor (e.g., VL53L0X) and trigger an action.

Requirements:

- Read distance as uint16_t, trigger GPIO if < 100 mm.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read distance register.
- **Action Trigger:** Set GPIO based on threshold.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Configure GPIO
    Write sensor config
FUNCTION read_proximity
    Read 2 bytes
    distance = (byte0 << 8) | byte1
    IF distance < 100:
        Set GPIO high
    ELSE:
        Set GPIO low
    RETURN distance
MAIN
    init_system()
    WHILE true:
        distance = read_proximity()
        process_distance(distance)

```

C Code Implementation

```
#include <stdint.h>

#define THRESHOLD 100

void configure_i2c(void) { /* I2C config */ }

void configure_gpio(void) { /* GPIO config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void set_gpio(uint8_t state) { /* Set GPIO */ }

void process_distance(uint16_t distance) { /* Process distance */ }

void init_system(void) {
    configure_i2c();
    configure_gpio();
    i2c_write(0x29, 0x00, 0x01); // Configure VL53L0X
}

uint16_t read_proximity(void) {
    uint8_t buffer[2];
    i2c_read(0x29, 0x14, buffer, 2); // Read distance
    uint16_t distance = (buffer[0] << 8) | buffer[1];
    set_gpio(distance < THRESHOLD ? 1 : 0);
    return distance;
}

int main(void) {
    init_system();
    while (1) {
        uint16_t distance = read_proximity();
        process_distance(distance);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Adjust threshold per application.
- **Testing:** Verify with known distances.
- **Optimization:** Use interrupt-driven reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add range configuration.

Project 185: Use SPI to Control an RGB LED Strip

Description: Create a project to use SPI to control an RGB LED strip (e.g., WS2812).

Requirements:

- Set colors for 4 LEDs.
- Assumptions: SPI interface, bit-banging for WS2812 protocol.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send RGB data using SPI for WS2812 timing.

- **Color Formatting:** Convert RGB values to WS2812 bitstream.

Complexity

- **Time Complexity:** $O(n)$ for n LEDs.
- **Space Complexity:** $O(n)$ for data buffer.

Pseudocode

```

FUNCTION init_led_strip
    Configure SPI for WS2812 timing
FUNCTION set_colors(colors)
    Convert RGB to WS2812 bitstream
    Send via SPI
MAIN
    init_led_strip()
    colors = {RGB values}
    set_colors(colors)

```

C Code Implementation

```

#include <stdint.h>

#define LED_COUNT 4

typedef struct {
    uint8_t r, g, b;
} Color;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint16_t len) { /* SPI write */ }

void init_led_strip(void) {
    configure_spi(); // Configure for ~800 kHz
}

void set_colors(Color* colors) {
    uint8_t buffer[LED_COUNT * 24];
    for (uint8_t i = 0; i < LED_COUNT; i++) {
        for (uint8_t j = 0; j < 8; j++) {
            buffer[i * 24 + j] = (colors[i].g >> (7 - j)) & 1 ? 0xFC : 0xC0;
            buffer[i * 24 + 8 + j] = (colors[i].r >> (7 - j)) & 1 ? 0xFC : 0xC0;
            buffer[i * 24 + 16 + j] = (colors[i].b >> (7 - j)) & 1 ? 0xFC : 0xC0;
        }
    }
    spi_write(buffer, LED_COUNT * 24);
}

int main(void) {
    init_led_strip();
    Color colors[LED_COUNT] = {{255, 0, 0}, {0, 255, 0}, {0, 0, 255}, {255, 255, 255}};
    set_colors(colors);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Timing:** Match WS2812 protocol timing.
- **Testing:** Verify colors with LEDs.
- **Optimization:** Use DMA for SPI.
- **Robustness:** Handle SPI errors.

- **Extensibility:** Support dynamic patterns.

Project 186: Use UART to Implement a Simple Handshake Protocol

Description: Implement a project to use UART to implement a simple handshake protocol.

Requirements:

- Send "RDY", wait for "ACK", then send data at 9600 baud.
- Assumptions: Commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Handshake:** Send "RDY", wait for "ACK", send data.

Complexity

- **Time Complexity:** $O(n)$ for n-byte data.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION handshake
    Send "RDY\r"
    Wait for "ACK\r"
    Send data
MAIN
    init_uart()
    WHILE true:
        handshake()
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

#define BAUD_RATE 9600
void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

void send_data(void) { /* Send data (stub) */ }

void init_uart(void) {
    configure_uart(BAUD_RATE);
}

void handshake(void) {
    uart_send("RDY\r");
    char buffer[8];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
```



```

        if (strcmp(buffer, "ACK") == 0) {
            send_data();
        }
        break;
    }
    buffer[i++] = c;
}

int main(void) {
    init_uart();
    while (1) {
        handshake();
    }
    return 0;
}

```

Expert Tips

- **Timeout:** Add timeout for ACK.
- **Testing:** Verify with a terminal.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle missing ACK.
- **Extensibility:** Add retry mechanism.

Project 187: Configure I2C to Interface with a Pressure Sensor and Log Data

Description: Configure I2C to interface with a pressure sensor (e.g., BMP280) and log data.

Requirements:

- Read pressure as `uint32_t`, store in buffer.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read pressure registers.
- **Data Logging:** Store readings in a circular buffer.

Complexity

- **Time Complexity:** $O(1)$ per read/log.
- **Space Complexity:** $O(n)$ for buffer size.

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Initialize circular buffer
FUNCTION read_pressure
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    Store pressure in buffer
MAIN
    init_system()
    WHILE true:
        read_pressure()

```

C Code Implementation

```
#include <stdint.h>

#define BUFFER_SIZE 10

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

uint32_t buffer[BUFFER_SIZE];

uint8_t buffer_index = 0;

void init_system(void) {
    configure_i2c();
    i2c_write(0x76, 0xF4, 0x27); // Configure BMP280
    for (uint8_t i = 0; i < BUFFER_SIZE; i++) buffer[i] = 0;
}

void read_pressure(void) {
    uint8_t data[3];
    i2c_read(0x76, 0xF7, data, 3); // Read pressure
    uint32_t pressure = ((uint32_t)data[0] << 16) | ((uint32_t)data[1] << 8) | data[2];
    buffer[buffer_index] = pressure;
    buffer_index = (buffer_index + 1) % BUFFER_SIZE;
}

int main(void) {
    init_system();
    while (1) {
        read_pressure();
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Apply compensation formulas.
- **Testing:** Verify with known pressures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add temperature logging.

Project 188: Use SPI to Read a Multi-Axis Accelerometer

Description: Create a project to use SPI to read a multi-axis accelerometer (e.g., ADXL345).

Requirements:

- Read 3-axis data (x, y, z as int16_t).
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read 6 bytes for accelerometer data.
- **Data Parsing:** Combine bytes into int16_t for each axis.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_accelerometer
    Configure SPI
    Write accelerometer config
FUNCTION read_accelerometer(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
MAIN
    init_accelerometer()
    WHILE true:
        read_accelerometer(&data)
        process_data(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    int16_t x, y, z;
} AccelData;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_data(AccelData* data) { /* Process data */ }

void init_accelerometer(void) {
    configure_spi();
    spi_write(0x2D, 0x08); // Enable ADXL345
}

void read_accelerometer(AccelData* data) {
    uint8_t buffer[6];
    spi_read(0x32 | 0x80, buffer, 6); // Auto-increment read
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_accelerometer();
    AccelData data;
    while (1) {
        read_accelerometer(&data);
        process_data(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Remove accelerometer bias.
- **Testing:** Verify with known movements.
- **Optimization:** Use burst reads.

- **Robustness:** Handle SPI errors.
- **Extensibility:** Add interrupt-based reads.

Project 189: Use UART to Send Sensor Data with Error Checking

Description: Implement a project to use UART to send sensor data with error checking.

Requirements:

- Send 2-byte sensor data with 1-byte checksum at 9600 baud.
- Assumptions: Timer for periodic sends, simple XOR checksum.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Checksum Calculation:** Compute XOR of data bytes.
- **Packet Sending:** Send data and checksum periodically.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    value = read_sensor()
    checksum = XOR of value bytes
    Send value and checksum
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send_bytes(uint8_t* data, uint8_t len) { /* Send bytes */ }

uint16_t read_sensor(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    uint8_t data[3] = {value >> 8, value & 0xFF, (value >> 8) ^ (value & 0xFF)};
    uart_send_bytes(data, 3);
}
```

```

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Checksum:** Use CRC for better error detection.
- **Testing:** Verify with a receiver checking checksum.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle transmission errors.
- **Extensibility:** Add header for framing.

Project 190: Configure I2C to Control a Multi-Channel LED Driver

Description: Configure I2C to control a multi-channel LED driver (e.g., PCA9685).

Requirements:

- Set 4 LED channels to specified 12-bit PWM values.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send PWM values to driver registers.
- **PWM Formatting:** Format 12-bit values per channel.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_driver
    Configure I2C
    Write driver config
FUNCTION set_leds(values)
    FOR each channel:
        Send LED register address and 12-bit value
MAIN
    init_driver()
    values = {val1, val2, val3, val4}
    set_leds(values)

```

C Code Implementation

```

#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

```

```

void init_driver(void) {
    configure_i2c();
    uint8_t config[] = {0x00, 0x00, 0x01, 0x04}; // Enable PCA9685, set prescale
    i2c_write(0x40, config, sizeof(config));
}

void set_leds(uint16_t* values) {
    uint8_t buffer[16];
    for (uint8_t i = 0; i < 4; i++) {
        buffer[i * 4] = 0x06 + i * 4; // LED register
        buffer[i * 4 + 1] = 0x00; // On time low
        buffer[i * 4 + 2] = 0x00; // On time high
        buffer[i * 4 + 3] = values[i] & 0xFF; // Off time low
        buffer[i * 4 + 4] = values[i] >> 8; // Off time high
    }
    i2c_write(0x40, buffer, 16);
}

int main(void) {
    init_driver();
    uint16_t values[4] = {1000, 2000, 3000, 4000};
    set_leds(values);
    while (1) {
        /* Idle */
    }
    return 0;
}

```

Expert Tips

- **Brightness:** Verify PWM output with LEDs.
- **Testing:** Test with known values.
- **Optimization:** Minimize I2C transactions.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support more channels.

Project 191: Use SPI to Interface with a Touch Screen Controller

Description: Create a project to use SPI to interface with a touch screen controller (e.g., XPT2046).

Requirements:

- Read touch coordinates (x, y) as uint16_t values.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send command to read X and Y coordinates.
- **Data Parsing:** Extract 12-bit coordinates from received bytes.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_touch
    Configure SPI

```

```

FUNCTION read_touch(x, y)
    Send X command, read 2 bytes
    x = (byte0 << 5) | (byte1 >> 3)
    Send Y command, read 2 bytes
    y = (byte0 << 5) | (byte1 >> 3)
MAIN
    init_touch()
    WHILE true:
        read_touch(&x, &y)
        process_touch(x, y)

```

C Code Implementation

```

#include <stdint.h>

typedef struct {
    uint16_t x, y;
} TouchData;

void configure_spi(void) { /* SPI config */ }

void spi_write_read(uint8_t* tx, uint8_t* rx, uint8_t len) { /* SPI write/read */ }

void process_touch(TouchData* data) { /* Process touch */ }

void init_touch(void) {
    configure_spi();
}

void read_touch(TouchData* data) {
    uint8_t tx[3] = {0xD0, 0x00, 0x00}; // X command
    uint8_t rx[3];
    spi_write_read(tx, rx, 3);
    data->x = ((rx[1] << 5) | (rx[2] >> 3)) & 0xFFF;
    tx[0] = 0x90; // Y command
    spi_write_read(tx, rx, 3);
    data->y = ((rx[1] << 5) | (rx[2] >> 3)) & 0xFFF;
}

int main(void) {
    init_touch();
    TouchData data;
    while (1) {
        read_touch(&data);
        process_touch(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Map coordinates to screen pixels.
- **Testing:** Verify with known touch points.
- **Optimization:** Use interrupt-driven SPI.
- **Robustness:** Filter noisy readings.
- **Extensibility:** Add pressure sensing.

Project 192: Use UART to Implement a Basic Terminal Interface

Description: Implement a project to use UART to implement a basic terminal interface.

Requirements:

- Accept commands like "ON" or "OFF" to control a GPIO at 9600 baud.

- Assumptions: Commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud, 8N1.
- **Command Parsing:** Read commands, toggle GPIO accordingly.

Complexity

- **Time Complexity:** $O(n)$ for n-character command.
- **Space Complexity:** $O(n)$ for input buffer.

Pseudocode

```

FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure GPIO
FUNCTION handle_command
    Read UART until '\r'
    IF command = "ON":
        Set GPIO high
    ELSE IF command = "OFF":
        Set GPIO low
MAIN
    init_system()
    WHILE true:
        handle_command()

```

C Code Implementation

```

#include <stdint.h>
#include <string.h>

#define BAUD_RATE 9600

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_gpio(void) { /* GPIO config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

void set_gpio(uint8_t state) { /* Set GPIO */ }

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_gpio();
}

void handle_command(void) {
    char buffer[8];
    uint8_t i = 0;
    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
            if (strcmp(buffer, "ON") == 0) {
                set_gpio(1);
                uart_send("GPIO ON\r\n");
            } else if (strcmp(buffer, "OFF") == 0) {
                set_gpio(0);
                uart_send("GPIO OFF\r\n");
            }
        }
        i++;
    }
}

```



```

        break;
    }
    buffer[i++] = c;
}
}

int main(void) {
    init_system();
    while (1) {
        handle_command();
    }
    return 0;
}

```

Expert Tips

- **Validation:** Check for valid commands.
- **Testing:** Verify with a terminal.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle invalid commands.
- **Extensibility:** Add more commands.

Project 193: Configure I2C to Read a Gas Sensor and Trigger an Alarm

Description: Configure I2C to read a gas sensor (e.g., CCS811) and trigger an alarm.

Requirements:

- Read CO2 as uint16_t, trigger GPIO if > 1000 ppm.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read CO2 register.
- **Alarm Trigger:** Set GPIO if threshold exceeded.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Configure GPIO
    Write sensor config
FUNCTION read_gas
    Read 2 bytes for CO2
    co2 = (byte0 << 8) | byte1
    IF co2 > 1000:
        Set GPIO high
    ELSE:
        Set GPIO low
    RETURN co2

MAIN
    init_system()
    WHILE true:

```

```
co2 = read_gas()
process_co2(co2)
```

C Code Implementation

```
#include <stdint.h>

#define THRESHOLD 1000

void configure_i2c(void) { /* I2C config */ }

void configure_gpio(void) { /* GPIO config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void set_gpio(uint8_t state) { /* Set GPIO */ }

void process_co2(uint16_t co2) { /* Process CO2 */ }

void init_system(void) {
    configure_i2c();
    configure_gpio();
    i2c_write(0x5A, 0x01, 0x10); // Start CCS811
}

uint16_t read_gas(void) {
    uint8_t buffer[2];
    i2c_read(0x5A, 0x02, buffer, 2); // Read CO2
    uint16_t co2 = (buffer[0] << 8) | buffer[1];
    set_gpio(co2 > THRESHOLD ? 1 : 0);
    return co2;
}

int main(void) {
    init_system();
    while (1) {
        uint16_t co2 = read_gas();
        process_co2(co2);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Initialize sensor per datasheet.
- **Testing:** Verify with known gas levels.
- **Optimization:** Use interrupt-driven reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add VOC reading.

Project 194: Use SPI to Control a High-Speed DAC

Description: Create a project to use SPI to control a high-speed DAC (e.g., AD5662).

Requirements:

- Set 16-bit DAC output value.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send 16-bit value to DAC.
- **Command Formatting:** Format per datasheet.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_dac
    Configure SPI
FUNCTION set_dac(value)
    Send 3 bytes (command + 16-bit value)
MAIN
    init_dac()
    set_dac(value)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void init_dac(void) {
    configure_spi();
}

void set_dac(uint16_t value) {
    uint8_t data[3] = {0x20, value >> 8, value & 0xFF}; // Write command
    spi_write(data, 3);
}

int main(void) {
    init_dac();
    set_dac(32768); // Mid-scale
    while (1) {
        /* Idle */
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify output with a multimeter.
- **Testing:** Test with known values.
- **Optimization:** Use high-speed SPI mode.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support multiple channels.

Project 195: Use UART to Send a Compressed Data Packet

Description: Implement a project to use UART to send a compressed data packet.

Requirements:

- Compress 4-byte sensor data to 2 bytes, send at 9600 baud.

- Assumptions: Simple run-length encoding (RLE), timer for sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Compression:** Use RLE to compress data.
- **Timer Trigger:** Send compressed packet periodically.

Complexity

- **Time Complexity:** $O(n)$ for n-byte data compression.
- **Space Complexity:** $O(n)$ for buffer.

Pseudocode

```

FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION compress_data(data, compressed)
    Apply RLE to data
    RETURN compressed size
FUNCTION timer_interrupt
    data = read_sensor()
    size = compress_data(data, compressed)
    Send compressed bytes
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send_bytes(uint8_t* data, uint8_t len) { /* Send bytes */ }

void read_sensor(uint8_t* data) { /* Read 4 bytes (stub) */ }

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

uint8_t compress_data(uint8_t* data, uint8_t* compressed) {
    compressed[0] = data[0]; // Simple RLE: take first byte
    compressed[1] = data[1]; // and second byte
    return 2;
}

void timer_isr(void) {
    uint8_t data[4], compressed[2];
    read_sensor(data);
    uint8_t size = compress_data(data, compressed);
    uart_send_bytes(compressed, size);
}

```

```

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Compression:** Use better algorithms for efficiency.
- **Testing:** Verify decompression on receiver.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle compression errors.
- **Extensibility:** Support variable-length data.

Project 196: Configure I2C to Interface with a UV Sensor and Log Data

Description: Configure I2C to interface with a UV sensor (e.g., VEML6070) and log data.

Requirements:

- Read UV index as uint8_t, store in buffer.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read UV register.
- **Data Logging:** Store readings in a circular buffer.

Complexity

- **Time Complexity:** $O(1)$ per read/log.
- **Space Complexity:** $O(n)$ for buffer size.

Pseudocode

```

FUNCTION init_system
    Configure I2C
    Initialize circular buffer
FUNCTION read_uv
    Read 1 byte
    Store UV index in buffer
MAIN
    init_system()
    WHILE true:
        read_uv()

```

C Code Implementation

```

#include <stdint.h>

#define BUFFER_SIZE 10

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

```

```

uint8_t buffer[BUFFER_SIZE];

uint8_t buffer_index = 0;

void init_system(void) {
    configure_i2c();
    i2c_write(0x38, 0x00, 0x02); // Configure VEML6070
    for (uint8_t i = 0; i < BUFFER_SIZE; i++) buffer[i] = 0;
}

void read_uv(void) {
    uint8_t data;
    i2c_read(0x39, 0x00, &data, 1); // Read UV
    buffer[buffer_index] = data;
    buffer_index = (buffer_index + 1) % BUFFER_SIZE;
}

int main(void) {
    init_system();
    while (1) {
        read_uv();
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Map UV index to real values.
- **Testing:** Verify with known UV sources.
- **Optimization:** Use burst reads if supported.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add timestamp logging.

Project 197: Use SPI to Read a Magnetometer and Calculate Direction

Description: Create a project to use SPI to read a magnetometer (e.g., LIS3MDL) and calculate direction.

Requirements:

- Read 3-axis data, compute heading in degrees.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read 6 bytes for magnetometer data.
- **Direction Calculation:** Use $\text{atan2}(y, x)$ for heading.

Complexity

- **Time Complexity:** $O(1)$ per read/calculation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_magnetometer
    Configure SPI
    Write magnetometer config
FUNCTION read_magnetometer(data)
    Read 6 bytes

```

```

    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
FUNCTION calculate_direction(data)
    heading = atan2(data.y, data.x) * 180 / PI
    RETURN heading
MAIN
    init_magnetometer()
    WHILE true:
        read_magnetometer(&data)
        heading = calculate_direction(data)
        process_heading(heading)

```

C Code Implementation

```

#include <stdint.h>
#include <math.h>

typedef struct {
    int16_t x, y, z;
} MagData;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_heading(float heading) { /* Process heading */ }

void init_magnetometer(void) {
    configure_spi();
    spi_write(0x20, 0x70); // Configure LIS3MDL
}

void read_magnetometer(MagData* data) {
    uint8_t buffer[6];
    spi_read(0x28 | 0x80, buffer, 6); // Auto-increment read
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

float calculate_direction(MagData* data) {
    return atan2(data->y, data->x) * 180.0f / M_PI;
}

int main(void) {
    init_magnetometer();
    MagData data;
    while (1) {
        read_magnetometer(&data);
        float heading = calculate_direction(&data);
        process_heading(heading);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Apply hard/soft iron corrections.
- **Testing:** Verify with known orientations.
- **Optimization:** Use fixed-point math for atan2.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add tilt compensation.

Project 198: Use UART to Implement a Simple ACK/NACK Protocol

Description: Implement a project to use UART to implement a simple ACK/NACK protocol.

Requirements:

- Send data, wait for "ACK" or "NACK" response at 9600 baud.
- Assumptions: Commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Protocol:** Send data, wait for ACK/NACK, resend on NACK.

Complexity

- **Time Complexity:** $O(n)$ for n-byte data.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION send_with_ack(data)
    Send data
    Wait for "ACK" or "NACK"
    IF "NACK":
        Resend data
    RETURN success
MAIN
    init_uart()
    WHILE true:
        send_with_ack(data)
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

#define BAUD_RATE 9600

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

void get_data(char* data) { strcpy(data, "DATA"); } // Stub

void init_uart(void) {
    configure_uart(BAUD_RATE);
}

uint8_t send_with_ack(void) {
    char data[16];
    get_data(data);
    uart_send(data);
    char buffer[8];
    uint8_t i = 0;
```



```

    while (i < sizeof(buffer) - 1) {
        char c = uart_read();
        if (c == '\r') {
            buffer[i] = '\0';
            if (strcmp(buffer, "ACK") == 0) return 1;
            if (strcmp(buffer, "NACK") == 0) return 0;
            break;
        }
        buffer[i++] = c;
    }
    return 0;
}

int main(void) {
    init_uart();
    while (1) {
        if (!send_with_ack()) {
            // Resend on NACK
        }
    }
    return 0;
}

```

Expert Tips

- **Timeout:** Add timeout for response.
- **Testing:** Verify with a terminal.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle missing responses.
- **Extensibility:** Add retry limit.

Project 199: Configure I2C to Control a Digital-to-Analog Converter for Audio

Description: Configure I2C to control a digital-to-analog converter (e.g., PCM5102) for audio.

Requirements:

- Set 16-bit audio samples for playback.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Send 16-bit audio samples to DAC.
- **Sample Formatting:** Format per DAC datasheet.

Complexity

- **Time Complexity:** $O(1)$ per sample.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_dac
    Configure I2C
    Write DAC config
FUNCTION play_audio(sample)
    Send 16-bit sample via I2C

```

```

MAIN
    init_dac()
    WHILE true:
        sample = get_audio_sample()
        play_audio(sample)

```

C Code Implementation

```

#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t* data, uint8_t len) { /* I2C write */ }

uint16_t get_audio_sample(void) { return 0; } // Stub

void init_dac(void) {
    configure_i2c();
    uint8_t config[] = {0x01, 0x00}; // Configure PCM5102
    i2c_write(0x4C, config, sizeof(config));
}

void play_audio(uint16_t sample) {
    uint8_t data[2] = {sample >> 8, sample & 0xFF};
    i2c_write(0x4C, data, 2);
}

int main(void) {
    init_dac();
    while (1) {
        uint16_t sample = get_audio_sample();
        play_audio(sample);
    }
    return 0;
}

```

Expert Tips

- **Timing:** Ensure sample rate matches audio.
- **Testing:** Verify with an oscilloscope.
- **Optimization:** Use DMA for I2C.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support stereo channels.

Project 200: Use SPI to Interface with a High-Speed Pressure Sensor

Description: Create a project to use SPI to interface with a high-speed pressure sensor (e.g., MS5611).

Requirements:

- Read pressure as `uint32_t`.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read pressure registers.
- **Data Parsing:** Apply sensor-specific compensation.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_sensor
    Configure SPI
    Write sensor config
FUNCTION read_pressure
    Send read command
    Read 3 bytes
    pressure = (byte0 << 16) | (byte1 << 8) | byte2
    RETURN pressure
MAIN
    init_sensor()
    WHILE true:
        pressure = read_pressure()
        process_pressure(pressure)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_pressure(uint32_t pressure) { /* Process pressure */ }

void init_sensor(void) {
    configure_spi();
    spi_write(0x1E, 0x00); // Reset MS5611
}

uint32_t read_pressure(void) {
    uint8_t buffer[3];
    spi_read(0x48, buffer, 3); // Read pressure
    return ((uint32_t)buffer[0] << 16) | ((uint32_t)buffer[1] << 8) | buffer[2];
}

int main(void) {
    init_sensor();
    while (1) {
        uint32_t pressure = read_pressure();
        process_pressure(pressure);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Use datasheet compensation formulas.
- **Testing:** Verify with known pressures.
- **Optimization:** Use burst reads.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Add temperature reading.

Project 201: Use UART to Send Sensor Data with a Timestamp

Description: Implement a project to use UART to send sensor data with a timestamp.

Requirements:

- Send data like "Value: XX, Time: HH:MM:SS" at 9600 baud.
- Assumptions: Timer-based timestamp.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Timestamp Generation:** Use system timer for HH:MM:SS.
- **Data Formatting:** Format sensor value and timestamp as string.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for timestamp
FUNCTION send_data
    value = read_sensor()
    time = get_timestamp()
    Format string "Value: XX, Time: HH:MM:SS"
    Send via UART
MAIN
    init_system()
    WHILE true:
        send_data()
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

typedef struct {
    uint8_t hour, minute, second;
} Time;

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

uint16_t read_sensor(void) { return 0; } // Stub

void get_timestamp(Time* time) { /* Get time (stub) */ time->hour = time->minute = time->second = 0; }

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}
```

```

void send_data(void) {
    uint16_t value = read_sensor();
    Time time;
    get_timestamp(&time);
    char buffer[32];
    sprintf(buffer, sizeof(buffer), "Value: %u, Time: %02u:%02u:%02u\r\n",
        value, time.hour, time.minute, time.second);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        send_data();
    }
    return 0;
}

```

Expert Tips

- **Timestamp Source:** Use RTC for accuracy.
- **Testing:** Verify with a terminal.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add more sensor data.

Project 202: Configure I2C to Read a Temperature Sensor with Error Handling

Description: Configure I2C to read a temperature sensor (e.g., TMP102) with error handling.

Requirements:

- Read temperature as float, retry on I2C failure.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read temperature register with retry logic.
- **Error Handling:** Retry up to 3 times on failure.

Complexity

- **Time Complexity:** $O(1)$ per read (with retries).
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_sensor
    Configure I2C
FUNCTION read_temperature
    FOR retry = 0 to 2:
        Try read 2 bytes
        IF success:
            temp = (byte0 << 4) | (byte1 >> 4)
            RETURN temp / 16
        ELSE:
            Wait and retry
    RETURN error

```

```

MAIN
    init_sensor()
    WHILE true:
        temp = read_temperature()
        process_temperature(temp)

```

C Code Implementation

```

#include <stdint.h>

#define MAX_RETRIES 3

void configure_i2c(void) { /* I2C config */ }

uint8_t i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { return 0; } // Stub
(0=success)

void delay_ms(uint16_t ms) { /* Delay */ }

void process_temperature(float temp) { /* Process temp */ }

void init_sensor(void) {
    configure_i2c();
}

float read_temperature(void) {
    uint8_t buffer[2];
    for (uint8_t retry = 0; retry < MAX_RETRIES; retry++) {
        if (i2c_read(0x48, 0x00, buffer, 2) == 0) {
            uint16_t raw = (buffer[0] << 4) | (buffer[1] >> 4);
            return raw * 0.0625f; // TMP102 conversion
        }
        delay_ms(10);
    }
    return -999.0f; // Error
}

int main(void) {
    init_sensor();
    while (1) {
        float temp = read_temperature();
        process_temperature(temp);
    }
    return 0;
}

```

Expert Tips

- **Error Handling:** Log retry failures.
- **Testing:** Simulate I2C failures.
- **Optimization:** Minimize retry delay.
- **Robustness:** Handle sensor faults.
- **Extensibility:** Add configuration options.

Project 203: Use SPI to Control a Multi-Channel LED Driver

Description: Create a project to use SPI to control a multi-channel LED driver (e.g., TLC5947).

Requirements:

- Set 4 LED channels to 12-bit PWM values.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send 12-bit PWM values for each channel.
- **Data Formatting:** Pack values into SPI buffer.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_driver
    Configure SPI
FUNCTION set_leds(values)
    Pack 4x12-bit values into buffer
    Send via SPI
MAIN
    init_driver()
    values = {val1, val2, val3, val4}
    set_leds(values)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint16_t len) { /* SPI write */ }

void init_driver(void) {
    configure_spi();
}

void set_leds(uint16_t* values) {
    uint8_t buffer[12]; // 4 channels * 3 bytes
    for (uint8_t i = 0; i < 4; i++) {
        buffer[i * 3] = values[i] >> 4;
        buffer[i * 3 + 1] = (values[i] & 0x0F) << 4 | (values[i] >> 8);
        buffer[i * 3 + 2] = values[i] & 0xFF;
    }
    spi_write(buffer, 12);
}

int main(void) {
    init_driver();
    uint16_t values[4] = {1000, 2000, 3000, 4000};
    set_leds(values);
    while (1) {
        /* Idle */
    }
    return 0;
}
```

Expert Tips

- **Brightness:** Verify PWM output with LEDs.
- **Testing:** Test with known values.
- **Optimization:** Use DMA for SPI.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support more channels.

Project 204: Use UART to Implement a Basic Checksum Protocol

Description: Implement a project to use UART to implement a basic checksum protocol.

Requirements:

- Send 2-byte data with 1-byte XOR checksum at 9600 baud.
- Assumptions: Timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Checksum Calculation:** Compute XOR of data bytes.
- **Packet Sending:** Send data and checksum.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    data = read_sensor()
    checksum = XOR of data bytes
    Send data and checksum
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send_bytes(uint8_t* data, uint8_t len) { /* Send bytes */ }

uint16_t read_sensor(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    uint16_t data = read_sensor();
    uint8_t buffer[3] = {data >> 8, data & 0xFF, (data >> 8) ^ (data & 0xFF)};
    uart_send_bytes(buffer, 3);
}
```



```
int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}
```

Expert Tips

- **Checksum:** Use CRC for better error detection.
- **Testing:** Verify with a receiver.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle transmission errors.
- **Extensibility:** Add packet header.

Project 205: Configure I2C to Interface with a Multi-Sensor Module and Parse Data

Description: Configure I2C to interface with a multi-sensor module (e.g., BME680) and parse data.

Requirements:

- Read temperature, pressure, and humidity as float values.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read sensor registers.
- **Data Parsing:** Apply compensation formulas for each sensor.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_sensor
    Configure I2C
    Write sensor config
FUNCTION read_sensors(data)
    Read 8 bytes for temp, pressure, humidity
    Apply compensation formulas
    data.temp = calculated temp
    data.pressure = calculated pressure
    data.humidity = calculated humidity
MAIN
    init_sensor()
    WHILE true:
        read_sensors(&data)
        process_data(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    float temp, pressure, humidity;
} SensorData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(SensorData* data) { /* Process data */ }

void init_sensor(void) {
    configure_i2c();
    i2c_write(0x76, 0xF4, 0x27); // Configure BME680
}

void read_sensors(SensorData* data) {
    uint8_t buffer[8];
    i2c_read(0x76, 0xF7, buffer, 8); // Read temp, pressure, humidity
    uint32_t raw_press = ((uint32_t)buffer[0] << 12) | ((uint32_t)buffer[1] << 4) | (buffer[2] >> 4);
    uint32_t raw_temp = ((uint32_t)buffer[3] << 12) | ((uint32_t)buffer[4] << 4) | (buffer[5] >> 4);
    uint16_t raw_hum = (buffer[6] << 8) | buffer[7];
    data->temp = raw_temp / 16384.0f; // Simplified
    data->pressure = raw_press / 256.0f; // Simplified
    data->humidity = raw_hum / 1024.0f; // Simplified
}

int main(void) {
    init_sensor();
    SensorData data;
    while (1) {
        read_sensors(&data);
        process_data(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Use datasheet formulas.
- **Testing:** Verify with known conditions.
- **Optimization:** Cache calibration data.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add gas sensor reading.

Project 206: Use SPI to Read a High-Resolution ADC

Description: Create a project to use SPI to read a high-resolution ADC (e.g., ADS1256).

Requirements:

- Read 24-bit ADC data.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send read command, receive 3 bytes.
- **Data Parsing:** Combine bytes into 24-bit value.

Complexity

- **Time Complexity:** $O(1)$ per read.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_adc
    Configure SPI
    Write ADC config
FUNCTION read_adc
    Send read command
    Read 3 bytes
    value = (byte0 << 16) | (byte1 << 8) | byte2
    RETURN value
MAIN
    init_adc()
    WHILE true:
        value = read_adc()
        process_value(value)
```

C Code Implementation

```
#include <stdint.h>

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_value(uint32_t value) { /* Process ADC value */ }

void init_adc(void) {
    configure_spi();
    spi_write(0x50, 0x03); // Configure ADS1256
}

uint32_t read_adc(void) {
    uint8_t buffer[3];
    spi_read(0x01, buffer, 3); // Read data
    return ((uint32_t)buffer[0] << 16) | ((uint32_t)buffer[1] << 8) | buffer[2];
}

int main(void) {
    init_adc();
    while (1) {
        uint32_t value = read_adc();
        process_value(value);
    }
    return 0;
}
```

Expert Tips

- **Accuracy:** Verify with known voltages.
- **Testing:** Test with known inputs.
- **Optimization:** Use DMA for continuous reads.
- **Robustness:** Handle SPI errors.

- **Extensibility:** Support multiple channels.

Project 207: Use UART to Send Sensor Data in a JSON Format

Description: Implement a project to use UART to send sensor data in JSON format.

Requirements:

- Send data like {"value":XX} at 9600 baud.
- Assumptions: Timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **JSON Formatting:** Format sensor data as JSON string.
- **Timer Trigger:** Send data periodically.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
FUNCTION timer_interrupt
    value = read_sensor()
    Format JSON string {"value":XX}
    Send via UART
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */
```

C Code Implementation

```
#include <stdint.h>
#include <stdio.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

uint16_t read_sensor(void) { return 0; } // Stub

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    char buffer[32];
    snprintf(buffer, sizeof(buffer), "{\"value\":%u}\r\n", value);
    uart_send(buffer);
}
```

```

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Formatting:** Ensure valid JSON syntax.
- **Testing:** Verify with a JSON parser.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add more fields.

Project 208: Configure I2C to Control a Real-Time Clock with Alarm Settings

Description: Configure I2C to control a real-time clock (e.g., DS3231) with alarm settings.

Requirements:

- Set and check alarm for a specific time.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Set alarm registers, read status.
- **Alarm Check:** Trigger action on alarm match.

Complexity

- **Time Complexity:** $O(1)$ per operation.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_rtc
    Configure I2C
    Set alarm time
FUNCTION check_alarm
    Read alarm status
    IF alarm triggered:
        Clear alarm
        Trigger action
    RETURN status
MAIN
    init_rtc()
    WHILE true:
        check_alarm()

```

C Code Implementation

```

#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

```

```

uint8_t i2c_read(uint8_t addr, uint8_t reg) { return 0; } // Stub

void trigger_action(void) { /* Trigger action */ }

void init_rtc(void) {
    configure_i2c();
    i2c_write(0x68, 0x07, 0x00); // Set alarm (example: 00:00:00)
    i2c_write(0x68, 0x0E, 0x80); // Enable alarm
}

uint8_t check_alarm(void) {
    uint8_t status = i2c_read(0x68, 0x0F);
    if (status & 0x01) {
        i2c_write(0x68, 0x0F, status & ~0x01); // Clear alarm
        trigger_action();
        return 1;
    }
    return 0;
}

int main(void) {
    init_rtc();
    while (1) {
        check_alarm();
    }
    return 0;
}

```

Expert Tips

- **Accuracy:** Verify RTC and alarm timing.
- **Testing:** Test with known times.
- **Optimization:** Use interrupts for alarms.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Support multiple alarms.

Project 209: Use SPI to Interface with Flash Memory for Data Logging

Description: Create a project to use SPI to interface with flash memory (e.g., W25Q64) for data logging.

Requirements:

- Log sensor data to flash.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Write data to flash memory.
- **Data Logging:** Append sensor data to sequential addresses.

Complexity

- **Time Complexity:** $O(n)$ for n-byte write.
- **Space Complexity:** $O(n)$ for data buffer.

Pseudocode

```
FUNCTION init_flash
    Configure SPI
FUNCTION log_data(data)
    Send write enable
    Send write command
    Send address
    Send data
    Wait for write completion
MAIN
    init_flash()
    WHILE true:
        data = read_sensor()
        log_data(data)
```

C Code Implementation

```
#include <stdint.h>

#define PAGE_SIZE 256

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void delay_ms(uint16_t ms) { /* Delay */ }

uint8_t read_status(void) { return 0; } // Stub

uint16_t read_sensor(void) { return 0; } // Stub

void init_flash(void) {
    configure_spi();
}

void log_data(uint16_t data, uint32_t addr) {
    uint8_t write_en = 0x06;
    spi_write(&write_en, 1); // Write enable
    uint8_t cmd[5] = {0x02, (addr >> 16) & 0xFF, (addr >> 8) & 0xFF, addr & 0xFF, data & 0xFF};
    spi_write(cmd, 5);
    while (read_status() & 0x01) delay_ms(1);
}

int main(void) {
    init_flash();
    uint32_t addr = 0;
    while (1) {
        uint16_t data = read_sensor();
        log_data(data, addr);
        addr += 1;
    }
    return 0;
}
```

Expert Tips

- **Verification:** Read back data to confirm.
- **Testing:** Verify with known patterns.
- **Optimization:** Use page writes.
- **Robustness:** Handle write timeouts.
- **Extensibility:** Add erase commands.

Project 210: Use UART to Implement a Simple Retry Mechanism

Description: Implement a project to use UART to implement a simple retry mechanism.

Requirements:

- Send data, retry up to 3 times if no ACK at 9600 baud.
- Assumptions: Commands end with '\r'.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Retry Protocol:** Send data, wait for ACK, retry on timeout.

Complexity

- **Time Complexity:** $O(n)$ for n retries.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_uart
    Configure UART (9600 baud, 8N1)
FUNCTION send_with_retry(data)
    FOR retry = 0 to 2:
        Send data
        Wait for "ACK\r"
        IF received ACK:
            RETURN success
    RETURN failure
MAIN
    init_uart()
    WHILE true:
        send_with_retry(data)
```

C Code Implementation

```
#include <stdint.h>
#include <string.h>

#define BAUD_RATE 9600
#define MAX_RETRIES 3

void configure_uart(uint32_t baud) { /* UART config */ }

char uart_read(void) { return 0; } // Stub

void uart_send(const char* str) { /* Send string */ }

void delay_ms(uint16_t ms) { /* Delay */ }

void get_data(char* data) { strcpy(data, "DATA"); } // Stub

void init_uart(void) {
    configure_uart(BAUD_RATE);
}

uint8_t send_with_retry(void) {
    char data[16];
    get_data(data);
```



```

    for (uint8_t retry = 0; retry < MAX_RETRIES; retry++) {
        uart_send(data);
        char buffer[8];
        uint8_t i = 0;
        uint16_t timeout = 100;
        while (i < sizeof(buffer) - 1 && timeout--) {
            char c = uart_read();
            if (c == '\r') {
                buffer[i] = '\0';
                if (strcmp(buffer, "ACK") == 0) return 1;
                break;
            }
            buffer[i++] = c;
            delay_ms(1);
        }
    }
    return 0;
}

int main(void) {
    init_uart();
    while (1) {
        send_with_retry();
    }
    return 0;
}

```

Expert Tips

- **Timeout:** Adjust timeout for reliability.
- **Testing:** Simulate missing ACKs.
- **Optimization:** Use interrupt-driven UART.
- **Robustness:** Handle partial responses.
- **Extensibility:** Add NACK handling.

Project 211: Use SPI to Interface with a Gyroscope and Calculate Angular Velocity

Description: Create a project to use SPI to interface with a gyroscope (e.g., L3GD20) and calculate angular velocity.

Requirements:

- Read 3-axis data (x, y, z) as int16_t and compute angular velocity in degrees per second.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Read 6 bytes for gyroscope data.
- **Data Processing:** Convert raw data to angular velocity using sensitivity factor.

Complexity

- **Time Complexity:** $O(1)$ per read/calculation.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_gyroscope
    Configure SPI
    Write gyroscope config
FUNCTION read_gyroscope(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
    Convert to angular velocity (multiply by sensitivity)
MAIN
    init_gyroscope()
    WHILE true:
        read_gyroscope(&data)
        process_angular_velocity(data)
```

C Code Implementation

```
#include <stdint.h>

typedef struct {
    float x, y, z;
} GyroData;

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t reg, uint8_t value) { /* SPI write */ }

void spi_read(uint8_t reg, uint8_t* data, uint8_t len) { /* SPI read */ }

void process_angular_velocity(GyroData* data) { /* Process data */ }

void init_gyroscope(void) {
    configure_spi();
    spi_write(0x20, 0x0F); // Enable L3GD20, 250 dps
}

void read_gyroscope(GyroData* data) {
    uint8_t buffer[6];
    spi_read(0x28 | 0x80, buffer, 6); // Auto-increment read
    int16_t raw_x = (buffer[0] << 8) | buffer[1];
    int16_t raw_y = (buffer[2] << 8) | buffer[3];
    int16_t raw_z = (buffer[4] << 8) | buffer[5];
    data->x = raw_x * 0.00875f; // 250 dps sensitivity
    data->y = raw_y * 0.00875f;
    data->z = raw_z * 0.00875f;
}

int main(void) {
    init_gyroscope();
    GyroData data;
    while (1) {
        read_gyroscope(&data);
        process_angular_velocity(&data);
    }
    return 0;
}
```

Expert Tips

- **Calibration:** Remove bias from gyroscope readings.
- **Testing:** Verify with known rotations.
- **Optimization:** Use burst reads for efficiency.
- **Robustness:** Handle SPI errors.

- **Extensibility:** Add temperature compensation.

Project 212: Use UART to Send Sensor Data with Priority Levels

Description: Implement a project to use UART to send sensor data with priority levels.

Requirements:

- Send data with priority (e.g., "P1:XX" or "P2:XX") at 9600 baud.
- Assumptions: Timer-based sends, higher priority sent first.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Priority Queue:** Maintain a queue for high/low priority data.
- **Timer Trigger:** Send data based on priority.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(n)$ for queue size.

Pseudocode

```

FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for periodic sends
    Initialize priority queue
FUNCTION timer_interrupt
    IF high-priority queue not empty:
        Send high-priority data
    ELSE IF low-priority queue not empty:
        Send low-priority data

MAIN
    init_system()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>
#include <stdio.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000
#define QUEUE_SIZE 10

typedef struct {
    uint8_t priority;
    uint16_t value;
} DataEntry;

void configure_uart(uint32_t baud) { /* UART config */ }

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

uint8_t get_data(uint16_t* value, uint8_t* priority) { return 0; } // Stub

DataEntry queue[QUEUE_SIZE];

```

```

uint8_t queue_index = 0;

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
    for (uint8_t i = 0; i < QUEUE_SIZE; i++) queue[i].priority = 0;
}

void timer_isr(void) {
    char buffer[16];
    for (uint8_t i = 0; i < queue_index; i++) {
        if (queue[i].priority == 1) {
            snprintf(buffer, sizeof(buffer), "P1:%u\r\n", queue[i].value);
            uart_send(buffer);
            queue[i].priority = 0;
            return;
        }
    }
    for (uint8_t i = 0; i < queue_index; i++) {
        if (queue[i].priority == 2) {
            snprintf(buffer, sizeof(buffer), "P2:%u\r\n", queue[i].value);
            uart_send(buffer);
            queue[i].priority = 0;
            return;
        }
    }
}

int main(void) {
    init_system();
    while (1) {
        uint16_t value;
        uint8_t priority;
        if (get_data(&value, &priority) && queue_index < QUEUE_SIZE) {
            queue[queue_index++] = (DataEntry){priority, value};
        }
    }
    return 0;
}

```

Expert Tips

- **Queue Management:** Compact queue after sends.
- **Testing:** Verify priority order with a terminal.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle queue overflow.
- **Extensibility:** Add more priority levels.

Project 213: Configure I2C to Read a Light Sensor and Adjust PWM Output

Description: Configure I2C to read a light sensor (e.g., TSL2561) and adjust PWM output.

Requirements:

- Read light level as uint16_t, adjust PWM duty cycle proportionally.
- Assumptions: I2C interface, 100 kHz, PWM on GPIO.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read light intensity register.
- **PWM Adjustment:** Scale light level to PWM duty cycle.

Complexity

- **Time Complexity:** $O(1)$ per read/adjustment.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_system
    Configure I2C
    Configure PWM
FUNCTION read_light
    Read 2 bytes
    light = (byte0 << 8) | byte1
    duty = scale light to PWM range
    Set PWM duty cycle
    RETURN light
MAIN
    init_system()
    WHILE true:
        light = read_light()
        process_light(light)
```

C Code Implementation

```
#include <stdint.h>

void configure_i2c(void) { /* I2C config */ }

void configure_pwm(void) { /* PWM config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void set_pwm_duty(uint16_t duty) { /* Set PWM duty */ }

void process_light(uint16_t light) { /* Process light */ }

void init_system(void) {
    configure_i2c();
    configure_pwm();
    i2c_write(0x39, 0x80, 0x03); // Enable TSL2561
}

uint16_t read_light(void) {
    uint8_t buffer[2];
    i2c_read(0x39, 0x8C, buffer, 2); // Read channel 0
    uint16_t light = (buffer[0] << 8) | buffer[1];
    set_pwm_duty(light / 64); // Scale to 0-1023
    return light;
}

int main(void) {
    init_system();
    while (1) {
        uint16_t light = read_light();
        process_light(light);
    }
    return 0;
}
```

Expert Tips

- **Scaling:** Adjust PWM range for application.
- **Testing:** Verify with known light levels.
- **Optimization:** Use interrupt-driven reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add multiple channels.

Project 214: Use SPI to Control a Stepper Motor Driver

Description: Create a project to use SPI to control a stepper motor driver (e.g., DRV8825).

Requirements:

- Set motor steps and direction.
- Assumptions: SPI interface, 4-wire mode.

Ways to Solve

Algorithm Used

- **SPI Communication:** Send step count and direction commands.
- **Command Formatting:** Format per driver datasheet.

Complexity

- **Time Complexity:** $O(1)$ per update.
- **Space Complexity:** $O(1)$.

Pseudocode

```
FUNCTION init_driver
    Configure SPI
FUNCTION set_stepper(steps, direction)
    Send step count and direction via SPI
MAIN
    init_driver()
    set_stepper(steps, direction)
```

C Code Implementation

```
#include <stdint.h>

#define FORWARD 0x01
#define REVERSE 0x00

void configure_spi(void) { /* SPI config */ }

void spi_write(uint8_t* data, uint8_t len) { /* SPI write */ }

void init_driver(void) {
    configure_spi();
}

void set_stepper(uint16_t steps, uint8_t direction) {
    uint8_t data[3] = {direction, steps >> 8, steps & 0xFF};
    spi_write(data, 3);
}

int main(void) {
    init_driver();
    set_stepper(100, FORWARD);
}
```

```

while (1) {
    /* Idle */
}
return 0;
}

```

Expert Tips

- **Timing:** Ensure step timing matches motor.
- **Testing:** Verify motor movement.
- **Optimization:** Use high-speed SPI.
- **Robustness:** Handle SPI errors.
- **Extensibility:** Support microstepping.

Project 215: Use UART to Send Data with a Sequence Number

Description: Implement a project to use UART to send data with a sequence number.

Requirements:

- Send data like "SEQ:XX,DATA:YY" at 9600 baud.
- Assumptions: Timer for periodic sends.

Ways to Solve

Algorithm Used

- **UART Configuration:** Set UART for 9600 baud.
- **Sequence Number:** Increment for each send.
- **Data Formatting:** Format sequence and data as string.

Complexity

- **Time Complexity:** $O(1)$ per send.
- **Space Complexity:** $O(1)$.

Pseudocode

```

FUNCTION init_system
    Configure UART (9600 baud, 8N1)
    Configure timer for 1s interrupt
    seq_num = 0
FUNCTION timer_interrupt
    value = read_sensor()
    Format string "SEQ:XX,DATA:YY"
    Send via UART
    Increment seq_num
MAIN
    init_system()
    WHILE true:
        /* Wait for timer */

```

C Code Implementation

```

#include <stdint.h>
#include <stdio.h>

#define BAUD_RATE 9600
#define CLOCK_FREQ 1000000

void configure_uart(uint32_t baud) { /* UART config */ }

```

```

void configure_timer(uint32_t period) { /* Timer config */ }

void uart_send(const char* str) { /* Send string */ }

uint16_t read_sensor(void) { return 0; } // Stub

static uint16_t seq_num = 0;

void init_system(void) {
    configure_uart(BAUD_RATE);
    configure_timer(CLOCK_FREQ); // 1s
}

void timer_isr(void) {
    uint16_t value = read_sensor();
    char buffer[32];
    snprintf(buffer, sizeof(buffer), "SEQ:%u,DATA:%u\r\n", seq_num++, value);
    uart_send(buffer);
}

int main(void) {
    init_system();
    while (1) {
        /* Wait for timer */
    }
    return 0;
}

```

Expert Tips

- **Sequence Wrap:** Handle sequence number overflow.
- **Testing:** Verify with a terminal.
- **Optimization:** Use DMA for UART.
- **Robustness:** Handle buffer overflow.
- **Extensibility:** Add more data fields.

Project 216: Configure I2C to Read a Multi-Axis Magnetometer

Description: Configure I2C to read a multi-axis magnetometer (e.g., MAG3110).

Requirements:

- Read 3-axis data (x, y, z) as int16_t.
- Assumptions: I2C interface, 100 kHz.

Ways to Solve

Algorithm Used

- **I2C Communication:** Read 6 bytes for magnetometer data.
- **Data Parsing:** Combine bytes into int16_t for each axis.

Complexity

- **Time Complexity:** O(1) per read.
- **Space Complexity:** O(1).

Pseudocode

```

FUNCTION init_magnetometer
    Configure I2C
    Write magnetometer config

```



```

FUNCTION read_magnetometer(data)
    Read 6 bytes
    data.x = (byte0 << 8) | byte1
    data.y = (byte2 << 8) | byte3
    data.z = (byte4 << 8) | byte5
MAIN
    init_magnetometer()
    WHILE true:
        read_magnetometer(&data)
        process_data(data)

```

C Code Implementation

```

#include <stdint.h>

typedef struct {
    int16_t x, y, z;
} MagData;

void configure_i2c(void) { /* I2C config */ }

void i2c_write(uint8_t addr, uint8_t reg, uint8_t value) { /* I2C write */ }

void i2c_read(uint8_t addr, uint8_t reg, uint8_t* data, uint8_t len) { /* I2C read */ }

void process_data(MagData* data) { /* Process data */ }

void init_magnetometer(void) {
    configure_i2c();
    i2c_write(0x0E, 0x10, 0x01); // Enable MAG3110
}

void read_magnetometer(MagData* data) {
    uint8_t buffer[6];
    i2c_read(0x0E, 0x01, buffer, 6); // Read XYZ
    data->x = (buffer[0] << 8) | buffer[1];
    data->y = (buffer[2] << 8) | buffer[3];
    data->z = (buffer[4] << 8) | buffer[5];
}

int main(void) {
    init_magnetometer();
    MagData data;
    while (1) {
        read_magnetometer(&data);
        process_data(&data);
    }
    return 0;
}

```

Expert Tips

- **Calibration:** Apply hard/soft iron corrections.
- **Testing:** Verify with known magnetic fields.
- **Optimization:** Use burst reads.
- **Robustness:** Handle I2C errors.
- **Extensibility:** Add heading calculation.